UNIVERSIDAD ESAN

FACULTAD DE INGENIERÍA

INGENIERÍA DE SISTEMAS

Modelo híbrido para la detección de la anemia a partir de imágenes de las palmas de las manos utilizando Vision Transformer y Machine Learning

Trabajo Final del curso de Tesis

Katherine Alejandra Ascurra Risco

Melissa Maybel Taipe Vitanzo

Asesor: Wilfredo Mamani Ticona

Lima, junio de 2025

DEDICATORIA

A Dios, a mi Pia, a mis queridos padres Betty y Alejandro y a Jaime F. por su apoyo incondicional para concretar este proyecto en mi vida profesional.

Katherine Alejandra Ascurra Risco

A Dios, a mi padre Alejandro y a mis abuelitos, Papavita y Mamita, en el cielo. También a mi querida madre Yolanda y a mi pequeño Muffin, quienes con su amor y compañía me dieron la fuerza para no rendirme nunca.

Melissa Maybel Taipe Vitanzo

Índice

Resumen……7

Introducción..... 23

CAPÍTULO I: Planteamiento del Problema 25

1.1 Contextualización de la Realidad Problemática 25
1.2 Definición del Problema de Investigación 34
1.2.1 Problema Principal 34
1.2.2 Problemas Secundarios 34
1.3 Objetivos de la investigación 35
1.3.1 Objetivo general 35
1.3.2 Objetivos específicos 35
1.4. Justificación de la investigación 35
1.4.1. Fundamentación Teórica 35
1.4.2 Contribución Práctica 36
1.4.3 Aporte Metodológico 37
1.5 Delimitación de la investigación 37
1.5.1 Ámbito Espacial 37
1.5.2 Ámbito Temporal 38
1.5.3 Ámbito Conceptual 38
CAPÍTULO II: Marco Teórico 39

2.1 Antecedentes de la Investigación 39
2.2 Bases Teóricas 58
2.2.1 Anemia 58
2.2.2 Machine Learning (Aprendizaje automático) 62
2.2.3 Vision Transformer (Transformadores de visión) 95
2.3 Marco Conceptual 114
2.3.1 Hemoglobina: Definición 114
2.3.2 Detección preliminar de la anemia 114
2.3.3 Métodos de Diagnóstico Invasivo 115
2.3.4 Alternativas de Diagnóstico No Invasivo 115
2.3.5 Segmentación de Imágenes 116

2.3.6 Región de Interés (ROI - Region of Interest) 116
2.3.7 Preprocesamiento de Imágenes 116
2.3.8 Redimensionamiento de imágenes 116
2.3.9 Normalización de los valores de pixeles 117
2.3.10 Balanceo de clases 117
2.3.11 Extracción de características 117
2.3.12 Conjunto de entrenamiento y prueba 117
2.3.13 Clasificación de imágenes 118
2.3.14 Evaluación de Modelos 118
2.4 Hipótesis 118
2.4.1 Hipótesis general 118
2.4.2 Hipótesis específicas 118
CAPÍTULO III: Metodología 119

3.1 Diseño y Enfoque Metodológico 119
3.1.1 Estructura del Diseño de la Investigación 119
3.1.2 Tipo de Diseño Metodológico 120
3.1.3 Enfoque de la investigación 120
3.2 Población y Muestra de la investigación 121
3.2.1 Población 121
3.2.2 Muestra 122
3.2.3. Unidad de análisis 122
3.3 Operacionalización de Variables de la Investigación 122
3.4 Técnicas de Recolección de Datos 125
3.5 Metodología para la implementación de la solución. 126
3.5.1 Obtención del dataset 126
3.5.2 Preprocesamiento de datos 128
3.5.3 Modelado y clasificación 134
3.5.4 Evaluación de Modelos 147
3.5.5 Implementación Web 150
3.6 Cronograma de Actividades de la Investigación 151
CAPÍTULO IV: Desarrollo de la solución 153

4.1 Obtención del dataset 153
4.2 Preprocesamiento de datos 155
4.3 Modelado y Clasificación 173

4.4 Evaluación de Modelos 267
4.5 Resultados del proyecto 319
4.6 Despliegue del modelo en una aplicación web funcional 320
4.6.1 Modelo seleccionado para el despliegue 321
4.6.2 Arquitectura general del sistema 321
4.6.3 Flujo de funcionamiento del sistema 324
4.6.4 Interfaz funcional de la aplicación desplegada 326
CAPÍTULO V: Conclusiones y Recomendaciones 333

5.1 Conclusiones 333
5.2 Recomendaciones 334
Bibliografía 336

Anexos 342

Índice de Figuras

Figura 1 Porcentaje de la población con anemia en grupos de mayor riesgo, 2019 26

Figura 2 Distribución global de la anemia en niños en edad preescolar según país. 27

Figura 3 Perú: Proporción de niños de 6 a 35 meses con anemia, según zona de residencia. 29

Figura 4 Obtención de análisis de sangre 31

Figura 5 Una muestra del ROI de la palma de la mano 32

Figura 6 Diagrama de bloque de la metodología propuesta 40

Figura 7 Flujograma que ilustra el flujo de trabajo en los modelos propuestos 46

Figura 8 Un marco conceptual de la metodología 48

Figura 9 Diagrama de flujo de metodología propuesta para procesamiento de imágenes 51

Figura 10 Metodología propuesta 55

Figura 11 Conjuntiva pálida vs conjuntiva normal 59

Figura 12 Palidez de los pliegues palmares de la mano del paciente (flecha roja) en comparación con la mano del examinador (flecha azul) 60

Figura 13 Ejemplo de imágenes de palidez en lengua (Class0: normal y Class 1: pálido) 61

Figura 14 Imágenes ejemplares de manos de pacientes con diferentes niveles de hemoglobina total en sangre 62

Figura 15 Pasos de Machine Learning 64

Figura 16 Tipos de Machine Learning 65

Figura 17 Aprendizaje supervisado 67

Figura 18 Aprendizaje no supervisado 68

Figura 19 Aprendizaje de refuerzo 69

Figura 20 Gráfico de los vectores de soporte 71

Figura 21 Fórmula de hiperplano y los márgenes que lo acompañan 72

Figura 22 72

Figura 23 Trazado de los puntos en las 3 dimensiones 73

Figura 24 El hiperplano en 3D de dos sets de puntos 74

Figura 25 Uso de kernel lineal SVM 75

Figura 26 Uso de kernel RBF para el dataset público "Iris" 75

Figura 27 Clasificación del dataset público "Iris" usando kernel polinominal de varios grados 76

Figura 28 Representación del parámetro C con un valor alto 77

Figura 29 Representación del parámetro C con un valor bajo 77

Figura 30 Representación del parámetro Gamma con un valor bajo 78

Figura 31 Representación del parámetro Gamma con un valor alto 79

Figura 32 Visualización de algoritmos de Random Forest 80

Figura 33 Clasificación de una instancia en función de los vecinos más cercanos 84

Figura 34 Clasificación del conjunto de datos pública Iris para diferentes valores de K 85

Figura 35 Modificación de la frontera de decisión al variar el número de vecinos k 86

Figura 36 Un árbol de decisiones para distinguir entre varios animales 91

Figura 37 Visualización de proceso jerárquico de división de datos en un árbol de decisión 94

Figura 38. Comparación entre Aprendizaje Automático y Aprendizaje Profundo en la extracción de características y clasificación..... 96

Figura 39 Comparación estructural entre el bloque básico de ResNet y el Bottleneck Transfomer 99

Figura 40 Clasificación jerárquica de modelos de atención para reconocimiento visual 99

Figura 41 Diagrama de la capa de autoatención utilizada en BOTNet 100

Figura 42 103

Figura 43 Arquitectura general jerárquica de Swin Transformer (Swin-T)..... 105

Figura 44 Dos bloques Swin Transformer sucesivos con W-MSA y SW-MSA 106

Figura 45 Estructura de la capa de pooling en la arquitectura PiT 108

Figura 46 109

Figura 47 Cada píxel ve a todos los demás píxeles mediante atención en MobileViT..... 112

Figura 48 Comparación entre ViT estándar y MobileViT 113

Figura 49 Metodología de la implementación de la solución 126

Figura 50 Pseudocódigo de la preparación y normalización de imágenes..... 138

Figura 51 Pseudocódigo para la definición de arquitecturas 140

Figura 52 Pseudocódigo del entrenamiento de modelos de Deep Learning 141

Figura 53 Pseudocódigo de la evaluación del modelo de Deep Learning 143

Figura 54 Cronograma de actividades..... 151

Figura 55 ROI de la Palma de la mano 153

Figura 56 Imagen fotográfica encontrada en el dataset de título “Anemic-268”..... 155

Figura 57 Script en Python para la detección automática de imágenes aumentadas 157

Figura 58 Nombres de imágenes originales y aumentadas 157

Figura 59 Script en Python para el filtrado, copia y resumen de imágenes originales 158

Figura 60 Imágenes detectadas con sombras oscuras 159

Figura 61 Imágenes con baja resolución o nitidez 160

Figura 62 Imágenes detectadas con recorte excesivo 160

Figura 63 Script para identificación y conteo de imágenes por clase 162

Figura 64 Script para el renombramiento y organización de imágenes por clase 163

Figura 65 Script para la verificación de resoluciones originales de las imágenes 164

Figura 66 Resultados de las resoluciones detectadas en las imágenes originales 165

Figura 67 Script para el redimensionamiento de imágenes a 224x224 píxeles 166

Figura 68 Script para la verificación de resoluciones después del redimensionamiento 167

Figura 69 Script para el aumento de datos y generación de imágenes por clase 169

Figura 70 Script de validación del número total de imágenes por clase después del aumento 170

Figura 71 Script para la creación de carpetas y conteo inicial de imágenes por clase 171

Figura 72 Script para la división aleatoria de imágenes y verificación del conteo final 172

Figura 73 Script sobre transformación de Datos BOTNet 174

Figura 74 Script sobre la carga del modelo BOTNet 175

Figura 75 Script de entrenamiento inicial del modelo BOTNet base 177

Figura 76 Script de Validación y early stopping del modelo BOTNet base. 178

Figura 77 Configuración y preparación del modelo BOTNet con Tuning 180

Figura 78 Entrenamiento y validación del modelo BOTNet con Tuning 181

Figura 79 Script de evaluación estructurada del modelo BOTNet base sobre el conjunto de prueba 183

Figura 80 Script de evaluación estructurada del modelo BOTNet con Tuning 184

Figura 81 Script para guardar el modelo BOTNet sin tuning en formato .pth..... 185

Figura 82 Script para guardar el modelo BOTNet con tuning en formato pth. 185

Figura 83 Script para exportar métricas comparativas a archivo .csv..... 186

Figura 84 Script sobre transformación de Datos ViT 187

Figura 85 Definición e inicialización del modelo ViT personalizado. 188

Figura 86 Script de entrenamiento inicial del modelo ViT base..... 190

Figura 87 Script de Validación y early stopping del modelo ViT base 191

Figura 88 Configuración e inicialización del modelo ViT con Tuning 192

Figura 89 Entrenamiento y validación del modelo ViT con Tuning 194

Figura 90 Script de evaluación del modelo ViT base sobre el conjunto de prueba 196

Figura 91 Script de evaluación del modelo ViT con tuning 197

Figura 92 Script para guardar el modelo ViT base en formato .pth..... 198

Figura 93 Script para guardar el modelo ViT con tuning en formato .pth..... 199

Figura 94 Script para exportar métricas comparativas de ViT en archivo .csv 199

Figura 95 Script de transformación de datos para el modelo Swin Transformer..... 200

Figura 96 Script de carga y adaptación del modelo Swin Transformer..... 201

Figura 97 202

Figura 98 Script de validación, control de early stopping y restauración del mejor modelo Swin base..... 203

Figura 99 Configuración y preparación del modelo Swin Transformer con Tuning..... 205

Figura 100 Entrenamiento y validación del modelo SwinTransformer con Tuning..... 206

Figura 101 Script de evaluación estructurada del modelo Swin Transformer base sobre el conjunto de prueba 207

Figura 102 Script de evaluación estructurada del modelo Swin Transformer con Tuning.. 208

Figura 103 Script para guardar el modelo Swin Transformer sin tuning en formato .pth ... 209

Figura 104 Script para guardar el modelo Swin con tuning en formato pth. 210

Figura 105 Script para exportar métricas comparativas a archivo .csv..... 210

Figura 106 211

Figura 107 Script de carga y adaptación del modelo PiT Transformer 212

Figura 108 Script de entrenamiento inicial del modelo PiT Transformer base 214

Figura 109 Script de validación, control de early stopping y restauración del mejor modelo PiT 215

Figura 110 Configuración y preparación del modelo PiT Transformer con Tuning 217

Figura 111 Entrenamiento y validación del modelo PiT Transformer con Tuning 218

Figura 112 Script de evaluación estructurada del modelo PiT Transformer base sobre el conjunto de prueba 219

Figura 113 Script de evaluación estructurada del modelo PiT Transformer con Tuning 220

Figura 114 Script para guardar el modelo PiT Transformer sin tuning en formato .pth..... 221

Figura 115 Script para guardar el modelo PiT Transformer con tuning en formato pth. 221

Figura 116 Script para exportar métricas comparativas a archivo .csv..... 222

Figura 117 Script de transformación de datos para el modelo MobileViT..... 223

Figura 118 Script de carga y adaptación del modelo MobileViT 223

Figura 119 Script de entrenamiento inicial del modelo MobileViT base 225

Figura 120 Script de validación, control de early stopping y restauración del mejor modelo MobileViT base..... 226

Figura 121 Configuración y preparación del modelo MobileViT con Tuning 228

Figura 122 Entrenamiento y validación del modelo MobileViT con Tuning..... 229

Figura 123 Script de evaluación estructurada del modelo MobileViT base sobre el conjunto de prueba 231

Figura 124 Script de evaluación estructurada del modelo MobileViT con Tuning..... 232

Figura 125 Script para guardar el modelo MobileViT sin tuning en formato .pth 233

Figura 126 Script para guardar el modelo MobileViT con tuning en formato .pth 233

Figura 127 Script para exportar métricas comparativas a archivo .csv..... 234

Figura 128 Importación de librerías necesarias para la implementación del modelo BOTNet como extractor y su evaluación con clasificadores de Machine Learning. 235

Figura 129 Configuración del modelo BOTNet como extractor mediante carga de pesos desde archivo .pth..... 235

Figura 130 Preprocesamiento de imágenes para BoTNet 236

Figura 131 Función para extraer features usando BoTNet 236

Figura 132 Función para generar los conjuntos de entrenamiento, validación y prueba con vectores de características extraídos por BOTNet 237

Figura 133 Entrenamiento y evaluación de clasificadores tradicionales con los vectores generados por BOTNet..... 238

Figura 134 Generación de curvas ROC para cada modelo clasificador..... 239

Figura 135 Matriz de confusión para el modelo aplicado al conjunto de prueba 240

Figura 136 Exportación de resultados de métricas comparativas a un archivo .CSV con timestamp automático 241

Figura 137 Importación de librerías necesarias para la implementación del modelo ViT como extractor y su evaluación con clasificadores de Machine Learning. 242

Figura 138 Carga del modelo ViT entrenado en formato .pth como extractor de características 243

Figura 139 Definición de transformaciones de redimensionamiento y normalización para las imágenes, y creación de los DataLoaders para cada partición del dataset 244

Figura 140 Extracción de características con ViT mediante hook..... 245

Figura 141. Evaluación de clasificadores tradicionales mediante métricas y visualización de resultados (matriz de confusión y curva ROC) 247

Figura 142 Exportación de métricas de desempeño de los clasificadores tradicionales. 248

Figura 143 Importación de librerías necesarias para la extracción de características con Swin Transformer y la evaluación con clasificadores tradicionales de Machine Learning 249

Figura 144 Carga del modelo Swin Transformer previamente entrenado desde archivo .pth para su uso como extractor de características. 250

Figura 145 Transformaciones aplicadas a las imágenes de palmas y extracción de características latentes mediante Swin Transformer 251

Figura 146 Extracción de características latentes a partir de imágenes clasificadas de palmas de manos mediante el modelo Swin Transformer. 252

Figura 147 Entrenamiento y evaluación de clasificadores tradicionales aplicados a las características extraídas mediante Swin Transformer. 254

Figura 148 Exportación de métricas de evaluación de los clasificadores tradicionales utilizando características extraídas con Swin Transformer. 255

Figura 149 Importación de librerías necesarias para la implementación del modelo PiT y evaluación con clasificadores tradicionales. 257

Figura 150 Configuración del modelo PiT como extractor de características mediante la carga de pesos desde archivo .pth. 258

Figura 151 Transformaciones aplicadas a las imágenes de palmas y extracción de características mediante el modelo PiT. 259

Figura 152 Organización de los vectores de características por conjunto y clase 260

Figura 153 Entrenamiento, evaluación y visualización de clasificadores clásicos a partir de características extraídas con PiT 261

Figura 154 Exportación automática de métricas de evaluación del modelo PiT a un archivo CSV 262

Figura 155 Importación de librerías necesarias para MobileViT híbrido. 263

Figura 156 Definición de rutas y modelo entrenado para MobileViT. 264

Figura 157 Función para transformar imágenes y extraer vectores característicos usando MobileViT 264

Figura 158 Extracción de vectores de características desde el dataset clasificado. 265

Figura 159 Entrenamiento y evaluación de modelos de Machine Learning usando vectores extraídos por MobileViT 266

Figura 160 Exportación automática de métricas de rendimiento en formato .csv para MobileViT híbrido 267

Figura 161 Métricas de clasificación obtenidas por modelo BOTNet sin Tuning..... 268

Figura 162 Matriz de confusión para el modelo BOTNet sin Tuning 269

Figura 163 Curva ROC del modelo BOTNet sin Tuning 270

Figura 164 Precisión del entrenamiento y validación por época del modelo BOTNet sin tuning..... 271

Figura 165 Pérdida de entrenamiento y validación por época del modelo BOTNet sin tuning. 271

Figura 166 Métricas de clasificación obtenidas por modelo BOTNet sin Tuning..... 272

Figura 167 Matriz de confusión para el modelo BOTNet con Tuning 273

Figura 168 Curva ROC del modelo BOTNet con Tuning 274

Figura 169 Precisión del entrenamiento y validación por época del modelo BOTNet con tuning..... 274

Figura 170 Pérdida de entrenamiento y validación por época del modelo BOTNet con tuning 275

Figura 171 Métricas de clasificación obtenidas por modelo ViT sin Tuning 276

Figura 172 Matriz de confusión para el modelo ViT sin Tuning..... 277

Figura 173 Curva ROC del modelo ViT sin Tuning..... 277

Figura 174 Precisión del entrenamiento y validación por época del modelo ViT sin tuning 278

Figura 175 Pérdida de entrenamiento y validación por época de modelo ViT sin tuning ... 279

Figura 176 Métricas de clasificación obtenidas por modelo ViT con Tuning 281

Figura 177 Matriz de confusión para el modelo ViT con Tuning 281

Figura 178 Curva ROC del modelo ViT con Tuning 282

Figura 179 Precisión del entrenamiento y validación por época del modelo ViT con Tuning 283

Figura 180 Pérdida de entrenamiento y validación por época de modelo ViT con Tuning. 284

Figura 181 Métricas de clasificación obtenidas por modelo Swin Transformer sin Tuning 285

Figura 182 Matriz de confusión para el modelo Swin Transformer sin Tuning 286

Figura 183 Curva ROC del modelo Swin Transformer sin Tuning 287

Figura 184 Precisión del entrenamiento y validación por época del modelo Swin Transformer sin tuning 288

Figura 185 Pérdida de entrenamiento y validación por época de modelo Swin Transformer sin tuning 288

Figura 186 Métricas de clasificación obtenidas por modelo Swin Transformer con Tuning 289

Figura 187 Matriz de confusión para el modelo Swin Transformer con Tuning 290

Figura 188 Curva ROC del modelo Swin Transformer con Tuning 291

Figura 189 Precisión del entrenamiento y validación por época del modelo Swin Transformer con Tuning 292

Figura 190 Pérdida de entrenamiento y validación por época de modelo Swin Transformer con Tuning 293

Figura 191 Métricas de clasificación obtenidas por modelo PiT Transformer sin Tuning.. 294

Figura 192 Matriz de confusión para el modelo PiT Transformer sin Tuning 295

Figura 193 Curva ROC del modelo PiT Transformer sin Tuning 296

Figura 194 Precisión del entrenamiento y validación por época del modelo PiT Transformer sin tuning 297

Figura 195 Pérdida de entrenamiento y validación por época de modelo PiT Transformer sin tuning..... 297

Figura 196 Métricas de clasificación obtenidas por modelo PiT con Tuning..... 299

Figura 197 Matriz de confusión para el modelo PiT Transformer con Tuning 299

Figura 198 Curva ROC del modelo PiT Transformer con Tuning 300

Figura 199 Precisión del entrenamiento y validación por época del modelo PiT Transformer con Tuning..... 301

Figura 200 Pérdida de entrenamiento y validación por época de modelo PiT Transfomer con Tuning 302

Figura 201 Métricas de clasificación obtenidas por modelo MobileViT sin Tuning 303

Figura 202 Matriz de confusión para el modelo MobileViT sin Tuning 304

Figura 203 Curva ROC del modelo MobileViT sin Tuning 305

Figura 204 Precisión del entrenamiento y validación por época del modelo MobileViT sin tuning..... 306

Figura 205 306

Figura 206 Métricas de clasificación obtenidas por modelo MobileViT con Tuning 307

Figura 207 Matriz de confusión para el modelo MobileViT con Tuning 308

Figura 208 Curva ROC del modelo MobileViT con Tuning 309

Figura 209 Precisión del entrenamiento y validación por época del modelo MobileViT con Tuning 309

Figura 210 Pérdida de entrenamiento y validación por época de modelo MobileViT con Tuning 310

Figura 211 Matriz de confusión BotNet SVM 312

Figura 212 Curva ROC BotNet SVM 312

Figura 213 Matriz de confusión ViT Random Forest 313

Figura 214 Curva ROC ViT Random Forest 314

Figura 215 Matriz de Confusión Swin SVM 315

Figura 216 Curva ROC Swin SVM 315

Figura 217 Matriz de Confusión PiT Random Forest 316

Figura 218 Curva ROC PiT - Random Forest 317

Figura 219 Matriz de Confusión Mobile SVM 318

Figura 220 Curva ROC Mobile SVM 318

Figura Arquitectura aplicación Web 324

Figura Inicio de sesión de usuario 326

- Después, el usuario tendrá la posibilidad de agregar un nuevo paciente, completando el formulario que se visualiza en la Figura XXFigura Registro de paciente 327

Figura Selección o toma de foto de palma de mano 328

Figura Selección de imagen 329

Figura Previsualización de imagen 330

Figura Imagen en proceso de análisis 331

Figura Resultado de análisis..... 332

Índice de Tablas

Tabla 1 Indicadores de desempeño para modelos de aprendizaje automático individuales ... 42

Tabla 2 Indicadores de desempeño para modelos de ensamble 43

Tabla 3 Desempeño de los modelos propuestos para la detección de anemia utilizando la palma palpable 47

Tabla 4 Rendimiento de los modelos propuestos para detectar anemia mediante la palma de la mano 49

Tabla 5 Comparaciones de los enfoques estudiados y los trabajos existentes 52

Tabla 6 Rendimiento del modelo usando característica fusión 53

Tabla 7 Resultados del estudio de ablación con el modelo AMSA propuesto..... 54

Tabla 8 Precisión de los modelos usando varios extractores de características 56

Tabla 9 Precisión de los modelos de Aprendizaje Profundo utilizando diferentes parámetros 57

Tabla 10 Definición y medición de variables..... 123

Tabla 11 Procedimientos en la fase de la obtención de la base de datos 126

Tabla 12 Procedimientos en la fase de preprocesamiento de datos 128

Tabla 15 Procedimientos en la fase de modelado y clasificación de la Metodología 1 136

Resumen

El avance de la inteligencia artificial ha revolucionado el diagnóstico médico, permitiendo desarrollar enfoques innovadores para la detección anticipada de enfermedades. La anemia, una condición prevalente en niños menores de cinco años, puede afectar significativamente su desarrollo físico y cognitivo; sin embargo, su diagnóstico sigue dependiendo de análisis de sangre invasivos, lo que limita su acceso en comunidades rurales. Este estudio propone el uso de técnicas de Aprendizaje Profundo para la clasificación preliminar de anemia a partir de imágenes de palmas de manos, ofreciendo una opción accesible y no invasiva. Se aplicaron arquitecturas avanzadas como BotNet, Vision Transformer, Swin Transformer, PiT y MobileViT, entrenadas con un conjunto de 2000 imágenes clasificadas en anémicas y no anémicas.

Asimismo, se desarrolló un enfoque híbrido que integra Aprendizaje Profundo como extractor de características visuales, combinándolos con algoritmos de Aprendizaje Automático como Support Vector Machines (SVM), Random Forest (RF), Decision Tree (DT), K-Nearest Neighbors (KNN) y Naïve Bayes (NB) para la clasificación final. La evaluación del desempeño de ambos enfoques se realizó mediante métricas estándar, incluyendo Accuracy, Precision, Recall, F1-Score, AUC. Los resultados obtenidos permiten analizar la efectividad de los modelos propuestos para el tamizaje temprano de anemia infantil mediante imágenes médicas, contribuyendo así a la implementación de soluciones tecnológicas para mejorar el acceso al diagnóstico en poblaciones vulnerables.

Palabras clave: Anemia, Aprendizaje Automático, Aprendizaje Profundo, Modelos híbridos, Detección no invasiva, Imágenes de palmas.

Abstract

The advance of artificial intelligence has revolutionized medical diagnostics, enabling the development of innovative approaches for the early detection of diseases. Anemia, a prevalent condition in children under five years of age, can significantly affect their physical and cognitive development; however, its diagnosis still relies on invasive blood tests, which limits its access in rural communities. This study proposes the use of Deep Learning techniques for the preliminary classification of anemia from palm images, offering an accessible and non-invasive option. Advanced architectures such as BotNet, Vision Transformer (ViT), Swin Transformer, PiT and MobileViT were applied, trained on a set of 2000 images classified into anemic and non-anemic.

Also, a hybrid approach integrating Deep Learning as a visual feature extractor was developed, combining it with Machine Learning algorithms such as Support Vector Machines (SVM), Random Forest (RF), Decision Tree (DT), K-Nearest Neighbors (KNN) and Naïve Bayes (NB) for the final classification. Performance evaluation of both approaches was performed using standard metrics, including Accuracy, Precision, Recall, F1-Score, Specificity and AUC. The results obtained allow us to analyze the effectiveness of the proposed models for the early screening of childhood anemia through medical imaging, thus contributing to the implementation of technological solutions to improve access to diagnosis in vulnerable populations.

Keywords: Anemia, Machine Learning, Deep Learning, Hybrid models, Non-invasive detection, Palm images.

Introducción

Actualmente, la tecnología avanza a un ritmo acelerado, impactando diversos ámbitos, incluida la salud. Uno de los problemas más comunes en la población infantil es la anemia, un trastorno caracterizado por niveles reducidos de hemoglobina en la sangre, lo que compromete el adecuado suministro de oxígeno a los tejidos y puede interferir en el crecimiento y la función cognitiva de los niños. A pesar de las iniciativas para mitigar sus efectos, los métodos convencionales de diagnóstico dependen de pruebas invasivas, como los análisis de sangre, cuya disponibilidad es limitada en áreas rurales o de difícil acceso. En este escenario, la inteligencia artificial surge como una solución innovadora para desarrollar métodos de detección no invasivos.

El avance en algoritmos de Aprendizaje Automático, en particular del Aprendizaje Profundo, ha permitido que la inteligencia artificial mejore la capacidad de diagnóstico médico. Estas metodologías han demostrado ser efectivas en el procesamiento de imágenes médicas para la detección de diversas enfermedades, alcanzando niveles de precisión equivalentes o incluso superiores a los de los especialistas humanos. En el caso de la anemia, investigaciones previas han evidenciado que la palidez en zonas específicas del cuerpo, como la palma de la mano, puede ser un indicador clave para su diagnóstico. Así, la integración del procesamiento de imágenes con modelos de inteligencia artificial representa una herramienta prometedora para optimizar la detección temprana de esta condición.

En este contexto, el presente trabajo de investigación propone el desarrollo de un enfoque híbrido para la detección preliminar de anemia a partir de imágenes de palmas de manos. Este enfoque contempla dos metodologías principales:

Por un lado, el uso de arquitecturas avanzadas de Deep Learning, como BOTNet, Vision Transformer (ViT), Swin Transformer, PiT y MobileViT, entrenadas para realizar la clasificación directa de imágenes en categorías de anémico y no anémico.

Por otro lado, se plantea una estrategia híbrida que utiliza dichas arquitecturas de Deep Learning como extractores de características visuales, las cuales son posteriormente clasificadas mediante algoritmos clásicos de Machine Learning, como Support Vector Machine (SVM), Random Forest (RF), Decision Tree (DT), K-Nearest Neighbors (KNN) y Naïve Bayes (NB).

El estudio se desarrolló en diversas fases. Primero, se utilizó una base de datos pública compuesta por imágenes de palmas de manos de niños menores de cinco años. Luego, se aplicaron técnicas de preprocesamiento para estandarizar los datos antes del análisis. Posteriormente, se entrenaron los modelos de Deep Learning para la clasificación directa y, en la metodología híbrida, se utilizaron como extractores de características para alimentar los algoritmos de Machine Learning. Finalmente, se evaluó el desempeño de ambas estrategias mediante métricas como Accuracy, Precision, Recall, F1-Score y AUC, con el fin de analizar su efectividad en la detección preliminar de anemia.

CAPÍTULO I: Planteamiento del Problema

Este capítulo presentará el contexto de la problemática, la formulación del problema principal, los objetivos específicos, así como la justificación y el alcance de la investigación.

1.1 Contextualización de la Realidad Problemática

La anemia infantil constituye un reto considerable para la salud pública a nivel global. Esta afección se distingue por una disminución en la cantidad de glóbulos rojos o en la concentración de hemoglobina en la sangre, situándose por debajo de los valores considerados normales (National Heart Lung and Blood Institute, 2021). Sus síntomas pueden incluir fatiga, debilidad, palidez y dificultad para respirar. Entre sus principales causas se encuentran las deficiencias nutricionales, como la falta de hierro, vitamina B12 o ácido fólico, así como pérdidas de sangre, enfermedades crónicas e infecciones.

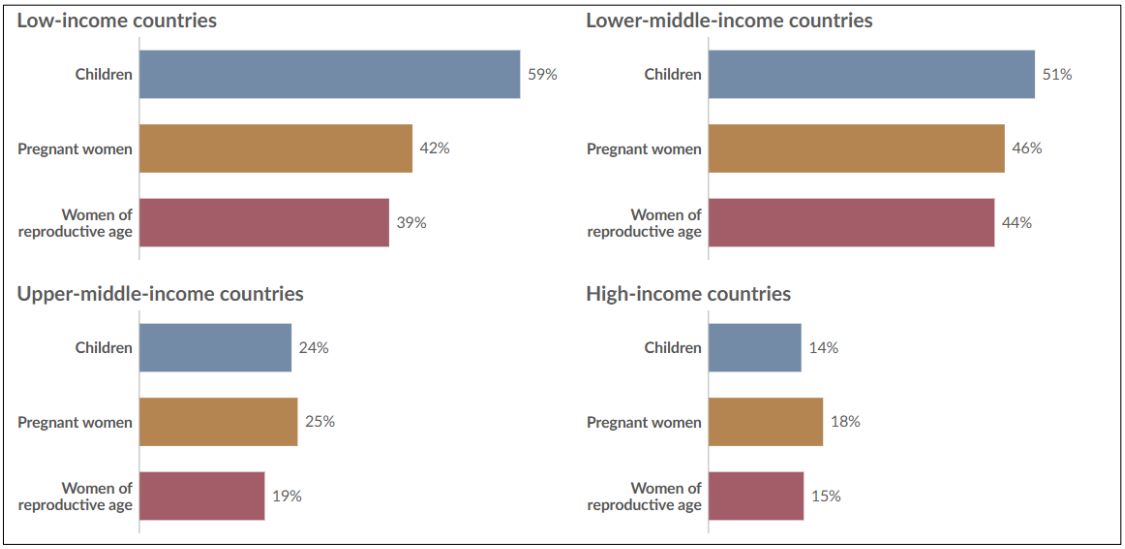
En 2021, se calculó que aproximadamente 1.920 millones de personas en todo el mundo padecían anemia, lo que equivale al 24.3% de la población mundial. Aunque este porcentaje ha disminuido en comparación con 1990, cuando la prevalencia era del 28.2%, la carga de la anemia sigue variando de manera significativa según factores como la edad, el sexo y la ubicación geográfica. En particular, afecta con mayor frecuencia a niños menores de cinco años, mujeres y comunidades en regiones como el África subsahariana y el sur de Asia (Gardner et al., 2023).

Tal como se visualiza en la Figura 1, la prevalencia de anemia es considerablemente más alta en países de bajos ingresos, donde afecta al 59% de los niños y al 42% de las mujeres embarazadas. En contraste, en naciones de altos ingresos, estos valores disminuyen al 14% y 18%, respectivamente. Investigaciones han identificado que factores como la inseguridad alimentaria, el acceso limitado a servicios de salud y la educación en nutrición contribuyen a la

incidencia de la anemia, lo que ayuda a explicar las diferencias entre regiones con distintos niveles de desarrollo (*World Health Organization, s/f*; Balarajan et al., 2011). Asimismo, la Organización Mundial de la Salud subraya que la anemia continúa siendo un problema crítico en países con recursos limitados, en la presencia de alimentos con gran contenido de hierro, y la implementación de programas de prevención son más reducidas ((World Health Organization, s/f)).

Figura 1

Porcentaje de la población con anemia en grupos de mayor riesgo, 2019



Nota. Extraído de Organización Mundial de la Salud (OMS), 2025. Recuperado de Ritchie & Roser (2017)

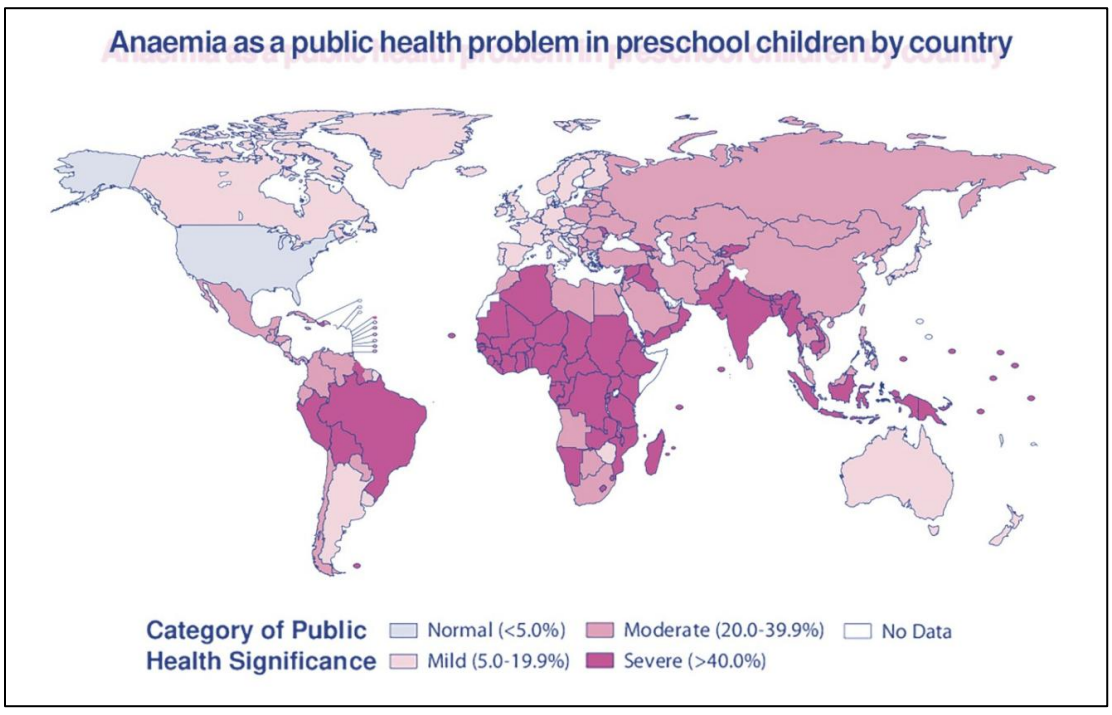
En el caso específico de los niños, se calcula que más de 269 millones de niños menores de cinco años sufren anemia (Organización Mundial de la Salud, 2023). A nivel global, la prevalencia de anemia en niños menores de cinco años alcanza el 39.8%, lo que la convierte en una de las principales causas de retraso en el crecimiento y en el desarrollo cognitivo durante la primera infancia.

En América Latina y el Caribe (LAC), la anemia sigue siendo un problema significativo. Según Iglesias Vázquez et al. (2019), el 32.93% de los niños menores de cinco años y el 17.49% de los niños en edad escolar presentan anemia, con diferencias marcadas entre países. Por ejemplo, mientras Costa Rica presenta la tasa más baja de anemia infantil con un 7.6%, Haití reporta una prevalencia alarmante del 65%, reflejando disparidades socioeconómicas en la región.

Tal como se observa en la Figura 2, la anemia en niños en edad preescolar varía ampliamente entre países, con las tasas más altas registradas en zonas de África y el sur de Asia, donde la prevalencia supera el 40%. En contraste, algunas regiones de América del Norte y Europa presentan niveles inferiores al 5%. Estos datos refuerzan la relación entre factores socioeconómicos, acceso a una alimentación adecuada y calidad de los servicios de salud en la prevalencia de anemia en la infancia.

Figura 2

Distribución global de la anemia en niños en edad preescolar según país.



Nota. La imagen muestra la cantidad de países en cada categoría de prevalencia de anemia en niños preescolares. Datos extraídos y adaptados de la Organización Mundial de la Salud (OMS). Anaemia as a public health problem in preschool children by country. Recuperado de Pedia-Start (s/f).

En Ghana, la anemia infantil continúa siendo un desafío significativo de salud pública. Según la Encuesta Demográfica y de Salud de Ghana de 2022, la prevalencia de anemia entre niños de 6 a 59 meses se redujo del 76.1% en 2003 al 48.9% en 2022. Sin embargo, persisten desigualdades notables. Los niños más pequeños (6–11 meses) presentan tasas más altas de anemia, alcanzando un 60.5% en 2022. Además, existen disparidades significativas según el nivel socioeconómico, la educación materna y la ubicación geográfica. Por ejemplo, la prevalencia de anemia en niños de hogares rurales fue del 56.2%, en comparación con el 41.4% en áreas urbanas (Osborne et al., 2024).

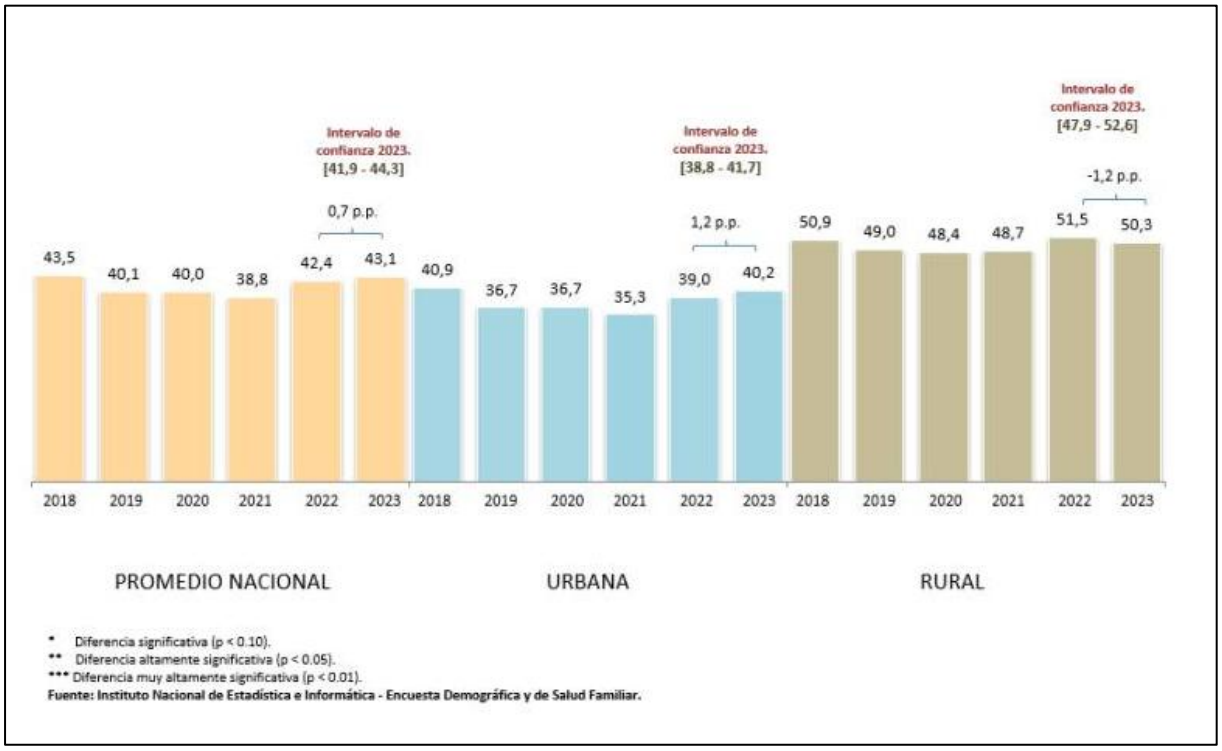
Estas cifras subrayan la necesidad de estrategias de detección temprana y accesibles, especialmente en comunidades rurales y de bajos recursos. El desarrollo de herramientas no invasivas basadas en inteligencia artificial, como el análisis de imágenes de las palmas de las manos, podría ser una solución prometedora para mejorar el diagnóstico y tratamiento oportuno de la anemia en estas poblaciones vulnerables.

En Perú, la anemia infantil también es preocupante. Tal como se puede ver en la Figura 3, según el Instituto Nacional de Estadística e Informática (INEI), el 43.1% de los niños peruanos entre 6 y 35 meses de edad padecen anemia, con un impacto mayor en áreas rurales, donde la prevalencia asciende al 50.3%. Algunas regiones muestran cifras alarmantes, como Puno (70.4%), Ucayali (59.4%) y Madre de Dios (58.3%), evidenciando la relación entre la anemia y factores como pobreza, acceso limitado a alimentos abundantes en hierro y

deficiencias en el sistema de salud pública (Instituto Nacional de Estadística e Informática (INEI), 2023).

Figura 3

Perú: Proporción de niños de 6 a 35 meses con anemia, según zona de residencia.



Nota. La imagen muestra el porcentaje de niños de 6 a 35 meses de edad con prevalencia de anemia, según área de residencia. Extraído de una encuesta demográfica y de salud familiar realizada por el INEI a en el año 2023.

La alta prevalencia de anemia en niños se asocia con múltiples factores, entre ellos deficiencias nutricionales, condiciones socioeconómicas adversas y restricciones en el acceso a servicios de salud.

La disponibilidad restringida de muestras de sangre en zonas rurales representa un desafío importante para la identificación temprana y el tratamiento adecuado de la anemia, lo que contribuye a su persistencia en estas comunidades. La insuficiencia de infraestructura

adecuada, la falta de personal de salud y la distancia de los centros médicos dificultan la realización periódica de exámenes hemoglobínicos en poblaciones vulnerables, especialmente en niños menores de cinco años. Como consecuencia, muchas familias desconocen el estado nutricional de sus hijos hasta que los signos de la anemia se hacen evidentes, lo que impacta negativamente su desarrollo físico y cognitivo. La ausencia de programas efectivos de diagnóstico en estas áreas refuerza el ciclo de desnutrición y anemia, perpetuando inequidades en el sistema de salud pública y poniendo de manifiesto la urgencia de intervenciones comunitarias que faciliten el acceso a servicios médicos esenciales (Reyes Narvaez et al., 2019).

A pesar de las iniciativas gubernamentales, como los programas de suplementación con hierro y la promoción de una dieta equilibrada, la disminución de la prevalencia de la anemia en los últimos años ha sido marginal. (Observatorio Medicina Salud y Sociedad del Colegio Médico del Perú, 2023)

La anemia en la infancia puede generar efectos a largo plazo, como retrasos en el desarrollo cognitivo y físico, disminución del desempeño escolar y un aumento en la vulnerabilidad a infecciones. Por ello, abordar este problema es fundamental para promover el bienestar y el crecimiento adecuado de la población infantil.

El diagnóstico de la anemia se lleva a cabo tradicionalmente mediante un examen de sangre, el cual permite evaluar los niveles de hemoglobina y hematocrito en el organismo. Durante el procedimiento, el profesional de salud desinfecta la piel y ajusta un torniquete en el brazo para facilitar la identificación de la vena. Luego, tal como se ve en la Figura 4, se introduce una aguja estéril en una vena del brazo, comúnmente en la parte interna del codo o el dorso de la mano, para extraer una pequeña muestra de sangre, que se deposita en un tubo de recolección o jeringa. Finalmente, la muestra es analizada en un laboratorio para confirmar si la persona presenta anemia (Nemours KidsHealth, s/f).

Figura 4

Obtención de análisis de sangre



Nota. Captura del video Blood Test: What to Expect, producido por Nemours KidsHealth. Extraído de Nemours KidsHealth (s/f).

Sin embargo, este método presenta limitaciones significativas, ya que es invasivo, lo que puede generar temor en niños pequeños debido a múltiples factores. Entre ellos, el miedo al dolor de la aguja, la ansiedad por lo desconocido, la presencia de un entorno médico intimidante y, en algunos casos, experiencias previas desagradables con procedimientos médicos. Incluso, si los padres o cuidadores muestran nerviosismo, esto puede aumentar la inseguridad del niño.

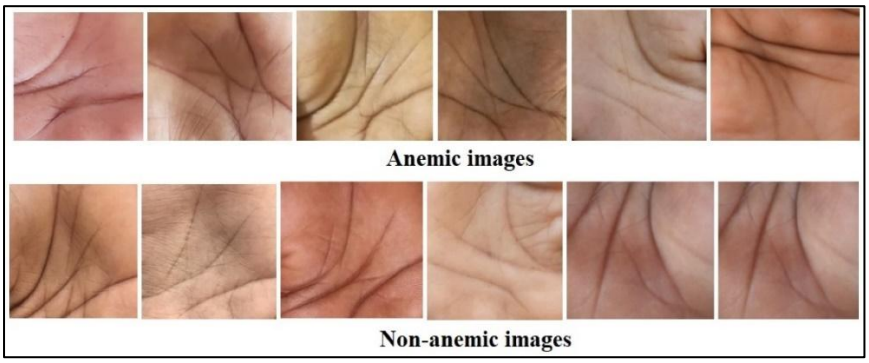
Más allá del temor que puede generar, este tipo de diagnóstico también presenta desafíos logísticos, ya que requiere infraestructura de laboratorio y personal capacitado para su diagnóstico, lo cual incrementa los costos y limita su disponibilidad en comunidades rurales o de escasos recursos. Además, no permite evaluaciones rápidas y masivas, lo que puede retrasar la identificación de casos y el inicio oportuno del tratamiento.

Ante estas barreras, surge la necesidad de explorar nuevas estrategias de detección que sean no invasivas, accesibles y de bajo costo. En ese sentido, los métodos no invasivos representan una alternativa prometedora, ya que no requieren extracción de sangre ni el uso de instrumentos que rompan la piel o penetren el cuerpo. Algunos de estos métodos incluyen el examen visual de la conjuntiva de los ojos, las uñas, la lengua y las palmas de las manos, los cuales pueden proporcionar indicios de anemia mediante la observación de cambios en la coloración de los tejidos.

En particular, el uso de imágenes de la palma de la mano para el diagnóstico de la anemia es una técnica poco común, pero que ha demostrado ser confiable. Tal como se muestra en la Figura 5, este procedimiento consiste en analizar qué tan pálida o amarillenta se encuentra la palma de una persona, lo que puede servir para realizar un prediagnóstico de anemia, el cual posteriormente se confirma con un examen de sangre (Appiahene et al., 2023a).

Figura 5

Una muestra del ROI de la palma de la mano



Nota. Adaptado de *Application of ensemble models approach in anemia detection using images of the palpable palm* por (Appiahene, Dogbe, et al., 2023)

Los avances tecnológicos han permitido optimizar los diagnósticos médicos mediante el uso de inteligencia artificial (IA), particularmente a través del uso de técnicas de Aprendizaje Automático, entre las que destaca el Aprendizaje Profundo. Estudios recientes han demostrado que el análisis de imágenes médicas mediante IA puede alcanzar niveles de precisión similar a los de especialistas humanos, lo que abre nuevas posibilidades para mejorar la detección y el diagnóstico de diversas enfermedades, incluida la anemia.

En este contexto, la implementación de modelos de inteligencia artificial sustentados con en el análisis de imágenes de la palma de la mano ha surgido como una herramienta innovadora y prometedora para la detección de la anemia. Estos modelos podrían hacer posible la realización de evaluaciones a gran escala y de fácil acceso, sin requerir laboratorios especializados, lo que ayudaría a detectar casos de anemia de manera temprana, especialmente en comunidades con recursos limitados. En contraste con los enfoques convencionales, que requieren análisis de sangre, estas tecnologías ofrecen una alternativa no invasiva, económica y rápida, permitiendo diagnósticos más eficientes en poblaciones con acceso restringido a servicios médicos.

Así pues, considerando que este sigue siendo un problema de salud pública de gran importancia en distintos ámbitos y que las estrategias convencionales han logrado avances limitados en la disminución de su prevalencia, surge la necesidad de incorporar estas nuevas tecnologías relacionadas a la inteligencia artificial ya que representan una oportunidad para mejorar la detección y el tratamiento oportuno.

En contextos como el del Perú, donde la anemia persiste especialmente en zonas rurales y afecta de manera significativa a la población infantil, el acceso limitado a servicios médicos constituye una de las principales barreras para su abordaje. En ese sentido, el uso de herramientas de diagnóstico temprano basadas en inteligencia artificial podría ser de gran

utilidad, al permitir la implementación de sistemas en aplicaciones móviles o plataformas web. Esto haría posible que, por ejemplo, con una capacitación previa, los propios padres o cuidadores puedan tomar fotografías de la palma de la mano de sus hijos y recibir una estimación preliminar sobre la posible presencia de anemia, facilitando así una intervención médica temprana.

1.2 Definición del Problema de Investigación

Se ha utilizado una técnica de matriz de consistencia para establecer preguntas generales y específicas.

1.2.1 Problema Principal

¿Cómo desarrollar un modelo híbrido para la detección de la anemia a partir de imágenes de las palmas de las manos utilizando Vision Transformer y Machine Learning?

1.2.2 Problemas Secundarios

PS1: ¿Cuáles son las características de la base de datos de imágenes de palmas de manos de personas con y sin anemia utilizada en el estudio?

PS2: ¿Qué técnicas de preprocesamiento se pueden aplicar para estandarizar las imágenes de palmas de manos utilizadas en la detección de la anemia?

PS3: ¿Qué técnicas de Vision Transformer y Machine Learning pueden utilizarse para construir un modelo híbrido de clasificación capaz de reconocer imágenes de palmas de manos de personas con y sin anemia?

PS4: ¿Qué métricas permiten evaluar el desempeño de la clasificación de imágenes de palmas de manos de personas con anemia y sin anemia?

1.3 Objetivos de la investigación

1.3.1 Objetivo general

Desarrollar un modelo híbrido basado en Vision Transformer y Machine Learning para detectar anemia a partir de imágenes de las palmas de las manos.

1.3.2 Objetivos específicos

OE1: Identificar y describir las características de la base de datos de imágenes de palmas de manos de personas con y sin anemia utilizada en el estudio.

OE2: Aplicar técnicas de preprocesamiento para estandarizar las imágenes de palmas de manos para la detección de la anemia.

OE3: Aplicar técnicas de Vision Transformer y Machine Learning para construir un modelo híbrido de clasificación que reconozca imágenes de palmas de manos de personas con y sin anemia.

OE4: Seleccionar y aplicar métricas que permiten evaluar el desempeño del modelo en la clasificación de imágenes de palmas de manos de personas con y sin anemia.

1.4. Justificación de la investigación

1.4.1. Fundamentación Teórica

Se busca aportar al campo de la salud digital mediante la implementación de técnicas de Aprendizaje Automático, incluyendo Aprendizaje Profundo como alternativa no invasiva a las pruebas de laboratorio tradicionales para la detección de anemia. El objetivo es facilitar un diagnóstico temprano y accesible, permitiendo la identificación de patrones en datos biomédicos sin la necesidad de extracción de sangre, lo que favorecería la toma de decisiones médicas y mejoraría la atención en comunidades con acceso limitado a servicios de salud. Además, se espera que los

resultados sean generalizables a diversas poblaciones, adaptando el modelo a diferentes condiciones fisiológicas y contextos ambientales. La relevancia del estudio radica en su potencial para mejorar la detección oportuna de la anemia, reducir complicaciones a largo plazo y contribuir a una gestión más eficiente de los recursos del sistema de salud.

1.4.2 Contribución Práctica

La principal utilidad práctica de esta investigación es la posibilidad de desarrollar un sistema de detección de anemia en niños que sea accesible a nivel global, particularmente en comunidades con limitado acceso a servicios de salud. Este sistema permitirá realizar una evaluación preliminar del estado de hemoglobina a partir de imágenes de las palmas de las manos, facilitando la identificación de casos de riesgo y promoviendo la toma de medidas preventivas o la búsqueda de atención médica en caso necesario.

Los beneficiarios directos serán los niños menores de cinco años, el grupo más afectado por la anemia, en quienes un diagnóstico tardío puede generar consecuencias significativas en su crecimiento físico, el desarrollo cognitivo y el bienestar general. Además, la implementación de esta tecnología en centros de salud, campañas médicas comunitarias o incluso en dispositivos móviles permitirá ampliar el alcance de los programas de salud y mejorar la eficiencia en la detección de anemia en poblaciones vulnerables.

En términos de utilidad, esta propuesta ofrece una solución innovadora que permitirá reducir los tiempos y costos de diagnóstico, optimizando la capacidad de respuesta del sistema de salud ante esta condición prioritaria de salud pública.

Si bien esta investigación se desarrolla utilizando datos de Ghana, los hallazgos podrían ser aplicables en contextos similares, como las zonas rurales del Perú, donde

también se presentan altas tasas de anemia infantil y dificultades de acceso a servicios de salud. La implementación de modelos de inteligencia artificial no invasivos para la detección preliminar de la anemia representa una herramienta valiosa para países de ingresos bajos y medianos, donde las limitaciones en la infraestructura de salud son comunes y el diagnóstico oportuno sigue siendo un desafío persistente.

1.4.3 Aporte Metodológico

Este estudio contribuirá a la validación de la inteligencia artificial como herramienta para el diagnóstico biomédico, específicamente en la detección preliminar de la anemia. Para ello, se utilizará una base de imágenes de palmas de manos, las cuales serán procesadas mediante técnicas de Aprendizaje Automático, destacando el uso de Aprendizaje Profundo, para identificar patrones asociados con niveles anormales de hemoglobina.

Como instrumento de investigación, se implementará un sistema de clasificación basado en dos enfoques complementarios: por un lado, el uso de arquitecturas avanzadas de Aprendizaje Profundo capaces de realizar la clasificación directa de las imágenes; y por otro, un modelo híbrido en el que dichas redes neuronales profundas actuarán como extractores de características alimentando posteriormente algoritmos clásicos de Machine Learning para la clasificación de casos como anémicos o no anémicos.

De esta manera, se busca optimizar la detección automatizada de la anemia, garantizando un análisis eficiente, no invasivo y de bajo costo (Zhang et al., 2023).

1.5 Delimitación de la investigación

1.5.1 Ámbito Espacial

Este estudio define su delimitación espacial utilizando la base de datos “Anemia Detection Using Palpable Palm Image datasets from Ghana”, accesible mediante el

portal Mendeley Data. Esta base de datos incluye 4260 imágenes de palmas de manos de niños menores de cinco años provenientes de diferentes hospitales de Ghana en África. Esta base de datos está dividida en dos secciones: infantes con anemia e infantes sin anemia. Las imágenes se encuentran en formato “png” y fueron tomadas por el personal de laboratorio utilizando cámaras estándar de alta calidad con un mínimo de 12MP.

1.5.2 Ámbito Temporal

Este estudio se basa en el conjunto de datos “Anemia Detection Using Palpable Palm Image datasets from Ghana”, el cual fue publicado el 27 de junio de 2022.

1.5.3 Ámbito Conceptual

Este estudio se enmarca en el uso de técnicas de aprendizaje automático para la detección no invasiva de anemia a partir del análisis de imágenes biomédicas. En particular, se emplean arquitecturas avanzadas de Aprendizaje Profundo para clasificar para clasificar imágenes de la palma de la mano según su posible condición anémica. Asimismo, se contempla un enfoque híbrido en el que estos modelos de Deep Learning se emplean como extractores de características visuales, las cuales son posteriormente clasificadas mediante algoritmos tradicionales de Machine Learning.

El objetivo es evaluar el desempeño de ambos enfoques, optimizando métricas como precisión, sensibilidad y el área bajo la curva ROC (AUC).

CAPÍTULO II: Marco Teórico

Este capítulo abordará los antecedentes, bases y conceptos teóricos, que facilitarán la comprensión del problema y las definiciones necesarias. Adicionalmente, en este capítulo se examinarán los estudios previos de autores que ya han llevado a cabo investigaciones relacionadas con el asunto.

2.1 Antecedentes de la Investigación

A continuación, se exponen algunas referencias recolectadas para comprender cómo se ha abordado en diversos ámbitos el contenido de estudio seleccionado, los objetivos, las conclusiones y las respuestas obtenidas.

a) Artículo de investigación titulado «*Application of ensemble models approach in anemia detection using images of the palpable palm*» de los autores Peter Appiahene, Samuel Según Dzifa Dogbe, Emmanuel Edem Yaw Kobina, Philip Sackey Dartey, Stephen Afrifa, Emmanuel Timmy Donkoh y Justice Williams Asare.

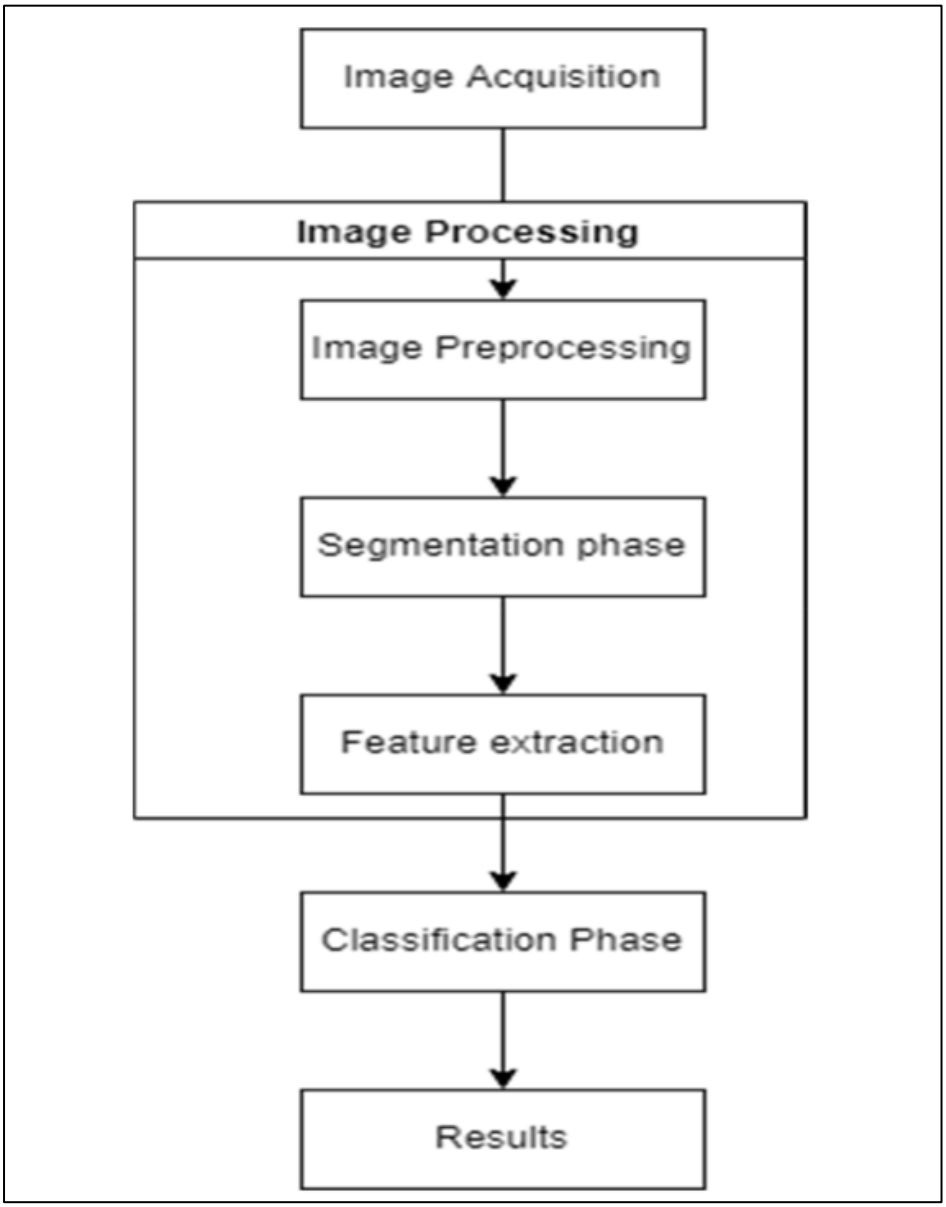
En dicho artículo de investigación se presenta un enfoque innovador para la detección de anemia a través de imágenes de la palma de la mano, en lugar de recurrir a los métodos tradicionales que implican la extracción de sangre y análisis clínicos. Este trabajo se sustenta en la necesidad de contar con opciones no invasivas y accesibles, especialmente en zonas donde los recursos médicos son limitados. Para llevar a cabo la investigación, se recolectaron imágenes de las palmas de 710 participantes en hospitales de Ghana utilizando un dispositivo digital llamado Kobo Collect, lo que permitió a profesionales en Ciencias Biomédicas realizar capturas de imágenes de manera organizada y estructurada para su posterior análisis.

La metodología propuesta en la investigación, antes mencionada, se detalla en la Figura 6 e incluye las fases de adquisición de imágenes; la fase procesamiento de imágenes, la cual

incluye las sub-fases de preprocesamiento de imágenes, segmentación y extracción de características; fase de clasificación y fase de resultados.

Figura 6

Diagrama de bloque de la metodología propuesta



Nota. Adaptado de *Application of ensemble models approach in anemia detection using images of the palpable palm* [Fig. 1] por Appiahene, et al.(2023a).

Tras obtener las imágenes, se ejecutó un proceso de preprocesamiento y segmentación para aislar la región de interés (ROI) de las palmas. Posteriormente, se llevó a cabo la extracción de características, analizando las imágenes en sus componentes de color RGB (rojo, verde y azul). Este análisis permitió obtener características numéricas a partir de la intensidad de cada color, las cuales fueron empleadas directamente en el entrenamiento de distintos algoritmos de aprendizaje automático para determinar si un participante presentaba o no anemia. Los algoritmos empleados incluyeron redes neuronales artificiales (ANN), máquinas de vectores de soporte (SVM), árboles de decisión (DT), Naïve Bayes (NB) y Random Forest (RF), cada uno con enfoques distintos para el aprendizaje y la predicción a partir de los datos.

A continuación, se aplicaron técnicas de aprendizaje por conjunto (ensemble learning), destinadas a combinar los resultados de estos modelos individuales y mejorar así la precisión general del diagnóstico. Las técnicas utilizadas fueron: apilamiento (stacking), que combina las predicciones de diferentes modelos base para generarse un modelo superior; votación (voting), que selecciona la clase predicha por la mayoría de los modelos; refuerzo (boosting), que ajusta iterativamente los errores cometidos por modelos previos; y agregación por muestreo (bagging), que entrena múltiples modelos en subconjuntos aleatorios del mismo conjunto de datos para mejorar la estabilidad y precisión.

Los resultados indicaron, como se muestra en la Tabla 1, indicaron que entre los modelos individuales, Random Forest fue el más eficaz, alcanzando un Accuracy de 99.53%. Asimismo, en la Tabla 2 se evidencia que, entre las técnicas de aprendizaje por conjunto, el método más exitoso fue el Apilamiento (stacking), logrando la mayor Accuracy del 99.98% cuando combina Naïve Bayes con Random Forest, así como cuando al integrar los cinco clasificadores (ANN+DT+RF+SVM+NB).

El estudio concluyó que la implementación de modelos de aprendizaje por conjunto en este ámbito no solo aumenta la precisión (Accuracy) del diagnóstico, sino que también contribuye a agilizar el proceso. Asimismo, los autores se destaca el uso de imágenes de la palma como una alternativa viable para la detección no invasiva de la anemia, especialmente en contextos con recursos médicos limitados. Además, el conjunto de datos generado ha sido puesto a disposición pública, lo que abre la posibilidad de ser reutilizado en investigaciones futuras orientadas a la mejora de los métodos de detección poblaciones vulnerables.

Tabla 1

Indicadores de desempeño para modelos de aprendizaje automático individuales

Clasificadores	Métricas de evaluación				
	Accuracy	Specificy	Precision	Sensisivity	F1 Score
Random Forest	99,53%	100%	100%	99,22%	99,61%
SVM	81,80%	82,62%	84,24%	87,49%	80,72%
Naïve Bayes	78,16%	79,19%	73,09%	77,45%	74,13%
ANN	74,45%	76,47%	92,71%	78,14%	75,83%
Decision Tree	99,20%	100%	100%	98,71%	99,35%

Nota. Adaptado de *Application of ensemble models approach in anemia detection using images of the palpable palm* [Tabla 2] por Appiahene, et al.(2023a).

Tabla 2

Indicadores de desempeño para modelos de ensamble

	Método de ensamble		Métricas de evaluación				
			Accuracy	Specificity	Precision	Recall/Sensivity	F1 Score
1	Bagging	NB	79,75%	59,68%	70,18%	76,02%	72,99%
		SVM	80,48%	86,75%	82,94%	22,14%	77,17%
		ANN	78,56%	75,79%	79,56%	81,36%	83,36%
		DT	79,24%	100%	100%	60,81%	75,63%
2	Boosting		74,67%	71,39%	86,07%	73,20%	79,11%
3	Voting	RF+SVM+ANN	82,46%	88%	94,14%	80,16%	86,59%
		RF+DT+NB	99,22%	100%	100%	98,71%	99,35%
		SVM+NB+RF	83,16%	80,50%	87,76%	84,78%	86,24%
		ANN+SVM+NB+DT+RF	87,30%	90,09%	94,40%	86%	90,01%
		SVM+ANN+NB	82%	87,67%	81,90%	78,22%	87,89%
4	Stacking	NB+RF	99,98%	100%	100%	100%	100%
		RF+NB+SVM	99,53%	100%	100%	99,22%	99,61%
		NB+RF+SVM+ANN	99,30%	100%	100%	98,84%	99,42%
		ANN+DT+RF+SVM+NB	99,98%	100%	100%	100%	100%
		SVM+NB+ANN	82,12%	83,99%	89,35%	88,22%	89,23%

Nota. Adaptado de *Application of ensemble models approach in anemia detection using images of the palpable palm* [Tabla 3] por Appiahene, et al. (2023a).

b) Artículo de investigación «*Iron deficiency anemia detection using machine learning models: A comparative study of fingernails, palm and conjuctiva of the eye images*» de los autores Justice Williams Asare, Peter Appiahene, Emmanuel Timmy Donkoh y Giovanni Dimauro.

En el estudio realizado, se buscó detectar la anemia a través de un enfoque no invasivo utilizando imágenes médicas y técnicas de aprendizaje automático. Se tomaron imágenes de 710 participantes, principalmente niños de 5 a 59 meses, provenientes de 10 hospitales en Ghana. La recopilación de datos se llevó a cabo utilizando protocolos estandarizados para asegurar la calidad de las imágenes. Las fotos se tomaron de las conjuntivas de los ojos, las palmas y las uñas, y los niveles de hemoglobina se clasificaron para determinar si un participante era anémico (menos de 11 g/dL) o no anémico (11 g/dL o más).

Tal como se muestra en la Figura 7 y según la metodología propuesta por los autores, los datos recolectados fueron posteriormente preprocesados para optimizar su uso en el análisis. Este proceso incluyó una segmentación efectiva de las imágenes, donde se utilizaron algoritmos de umbral para aislar las áreas relevantes de interés. Se aplicó el espacio de color CIE Lab*, el cual posibilita la descomposición de las imágenes en sus componentes cromáticos, permitiendo un análisis más detallado de atributos como el color y la luminosidad de las áreas del cuerpo.

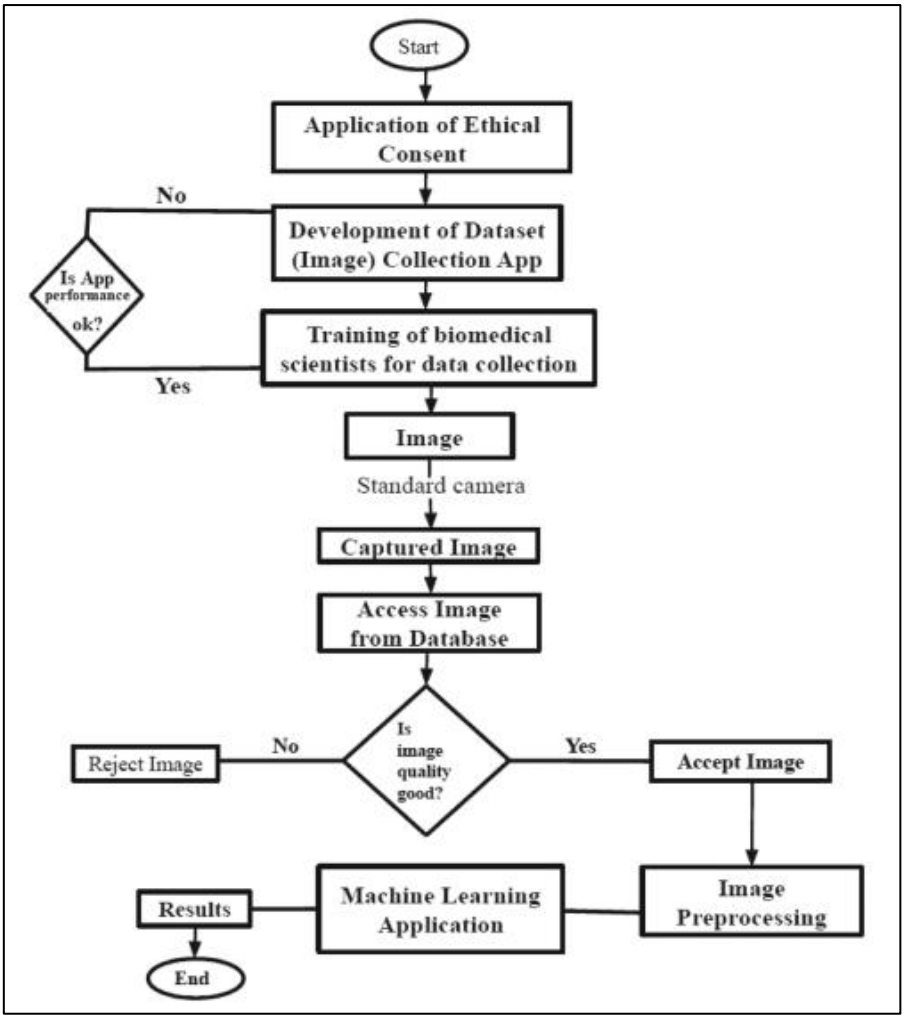
Dado que el tamaño de conjunto de datos original podía limitar la capacidad de generalización de los modelos, se aplicaron técnicas de ampliación de datos (data augmentation) para aumentar artificialmente la cantidad de imágenes disponibles. Esta etapa consistió en operaciones como rotación (90° y 270°), inversión vertical y horizontal (Flip), y traslación sobre los ejes X y Y. Con estas transformaciones se generaron nuevas instancias a partir de las imágenes originales, incrementando el número de muestras a 2,625 por cada tipo

de imagen (conjuntiva, palma y uñas), lo que permitió mejorar el entrenamiento y la robustez de los modelos.

Con los datos preprocesados, se implementaron cinco algoritmos de aprendizaje automático para la detección de anemia: Naïve Bayes, k-Vecinos Más Cercanos (k-NN), Máquina de Vectores de Soporte (SVM), árboles de decisión y Red Neuronal Convolucional (CNN). Cada uno de estos métodos presenta diferentes enfoques para clasificar los datos. Naïve Bayes utiliza probabilidades basadas en información previa, mientras que k-NN clasifica imágenes comparando sus características con las de sus vecinos más cercanos. SVM intenta encontrar la frontera ideal que distingue las clases dentro de un espacio de alta dimensión, mientras que los árboles de decisión generan un modelo gráfico comprensible, segmentando los datos mediante decisiones binarias. La CNN, más avanzada, utiliza múltiples capas para extraer características significativas de las imágenes, lo que mejora la precisión del diagnóstico.

Figura 7

Flujograma que ilustra el flujo de trabajo en los modelos propuestos



Nota. Adaptado de *Iron deficiency anemia detection using machine learning models: A comparative study of fingernails, palm and conjuctiva of the eye images* [Fig. 2] por Asare et al. (2023).

Los modelos fueron validados mediante técnicas de validación cruzada, y se ajustaron para optimizar su rendimiento. Como se muestra en la Tabla 3, sobre la evaluación de los modelos propuestos en la detección de anemia mediante la palma de la mano, la técnica CNN alcanzó la mayor precisión (99.12%) entre los algoritmos evaluados, indicando su capacidad para clasificar correctamente las imágenes de los participantes. El estudio determinó que la

aplicación de técnicas de machine learning en el análisis de imágenes médicas constituye una alternativa eficaz para mejorar la identificación de la anemia, especialmente en escenarios donde los recursos son escasos. Finalmente, en la investigación se resalta que los hallazgos obtenidos a partir de los modelos desarrollados en este trabajo evidencian que la palma de la mano es una región precisa del cuerpo humano que puede emplearse para la detección de anemia causada por deficiencia de hierro, particularmente en niños menores de seis años.

Tabla 3

Desempeño de los modelos propuestos para la detección de anemia utilizando la palma palpable

S/N	Algoritmo	Palmas de la mano (%)				
		Accuracy	F1-Score	AUC	Precision	Recall
1	CNN	99,12	99,89	99,95	99,79	99,98
2	Naïve Bayes	98,96	99,97	99,98	99,97	99,93
3	Decision Tree	98,29	98,97	99,38	98,77	99,18
4	k-NN	98,92	99,89	99,98	99,79	99,92
5	SVM	95,34	94,59	98,97	95,99	93,23

Nota. Adaptado *de Iron deficiency anemia detection using machine learning models: A comparative study of fingernails, palm and conjuctiva of the eye images* [Tabla 8] por Asare et al. (2023)

c) Artículo de investigación titulado «*Detection of iron deficiency anemia by medical images: a comparative study of machine learning algorithms*» de los autores Peter Appiahene, Justice Williams Asare, Emmanuel Timmy Donkoh, Giovanni Dimauro y Rosalia Maglietta.

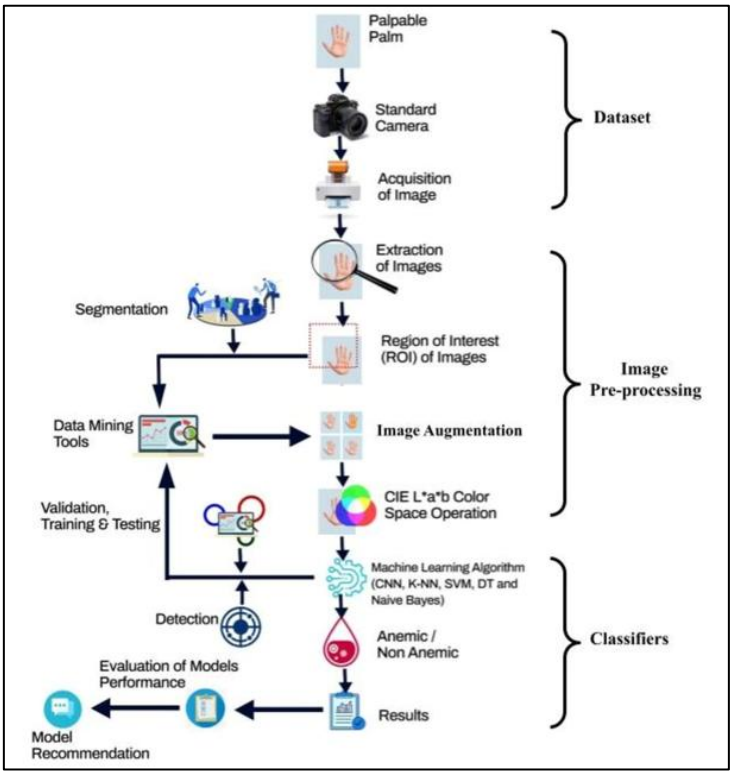
Este estudio presenta un enfoque no invasivo para la detección de anemia a través del análisis de imágenes de la palma de la mano, empleando modelos de aprendizaje automático.

Su objetivo es comparar la efectividad de distintos algoritmos en la detección de anemia sin tener que realizar pruebas invasivas, promoviendo así una alternativa accesible y rápida.

Tal como se muestra en la Figura 8, para ejecutar la investigación, se recolectaron imágenes de las palmas de 527 participantes. Posteriormente, se adoptaron técnicas de aumento de datos, como rotación, volteo y traslación, incrementando el dataset hasta 2635 imágenes. Estas imágenes fueron procesadas en el espacio de color CIE Lab* (CIELAB), permitiendo extraer características específicas de la tonalidad de la piel y la palidez de la palma, indicadores relevantes para la detección de anemia.

Figura 8

Un marco conceptual de la metodología



Nota. Adaptado de *Detection of iron deficiency anemia by medical images: a comparative study of machine learning algorithms* [Fig. 1] por Appiahene, et al.(2023b).

Además, las imágenes fueron segmentadas para aislar la región de interés y se aplicaron distintos modelos de aprendizaje automático: Convolutional Neural Network (CNN), k-Nearest Neighbors (k-NN), Naïve Bayes, Support Vector Machines (SVM) y Árbol de Decisión. La división de datos se realizó en 70% para entrenamiento, 10% para validación y 20% para prueba. Se aplicaron métodos de preprocesado para incrementar la calidad de las imágenes previo a su categorización.

Tal como se muestra en la Tabla 4, los resultados indicaron que el modelo de Naïve Bayes fue el más preciso, alcanzando una exactitud del 99.96%, seguido por la CNN y k-NN, ambos con 99.92%, mientras que SVM obtuvo la menor precisión con 96.34%. Este estudio reafirma la viabilidad del uso de imágenes de la palma de la mano para el diagnóstico temprano de anemia mediante métodos no invasivos, representando una alternativa eficiente y económica (Appiahene, et al., 2023b).

Tabla 4

Rendimiento de los modelos propuestos para detectar anemia mediante la palma de la mano

S/N	Algoritmo	Palmas (%)				
		Accuracy	F1-Score	AUC	Precision	Recall
1	CNN	99,92	99,89	99,95	99,79	99,98
2	Naïve Bayes	99,96	99,97	99,98	99,79	99,93
3	Decision Tree	99,29	98,97	99,38	98,77	99,18
4	k-NN	99,92	99,89	99,98	99,79	99,92
5	SVM	96,34	94,59	98,97	95,99	93,23

Nota. Adaptado de *Detection of iron deficiency anemia by medical images: a comparative study of machine learning algorithms* [Tabla. 5] por Appiahene, et al.(2023b).

d) Artículo de investigación titulado «*Joint Image Processing with Learning-Drive Data Representation and Model Behavior for Non-Intrusive Anemia Diagnosis in Pediatric Patients*» del autorTarek Berghout.

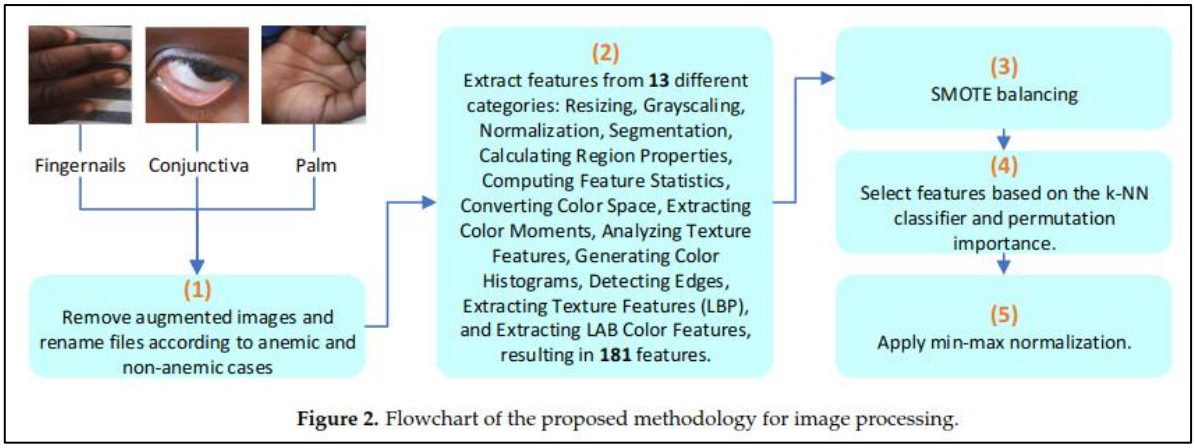
El estudio propone un sistema sustentado en aprendizaje profundo para la detección no invasiva de anemia en niños, mediante el análisis de imágenes de la conjuntiva ocular, la palma de la mano y las uñas. Su objetivo es mejorar la precisión del diagnóstico sin necesidad de realizar análisis de sangre frecuentes, especialmente en poblaciones pediátricas.

Para ello, se utilizaron tres datasets públicos, los cuales contenían 710 imágenes de niños menores de 6 años. Estas imágenes fueron adquiridas en hospitales de África Occidental y abarcan 424 imágenes de niños con anemia y 286 de niños sin anemia. Las imágenes fueron sometidas a técnicas de segmentación para extraer regiones de interés y se transformaron a espacios de color como CIELAB y HSV, permitiendo una mejor caracterización de los patrones de palidez en la piel.

En la metodología, como se muestra en la Figura 9, se implementó un pipeline de procesamiento de imágenes que permitió extraer 181 características divididas en 13 categorías. Posteriormente, se entrenó un modelo basado en Long Short-Term Memory (LSTM), optimizado mediante técnicas bayesianas. Para mejorar la capacidad del modelo, se integró la arquitectura LSTM en una nueva red profunda llamada Recurrent Expansion Network (RexNet), la cual busca mejorar la representación de datos y el comportamiento del modelo en la detección de anemia.

Figura 9

Diagrama de flujo de metodología propuesta para procesamiento de imágenes



Nota. Adaptado de *Joint Image Processing with Learning-Drive Data Representation and Model Behavior for Non-Intrusive Anemia Diagnosis in Pediatric Patients* [Fig. 2] por Berghout (2024).

Tal como se muestra en la Tabla 5, para el diagnóstico considerando el dataset de las palmas de manos, en la fase de Testing, los resultados indicaron que tanto LSTM y RexNet, alcanzaron una precisión de 99.18%, lo que podría significar que ambos modelos generalizan bien a nuevos datos. Sin embargo, la técnica de RexNet toma más tiempo de entrenamiento, lo que podría ser una desventaja si se busca eficiencia computacional. Así pues, de manera general, el estudio representa una alternativa viable y robusta para la detección de anemia sin necesidad de pruebas invasivas, con aplicaciones potenciales en el ámbito clínico.

Tabla 5

Comparaciones de los enfoques estudiados y los trabajos existentes

Método	Dataset	Testing			
		Accuracy	Precision	Recall	F1 Score
LSTM	Palma	0,9918	0,9910	0,9926	0,9917
RexNet	Palma	0,9918	0,9913	0,9924	0,9918
[20]*	Palma	0,9912	0,9979	0,9998	0,9989
[19]**	Palma	0,9996	0,9979	0,9998	0,9997

Nota. Adaptado de *Joint Image Processing with Learning-Drive Data Representation and Model Behavior for Non-Intrusive Anemia Diagnosis in Pediatric Patients* [Tabla 4] por Berghout (2024).

e) Artículo de investigación titulado «*Revolutionizing anemia detection: integrative machine learning models and advanced attention mechanisms*» de los autores Muhammad Ramzam, Jinfang Sheng, Mushammad Usman Saeed, Bing Wang y Faisal Z. Duraihem.

El estudio explora la adopción de modelos avanzados de aprendizaje automático, combinando atención espacial y técnicas multimodales para mejorar la detección de anemia. Su objetivo es desarrollar un modelo que supere las limitaciones de los enfoques tradicionales, optimizando la identificación de patrones en imágenes y datos clínicos.

Para la investigación, se utilizaron datasets de imágenes y datos textuales, incluyendo imágenes del lecho ungueal, la conjuntiva ocular y la palma de la mano. Se aplicaron modelos tradicionales como regresión logística, árboles de decisión, Random Forest, Support Vector Machines (SVM), Naïve Bayes y k-NN, junto con un modelo basado en la arquitectura de AlexNet, modificado con módulos de atención.

La metodología incluyó la conversión de las imágenes a los canales BGR (Blue-Green-Red), lo que permitió mejorar la segmentación y extracción de características relevantes. Se aplicaron técnicas de aprendizaje profundo que incorporan módulos de atención espacial, junto con estudios de ablación, con el objetivo de analizar el efecto de diversas configuraciones del modelo. Además, se compararon distintos métodos de clasificación para evaluar la robustez del enfoque propuesto.

Los resultados obtenidos reflejan el impacto de diferentes configuraciones del modelo AlexNet al integrar fusión de características. Como se observa en la Tabla 6, se compararon cuatro variantes del modelo: AlexNet+BGR, que combinó imágenes y texto logrando una precisión de 94.51%; AlexNet con atención simple, con 93.78% de precisión; AlexNet con atención múltiple por canal, que aunque tuvo la menor precisión (84.72%), logró un alto F1 Score (98.06%); y finalmente, AlexNet con atención espacial, que obtuvo el mejor rendimiento global con 95.36% de precisión y un F1 Score de 98.65%, validando la efectividad de la atención espacial en la detección de anemia.

Tabla 6

Rendimiento del modelo usando característica fusión

Modelo	Dataset	Accuracy	Precision	Recall	F1Score
AlexNet+BGR	Image+text	94,51%	96,78%	95,98%	96,35%
AlexNet+attention		93,78%	95,79%	96,04%	96,38%
AlexNet+multiple channel attention		84,72%	97,62%	97,18%	98,06%
AlexNet+Spatial attention		95,36%	98,47%	97,83%	98,65%

Nota. Adaptado de *Revolutionizing anemia detection: integrative machine learning models and advanced attention mechanisms* [Tabla 2] por (Ramzan et al., 2024).

Finalmente, de acuerdo con la Tabla 7, los resultados mostraron que el modelo AlexNet Multiple Spatial Attention (AMSA) obtuvo la mayor precisión con 99.58%, destacando la efectividad de la integración de atención espacial y la fusión de características. Se concluyó que este enfoque es altamente prometedor para la automatización de la detección de anemia, con potencial de implementación en entornos clínicos.

Tabla 7

Resultados del estudio de ablación con el modelo AMSA propuesto

Método	Accuracy	Precision	Recall	F1 score
AMSA	99,58%	99,97%	99,95%	99,97%
No BGR	99,15%	99,85%	99,82%	99,83%
No attention	92,64%	95,12%	95,24%	96,01%
No multiple attention	93,75%	96,37%	96,15%	95,97%
No spatial attention	90,95%	94,72%	94,51%	94,42%

Nota. Adaptado de *Revolutionizing anemia detection: integrative machine learning models and advanced attention mechanisms* [Tabla 3] por (Ramzan et al., 2024).

f) Artículo de investigación titulado «*Proposal of a Computational Vision Model for the Prediagnosis of Anemia based on the image of the ocular conjuctiva*» de los autores Alejandro Idelfonso y Wilfredo Ticona.

Este estudio propone un modelo basado en visión por computadora para el prediagnóstico de anemia mediante imágenes de la conjuntiva ocular. Su objetivo es proporcionar una herramienta rápida y accesible para detectar anemia en regiones con acceso limitado a laboratorios clínicos.

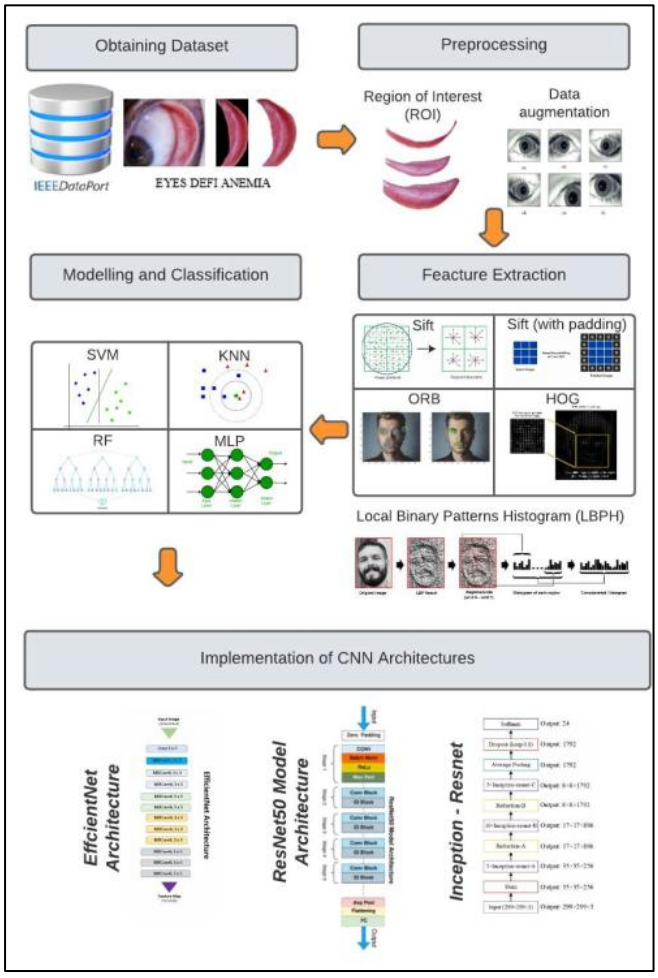
Se utilizó el dataset EYES-DEFI-ANEMIA de IEEE DataPort, el cual contiene 217 imágenes de la conjuntiva ocular de pacientes con y sin anemia. Se aplicaron técnicas de

segmentación para aislar la región de interés (fornix palpebral) y mejorar la precisión del análisis.

Como se visualiza en la Figura 10, la metodología del estudio incluyó cinco fases: obtención del dataset, preprocesamiento, extracción de características (Scale Invarient feature transform o SIFT, Oriented fast and rotated brief o ORB, Speeded-Up Robust Features o SURF y Histogram of Oriented Gradients o HOG), clasificación con modelos de aprendizaje automático (SVM, Random Forest, MLP, RNN) y experimentación con redes neuronales profundas como Inception-ResNet-v2. Se evaluaron múltiples combinaciones de modelos y técnicas de preprocesamiento para mejorar la precisión de la detección.

Figura 10

Metodología propuesta



Nota. Adaptado de *Proposal of a Computational Vision Model for the Prediagnosis of Anemia based on the image of the ocular conjuctiva* [Fig.1] por Ildefonso & Ticona (2024).

Los resultados evidencian la importancia del extractor de características en el desempeño de los modelos de clasificación. Como se observa en la Tabla 8, el modelo Random Forest con HOG alcanzó la mayor precisión con 79%, superando a otros enfoques basados en SIFT, LBPH y ORB. En particular, los modelos basados en SVM y KNN mostraron mejores resultados con el extractor LBPH y SIFT Padding, mientras que HOG demostró ser el más eficaz para el diagnóstico de anemia en este conjunto de datos. Estos hallazgos destacan la relevancia de una adecuada selección de técnicas de preprocesamiento y extracción de características para mejorar la capacidad predictiva de los modelos.

Tabla 8

Precisión de los modelos usando varios extractores de características

Modelo	Extractor de características				
	SIFT	SIFT(Padding)	LBPH	HOG	ORB
SVM	64%	65%	68%	76%	62%
KNN	67%	68%	65%	75%	63%
Random Forest	65%	70%	67%	79%	64%
MLP	69%	62%	66%	71%	64%

Nota. Adaptado de *Proposal of a Computational Vision Model for the Prediagnosis of Anemia based on the image of the ocular conjuctiva* [Tabla 8] por Ildefonso & Ticona (2024).

Por otro lado, los modelos de Aprendizaje Profundo presentaron un rendimiento significativamente superior en comparación con los enfoques tradicionales. Como se muestra en la Tabla 9, el modelo Inception-ResNet-v2 logró una precisión del 90.69% con una pérdida de 0.2788, consolidándose como una de las opciones más robustas. Sin embargo, el modelo

personalizado (Personalized), diseñado con una configuración específica de 1024 neuronas y 1000 muestras, obtuvo la mayor precisión con 93.18%, aunque con una pérdida algo superior (0.3563) en comparación con Inception-ResNet-v2. Estos resultados sugieren que la personalización de parámetros en redes profundas puede optimizar aún más la precisión en la detección de anemia, posicionando el uso de redes neuronales como una alternativa viable y más precisa que los modelos convencionales.

Tabla 9

Precisión de los modelos de Aprendizaje Profundo utilizando diferentes parámetros

Modelo	Neurons	Epochs	Samples	Accuracy	Loss
Inception-ResNet-v2	2048	50	1500	90,69%	0,2788
Resnet	512	50	500	84,09%	0,6131
EfficientNet	2048	50	1500	86,04%	0,4077
Personalized	1024	50	1000	93,18%	0,3563

Nota. Adaptado de *Proposal of a Computational Vision Model for the Prediagnosis of Anemia based on the image of the ocular conjuctiva* [Tabla 8] por Ildefonso & Ticona (2024).

2.2 Bases Teóricas

2.2.1 Anemia

La anemia es un trastorno definido por una reducción en los niveles de hemoglobina o en la cantidad de glóbulos rojos, lo que compromete la capacidad de la sangre para transportar oxígeno de manera eficiente, manifestándose en síntomas como fatiga y palidez (Gardner et al., 2023). Las causas más comunes incluyen deficiencia de hierro, problemas nutricionales y enfermedades crónicas (World Health Organization, s/f).

Además de la falta de hierro y los problemas nutricionales, ciertas enfermedades crónicas pueden contribuir al desarrollo de la anemia al afectar la absorción de hierro o provocar pérdidas sanguíneas. Afecciones como la celiaquía y la enfermedad inflamatoria intestinal pueden dificultar la asimilación de nutrientes esenciales, mientras que infecciones persistentes y parásitos intestinales pueden agravar el déficit de hierro. En algunos casos, la anemia tiene un origen genético, como en la anemia falciforme o las talasemias, donde la producción o estructura de los glóbulos rojos se ve comprometida (Camaschella, 2015).

La anemia por deficiencia de hierro es frecuente en la niñez y suele estar relacionada con una alimentación insuficiente en este mineral. Es particularmente común en etapas de rápido crecimiento, cuando el organismo requiere mayores cantidades de hierro para su adecuado desarrollo. Si no se detecta y trata a tiempo, esta condición puede influir negativamente en diversas áreas del desarrollo infantil, afectando el aprendizaje, la coordinación motora y la interacción social, con posibles repercusiones a largo plazo (Lozoff, 2007).

Tradicionalmente, el diagnóstico de la anemia se ha realizado a través de análisis sanguíneos, evaluando los niveles de hemoglobina y hematocrito. Sin embargo, existen

métodos más sencillos y accesibles que pueden sugerir la presencia de anemia, especialmente útiles en entornos con recursos limitados.

La evaluación visual de la palidez en distintas partes del cuerpo es un método clínico que puede ayudar a identificar casos de anemia sin necesidad de pruebas de laboratorio inmediatas (Kalantri et al., 2010). Entre los signos más comunes utilizados en la exploración médica se encuentra la observación de la conjuntiva, la cual se examina levantando suavemente el párpado inferior para evaluar su coloración. En individuos sanos, esta estructura debe presentar un tono rosado debido a una adecuada perfusión sanguínea. Sin embargo, en personas con anemia, la conjuntiva palpebral adquiere una apariencia más pálida, lo que sugiere niveles bajos de hemoglobina. La Figura 11 ilustra esta diferencia, mostrando la comparación entre una conjuntiva normal y otra pálida.

Figura 11

Conjuntiva pálida vs conjuntiva normal



Nota. Adaptado de *The relation of conjunctival pallor to the presence of anemia* [Figs.1 y 2] por Sheth et al. (1997).

Otro indicador visual útil es la palidez en las palmas de las manos, ya que la baja concentración de melanina en esta zona facilita la detección de cambios en la oxigenación de los tejidos. En personas sin anemia, las palmas presentan un tono rosado debido a la adecuada

circulación sanguínea. No obstante, cuando los niveles de hemoglobina son bajos, la piel en esta área puede volverse pálida, reflejando una disminución en la oxigenación celular. La Figura 12 muestra la diferencia de tonalidad en los pliegues palmares entre la mano de un paciente con anemia severa (hemoglobina de 4.7 g/dl) y la del examinador, en la que se observa una coloración más intensa.

Figura 12

Palidez de los pliegues palmares de la mano del paciente (flecha roja) en comparación con la mano del examinador (flecha azul)



Nota. Adaptado de *Palmar Crease Pallor* [Fig.1] por Manesh & Kohlwes (2015).

Además, la palidez puede manifestarse en el lecho ungueal, ubicado bajo las uñas. En condiciones normales, esta zona muestra una coloración rosada gracias a la hemoglobina en los capilares subyacentes. Cuando la hemoglobina disminuye significativamente, la palidez en esta área se hace más notoria, lo que puede ser un indicio de anemia. Aunque este método puede ser

útil en la evaluación clínica, su precisión es menor en comparación con otros signos como la observación de la lengua o la conjuntiva (Kalantri et al., 2010).

La lengua también es un área clave en la detección de anemia, ya que su palidez puede estar directamente relacionada con niveles bajos de hemoglobina. En individuos saludables, la lengua tiene un color rosado uniforme, mientras que en casos de anemia severa adquiere un tono pálido. La Figura 13 presenta un sistema automatizado basado en inteligencia artificial que clasifica imágenes de lenguas normales y pálidas, proporcionando una alternativa innovadora para la identificación temprana de esta condición.

Figura 13

Ejemplo de imágenes de palidez en lengua (Class0: normal y Class 1: pálido)



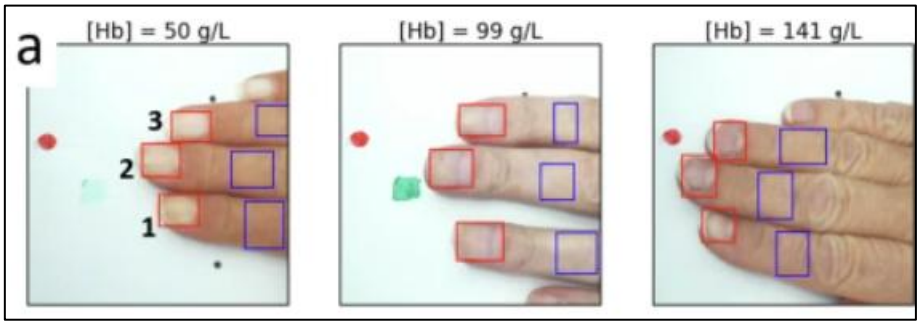
Nota. Adaptado de *Azure-Based Smart Monitoring System for Anemia-Like Pallor* [Fig.1] por Roychowdhury et al. (2017)

Por último, evaluar la tonalidad de la piel en diversas zonas del cuerpo puede ser una herramienta útil para la detección de anemia. La Figura 14 muestra imágenes de manos de pacientes con diferentes niveles de hemoglobina total en sangre, lo que permite visualizar la relación entre la oxigenación de los tejidos y la presencia de anemia. Este tipo de evaluación

puede contribuir al desarrollo de herramientas no invasivas para la medición de hemoglobina en la práctica médica.

Figura 14

Imágenes ejemplares de manos de pacientes con diferentes niveles de hemoglobina total en sangre



Nota. Adaptado de *Dataset of human skin and fingernails images for non-invasive haemoglobin level assessment* [Fig.1a] por Yakimov et al. (2024)

Estos métodos de observación son herramientas valiosas, especialmente en comunidades donde el acceso a laboratorios es limitado. No obstante, siempre deben complementarse con una evaluación médica completa y, si es posible, análisis de sangre para reafirmar el diagnóstico y determinar la causa específica.

2.2.2 Machine Learning (Aprendizaje automático)

a) Definición

Es un campo de la inteligencia artificial que optimiza el rendimiento mediante la programación basada en datos y experiencias previas, permitiendo que los programas mejoren su desempeño a partir del análisis y ajuste de patrones en la información disponible, ya sea con o sin supervisión humana (Dulhare et al., 2020).

Mitchell (1997) proporciona una definición formal de Machine Learning, indicando que un programa de computadora aprende de la experiencia E en relación con

una clase de tareas T y una medida de desempeño P, cuando, a partir de la experiencia E, su rendimiento en las tareas de T mejora según P. Esta definición enfatiza el carácter operativo de Machine Learning, diferenciándolo de una simple interpretación cognitiva. Siguiendo esta línea, Dulhare et al. (2020) destacan que el machine learning busca desarrollar programas que, mediante el uso de datos pasados, mejoren su rendimiento sin intervención humana directa.

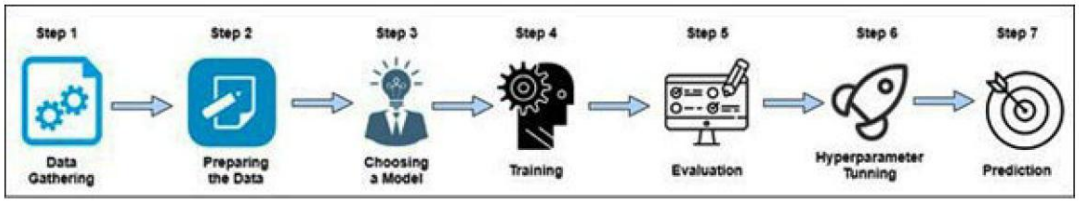
Esta visión sigue el planteamiento de Alan Turing, quien reformuló la pregunta "¿Pueden las máquinas pensar?" por "¿Pueden las máquinas hacer lo que los humanos pueden hacer?" (Dulhare et al., 2020), lo que refleja el objetivo último de la disciplina: lograr sistemas capaces de mejorar su desempeño de manera autónoma mediante la experiencia.

El proceso de Machine Learning puede incluir siete pasos, como se muestran en la Figura 15. Comienza con la recolección de datos, asegurando que la cantidad y calidad sean adecuadas para entrenar el modelo de manera efectiva. Errores en la selección de características o en la diversidad del conjunto de datos pueden afectar su rendimiento. Luego, en la preparación de datos, se organizan, limpian y corrigen inconsistencias, además de dividirse en dos partes: una para el entrenamiento (~80%) y otra para la evaluación (~20%). Posteriormente, se elige el modelo de aprendizaje adecuado basados en el tipo de datos y el problema a solucionar. En la etapa de entrenamiento, los parámetros iniciales del modelo se ajustan iterativamente mediante optimización para minimizar el error. Luego, en la evaluación, se mide su desempeño con datos nuevos para verificar su capacidad de generalización; si la precisión es baja, se revisan pasos previos. Para mejorar la eficacia del modelo, se realiza la tuning de hiperparámetros, optimizando factores como la tasa de aprendizaje. Finalmente, en la etapa de predicción, el modelo se implementa en aplicaciones prácticas, permitiendo realizar inferencias y

tomar decisiones basadas en los datos analizados. Este flujo de trabajo garantiza el desarrollo de modelos de Machine Learning robustos y confiables (Hiran et al., 2021).

Figura 15

Pasos de Machine Learning



Nota: Adaptado de *Master Supervised and Unsupervised Learning Algorithms with Real Examples* [Fig. 1.5] por (Hiran et al., 2021).

b) Historia

Machine Learning ha evolucionado más allá del alcance original de la inteligencia artificial (AI). En sus inicios, los investigadores descubrieron que las máquinas podían aprender a partir de los datos y exploraron diversas estrategias simbólicas, incluyendo los llamados "redes neuronales". Estas técnicas, como el perceptrón y otros modelos primitivos, fueron más tarde reformuladas en modelos lineales generalizados (Dulhare et al., 2020).

Con el tiempo, la inteligencia artificial (AI) se inclinó hacia enfoques basados en el conocimiento y la lógica, creando una brecha con el machine learning. En los años 80, los sistemas expertos se convirtieron en el enfoque predominante dentro de la AI, desplazando el interés por la estadística y llevando la investigación hacia la programación lógica deductiva. Sin embargo, los enfoques basados en estadística y probabilidad continuaron evolucionando al margen del ámbito tradicional de la inteligencia artificial. Fue durante la década de 1990 cuando el Machine Learning resurgió como una disciplina independiente, distanciándose de los métodos simbólicos

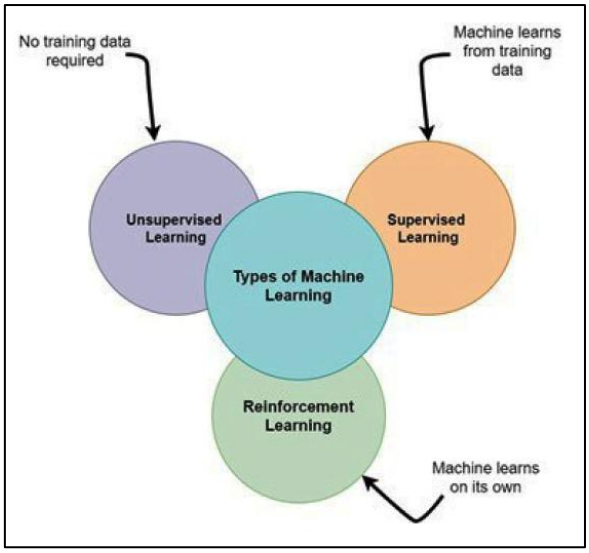
heredados de la AI y adoptando modelos fundamentados en principios estadísticos y probabilísticos, impulsados por el auge de la digitalización y el acceso masivo a la información a través de internet (Dulhare et al., 2020).

c) Tipos de Machine Learning

Machine Learning puede implementarse de diversas formas, ya sea mediante un proceso supervisado, donde la máquina es entrenada con datos etiquetados, o a través de enfoques en los que el modelo aprende de manera autónoma a partir de patrones en los datos. En términos generales, Machine Learning se clasifica en tres categorías principales: aprendizaje supervisado, no supervisado y por refuerzo, como se ilustra en la Figura 16. En esta parte, se analizarán en profundidad estas categorías para comprender sus características y aplicaciones (Hiran et al., 2021).

Figura 16

Tipos de Machine Learning



Nota: Adaptado de *Master Supervised and Unsupervised Learning Algorithms with Real Examples* [Fig. 1.6] por Hiran et al. (2021).

- **Supervised Learning (Aprendizaje supervisado)**

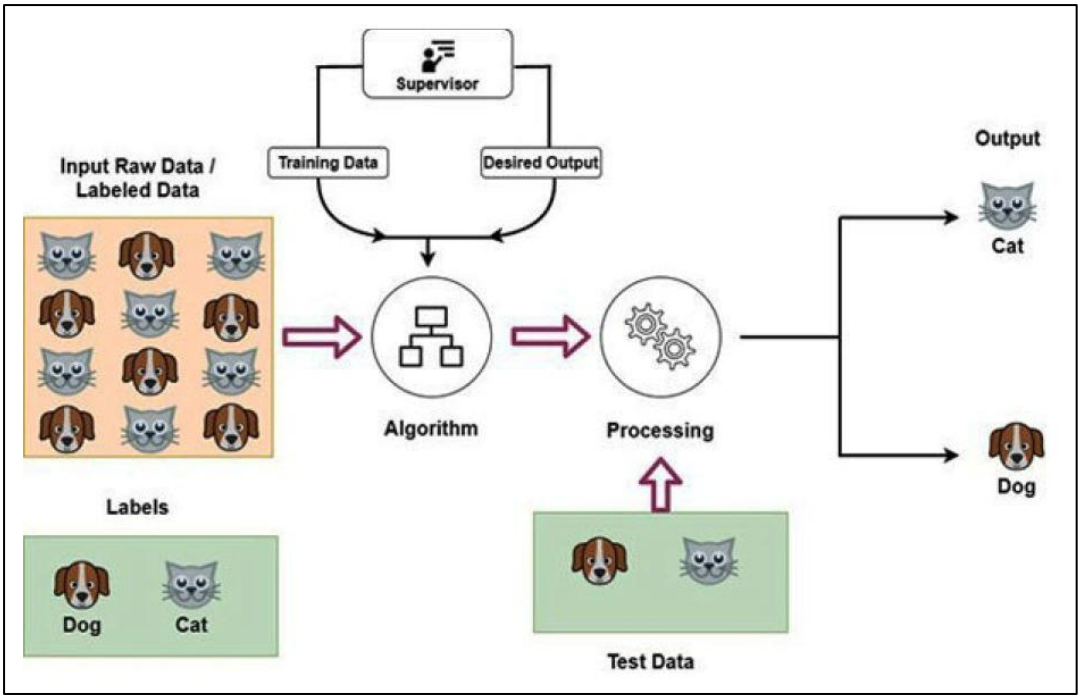
El aprendizaje supervisado en Aprendizaje Automático se basa en la enseñanza a través de ejemplos, facilitando que el algoritmo adquiera conocimientos a partir de un conjunto de datos con etiquetas predefinidas. Se denomina "supervisado" porque el proceso de entrenamiento puede compararse con la supervisión de un instructor que guía el aprendizaje del modelo. En este enfoque, los datos incluyen variables de entrada y salida previamente definidas, lo que permite al algoritmo identificar patrones y establecer relaciones entre ambas (Hiran et al., 2021).

Durante el entrenamiento, el modelo analiza los datos de entrada y aprende a asociarlos con las salidas correspondientes. Una vez finalizado este proceso, el algoritmo puede recibir nuevos datos sin etiquetar y predecir su clasificación basándose en el conocimiento adquirido. El objetivo principal del aprendizaje supervisado es generar predicciones precisas sobre datos desconocidos utilizando la experiencia previa (Hiran et al., 2021).

Como se muestra en la figura 17, un ejemplo común de aprendizaje supervisado es la clasificación de imágenes de gatos y perros, donde el modelo recibe un conjunto de imágenes etiquetadas y aprende a distinguir entre ambas categorías. Si el entrenamiento es exitoso, cuando se le presenta una nueva imagen sin etiqueta, el modelo podrá identificar correctamente si representa a un gato o un perro. En el aprendizaje supervisado, los problemas se clasifican en dos categorías principales: la regresión, utilizada para predecir valores continuos, y la clasificación, que consiste en asignar una etiqueta a una categoría determinada (Hiran et al., 2021).

Figura 17

Aprendizaje supervisado



Nota: Adaptado de *Machine Learning: Master Supervised and Unsupervised Learning Algorithms with Real Examples* [Fig. 1.7] por Hiran et al. (2021).

- **Unsupervised learning (Aprendizaje no supervisado)**

El aprendizaje no supervisado se fundamenta en la exploración de datos sin etiquetas, lo que implica la ausencia de una variable de salida predefinida y de un proceso de supervisión en el entrenamiento del modelo. A diferencia del aprendizaje supervisado, en este enfoque no es necesario proporcionar ejemplos con respuestas correctas, ya que el algoritmo identifica patrones y estructuras dentro de los datos por sí mismo (Hiran et al., 2021).

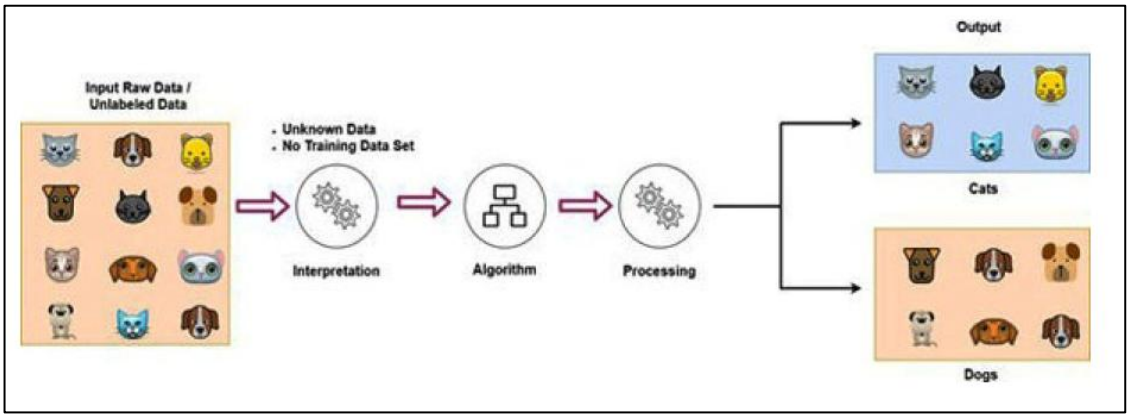
El modelo examina las características de los datos de entrada y detecta similitudes para agruparlos en categorías, sin conocimiento previo sobre su contenido. Por ejemplo, tal como se ve en la Figura 18, si se brinda un conjunto de imágenes de

gatos y perros sin etiquetas, el algoritmo no sabe qué imágenes pertenecen a cada categoría, pero puede organizarlas en grupos según sus semejanzas. Este proceso, conocido como clustering, permite segmentar los datos sin intervención humana (Hiran et al., 2021).

Los algoritmos de aprendizaje no supervisado pueden clasificarse en dos tipos principales: clustering, que organiza los datos en grupos según patrones comunes, y asociación, que busca relaciones ocultas entre las variables de los datos (Hiran et al., 2021).

Figura 18

Aprendizaje no supervisado



Nota: Adaptado de *Master Supervised and Unsupervised Learning Algorithms with Real Examples* [Fig. 1.9] por Hiran et al. (2021).

- **Reinforcement Learning (Aprendizaje de refuerzo)**

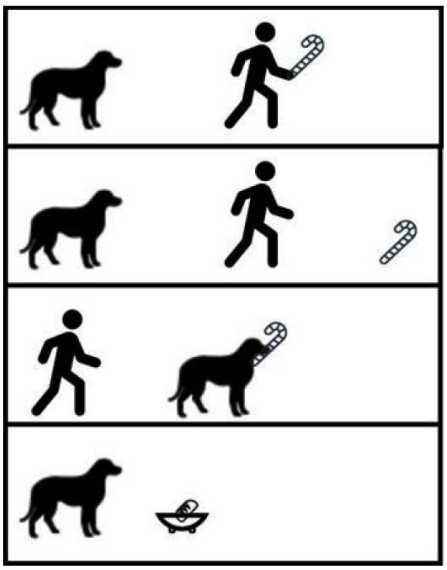
Es el tercer y último tipo de Machine Learning y se basa en un sistema de retroalimentación en el que un agente aprende de su propia experiencia sin necesidad de datos etiquetados. En este método, el modelo perfecciona su rendimiento mediante un proceso de ensayo y error, obteniendo recompensas por decisiones acertadas y

penalizaciones por aquellas incorrectas. Su objetivo se basa en maximizar la acumulación de recompensas a lo largo del tiempo para optimizar su comportamiento (Hiran et al., 2021).

Un ejemplo ilustrativo es el entrenamiento de un perro, como el mostrado en la Figura 19. Dado que el animal no comprende el lenguaje humano, no se le pueden dar instrucciones directas. En su lugar, se le coloca en una situación en la que intenta responder de diversas maneras. Si su acción es deseada, recibe un incentivo, como comida. Con el tiempo, el perro repite la acción esperando obtener la misma recompensa, mientras que evita conductas que le generen resultados negativos. De manera similar, en el aprendizaje por refuerzo, el modelo ajusta su comportamiento en función de las recompensas obtenidas, desarrollando estrategias óptimas a partir de su interacción con el entorno (Hiran et al., 2021).

Figura 19

Aprendizaje de refuerzo



Nota: Adaptado de *Master Supervised and Unsupervised Learning Algorithms with Real Images* [Fig. 1.9] por Hiran et al. (2021).

d) Machine Learning en la clasificación de imágenes

Uno de los usos del Machine Learning es la identificación de imágenes; por ejemplo, para identificar elementos, personas, lugares, fotos digitales y más en una imagen, se utiliza el procesamiento de imágenes. Este método se emplea para realizar análisis adicionales, como la detección de rostros, el reconocimiento de caracteres, el reconocimiento de patrones y el reconocimiento facial (Hiran et al., 2021).

A continuación, se presentan cinco algoritmos de Machine Learning usados en clasificación de imágenes:

- **Support Vector Machine (SVM)**

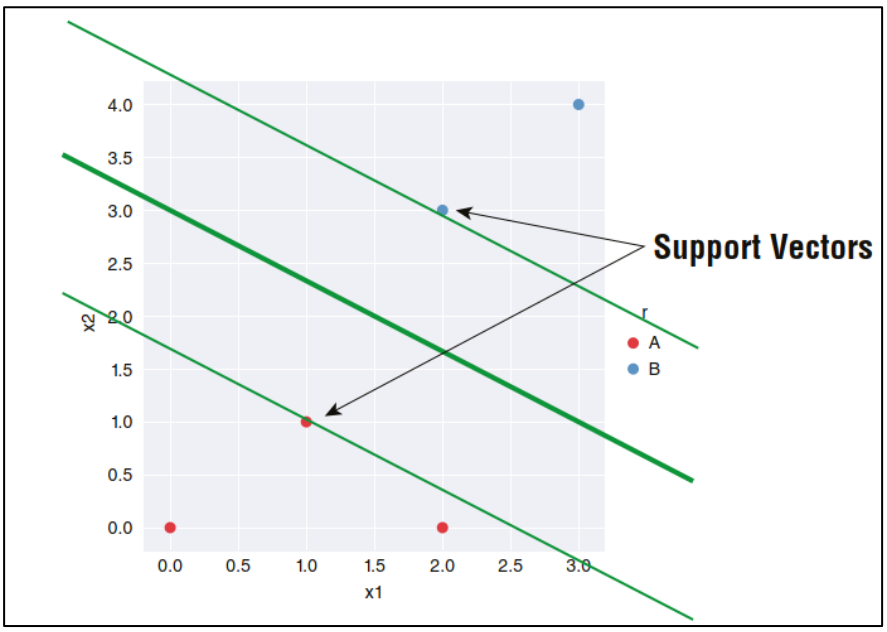
La máquina de vectores de soporte es un método de aprendizaje supervisado que aborda principalmente tareas de clasificación, aunque también puede ser aplicado a problemas de regresión. Su funcionamiento se basa en la identificación del hiperplano de separación óptimo que maximiza el margen entre distintas clases. Para ello, cada instancia es representada en un espacio n-dimensional, donde n es el número de características, y se determina la frontera que mejor distingue las categorías. En dos dimensiones, esta frontera es una línea recta; en tres dimensiones, se convierte en un plano. Los puntos más próximos al hiperplano, denominados “vectores de soporte” tienen un papel fundamental ya que definen su ubicación y orientación. La eliminación de alguno de estos puntos puede alterar significativamente dicha frontera (Dulhare et al., 2020).

A partir de este punto, se sigue la explicación desarrollada por Lee (2019), quien desarrolla con detalle los fundamentos algebraicos y geométricos de la Máquina de Vectores de Soporte, así como sus extensiones mediante funciones núcleo (kernel) y los parámetros ajustables que afectan su comportamiento.

Cuando se trata de clasificación lineal, los “vectores de soporte”, se encuentran en los márgenes definidos por líneas paralelas, como se observa en la Figura 20. Estos puntos, al estar más próximo al hiperplano central, determinan directamente su posición.

Figura 20

Gráfico de los vectores de soporte



Nota. Adaptado de “Python ® Machine Learning» [Fig.8.5] por Lee (2019).

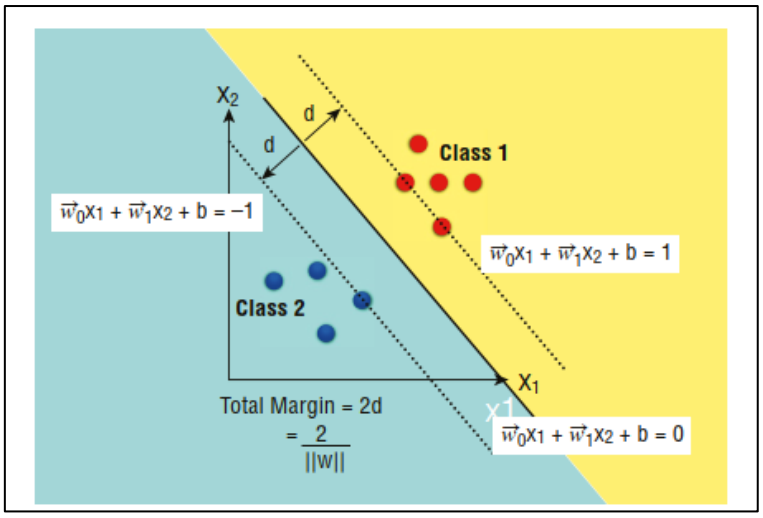
El hiperplano puede expresarse mediante la función:

$$g(x) = \vec{W}_0.x1 + \vec{W}_1.x2 + b$$

Donde x_1 y x_2 son las variables de entrada, $\vec{W}_0$ y $\vec{W}_1$ son los vectores de peso asociados, y b es el sesgo o término de ajuste. Cuando el valor de g(x) es mayor o igual que 1, entonces el punto especificado está en la Clase 1, si es menor o igual que -1, pertenece a la Clase 2. El objetivo de SVM es encontrar los márgenes más amplios posibles entre las clases, lo que se logra minimizando el valor de la norma de los pesos ||w||. La Figura 21 ilustra esta relación, en la que el margen total entre las clases es igual a 2/ ||w||.

Figura 21

Fórmula de hiperplano y los márgenes que lo acompañan

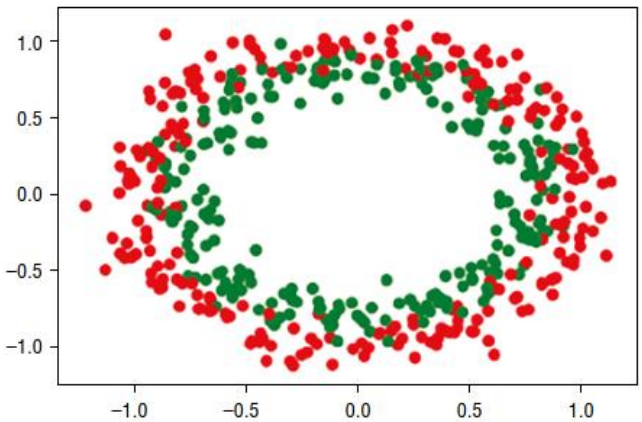


Nota. Adaptado de “Python ® Machine Learning» [Fig.8.6] por Lee (2019).

En situaciones en que los datos no pueden separarse mediante una línea recta como en la Figura 22, se utiliza el denominado “truco del núcleo” (Kennel Trick), que transforma los datos a un espacio de mayor dimensión en el que se pueda lograr una separación lineal clara.

Figura 22

Diagrama de dispersión de dos grupos de puntos distribuidos de forma circular



Nota. Adaptado de “Python ® Machine Learning» [Fig.8.11] por Lee (2019).

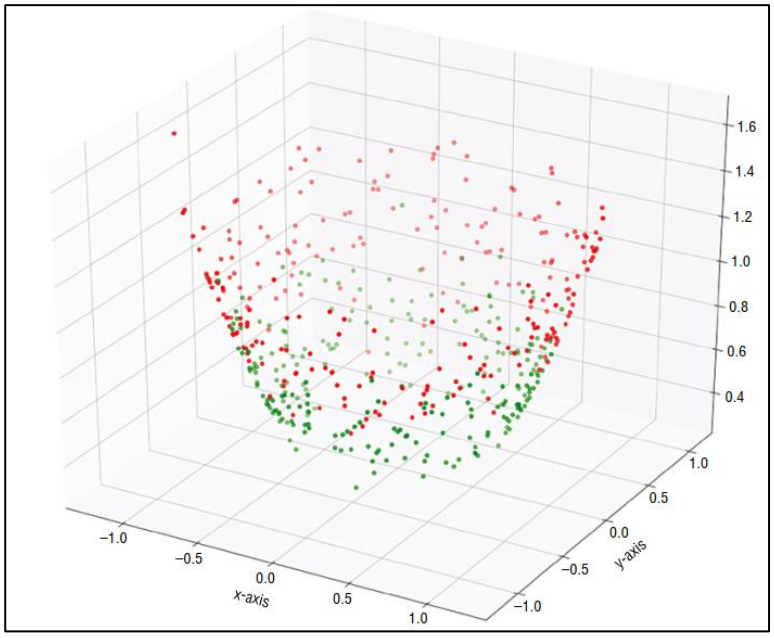
Una técnica común consiste en agregar una tercera dimensión, definida como:

$$z = x^2+y^2$$

lo cual permite visualizar los datos como una superficie en 3D. Esto facilita la identificación de un plano que separe ambos conjuntos de puntos, como se observa en la Figura 23.

Figura 23

Trazado de los puntos en las 3 dimensiones



Nota. Adaptado de “Python ® Machine Learning» [Fig.8.12] por Lee (2019).

Con esta transformación, la fórmula del hiperplano se expresa como:

$$g(x) = \vec{W}_0.x1 + \vec{W}_1.x2 + \vec{W}_2.x3 + b$$

Donde:

x3 representa la nueva dimensión. Para obtener el valor de x3 que defina el hiperplano, se despeja:

$$\vec{W}_0.x1 + \vec{W}_1.x2 + \vec{W}_2.x3 + b = 0$$

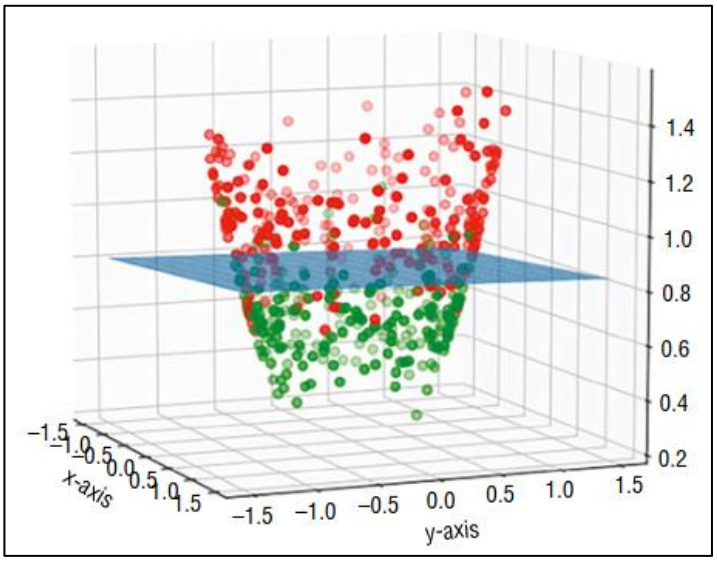
$$\vec{W}_2.x3 = -\vec{W}_0.x1 - \vec{W}_1.x2 - b$$

$$\Rightarrow x3 = \frac{(-\vec{W}_0.x1 - \vec{W}_1.x2 - b)}{\vec{W}_2}$$

Este plano de separación en 3D se puede observar en la Figura 24, donde queda evidenciado cómo divide dos grupos de datos.

Figura 24

El hiperplano en 3D de dos sets de puntos



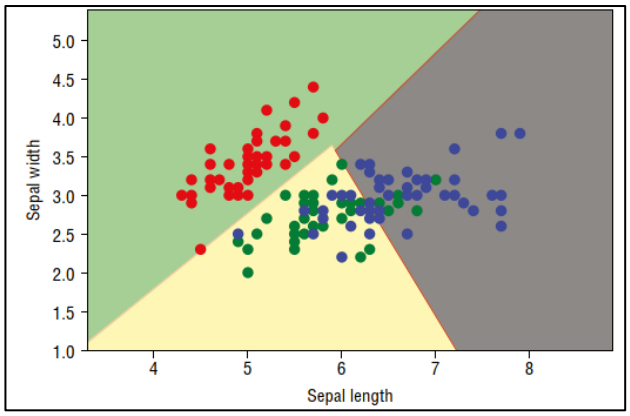
Nota. Adaptado de “Python ® Machine Learning» [Fig.8.16] por Lee (2019).

En algunos casos, los datos siguen sin poder separarse adecuadamente, incluso tras la transformación. En tales situaciones, se emplean funciones núcleo (kernels), que se permiten trazar fronteras no lineales de decisión. Entre los tipos más comunes se encuentran:

- Kernel Lineal, que utiliza una línea recta como frontera de separación, como en la figura 25

Figura 25

Uso de kernel lineal SVM

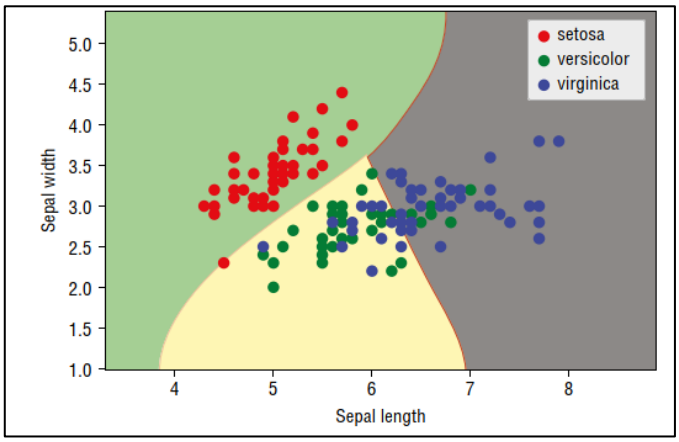


Nota. Adaptado de “Python ® Machine Learning» [Fig.8.19] por Lee (2019).

- Kernel de base Radial (Radial Basis Function, RBF) también conocido como núcleo gaussiano, asigna un valor a cada punto según su distancia al centro. Su uso sobre el conjunto de datos Iris se muestra en la Figura 26.

Figura 26

Uso de kernel RBF para el dataset público "Iris"

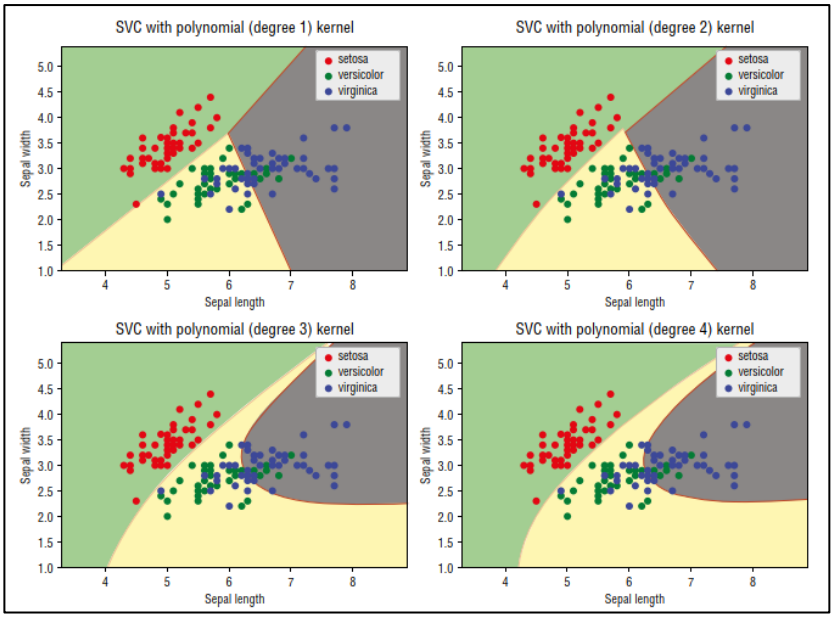


Nota. Adaptado de “Python ® Machine Learning» [Fig.8.23] por Lee (2019).

- Kernel Polinomial, que permite un límite de decisión más complejo. En la Figura 27 se observa su efecto al variar el grado del polinomio de 1 a 4. En las gráficas, las siglas SVC hace referencia a Support Vector Classification, una clase de la biblioteca Scikit-learn.

Figura 27

Clasificación del dataset público "Iris" usando kernel polinominal de varios grados

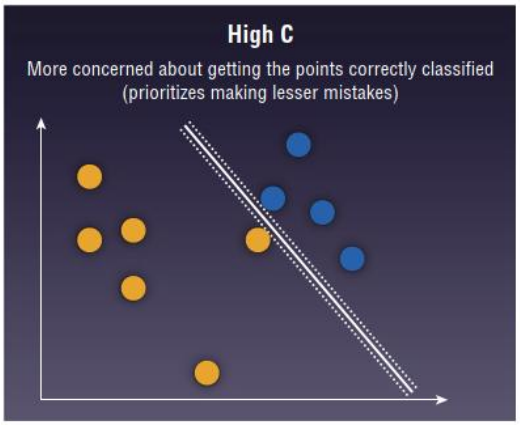


Nota. Adaptado de “Python ® Machine Learning» [Fig.8.28] por Lee (2019).

Asimismo, SVM cuenta con hiperparámetros que controlan su comportamiento. Uno de ellos es el parámetro es el parámetro C, que regula el equilibrio entre la amplitud del margen y la cantidad de errores permitidos. Cuando un valor alto de C, el algoritmo prioriza la clasificación correcta de los puntos, como se observa en la Figura 28.

Figura 28

Representación del parámetro C con un valor alto

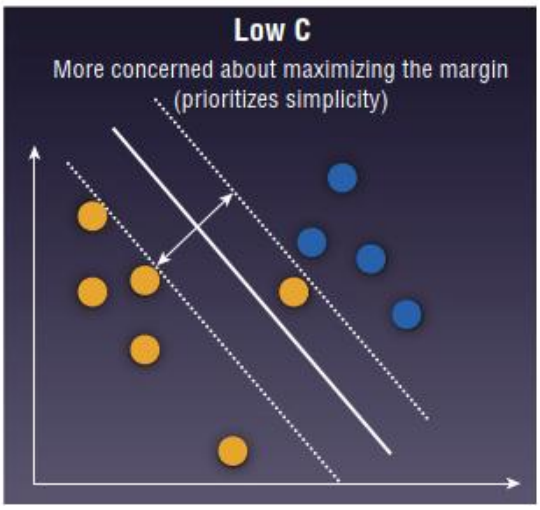


Nota. Adaptado de “Python ® Machine Learning» [Fig.8.20] por Lee (2019).

Por el contrario, como se indica en la figura 29, un valor bajo de C genera márgenes amplios, aceptando posibles errores de clasificación.

Figura 29

Representación del parámetro C con un valor bajo

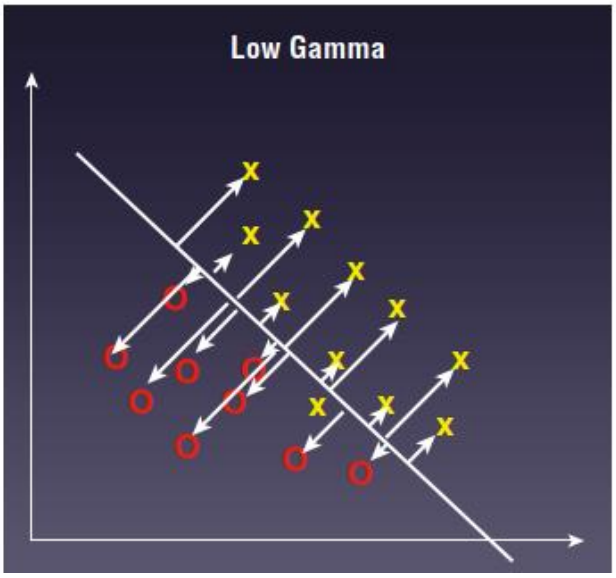


Nota. Adaptado de “Python ® Machine Learning» [Fig.8.21] por Lee (2019).

Otro hiperparámetro importante es Gamma que define el alcance de la influencia de cada ejemplo de entrenamiento. Un Gamma bajo, como se muestra en la Figura 30, implica un mayor alcance; mientras que un Gamma alto, como se muestra en la Figura 31, limita la influencia a los puntos cercanos, lo cual puede conducir a sobreajuste.

Figura 30

Representación del parámetro Gamma con un valor bajo

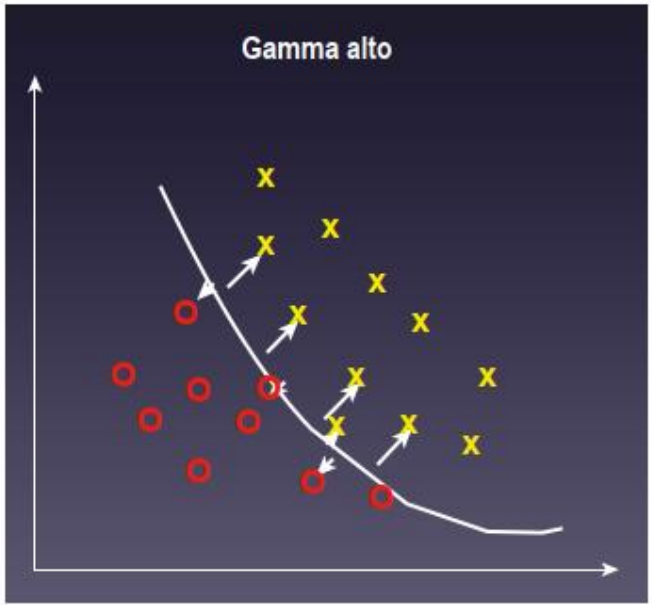


Nota. Adaptado de “Python ® Machine Learning» [Fig.8.25] por Lee (2019).

Por el contrario, un Gamma alto restringe la influencia a zonas cercanas al punto, como se aprecia en la Figura 31, lo que puede llevar a un sobreajuste si no se controla adecuadamente.

Figura 31

Representación del parámetro Gamma con un valor alto



Nota. Adaptado de "Python ® Machine Learning» [Fig.8.26] por Lee (2019).

- **Random Forest (RF)**

Es un método de Aprendizaje Automático basado en el aprendizaje supervisado, empleado en tareas de clasificación y regresión. Su principio fundamental se basa en el aprendizaje por conjuntos (ensemble learning), una técnica que combina múltiples clasificadores – en este caso, árboles de decisión – para resolver problemas complejos y mejorar la precisión del modelo (Hiran et al., 2021).

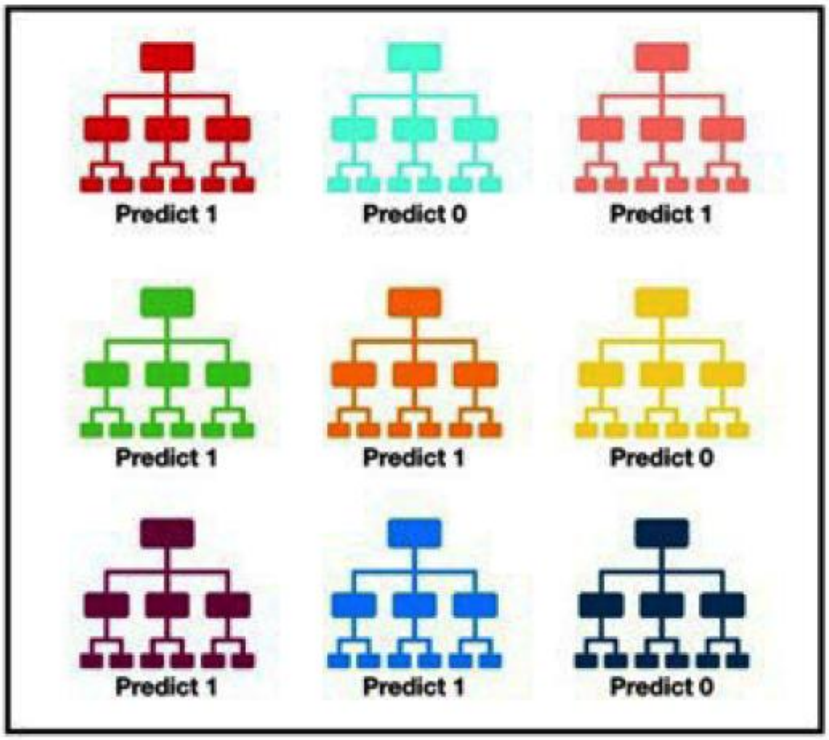
A diferencia de un árbol de decisión individual, Random Forest genera múltiples árboles a partir de diferentes subconjuntos de un conjunto de datos y combina sus resultados. En lugar de depender de un único árbol, el algoritmo toma en cuenta las predicciones de todos ellos y determina el resultado final mediante el voto mayoritario en problemas de clasificación o el promedio en regresión. Este enfoque contribuye a

aumentar la exactitud del modelo y a minimizar la posibilidad de sobreajuste (overfitting) (Hiran et al., 2021).

El riesgo de sobreajuste disminuye y la precisión del modelo mejora a medida que se incrementa la cantidad de árboles en el bosque. Cada árbol individual genera una predicción de clase, y la opción con mayor número de votos se considera la salida final del modelo. Como se muestra en la Figura 20, la predicción definitiva se determina mediante el principio de mayoría de votos (Hiran et al., 2021).

Figura 32

Visualización de algoritmos de Random Forest



Nota: Adaptado de *Machine Learning: Master Supervised and Unsupervised Learning Algorithms with Real Examples* [Fig. 21] por Hiran et al. (2021).

Además, de su arquitectura basada en el principio del voto mayoritario, Random Forest presenta una fundamentación matemática que explica su eficacia y su resistencia

al sobreajuste. Para ello, Breiman (2001) introduce el concepto de función margen, la cual mide la diferencia entre la probabilidad de que un conjunto de árboles del bosque clasifique correctamente una instancia X en su clase real Y, y la probabilidad más alta de clasificarla erróneamente en cualquier otra clase j distinta de Y. Esta función se expresa como:

$$mg\ (X,Y) = P_{\Theta}(h(X,\Theta) = Y) - \max_{j \neq Y} P_{\Theta}(h(X,\Theta) = j)$$

En esta expresión $h(X,\Theta)$ representa la salida de un clasificador individual (un árbol) entrenado con un vector aleatorio Θ, el cual define las decisiones aleatorias involucradas en la construcción del árbol, como la selección de subconjuntos de datos o de variables. Cada valor de Θ genera un árbol diferente, y el conjunto total de estos árboles constituye el Random Forest. La notación $P_{\Theta}(.)$ indica que la probabilidad se calcula sobre la distribución de todos los árboles construidos a partir de diferentes realizaciones del vector aleatorio Θ.

La comparación entre la clase verdadera Y y cualquier otra clase j ≠ Y (es decir, todas las clases distintas de la correcta) se justifica porque, en el contexto de clasificación multiclase, es necesario evaluar no solo si el modelo acierta, sino también cuán cercano está a equivocarse, identificando la clase alternativa más votada. Por ello, se toma el valor máximo de todas la probabilidades de clasificación incorrecta para contrastarlo con la probabilidad de clasificación correcta.

Un valor de margen positivo indica que la mayoría de los árboles del bosque predicen correctamente la case de una instancia, mientras que un valor negativo significa que otra clase distinta recibió más votos. A partir de esta función, se define el error de generalización del modelo como la probabilidad de que el margen sea menor que cero, es decir, que el modelo se equivoque:

$$PE^* = P_{X,Y}\,(mg(X,Y) < 0)$$

Un aspecto fundamental señalado por Breiman (2001)es que, a medida que se incrementa el número de árboles en el bosque, el valor de PE* tiende a estabilizarse, convergiendo casi seguramente a un límite, gracias a la Ley Fuerte de los Grandes Números. Esto significa que, contrariamente a lo que ocurre con otros modelos más complejos, Random Forest no sobreajusta se añadan muchos árboles, siempre que se mantenga una adecuada aleatorización en su construcción.

Adicionalmente, se identifica que el rendimiento del bosque está condicionado por dos factores clave: la fuerza de los clasificadores individuales y la correlación entre ellos. La fuerza se refiere a la capacidad promedio de los árboles para clasificar correctamente una instancia, mientras que la correlación mide qué tanto coinciden los errores entre árboles distintos. Breiman (2001) demuestra que el error de generalización está acotado por la siguiente relación:

$$PE^* \leq \frac{\bar{\rho}(1-s^2)}{s^2}$$

Donde s representa la fuerza promedio del conjunto de clasificadores y $\bar{\rho}$ es la correlación media entre ellos. Esta relación pone en evidencia que la eficacia de Random Forest depende de lograr una alta fuerza individual y una baja correlación entre árboles, lo cual se consigue mediante el muestreo aleatorio de datos (bagging) y la selección aleatoria de características en cada nodo del árbol.

En problemas de clasificación binaria, la función margen se simplifica a una expresión más directa:

$$mr\,(X,Y) = 2P_{\Theta}(h(X,\Theta) = Y) - 1$$

Esta formulación permite evaluar si el clasificador se comporta de manera significativamente mejor que al azar. Una fuerza promedio mayor a cero implica que el modelo supera el rendimiento aleatorio, condición mínima para que el conjunto de clasificadores sea considerado válido.

- **K-Nearest Neightbors (kNN)**

El algoritmo k-Nearest Neighbors (kNN) es una técnica de clasificación que se fundamenta en la cercanía entre los datos. Su funcionamiento se basa en identificar los k registros más similares dentro de un conjunto de entrenamiento y asignar la categoría mayoritaria a una nueva instancia sin etiquetar. Para ello, se utiliza un valor de k previamente definido, que determina cuántos vecinos cercanos serán considerados en la clasificación (Lantz, 2013).

En el proceso de clasificación, la proximidad entre las instancias se calcula utilizando una métrica de distancia, que permite cuantificar el grado de similitud entre cada instancia. Una de las métricas más utilizadas es la distancia Euclidiana, que se define como:

$$dist(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \cdots + (p_n - q_n)^2}$$

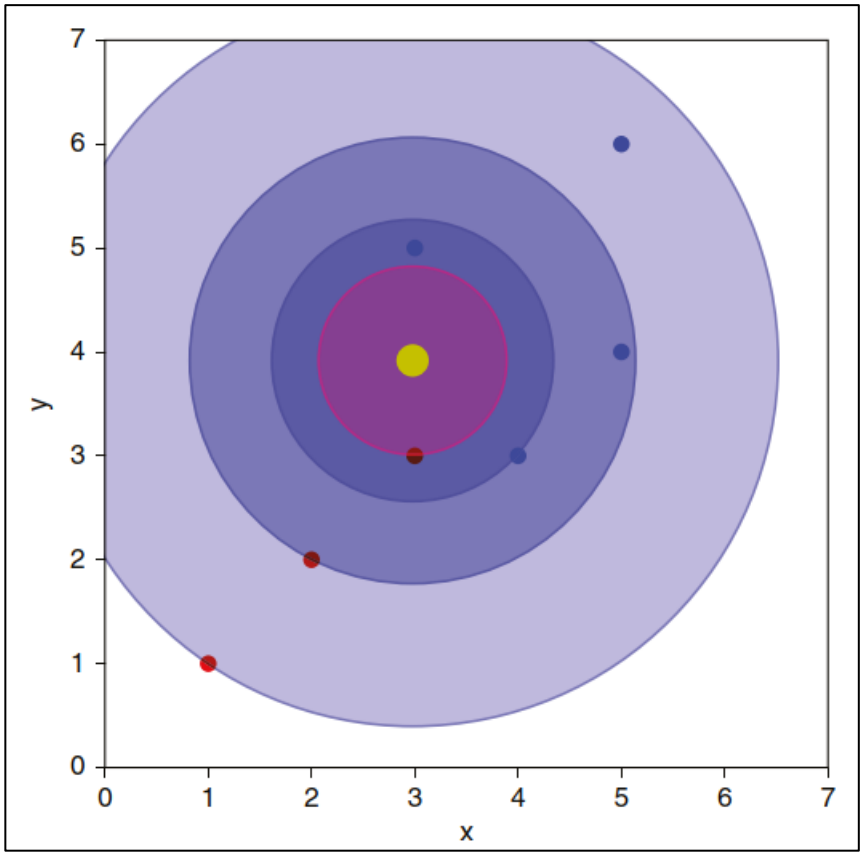
donde p y q representan dos instancias a comparar, cada una descrita mediante n características. Esta fórmula permite ubicar cada instancia como un punto en un espacio multidimensional y calcular la distancia "en línea recta" entre ellos.

Una vez obtenidas estas distancias, el algoritmo selecciona las **k instancias más cercanas** a la nueva instancia y asigna la clase que predomina entre esos vecinos. Por ejemplo, si se usa un **k=3**, se realiza una votación entre las tres instancias más cercanas para determinar la clase final (Lantz, 2013).

Este proceso de decisión puede visualizarse en la Figura 33, donde se representa gráficamente cómo la clasificación de una instancia depende del valor de k. A través de círculos concéntricos, se muestra cómo cambia la clase asignada al punto central conforme se consideran diferentes cantidades de vecinos más cercanos. Esta figura permite entender con claridad el principio de votación mayoritaria sobre el cual se fundamenta el algoritmo (Lee, 2019).

Figura 33

Clasificación de una instancia en función de los vecinos más cercanos



Nota. Adaptado de “Python ® Machine Learning» [Fig.9.3] por Lee (2019).

Entre sus ventajas, kNN destaca por su simplicidad y efectividad, además de no requerir suposiciones sobre la distribución de los datos. También presenta una fase de

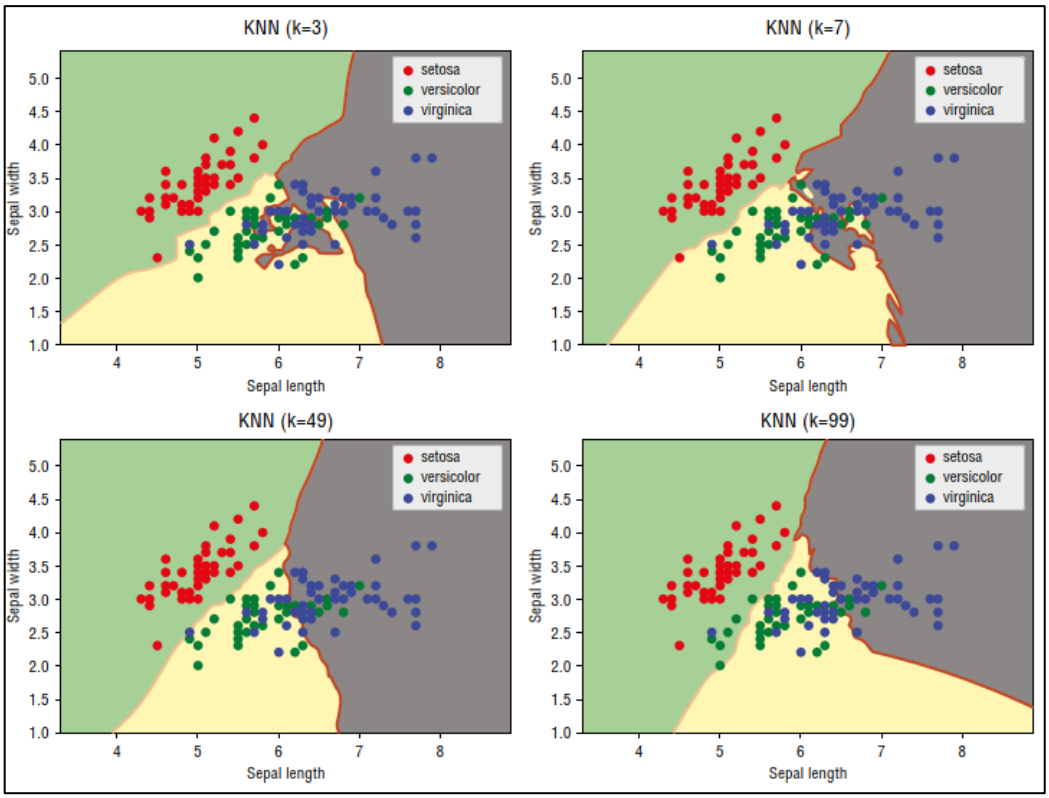
entrenamiento rápida, ya que el modelo solo almacena los datos sin realizar cálculos previos complejos. Sin embargo, tiene algunas limitaciones, como la falta de un modelo explícito, lo que impide obtener información sobre las relaciones entre las características. Asimismo, el proceso de clasificación puede volverse lento cuando se trabaja con grandes volúmenes de datos, debido a la necesidad de calcular la distancia entre cada nueva instancia y todos los ejemplos del conjunto de entrenamiento. También demanda un alto consumo de memoria y requiere procesamiento adicional en caso de datos nominales o valores faltantes (Lantz, 2013).

Una vez determinado el conjunto de vecinos más próximos, el valor de k se convierte en un factor crucial para la efectividad del algoritmo. Tal como señala Lee (2019), si se elige un valor demasiado pequeño, como k = 1, el modelo puede volverse excesivamente sensible a valores atípicos, lo que incrementa el riesgo de errores de sobreajuste. En contraste, un valor de k muy elevado tiende a suavizar la frontera de decisión, ignorando particularidades importantes del entorno inmediato de los datosy provocando subajuste. Por ello, la elección adecuada de k debe basarse en un equilibrio entre posición local y generalización global.

Este efecto puede visualizar en la Figura 34 donde muestran los resultados de aplicar distintos valores de k sobre el conjunto de datos públicos Iris. Cuando k =1, las regiones de decisión son altamente detalladas y específicas; sin embargo, al aumentar k a 49 a 99, dichas regiones se tornan más suaves y menos definidas, lo que refleja una pérdida de capacidad discriminativa del modelo.

Figura 34

Clasificación del conjunto de datos pública Iris para diferentes valores de K

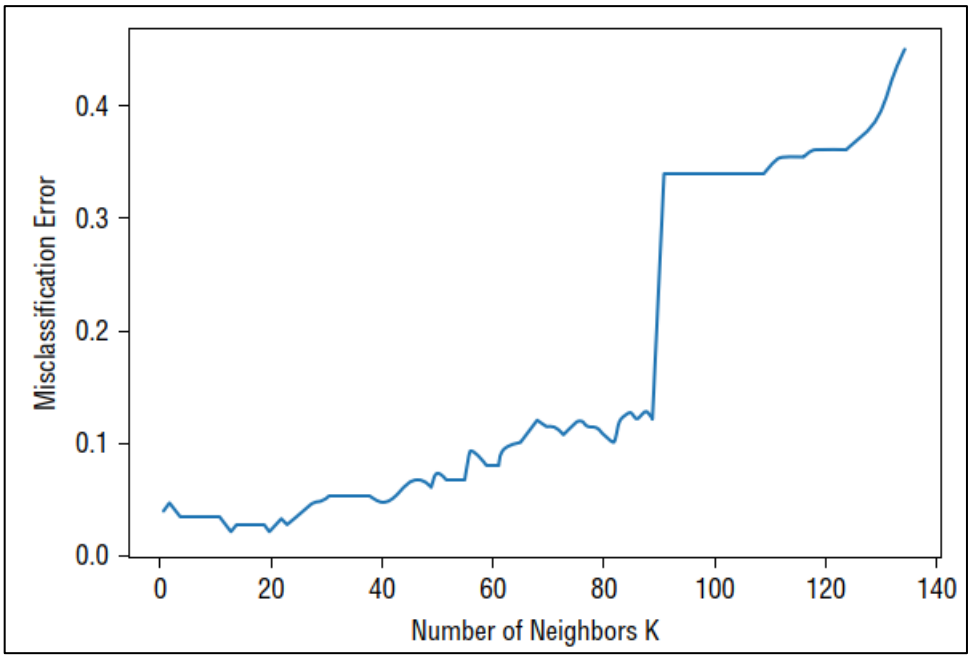


Nota. Adaptado de “Python ® Machine Learning» [Fig.9.7] por Lee (2019).

Asimismo, Lee (2019) presenta una representación gráfica del impacto que tiene el parámetro k sobre la forma de las regiones de decisión. A medida que aumenta este valor, se observa un cambio gradual en la configuración de las fronteras, que se hacen más amplias y generalizadas. La Figura 35 permite visualizar cómo el modelo responde a diferentes configuraciones del parámetro, lo cual resulta esencial para comprender su comportamiento en función del entorno de datos.

Figura 35

Modificación de la frontera de decisión al variar el número de vecinos k



Nota. Adaptado de “Python ® Machine Learning» [Fig.9.9] por Lee (2019).

En síntesis, aunque el algoritmo kNN destaca por su sencillez conceptual y facilidad de implementación, su rendimiento final depende en gran medida de la elección adecuada del parámetro k. Tal como lo demuestra Lee (2019), el análisis gráfico de este valor constituye una herramienta fundamental para orientar su configuración en función de la naturaleza del problema.

- **Naïve Bayes (NB)**

El algoritmo Naïve Bayes pertenece a un grupo de métodos de clasificación que se basan en la suposición de independencia estadística entre las características utilizadas para predecir una variable objetivo. Su fundamento teórico se encuentra en el Teorema de Bayes, el cual establece que cada característica contribuye a la predicción de manera independiente, sin influencias mutuas. Debido a esta suposición simplificadora, el modelo recibe el nombre de Naïve Bayes (Dulhare et al., 2020).

Aunque este algoritmo asume independencia entre las variables, es altamente eficiente en el manejo de grandes volúmenes de datos, ya que facilita un entrenamiento

y una predicción rápidos. Asimismo, muestra un desempeño destacado en el procesamiento de variables categóricas, lo que lo hace una alternativa adecuada para diversas tareas de clasificación (Dulhare et al., 2020).

A nivel matemático, el modelo se fundamenta en el Teorema de Bayes, el cual permite estimar la probabilidad posterior de una hipótesis h dada una observación x, utilizando la información previa y la verosimilitud de los datos. Mitchell (1997) señala que esta relación se expresa de la siguiente manera:

$$P(h|x) = \frac{P(x|h).P(h)}{P(x)}$$

Donde:

- $P(h|x)$ representa la probabilidad posterior de la hipótesis,
- $P(h)$ es la probabilidad previa de la hipótesis,
- $P(x|h)$ es la verosimilitud de los datos dados h,
- $P(x)$ es la probabilidad marginal de los datos.

Además, para Mitchell (1997), en el contexto de clasificación, como lo indica se desea predecir el valor más probable de una variable de clase $v_j \in V$ para una nueva instancia descrita por un conjunto de atributos $(a_1, a_2, \dots, a_n)$. La aproximación bayesiana sugiere seleccionar el valor v_j que maximice la probabilidad posterior $P(v_j|(a_1, a_2, \dots, a_n)$, lo cual se expresa mediante el criterio MAP (Maximum A Posteriori):

$$v_{MAP} = \arg\max_{v_j \in V} \frac{P(a_1, a_2, .., a_n|v_j).P(v_j)}{P(a_1, a_2, .., a_n)}$$

Dado que el denominador $P(a_1, a_2, .., a_n)$ es constante para todas las clases durante la comparación, puede omitirse en la maximización, lo que conduce a:

$$v_{MAP} = \arg\max_{v_j \in V} P\left(a_1, a_2, .., a_n \middle| v_j\right). P(v_j)$$

No obstante, estimar directamente $P\left(a_1, a_2, .., a_n \middle| v_j\right). P(v_j)$ puede resultar inviable si el número de combinaciones posibles de atributos es elevado. Para superar esta dificultad, el clasificador Naïve Bayes introduce la suposición de independencia condicional, según la cual los atributos son estadísticamente independientes entre sí dado el valor de la clase. Esta hipótesis permite descomponer la probabilidad conjunta como el producto de las probabilidades individuales:

$$P\left(a_1, a_2, .., a_n \middle| v_j\right) = \prod_i P(a_i | v_j)$$

Al sustituir esta expresión en la ecuación de v_{MAP}, se obtiene la fórmula del clasificador Naïve Bayes, que estima la clase más probable de la nueva instancia como:

$$v_{NB} = \arg\max_{v_j \in V} P\left(v_j\right) \prod_i P(a_i | v_j)$$

No obstante, una dificultad frecuente en la implementación práctica del clasificador Naïve Bayes surge cuando un valor específico de un atributo no ha sido observado en el conjunto de entrenamiento para una clase determinada. En estos casos, la probabilidad condicional $P(a_i | v_j)$ se evalúa como cero, lo cual provoca que todo el producto de probabilidades en la expresión v_{NB} también sea igual a cero, anulando el efecto de las demás evidencias incluso si son altamente indicativas. Para mitir este problema se recurre a una técnica conocida como suavizado de Laplace (Laplace smoothing), que consiste en ajustar las frecuencias observadas para evitar probabilidades nulas.

Como explica Lantz (2013), el suavizado de Laplace agrega una unidad a cada combinación de atributo y clase, de modo que incluso aquellas combinaciones no observadas reciban una probabilidad mínima diferente de cero. Matemáticamente, esta corrección se expresa como:

$$P(a_i|v_j) = \frac{N_{i,j} + 1}{N_j + k}$$

Donde:

- $N_{i,j}$ representa el número de veces que el atributo a_i ocurre en la clase v_j,
- N_j es el total de instancias en las que aparece la clase v_j,
- k es el número total de valores posibles del atributo a_i.

Este ajuste, aunque sencillo, contribuye significativamente a la estabilidad de modelo, especialmente cuando se trabaja con conjuntos de datos pequeños o con atributos de alta cardinalidad. Además, evita que una sola observación faltante invalide el resto de la evidencia, manteniendo así la robustez del clasificador. Por estas razones, el suavizado de Laplace se considera una práctica estándar al aplicar Naïve Bayes en contextos reales de clasificación supervisada (Lantz, 2013).

Finalmente según Hiran et al. (2021), existen tres variantes principales de Naïve Bayes, adaptada a distintos tipos de datos:

- Gaussian Naïve Bayes, para atributos continuos bajo distribución normal,
- Multinomial Naïve Bayes, para datos de conteo o frecuencia,
- Bernoulli Naïve Bayes, para variables binarias de presencia o ausencia.

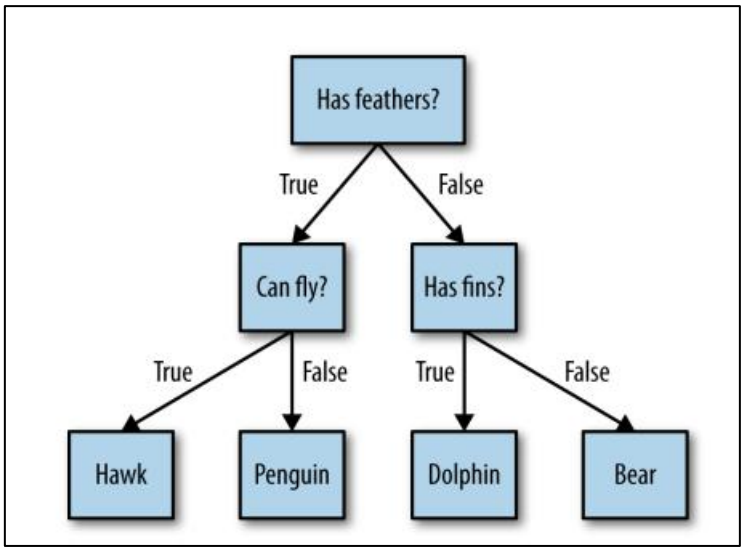
- **Decision Tree (DT)**

Es un modelo utilizado en tareas de clasificación y regresión que estructura una jerarquía de preguntas condicionales (if/else) para llegar a una conclusión. Este enfoque es similar a la estrategia empleada en juegos de preguntas y respuestas, donde el objetivo es identificar un elemento con la menor cantidad de preguntas posibles (Müller & Guido, 2017).

Por ejemplo, para diferenciar entre cuatro animales—oso, halcón, pingüino y delfín—se pueden formular preguntas estratégicas. Una posible primera pregunta sería si el animal tiene plumas, lo que reduciría las opciones a dos. En caso afirmativo, se podría preguntar si el animal puede volar para distinguir entre un halcón y un pingüino. Si la respuesta es negativa, las opciones se reducirían a un oso y un delfín, y una pregunta como "¿tiene aletas?" permitiría diferenciarlos. Este proceso se representa en la Figura 36**¡Error! No se encuentra el origen de la referencia.**, donde cada nodo corresponde a una pregunta y cada hoja representa una categoría final (Müller & Guido, 2017).

Figura 36

Un árbol de decisiones para distinguir entre varios animales



Nota: Adaptado de *Introduction to Machine Learning with Pyhton: a guide for data scientist* [Fig. 2-22] por Müller & Guido (2017).

En el contexto del Aprendizaje Automático, este método permite clasificar datos en función de sus características. En el ejemplo anterior, las variables utilizadas para la clasificación fueron "tiene plumas", "puede volar" y "tiene aletas". En lugar de construir manualmente estos modelos, un árbol de decisión puede construirse automáticamente a partir de los datos utilizando aprendizaje supervisado, optimizando la clasificación y permitiendo identificar patrones de manera automatizada (Müller & Guido, 2017).

Para construir automáticamente un árbol a partir de datos etiquetados, se emplean criterios matemáticos que permiten seleccionar, en cada nodo, el atributo más informativo para realizar la partición. Uno de los enfoques más comunes es el algoritmo ID3, basado en la teoría de la información. Este procedimiento utiliza la entropía como medida de impureza o incertidumbre de un conjunto de datos, la cual se calcula mediante la siguiente expresión (Lantz, 2013):

$$Entropy\ (S) = -\sum_{i=1}^{C} p_i \log_2(p_i)$$

donde:

- S representa el conjunto de entrenamiento,
- C es el número de clases
- p_i es la proporción de instancias pertenecientes a la clase i.

Una vez calculada la entropía del conjunto de entrenamiento, el siguiente paso es seleccionar el atributo que permita dividir los datos de manera que la impureza total se reduzca. Este proceso, fundamental para la construcción de árboles, tiene como objetivo agrupar los datos de forma que las clases queden lo más puras posible en cada subconjunto resultante. Según Lantz (2013) , este criterio permite estructurar divisiones que maximizan la homogeneidad dentro de los nodos hijos, optimizando así la clasificación.

Esta reducción de la incertidumbre se cuantifica a través de un valor denominado ganancia de información (Information Gain), que representa la diferencia entre la entropía del conjunto original y la entropía resultante tras la partición. En términos simples, cuanto mayor sea la ganancia de información, más útil será el atributo para segmentar correctamente las clases. Lantz (2013) presenta esta relación con la siguiente fórmula:

$$InfoGain\ (F) = Entropy(S_1) - Entropy(S_2)$$

Donde:

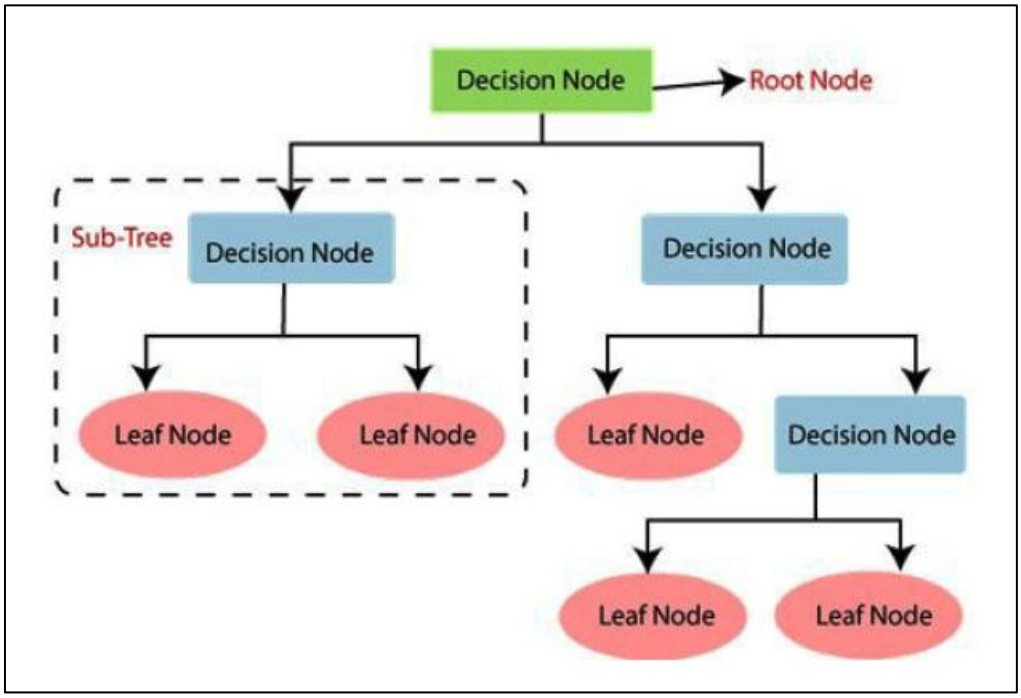
- F representa el atributo considerado para dividir,
- S_1 es el conjunto antes de la división,
- S_2 es el conjunto luego de aplicar la división según F.

Este procedimiento de selección del mejor atributo se repite de forma recursiva: una vez escogido el atributo más informativo, el conjunto de datos se divide según los valores de dicho atributo, y el mismo mecanismo se aplica sucesivamente a cada subconjunto generado. El proceso de construcción del árbol continúa hasta que se cumple algún criterio de detención, como alcanzar nodos completamente puros o con un número mínimo de instancias.

Este comportamiento puede observarse de manera gráfica en la figura 37, donde se ilustra cómo los datos son particionados de forma jerárquica para construir un árbol de decisión.

Figura 37

Visualización de proceso jerárquico de división de datos en un árbol de decisión



Nota. Adaptado de *Machine Learning: Master Supervised and Unsupervised Learning Algorithms with Real Examples* [Fig. 2.10], por Hiran et al. (2021).

En entornos reales, la construcción de árboles excesivamente profundos puede ocasionar sobreajuste, lo cual significa que el modelo se ajusta demasiado a los datos de entrenamiento y pierde capacidad para generalizar. Según Lantz (2013), los árboles más profundos, al contar con mayor número de nodos, tienen una mayor propensión a memorizar detalles del conjunto de entrenamiento, por lo que conviene restringir su complejidad. En este sentido, es posible controlar el crecimiento del árbol como una estrategia para mejorar su rendimiento predictivo.

2.2.3 Vision Transformer (Transformadores de visión)

a) Definición

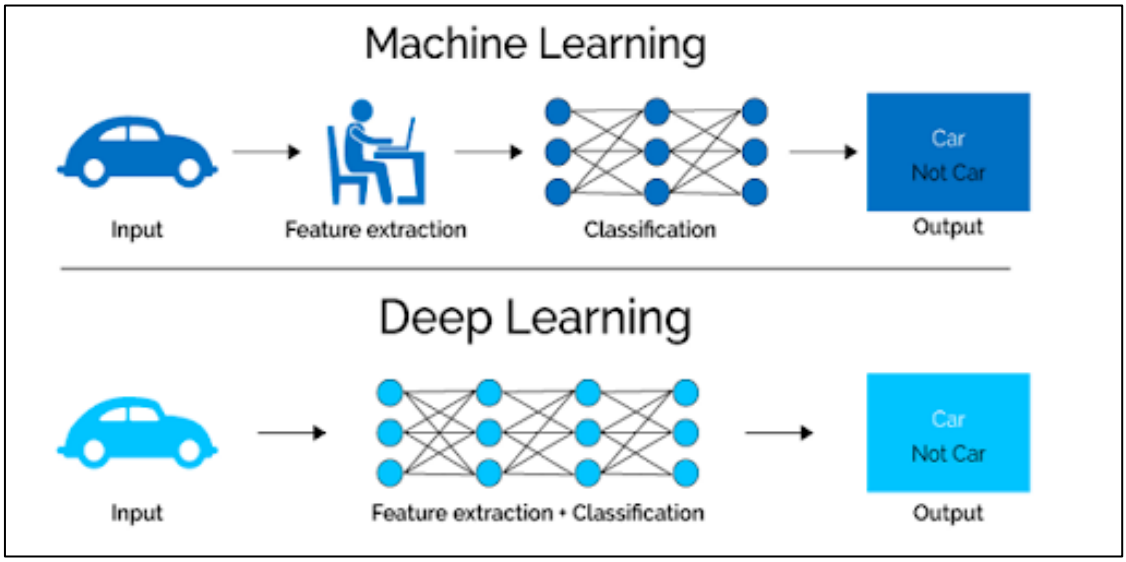
El Vision Transformer (ViT) forma parte de las arquitecturas más avanzadas del Deep Learning (Aprendizaje Profundo), una subárea del Machine Learning que permite construir representaciones jerárquicas de los datos mediante redes neuronales profundas. A diferencia de los métodos tradicionales que requieren una etapa previa de extracción manual de características, el Deep Learning automatiza este proceso, extrayendo directamente patrones significativos desde los datos crudos a través de múltiples capas de transformación (LeCun et al., 2015).

Según Oppermann (2023), en los algoritmos tradicionales de Machine Learning, como los árboles de decisión o las máquinas de soporte vectorial, la extracción de características es un proceso esencial que requiere ajustes manuales para mejorar el desempeño. En contraste, el aprendizaje profundo permite que las redes neuronales adquieran patrones directamente a partir de los datos, sin necesidad de intervención humana, lo que ha impulsado avances significativos en campos como el reconocimiento de imágenes, el procesamiento del lenguaje natural y la biomedicina. (LeCun et al., 2015).

Para visualizar esta diferencia fundamental entre el enfoque tradicional de Machine Learning y en enfoque automatizado de Deep Learning, se puede observar la comparación esquemática de la Figura 38, la cual ilustra cómo el Deep Learning integra la extracción de características dentro del propio modelo de clasificación.

Figura 38.

Comparación entre Aprendizaje Automático y Aprendizaje Profundo en la extracción de características y clasificación



Nota: Recuperado de Builtin en Oppermann (2023).

En el caso particular de Vision Transformer, este modelo adapta el mecanismo de autoatención, originalmente desarrollado para el procesamiento de lenguaje natural, al análisis de imágenes. Esto se logra dividiendo una imagen en pequeños parches que se tratan como secuencias de datos, permitiendo que el modelo capture relaciones espaciales a largo alcance. Esta capacidad ha marcado una evolución importante dentro del Deep Learning, al superar en algunos casos la precisión de las redes convolucionaels tradicionales en tareas de clasificación de imágenes (Dosovitskiy et al., 2020).

b) Historia

El origen de Deep Learning se remonta a la década de 1940 con el modelo teórico de neuronas artificiales propuesto por McCulloch & Pitts (1943). Posteriormente, Rosenblatt (1958) desarrolló el perceptrón, una red neuronal simple capaz de aprender patrones lineales mediante. No obstante, sus limitaciones teóricas fueron señaladas por Minsky & Papert (1969), quienes demostraron que el perceptrón no podía resolver problemas como el de la función XOR. Esta crítica plasma en su obra "Perceptrones" publicada originalmente en 1969 y reeditada posteriormente por MIT en el 2017, llevó a un descenso en el interés y financiamiento para la investigación en redes neuronales durante más de una década (Minsky & Papert, 2017)

El resurgimiento del Deep Learning ocurrió en los años 80 con la introducción del algoritmo de retropropagación del error (Rumelhart et al., 1986), que permitió entrenar eficientemente redes neuronales multicapa. A partir de entonces, el campo experimentó una evolución significativa, potenciada por el aumento de poder computacional, la disponibilidad de grandes conjuntos de datos y el desarrollo de nuevas arquitecturas como las redes convolucionales (CNN) y, más recientemente, los Transformers. En este último grupo destaca el Vision Trasnformer, que ha amplicado el alcance del Deep Learning hacia una representación más global y contextual de las imágenes (LeCun et al., 2015; Dosovitskiy et al., 2020).

c) Vision Transformer en la clasificación de imágenes

Uno de los usos más relevantes del Vision Transformer es su aplicación en tareas de clasificación de imágenes, particularmente en el ámbito médico. A diferencia de los métodos tradicionales, que requieren definir manualmente las características visuales a evaluar, el ViT permite aprender de forma automática representaciones discriminativas a partir de imágenes divididas en parches. Esta estrategia favorece la detección de

patrones sutiles y la captura de relaciones espaciales a largo alcance, fundamentales en entornos diagnósticos donde las señales visuales pueden ser tenues o poco evidentes (Dosovitskiy et al., 2020; Khan et al., 2022).

Gracias a su diseño estructurado en bloques de atención, el ViT permite una representación global de la imagen desde las primeras capas del modelo, lo que facilita su aplicación en escenarios de clasificación, segmentación y detección. Por estas razones, las arquitecturas derivadas de Vision Transformer se han consolidado como herramientas eficaces para el procesamiento automatizado de imágenes biomédicas.

d) Arquitecturas basadas en Vision Transformer

A continuación, se presentan cinco modelos basados en arquitecturas de Vision Transformer que han sido empleados en tareas de clasificación de imágenes en el ámbito biomédico:

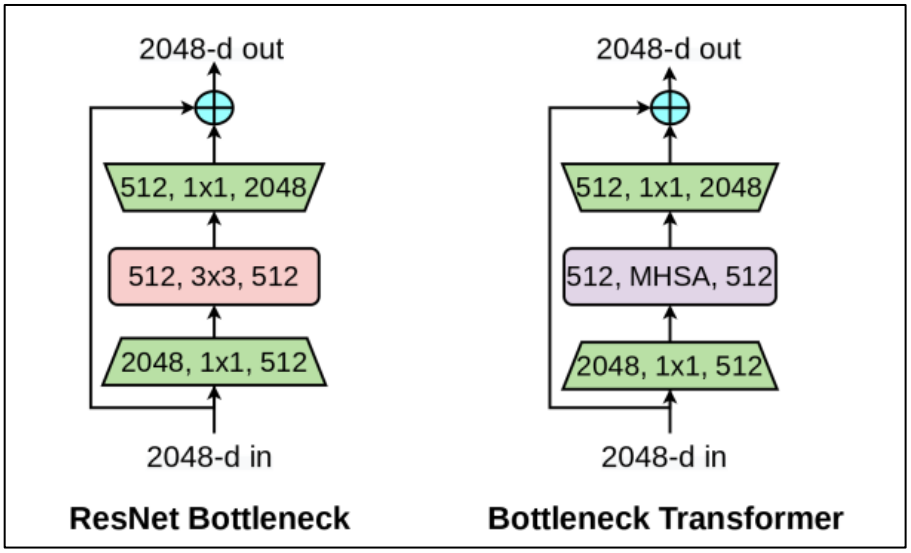
- **BOTNet**

BOTNet es un modelo de inteligencia artificial que combina dos enfoques para analizar imágenes. Estos son las redes neuronales convolucionales (CNNs) y los mecanismos de atención tipo Transformer. BOTNet toma como base las arquitectura de ResNet y sustituye las convoluciones 3x3 de los bloques finales por bloques de Multi-Head Self Attention (MSHA), incrementando así la capacidad de modelar relaciones espaciales de largo alcance (Srinivas et al., 2021).

Una diferencia clave entre ResNet y BOTNet se observa en la estructura de los bloques residuales. Mientras que ResNet emplea convolusiones locales para capturar información espcial, BOTNet reemplaza estas capas por mecanismos MHSA, como se ilustra en la figura 39.

Figura 39

Comparación estructural entre el bloque básico de ResNet y el Bottleneck Transfomer

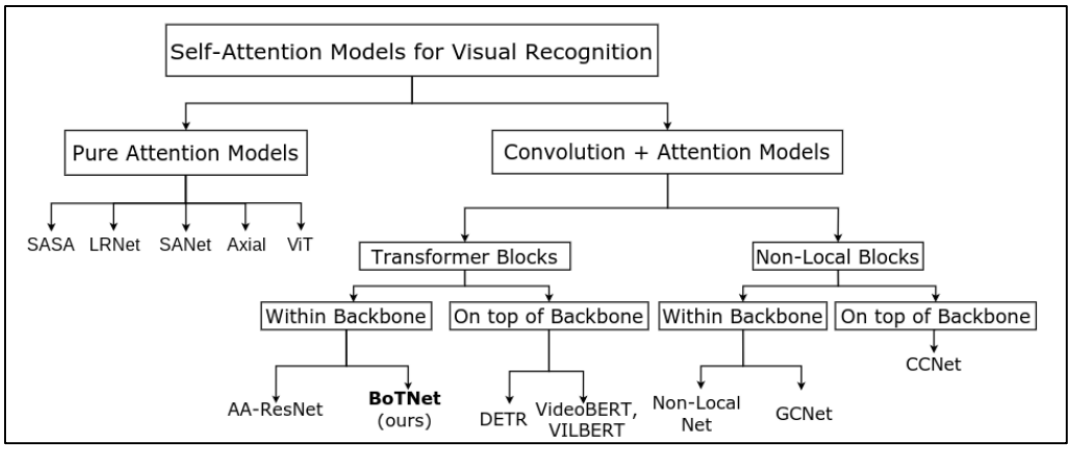


Nota. Adaptado de *Bottleneck Transformers for Visual Recognition* [Fig. 1], por (Srinivas et al., 2021).

A nivel conceptual, BOTNet se posiciona dentro de la categoría de modelos híbridos que integran atención y convolución. La figura 40 ubica a BOTNet dentro de un árbol jerárquico de arquitectura de autoatención aplicadas a visión por computadora.

Figura 40

Clasificación jerárquica de modelos de atención para reconocimiento visual

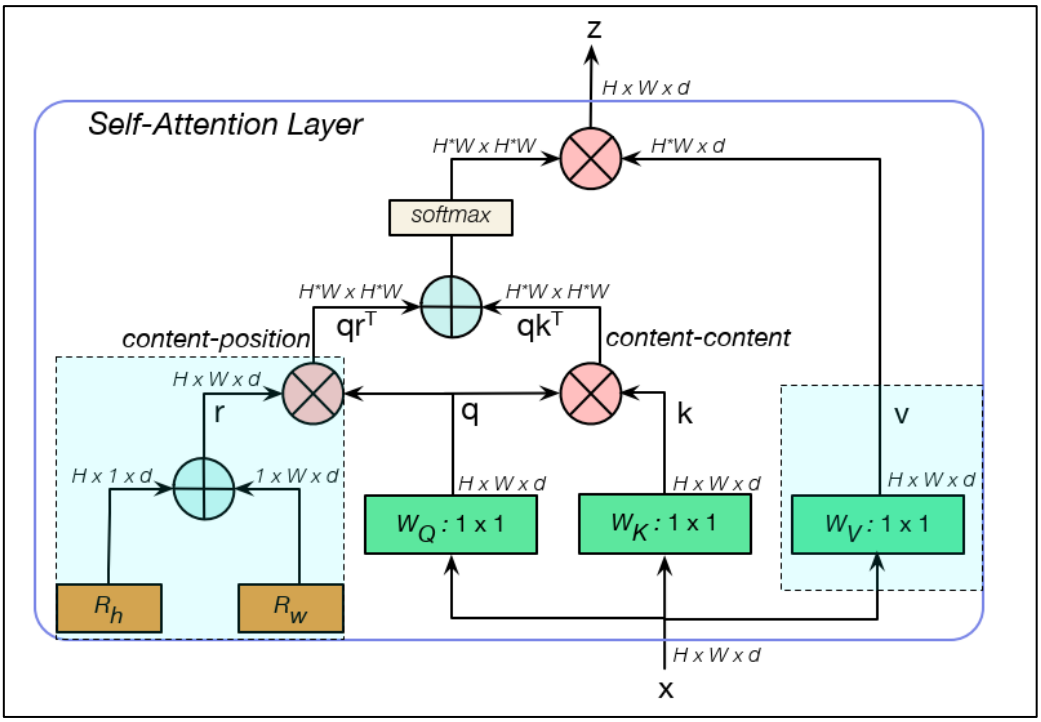


Nota. Adaptado de *Bottleneck Transformers for Visual Recognition* [Fig. 2], por (Srinivas et al., 2021).

Para comprender el mecanismo interno de BOTNet, se debe analizar su estructura de atención. Esta se basa en la descomposición del mecanismo MSHA en dos componentes: atención contenido-contenido y atención contenido-posición. Esta configuración permite capturar no solo relaciones semánticas entre regiones de la imagen (contenido-contenido) sino también la información sobre su disposición espacial relativa (contenido-posición), como se muestra en la Figura 41.

Figura 41

Diagrama de la capa de autoatención utilizada en BOTNet



Nota. Adaptado de *Bottleneck Transformers for Visual Recognition* [Fig. 4], por (Srinivas et al., 2021).

En este diseño, se realiza atención “all-to-all” sobre un mapa de características bidimensional, donde se emplean codificaciones posicionales relativas divididas en: R_h para la altura y R_w para el ancho. Las puntuaciones de atención (logits) se calculan mediante la suma $qk^T + qr^T$, en la que q, k y r representan, respectivamente, los vectores de consulta (query), clave(key) y codificación posicional. Las operaciones que intervienen incluyen suma elemento a elemento y multiplicación matricial, mientras que los bloques etiquetados como 1x1 indican convoluciones puntuales. Aunque en la figura no se muestran, el modelo implementa múltiples cabezas de atención en paralelo, lo que mejora su capacidad de captura de patrones espaciales complejos. Además, elementos clave como codificaciones posicionales y la proyección del valor, resaltados en azul,

contribuyen diferencias clave frente a las capas No-Local tradicionales (Srinivas et al., 2021).

Este rediseño, al sustituir el núcleo convolucional de los bloques básicos por mecanismos de atención, expande el campo perceptivo sin sacrificar la eficiencia computacional. Esta mejora se traduce en un rendimiento superior en tareas de clasificación y segmentación, con especial impacto en aplicaciones como la visión médica (Srinivas et al., 2021).

- **ViT**

El Vision Transformer (ViT) es una arquitectura que traslada el mecanismo de autoatención, propio de los Transformers en procesamiento de lenguaje natural, al dominio de imágenes. Su innovación radica en representar una imagen como una secuencia de parches planos que luego se tratan como tokens de entrada para el modelo Transformer. Esta conversión permite que la arquitectura capture relaciones espaciales a largo alance sin reducir a convoluciones (Dosovitskiy et al., 2020).

Desde el punto de vista matemático, Vision Transformer (ViT) transforma una iamgen $x \in \mathbb{R}^{HxWxC}$ en una secuencia de $N = \frac{HW}{P^2}$ parches de tamaño fijo PxP. Cada parche se aplana y se proyecta linealmente a una dimensión D mediante una matriz $E \in \mathbb{R}^{(P^2.C)\times D}$. Luego se añade un token de clasificación x_{class} al inicio de la secuencia, y se suma una codificación posicional $E_{POS} \in \mathbb{R}^{(N+1)\times D}$ que permite al modelo conservar la información de orden espacial. Esto da lugar a la siguiente secuencia de vectores de características de entrada:

$$z_0 = \left[x_{class}; x_p^1 E; x_p^2 E; \ldots; x_p^N E\right] + E_{pos}$$

Esta secuencia pasa por una pila de bloques Transformer estándar, cada uno compuesto por atención multi-cabeza (MSA) y una red neuronal de tipo feedforward (MLP). En cada capa $l = 1, \dots, L$, la actualización es:

$$z_l' = MSA\left(LN(z_{l-1})\right) + z_{l-1}$$

$$z_l = MLP\left(LN(z_l')\right) + z_l'$$

Finalmente, la salida del modelo se obtiene a partir del token de clasificación procesado:

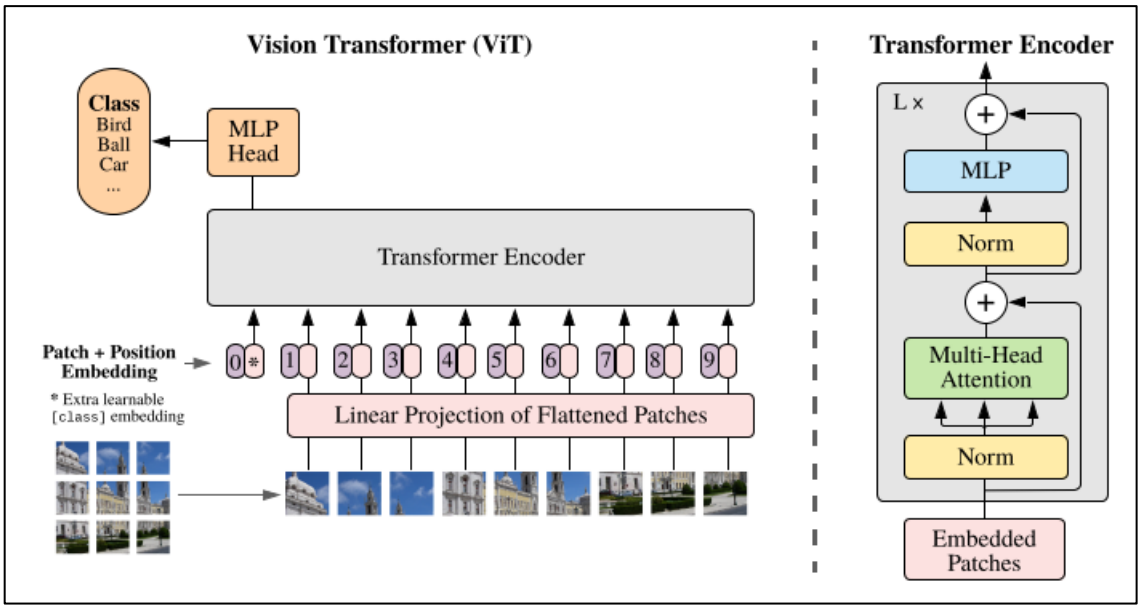
$$y = LN(Z_L^O)$$

La red MLP contiene dos capas y utiliza la función no lineal GELU como activación. Además, todas las capas utilizan normalización por capas (Layer Normalization) y los mecanismos de atención implementan múltiples cabezas en paralelo para enriquecer la capacidad de representación del modelo. ViT no emplea convoluciones ni mecanismos recurrentes, lo que simplifica su arquitectura en comparación con otros modelos de visión por computadora (Dosovitskiy et al., 2020).

Este proceso completo se resume en la Figura 42, que ilustra el flujo de datos desde la imagen de entrada hasta la predicción final del modelo.

Figura 42

Arquitectura general del Vision Transformer (ViT)



Nota. Adaptado de *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale* [Fig. 1], por (Dosovitskiy et al., 2020)

En su implementación original, ViT se preentrena con conjuntos de datos de gran escala como ImageNet-21k y JFT-300M, lo que permite un rendimiento competitivo incluso con una arquitectura puramente basada en Transformers (Dosovitskiy et al., 2020).

- **SwinTransformer**

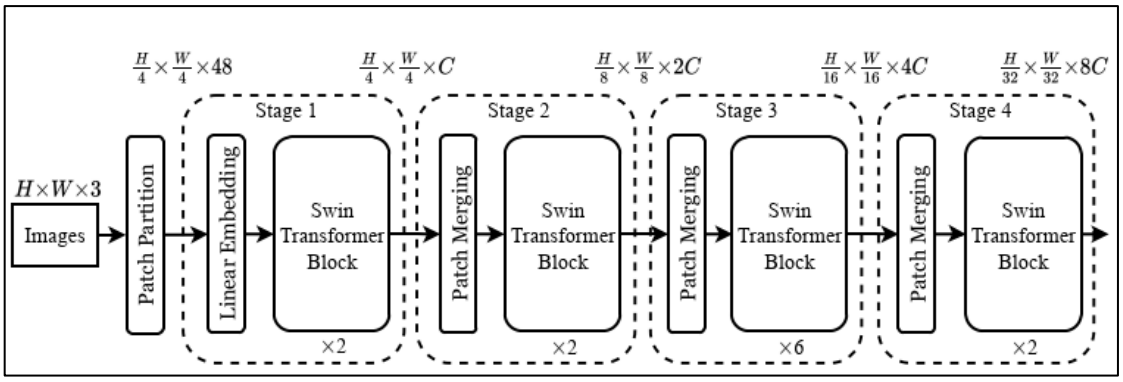
Swin Transformer es una arquitectura optimizada para tareas de visión por computadora, diseñada para superar las limitaciones computacionales de los Transformers estándar cuando se aplican a imágenes de alta resolución. A diferencia de la atención global títpica, que tiene una complejidad cuadrática con respecto al tamaño de la imagen, Swin Transformer reduce la complejidad al restringir el cálculo de atención a ventanas locales de tamaño fijo, manteniendo al mismo tiempo la capacidad

de modelar relaciones globales gracias al uso de ventanas desplazadas (*shifted Windows*) (Liu et al., 2021).

El modelo organiza su estructura en cuatro etapas jerárquicas, donde la resolución espacial disminuye progresivamente mediante una operación denominada Patch Merging. Esta operación agrupa regiones 2x2 de la imagen, concatena sus canales y reduce la resolución, permitiendo construir una representación análoga a las redes convolucionales profundas. Esta arquitectura jerárquica se muestra en la Figura 43.

Figura 43

Arquitectura general jerárquica de Swin Transformer (Swin-T)



Nota. Adaptado de *Swin Transformer: Hierarchical Vision Transformer using Shifted Windows* [Fig. 3a], por Liu et al. (2021).

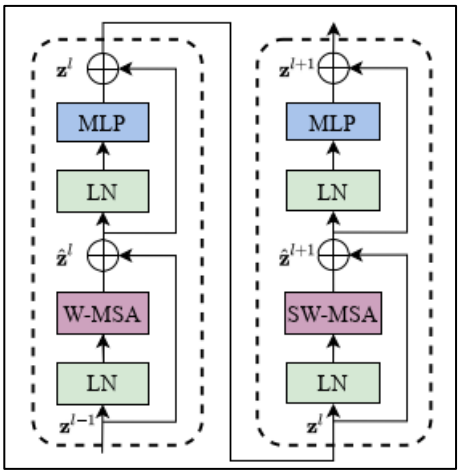
Cada etapa del modelo contiene bloques Swin Transfomer compuestos por dos tipos de mecanismos de atención:

- W-MSA (Window-based Multi-Head Self-Attention), donde se calcula atención dentro de ventanas locales sin superposición.
- SW-MSA (Shifted Window Multi-Head Self-Attention), donde las ventanas se desplazan entre capas para permitir el intercambio de información entre regiones adyacentes sin aplicar atención sobre toda la imagen.

Esta configuración permite capturar relaciones espaciales de corto y largo alcance de forma eficiente. La Figura 44 ilustra el funcionamiento de dos bloques Swin Transformer sucesivos, donde se alternan las operaciones W-MSA y SW-MSA.

Figura 44

Dos bloques Swin Transformer sucesivos con W-MSA y SW-MSA



Nota. Adaptado de *Swin Transformer: Hierarchical Vision Transformer using Shifted Windows* [Fig. 3a], por Liu et al. (2021).

Desde el punto de vista matemático, el mecanismo de atención dentro de cada ventana se expresa como:

$$Attention\ (Q,K,V) = Softmax\left(\frac{QK^T}{\sqrt{d}} + B\right)V$$

donde

- $Q,K,V \in \mathbb{R}^{M\times d}$ son las matrices de consulta (Query), clave(key) y value (value) por cada ventana de tamaño M,
- $d\ es\ la$ dimensión del espacio de característica d,

- B representa el sesgo de posición relativo (relative position bias), que es aprendido durante el entrenamiento.

Esta cálculo se realiza en paralelo sobre todas las ventanas, y al incorporar la variante SW-MSA, se permite la conexión entre regiones que, de otro modo, estarían aisladas.

Gracias a esta combinación de eficiencia y capacidad de modelado jerárquico, Swin Transformer ha demostrado un rendimiento competitivo en tareas como clasificación de imágenes, segmentación semántica y detección de objetos, superando a arquitecturas convolucionales tradicionales en varios benchmarks (Liu et al., 2021).

Por ejemplo, en una investigación centrada en la detección de nódulos tiroideos a partir de imágenes de ultrasonido, los investigadores emplearon Swin Transformer para llevar a cabo el análisis de dichas imágenes. Este modelo alcanzó una precisión promedio (mAP) del 44.8% y una sensibilidad del 90.5%, lo que indica que, aunque aún puede mejorar en exactitud, es altamente efectivo para detectar la mayoría de los nódulos sin dejarlos pasar por alto. (Tian et al., 2023).

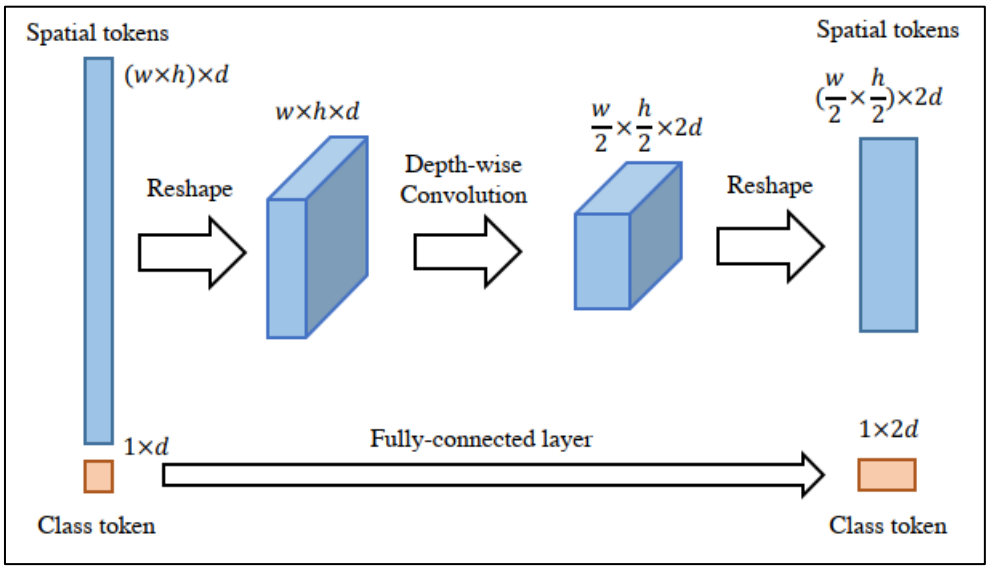
- **PiT**

PiT (Pooling-based Vision Transformer) es una arquitectura que combina la capacidad de representación de los Transformers con una técnica clásica de las redes convolucionales: el pooling. A diferencia de los Transformers tradicionales, que mantienen una resolución constante en todas sus capas, PiT introduce capas de reducción espacial mediante una operación de pooling especializada, lo que permite mejorar la eficiencia computacional sin sacrificar la capacidad de representación visual (Heo et al., 2021).

Desde el punto de vista estructural, PiT introduce una capa de pooling especializada para operar sobre secuencias de tokens generadas en ViT. Como se muestra en Figura 45, esta capa transforma la secuencia unidimensional de tokens espaciales $(w \times h) \times d$ en una representación tridimensional $w \times h \times d$, que refleja la estructural espacial original. Luego, se aplica una convolución Depth-wise, que reduce las dimensiones espaciales a la mitad mientras duplica el número de canales, resultando en una salida de tamaño $\frac{w}{2} \times \frac{h}{2} \times 2d$. Finalmente, esta representación es reformateada nuevamente como una secuencia de tokens, adaptada al procesamiento subsiguiente por los bloques Transformer. Además, el token de clase pasa por una capa totalmente conectada para ajustar su dimensión $1 \times 2d$, asegurando compatabilidad con la salida de la capa de pooling.

Figura 45

Estructura de la capa de pooling en la arquitectura PiT

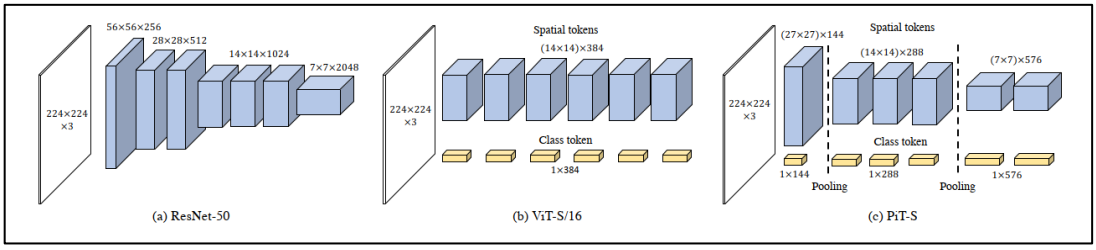


Nota. Adaptado de *Rethinking Spatial Dimensions of Vision Transformers* [Fig. 4], por Heo et al. (2021)

El efecto de esta estrategia en la arquitectura completa del modelo puede observase en la Figura 46, donde se comparan los esquemas de ResNet-50, ViT-S/16 y PiT-S. Mientras que ViT mantiene constantes las dimensiones espaciales de la imagen, PiT introduce una reducción progresiva mediante constantes las dimensiones espaciales de la imagen, PiT introduce una reducción progresiva mediante pooling, replicando la eficiencia estructural de ResNet. Esto permite disminuir el consumo de memoria y acelerar el entrenamiento sin comprometer el rendimiento del modelo (Heo et al., 2021).

Figura 46

Arquitectura general de PiT y comparación con ViT



Nota. Adaptado de *Rethinking Spatial Dimensions of Vision Transformers* [Fig. 1], por Heo et al. (2021)

Para caracterizar cuantitativamente el comportamiento del mecanismo de atención en PiT, Heo et al. (2021) emplearon dos métricas aplicadas sobre la matriz de atención $A \in \mathbb{R}^{M \times N}$: la entropía de atención y la distancia de atención. Estas métricas permiten evaluar tanto la distribución como la dispersión espacial de las relaciones atencionales entre tokens.

La primera métrica de atención se expresa mediante la siguiente fórmula:

$$Entropy = \frac{1}{N}\sum_{j=1}^{N}\sum_{i=1}^{M} \propto_{i,j} \log \propto_{i,j}$$

Esta expresión mide el grado de dispersión o concentración en la asignación de atención. Un valor bajo de entropía indica que el modelo focaliza su atención en unas pocas posiciones específicas (alta concentración), mientras que un valor elevado sugiere una atención más distribuida entre múltiples posiciones.

La segunda métrica, distancia de atención, se define como:

$$Entropy = \frac{1}{N}\sum_{j=1}^{N}\sum_{i=1}^{M} \propto_{i,j} ||p_i - p_j||_1$$

En esta fórmula, p_i y p_j representan las coordenadas espaciales relativas de los tokens. La distancia evalúa cuán separadas están las posiciones que reciben atención, en términos de su ubicación dentro del mapa de características, usando la norma L_1.

Ambas métricas se utilizaron para realizar un análisis comparativo entre ViT y PiT, revelando que este último tiende a mostrar una atención más dispersa en las primeras capas y más focalizada en las capas profundas. Esta evolución sugiere que PiT realiza un preprocesamiento visual más amplio en etapas tempranas y refina su foco atencional en etapas posteriores, lo que refleja una progresiva coordinación del aprendizaje jerárquico (Heo et al., 2021).

Por ejemplo, en una investigación orientada a la recuperación de imágenes médicas, se incorporó PiT (Pooling-based Vision Transformer) con el fin de mejorar tanto la eficacia como la precisión en la localización de imágenes diagnósticas dentro de repositorios clínicos. En el estudio de Ren et al. (2024), se aplicó un modelo basado en PiT junto con técnicas de reducción de dimensionalidad y atención de baja complejidad, optimizando la extracción de características visuales relevantes. Como resultado, el sistema logró una precisión del 92.62% en ImageNet100 y del 90.60% en

MS-COCO, demostrando que PiT puede superar a los modelos tradicionales de CNN y ViT en la recuperación de imágenes médicas, facilitando el acceso rápido a información relevante para el diagnóstico clínico.

- **MobileVit**

MobileViT es un modelo optimizado para dispositivos móviles y sistemas con baja capacidad computacional. Este arquitectura combina redes convolucionales (CNNs) y Transformers, lo que le permite capturar tanto relaciones locales como globales en imágenes, utilizando menos parámetros. Su diseño ligero lo convierte en una solución eficiente para tareas de visión por computadora en entornos con recursos limitados, permitiendo la ejecución de modelos avanzados sin depender de hardware especializado (Mehta & Rastegari, 2021).

En contraste con los Vision Transformers (ViT) tradicionales, que tienden a perder la estructura espacial de las imágenes al tratar los parches como secuencias planas, MobileVit conserva esta estructura utilizando operaciones convolucionales antes y después de la autoatención. Esto permite que cada píxel pueda acceder a información de toda la imagen sin necesidad de recurrir a mecanismos de atención global complejos.

En la arquitectura de MobileViT, se toma una imagen de entrada representada como un tensor $X \in \mathbb{R}^{H \times W \times C}$, donde H es la altura, W el ancho de la imagen y C es el número de canales. MobileViT aplica primero una convolución estándar de tamaño $n \times n$ sobre una convolución puntual 1 x 1, obteniéndose un tensor intermedio: $X_L \in \mathbb{R}^{H \times W \times d}$; donde d representa la dimensión proyectada, típicamente mayor que C, y se obtiene mediante combinaciones lineales de los canales de entrada.

Luego el tensor se reorganiza en una secuencia de N parches planos superpuestos $X_U \in \mathbb{R}^{P \times N \times d}$, donde $P = h \times w$ es el número de píxeles por parche y N= $\frac{HW}{P}$ representa el número total de parches. Cada parche $X_U(p)$ se introduce en un bloque Transformer, lo que da lugar a:

$$X_G(p) = Transformer\big(X_U(p)\big), 1 \leq p \leq P$$

Esto produce la representación global: $X_G \in \mathbb{R}^{P \times N \times d}$. Posteriormente, los parches enriquecidos se fusionan con la información local original. Se aplica una convolución adicional de tamaño nxn para generar la salida final del bloque MobileViT. Este proceso permite que cada píxel en X_G integre información proveniente de toda la imagen, como se ilustra en la Figura 47.

Figura 47

Cada píxel ve a todos los demás píxeles mediante atención en MobileViT



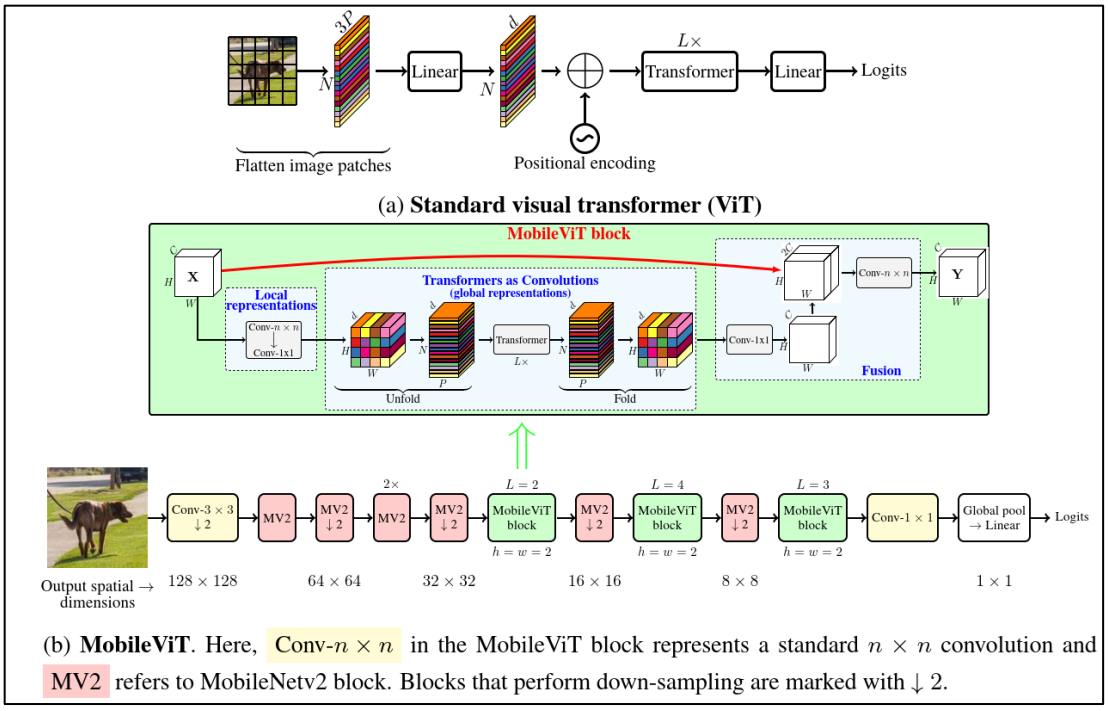
Nota. Adaptado de *MobileViT: Light-weight, General-purpose, and Mobile-friendly Vision Transformer* [Fig. 4], por Mehta & Rastegari (2021)

En comparación con Visión Transformer (ViT), MobileViT conserva el orden espacial de los píxeles dentro de cada parche, lo que mantiene el sesgo inductivo

espacial. Esta propiedad evita que el modelo requiera arquitecturas profundas o costosas; la arquitectura completa se resume en la Figura 48.

Figura 48

Comparación entre ViT estándar y MobileViT



(b) **MobileViT**. Here, Conv-$n \times n$ in the MobileViT block represents a standard $n \times n$ convolution and MV2 refers to MobileNetv2 block. Blocks that perform down-sampling are marked with $\downarrow 2$.

Nota. Adaptado de *MobileViT: Light-weight, General-purpose, and Mobile-friendly Vision Transformer* [Fig. 1], por Mehta & Rastegari (2021)

Además, MobileViT se caracteriza por ser "light-weighT". Por ejemplo, mientras que el modelo DeiT utiliza L = 12 capas y dimensión d = 192, MobileViT emplea solo L = {2,4,3} con d={96,120,144}, logrando una arquitectura más compacta y rápida (Mehta & Rastegari, 2021).

Por ejemplo, un estudio reciente sobre el diagnóstico del cáncer de páncreas aplicó MobileViT optimizado mediante DARTS para analizar imágenes de tomografía

computarizada (TC). En este proceso, los investigadores convirtieron las imágenes en grafos mediante la aplicación del algoritmo de detección de esquinas de Harris, con el fin de reflejar relaciones espaciales complejas. Estas representaciones gráficas fueron procesadas con modelos MobileViT optimizados, logrando una precisión del 97.33% y una puntuación F1 del 96.25%, superando a los modelos tradicionales de CNN y Vision Transformer. Este enfoque innovador demuestra el potencial de MobileViT para establecer un nuevo estándar en la detección anticipada del cáncer de páncreas y su escalabilidad para aplicaciones más amplias en imágenes médica

s (Alaca, 2025).

2.3 Marco Conceptual

2.3.1 Hemoglobina: Definición

Un examen de hemoglobina es un examen habitual de la sangre. Evalúa la presencia de una proteína en los glóbulos rojos, conocida como hemoglobina. Cuando inhalas, la hemoglobina lleva oxígeno a los órganos y tejidos corporales. Después, lleva el gas residual, el dióxido de carbono, a los pulmones para inhalarlo. Si un estudio de hemoglobina revela un nivel de hemoglobina inferior al que debería, esto indica una condición conocida como anemia. Para los hombres, la hemoglobina debería ser de 13.2 a 16.6 g/dl, mientras que, para las mujeres, debería ser de 11.6 a 15 g/dl (Mayo Clinic, s/f-a).

2.3.2 Detección preliminar de la anemia

La detección preliminar de la anemia permite identificar esta condición a través de una evaluación clínica y pruebas de laboratorio. Inicialmente, se analizan síntomas como fatiga, debilidad y palidez, que pueden sugerir la presencia de anemia (Mayo Clinic, s/f-b). Posteriormente, un hemograma completo establece los valores de hemoglobina, hematocrito y el recuento de glóbulos rojos para confirmar el diagnóstico. Si los resultados son irregulares, se

llevan a cabo exámenes adicionales, como la evaluación de hierro en sangre y ferritina, con el objetivo de identificar la causa raíz y guiar el tratamiento (National Heart Lung and Blood Institute, 2021) (RadiologyInfo.org (2024)).

2.3.3 Métodos de Diagnóstico Invasivo

Los procedimientos de análisis invasivos son esenciales para diagnosticar la anemia con precisión. El método más común es la extracción de sangre mediante punción venosa, conocida como venopunción, que permite realizar un hemograma completo. Este análisis mide parámetros como la cantidad de glóbulos rojos (hematocrito) y los niveles de hemoglobina, fundamentales para detectar la anemia (Mayo Clinic, s/f-a).

Además, se evalúan otros componentes sanguíneos, como los glóbulos blancos y las plaquetas, para identificar posibles causas subyacentes de la anemia (Huffington Post & Pérez de Pablos, 2024). En casos específicos, se pueden requerir procedimientos adicionales, como la biopsia de médula ósea, con el fin de verificar la formación de glóbulos rojos sanos y descartar trastornos hematológicos subyacentes (American Cancer Society, 2023).

2.3.4 Alternativas de Diagnóstico No Invasivo

Los procedimientos de análisis no invasivos para la detección de anemia han avanzado significativamente, permitiendo estimar los niveles de hemoglobina sin necesidad de extracción sanguínea. Una técnica destacada es la evaluación de la palidez en áreas como las palmas de las manos, las uñas y la lengua mediante imágenes digitales procesadas con inteligencia artificial. Por ejemplo, Sarah et al. (2020) desarrollaron un sistema que analiza la coloración del lecho ungueal, la conjuntiva palpebral y la lengua utilizando técnicas de aprendizaje automático, logrando estimaciones precisas de los niveles de hemoglobina. Estos métodos resultan útiles en el diagnóstico de poblaciones vulnerables y en entornos con recursos limitados.

2.3.5 Segmentación de Imágenes

Consiste en dividir una imagen en regiones homogéneas en cuanto a ciertas características, como intensidad de gris o textura. Esta técnica es fundamental para identificar y aislar objetos de interés dentro de una imagen, facilitando su análisis posterior. Por ejemplo, en el procesamiento de imágenes médicas, la segmentación permite resaltar estructuras anatómicas específicas para su estudio detallado (Gonzalez & Woods, 2017).

2.3.6 Región de Interés (ROI - Region of Interest)

Se refiere a una porción específica de una imagen que ha sido seleccionada para un análisis más detallado. Definir una ROI es crucial cuando solo una parte de la imagen contiene información relevante, permitiendo enfocarse en áreas específicas y mejorar la eficiencia del procesamiento(Campa Fernández, 2017).

2.3.7 Preprocesamiento de Imágenes

Se refiere al conjunto de técnicas aplicadas a imágenes con el objetivo de mejorar su calidad y facilitar su posterior análisis. Incluye la eliminación de ruido, la corrección de iluminaciones, el aumento de datos mediante transformaciones geométricas y fotométricas, y la reducción de la dimensionalidad para mejorar la eficiencia computacional en modelos de aprendizaje automático (Wang et al., 2021).

2.3.8 Redimensionamiento de imágenes

Implica el ajuste de las dimensiones de una imagen con el fin de adaptarla a requisitos específicos de procesamiento o visualización. Este proceso es común en la preparación de datos para algoritmos de aprendizaje automático, donde las imágenes deben tener tamaños uniformes para garantizar una correcta interpretación por parte del modelo (Gonzalez & Woods, 2017).

2.3.9 Normalización de los valores de pixeles

Consiste en ajustar las intensidades de una imagen a un rango estándar, por lo general entre 0 y 1. Este procedimiento resulta imprescindible para reforzar la coherencia y la eficacia de los algoritmos de procesamiento de imágenes, especialmente en redes neuronales, al disminuir la variabilidad y facilitar la convergencia durante la fase de entrenamiento (Gonzalez & Woods, 2017).

2.3.10 Balanceo de clases

En el contexto del Aprendizaje Automático, consiste en las técnicas utilizadas para abordar la desproporción en la cantidad de muestras entre diferentes clases en un conjunto de datos. El balanceo de clases es crucial para evitar sesgos en los modelos predictivos y garantizar un rendimiento equilibrado en todas las categorías (Gonzalez & Woods, 2017).

2.3.11 Extracción de características

Implica reconocer y medir atributos relevantes en las imágenes de palmas de manos, con el fin de detectar indicios de anemia. Más tarde, estos rasgos se incorporan como insumos para los algoritmos de aprendizaje automático, con el objetivo de llevar a cabo su clasificación correspondiente. La efectividad de este enfoque radica en su capacidad para capturar patrones sutiles que podrían no ser evidentes a simple vista, mejorando así la precisión del diagnóstico (Appiahene, et al., 2023a).

2.3.12 Conjunto de entrenamiento y prueba

Refiere a porciones de un mismo conjunto de datos. El modelo de aprendizaje automático se entrena utilizando el conjunto de entrenamiento, mientras que el conjunto de prueba se emplea para medir su rendimiento con datos no utilizados previamente (Harrington, 2012).

2.3.13 Clasificación de imágenes

Es el proceso de fijar una etiqueta o categoría a una imagen en función de sus características visuales. Por ejemplo, según Peksi et al. (2021), la clasificación de imágenes se aplicó a la detección de anemia utilizando imágenes digitales de uñas y palmas de las manos, donde las características de color extraídas de estas imágenes fueron clasificadas mediante el método Naïve Bayes para determinar si una persona es anémica o no.

2.3.14 Evaluación de Modelos

Consiste en evaluar el desempeño y la eficacia de un modelo de aprendizaje automático. Los procedimientos de evaluación pueden abarcar indicadores como exactitud, sensibilidad, precisión y el área bajo la curva - ROC (Fawcett, 2006).

2.4 Hipótesis

Se estableció la hipótesis general y las hipótesis específicas conforme a los problemas y metas detectados anteriormente.

2.4.1 Hipótesis general

HG: El desarrollo de un modelo híbrido basado en Vision Transformer y Machine Learning permite detectar la anemia a partir de imágenes de palmas de las manos.

2.4.2 Hipótesis específicas

He1: La base de datos de imágenes de palmas de manos utilizada en el estudio contiene características adecuadas para ser utilizada en el desarrollo de un modelo híbrido de detección de anemia.

He2: La aplicación de técnicas de preprocesamiento permite estandarizar adecuadamente las imágenes de palmas de manos para el desarrollo de un modelo híbrido de detección de anemia.

He3: La utilización de técnicas basadas en Vision Transformer y Machine Learning permite construir un modelo híbrido de clasificación que reconoce adecuadamente imágenes de palmas de manos de personas con y sin anemia.

He4: La aplicación de métricas como Accuracy, Precision, Recall, F1-score y AUC-ROC permite evaluar adecuadamente el desempeño del modelo híbrido de clasificación de imágenes de palmas de manos de personas con y sin anemia.

CAPÍTULO III: Metodología

El contenido de este capítulo aborda la planificación metodológica, especificando el diseño utilizado, el tipo de estudio y el enfoque aplicado.

3.1 Diseño y Enfoque Metodológico

3.1.1 Estructura del Diseño de la Investigación

El diseño experimental es una metodología en la que se manipulan variables para evaluar su impacto en una variable dependiente, permitiendo establecer relaciones de causa y efecto con alta validez. En el contexto de la inteligencia artificial orientada a la salud, este diseño es útil para probar diferentes enfoques en el análisis de imágenes médicas, garantizando condiciones controladas y replicables. En estudios de clasificación de imágenes biomédicas, el diseño experimental permite comparar diferentes modelos de Machine Learning y Deep Learning, analizar su precisión en la identificación de patrones visuales en estructuras visuales y optimizar su desempeño en entornos clínicos (Goodfellow et al., 2016).

En este estudio, se aplica diseño experimental para evaluar y comparar dos enfoques distintos: Machine Learning y Deep Learning, evaluados de manera independiente en la detección preliminar de anemia mediante imágenes de palmas de manos. Ambos métodos se utilizarán por separado, obteniendo resultados individuales que posteriormente serán cruzados

para analizar su complementariedad y su desempeño en la clasificación de imágenes médicas. Se evaluarán diversos algoritmos de aprendizaje automático, como SVM, Naïve Bayes, Random Forest, KNN y Decision Tree, junto con arquitecturas de aprendizaje profundo (BOTNet, ViT, Swin Transformer, PiT y MobileViT), con el propósito de identificar cuál de estas técnicas ofrece el desempeño más sólido en términos de precisión, eficiencia computacional y capacidad de adaptarse a distintas condiciones de generalización de los datos. (Bishop, 2006).

3.1.2 Tipo de Diseño Metodológico

El tipo de investigación aplicada se concibe para dar respuesta a problemas concretos y concentra sus esfuerzos en la implementación de teorías y conocimientos en contextos específicos. De este modo, persigue la obtención de resultados con una repercusión palpable en distintos ámbitos, como la salud, la tecnología o la educación. (Sabino, 1992).

La presente investigación es de tipo aplicado, ya que busca desarrollar y evaluar modelos computacionales que puedan ser implementados como herramientas de apoyo en sistemas de detección médica de anemia, a partir de análisis de imágenes de palmas de manos. Asimismo, el estudio posee un nivel explicativo, en tanto se propone analizar qué factores influyen en la mejora del desempeño de los modelos, cómo interactúan distintas técnicas de Machine Learning y Deep Learning, y de qué manera estas combinaciones optimizan los resultados. Se examinará la influencia de diferentes técnicas de preprocesamiento y balanceo de clases, así como la capacidad de cada modelo para generalizar en conjuntos de datos con variaciones en iluminación y tono de piel (Sheskin, 2020).

3.1.3 Enfoque de la investigación

El enfoque cuantitativo se apoya en reunir y examinar información numérica con ayuda de métodos estadísticos, con el fin de identificar patrones y vínculos entre variables. Gracias a

este planteamiento, es posible medir con precisión el rendimiento de distintas técnicas, al mismo tiempo que se obtienen resultados confiables y objetivos. En investigaciones sobre inteligencia artificial aplicada a la salud, el enfoque cuantitativo es fundamental para evaluar métricas como precisión y sensibilidad en modelos de clasificación de imágenes médicas (Kuhn & Johnson, 2013).

En esta investigación, optar por un enfoque cuantitativo facilita tanto la medición como el análisis minucioso de los datos, lo cual favorece una comprensión más profunda de la relación entre las características de las imágenes de las palmas y la detección de anemia. Además, esta perspectiva promueve la posibilidad de extrapolar los hallazgos obtenidos y de comparar diversas técnicas de diagnóstico de manera sistemática. (Hernández Sampieri et al., 2017).

3.2 Población y Muestra de la investigación

3.2.1 Población

La población o universo hace referencia al grupo de individuos o elementos sobre los cuales se pretende obtener información en el marco de una investigación (Pineda et al., 1994).

La población de la presente investigación está compuesta por imágenes de la palma de la mano de niños menores de cinco años. Estas imágenes fueron capturadas en los siguientes diez hospitales de Ghana: Komfo Anokye Teaching y Manhyia District, ambos en Kumasi; Sunyani Municipal y SDA, ambos en Sunyani; Bolgatanga Regional en Bolgatanga; Kintampo Municipal en Kintampo; Ahmadiyya Muslim de Techiman; Ejusu Government en Ejusu; Nkawie-Toase Government en Nkawie-Toase y Holy Family en Berekum (Asare et al., 2022)

En total, la población considerada en este estudio está conformada por 4,260 imágenes de palmas de manos de niños, las cuales fueron obtenidas a partir de 710 imágenes originales (426 de niños anémicos y 284 de niños no anémicos) y posteriormente ampliadas mediante

técnicas de aumento de datos (Data Augmentation) para garantizar un conjunto balanceado de datos (Asare et al., 2022).

3.2.2 Muestra

La muestra se define como un segmento representativo de la población o universo de estudio, sobre el cual se desarrollará la investigación con el propósito de extrapolar los resultados obtenidos al conjunto total (Pineda et al., 1994).

Se utilizó un muestreo no probabilístico, recolectando un total de 710 imágenes originales (426 de niños anémicos y 284 de niños no anémicos). Posteriormente, mediante técnicas de aumento de datos (*Data Augmentation*), el conjunto de imágenes se amplió a 4260 imágenes, asegurando un balance entre las categorías. Estas imágenes fueron etiquetadas según los valores de Hemoglobina (Hb) y se dividieron en conjuntos de entrenamiento y validación (Asare et al., 2022).

3.2.3. Unidad de análisis

Hace referencia a los componentes dentro de una población o conjunto de estudio en los cuales se examinan o cuantifican las variables relevantes (Pineda et al., 1994).

La unidad de análisis corresponde a imágenes de la palma de la mano de infantes menores de cinco años, capturadas con cámaras de alta resolución (mínimo 12 MP). Estas imágenes fueron clasificadas como "anémico" o "no anémico" de acuerdo con los niveles de hemoglobina (Hb) obtenidos en pruebas de laboratorio.

3.3 Operacionalización de Variables de la Investigación

Para analizar la relación entre las variables del estudio, es fundamental definir y estructurar su medición de manera precisa. En la Tabla 10 se establecen los elementos clave que intervienen en el análisis, especificando su conceptualización, indicadores y métodos de evaluación. Se detallan tanto las variables independientes, relacionadas con las técnicas de

Aprendizaje Automático y Aprendizaje Profundo, como la variable dependiente, que hace referencia a la detección de anemia. A través de esta operacionalización, se busca garantizar una medición objetiva y consistente que respalde la validez del estudio.

Tabla 10

Definición y medición de variables

	Definición (Referencia)	**Indicador**	**Fórmula del indicador**
Independiente			
Machine learning (ML) y Vision Transformers	Según Dulhare et al. (2020), Aprendizaje Automático es una rama de la inteligencia artificial que optimiza el rendimiento mediante la programación basada en datos y experiencias previas, permitiendo que los programas mejoren su desempeño con o sin supervisión humana. Según Dosovitskiy et al. (2020), es un modelo de red neuronal basado en la arquitectura Transformer que permite clasificar imágenes dividiéndolas en pequeños parches, los cuales son procesados como secuencias mediante mecanismos de autoatención, logrando así	Modelo de clasificación	Tipo de modelo de ML o DL utilizado
		Accuracy	$\frac{TP + TN}{TP + FP + FN + TN}$
		Precision	$\frac{TP}{TP + FP}$
		Recall	$\frac{TP}{TP + FN}$
		F1-Score	$2 * \frac{Precision * Recall}{Precision + Recall}$
		ROC AUC	$P(score(x^+) > score(x^-)$

	capturar relaciones espaciales sin el uso de convoluciones tradicionales, lo que facilita una comprensión global de la imagen desde las primeras capas del modelo.		
Dependiente			
Anemia	Según Kesarwani et al. (2023), la anemia puede detectarse de forma no invasiva analizando la palidez palmar utilizando aplicaciones móviles y algoritmos de procesamiento de imágenes.	Porcentaje de detecciones correctas	Número de detecciones correctas / número total de detecciones) * 100

De acuerdo con las definiciones propuestas por Kelleher et al. (2015) y adaptadas a esta investigación, los valores obtenidos a partir de la matriz de confusión se interpretan de la siguiente manera en el contexto de la predicción de anemia:

- TP (Verdaderos Positivos): Un paciente tiene anemia y el modelo predice correctamente que tiene anemia.
- FP (Falsos Positivos): Un paciente no tiene anemia, pero el modelo predice incorrectamente que sí tiene anemia.
- FN (Falsos Negativos): Un paciente sí tiene anemia, pero el modelo predice incorrectamente que no tiene anemia.
- TN (Verdaderos Negativos): Un paciente no tiene anemia y el modelo predice correctamente que no tiene anemia.

3.4 Técnicas de Recolección de Datos

Para este estudio se utilizó un conjunto de datos previamente publicado por Asare et al. (2022) en el repositorio Mendeley Data, bajo el título Anemia Detection using Palpable Palm Image Datasets from Ghana. El dataset está compuesto por imágenes de palmas de manos de niños entre 0 y 5 años, capturadas en hospitales de Ghana por profesionales de la salud utilizando cámaras de alta resolución (mínimo 12 MP). Dado que los participantes eran menores de edad, los autores del dataset aplicaron protocolos éticos y obtuvieron el consentimiento de los tutores legales.

El conjunto publicado contiene 4,260 imágenes en formato .png, generadas a partir de 710 imágenes reales mediante técnicas de aumento de datos (como rotación, traslación e inversión horizontal). Sin embargo, el dataset original de 710 imágenes reales no fue publicado de forma separada, por lo que el análisis inicial se realizó sobre la totalidad de las 4,260 imágenes disponibles.

Para garantizar la validez del estudio y trabajar únicamente con datos reales, se realizó un proceso de depuración manual, eliminando las imágenes que presentaban indicios de haber sido generadas artificialmente. Esta depuración se basó en los nombres de archivo: se identificó como “original” aquella imagen sin sufijos numéricos. Por ejemplo, “AnemicP-226” se consideró real, mientras que “AnemicP-226 (2)”, “AnemicP-226 (3)”, etc., se interpretaron como versiones aumentadas. A partir de este criterio, se seleccionaron 526 imágenes reales.

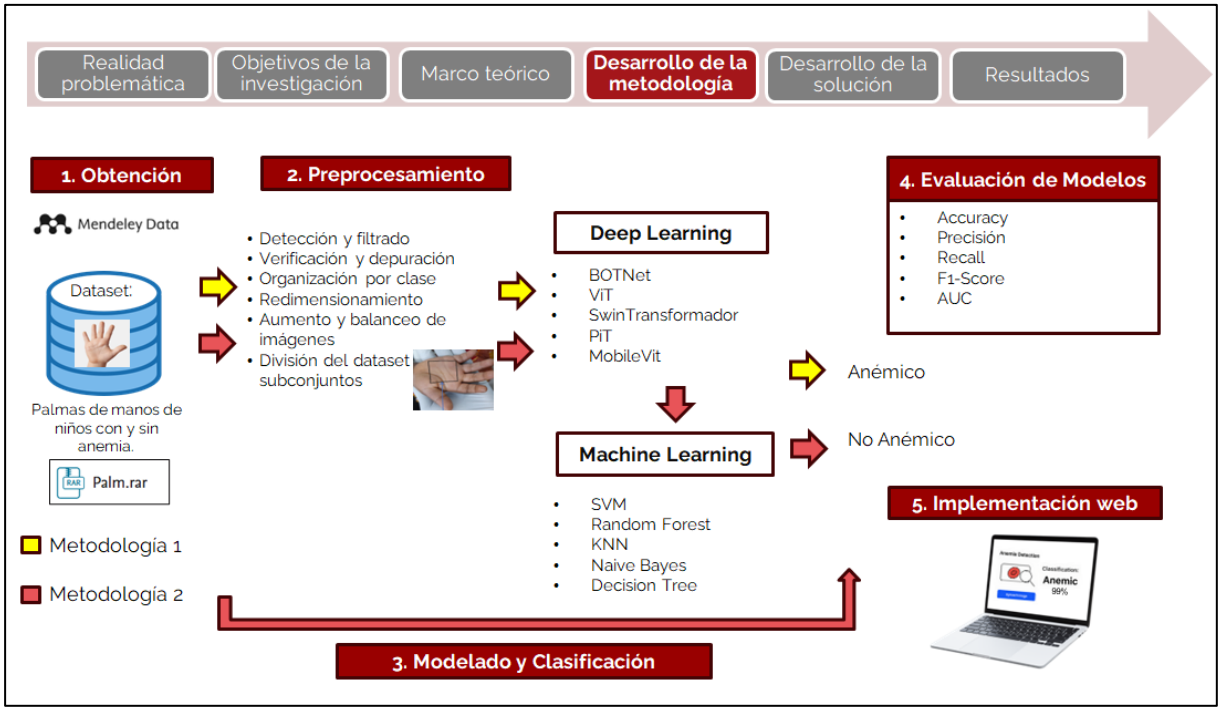
Posteriormente, se aplicó una segunda ronda de depuración, descartando imágenes que presentaban iluminación artificial (flash), desenfoques o errores en el recorte de la Región de Interés (ROI). Como resultado, se obtuvo una muestra final de 295 imágenes reales y limpias, que fueron utilizadas para los procesos de aumento controlado, balanceo de clases y entrenamiento de modelos de clasificación.

3.5 Metodología para la implementación de la solución.

El proceso metodológico desarrollado en esta investigación se sintetiza en la Figura 49.

Figura 49

Metodología de la implementación de la solución



3.5.1 Obtención del dataset

En esta etapa se describirá la adquisición del dataset con el cual se trabajó en el presente trabajo de investigación. A continuación, en la tabla 11 se presentan los procedimientos, su descripción y tareas donde se detalla lo referido a la adquisición del dataset:

Tabla 11

Procedimientos en la fase de la obtención de la base de datos

Procedimientos	Descripción	Tareas

1. Identificación del dataset	Búsqueda y análisis de datos con imágenes de palmas de manos de niños de 0 a 5 años.	- Buscar datasets en fuentes abiertas. - Analizar la información del dataset y su adecuación al estudio.
2. Descarga del dataset	Obtención del conjunto de datos para su posterior análisis.	- Descargar el dataset completo desde el repositorio Mendeley Data. - Verificar la integridad y estructura de los archivos.

En las siguientes líneas se especifican los procedimientos realizados y los productos obtenidos en cada una de ellas:

- **Procedimiento 1: Identificación del dataset.**

Se procedió a la identificación y selección del conjunto de datos necesario para el desarrollo de la presente investigación. El dataset seleccionado fue "Anemia Detection using Palpable Palm Image Datasets from Ghana", obtenido del repositorio en línea Mendeley Data, creado por Asare et al. (2022). Este conjunto de datos contiene imágenes fotográficas de palmas de manos de niños de 0 a 5 años, capturadas en hospitales de Ghana por un equipo de profesionales de la salud. La selección de este dataset se basó en su disponibilidad pública, calidad de las imágenes y relevancia para los objetivos del estudio.

- **Procedimiento 2: Descarga del dataset**

Una vez identificado el dataset, se procedió a su descarga completa en un formato comprimido .rar. Al descomprimir el archivo, se obtuvieron imágenes en formato .png con

nombres que indicaban la clase correspondiente “AnemiP-xxx” para imágenes de pacientes con anemia y “Non-AnemicP- xxx” para imágenes de pacientes sin anemia.

Se verificó la integridad del archivo descargando, asegurando que todas las imágenes se encontraran accesibles y en buen estado para ser utilizados en las siguientes etapas del trabajo.

Entregable: Dataset descomprimido y organizado en una única carpeta, con imágenes fotográficas de palmas de manos de niños de 0 a 5 años, provenientes del repositorio Mendeley Data (Asare et al., 2022).

3.5.2 Preprocesamiento de datos

Esta actividad contempla la preparación estructurada del conjunto de imágenes a utilizarse en el modelado. Incluye la estandarización de nombres de archivo, la clasificación automática en clases (anemic y non-anemic), y una revisión manual destinada a eliminar imágenes con problemas de calidad como desenfoque, flash o recortes inadecuados. Posteriormente, se aplicarán técnicas de aumento de datos para balancear las clases, y se organizará el conjunto final en tres subconjuntos —entrenamiento, validación y prueba— bajo una proporción del 70%, 10% y 20%, respectivamente.

Tabla 12

Procedimientos en la fase de preprocesamiento de datos

Procedimientos	Descripción	Tareas
1. Detección y filtrado	Identificación de imágenes generadas mediante técnicas previas de aumento, a partir del análisis de patrones en los nombres de archivo, con el objetivo de excluirlas y trabajar únicamente con imágenes originales. Este filtrado permite	• Leer todos los archivos dentro de la carpeta del dataset. • Utilizar una expresión regular para detectar imágenes aumentadas (ej. con paréntesis en el nombre).

		establecer un conjunto base limpio y libre de duplicaciones sintéticas.	• Crear una carpeta de destino para las imágenes originales. • Copiar únicamente las imágenes no aumentadas. • Verificar el total de archivos copiados.
2.	Verificación y depuración	Depuración visual del conjunto de imágenes originales para eliminar aquellas que presenten desenfoques, mala calidad o recortes excesivos que afecten la visibilidad de la región de interés.	• Revisar manualmente las imágenes originales. • Eliminar aquellas borrosas, con mala calidad o recortes inadecuados. • Documentar los archivos eliminados y conservar respaldo.
3.	Organización por clase	Identificación de las clases presentes en el dataset a partir de los nombres de los archivos y conteo de imágenes por clase, con el fin de conocer la distribución de datos entre categorías. Estandarización de nombres y organización de las imágenes en carpetas diferenciadas por clase, para garantizar una estructura clara y homogénea del dataset.	• Identificar los nombres de los archivos contenidos en el dataset. • Segmentar los nombres por prefijos que indiquen la posible clase. • Contabilizar la cantidad de imágenes con prefijos similares. • Detectar la cantidad de imágenes correspondientes a las clases. • Identificar clase según prefijo o nombre del archivo. • Renombrar de forma uniforme según clase. • Crear carpetas por clase.

			• Mover las imágenes renombradas a su carpeta correspondiente.
4.	Redimensionamiento	Redimensionamiento uniforme de todas las imágenes a una resolución estándar, asegurando coherencia estructural y compatibilidad con los modelos de aprendizaje profundo.	• Verificar las resoluciones originales por clase. • Redimensionar todas las imágenes. • Guardar en carpetas organizadas por clase.
5.	Aumento y balanceo de imágenes	Generación de imágenes sintéticas a partir de las originales mediante técnicas de Data Augmentation, con el propósito de equilibrar la cantidad de datos por clase y aumentar la diversidad del conjunto sin introducir sesgos.	• Determinar cuántas imágenes adicionales se requieren por clase para alcanzar un umbral predefinido. • Aplicar transformaciones como rotación, volteo, desenfoque, ajuste de color y contraste. • Generar y almacenar las imágenes aumentadas de forma estructurada por clase.
6.	División del dataset en subconjuntos	Distribución estructurada del conjunto de datos en tres subconjuntos independientes -entrenamiento, validación y prueba- en proporciones de 70 %, 10 % y 20 %, respectivamente. Esta segmentación busca asegurar una evaluación imparcial del modelo y mantener el equilibrio entre clases en cada conjunto.	• Mezclar aleatoriamente las imágenes de cada clase. • Calcular la cantidad de imágenes por subconjunto según la proporción definida. • Crear carpetas organizadas por subconjunto y clase. • Copiar las imágenes a su carpeta correspondiente.

- **Procedimiento 1: Detección y filtrado**

En esta primera actividad del preprocesamiento, se realiza un escaneo automatizado del conjunto inicial de imágenes con el objetivo de detectar aquellas que hubieran sido generadas mediante técnicas previas de data augmentation. Para ello, se implementó una expresión regular capaz de identificar patrones específicos en los nombres de archivo, como la presencia de números entre paréntesis, los cuales son comúnmente introducidos por sistemas automáticos de duplicación. Esta detección preliminar permitió documentar el estado inicial del conjunto de datos y establecer una base clara para las siguientes acciones de organización.

Posteriormente, se procedió a un proceso de filtrado controlado, destinado a aislar únicamente las imágenes originales. Esta actividad consistió en copiar, desde la carpeta fuente, únicamente aquellos archivos que no presentaban indicios de haber sido aumentados, conformando así un subconjunto limpio y sin duplicaciones sintéticas. Para ello, se creó una carpeta exclusiva destinada al almacenamiento de dichas imágenes originales. Esta medida fue clave para reducir el riesgo de sesgos en el análisis posterior y asegurar que las transformaciones futuras se apliquen sobre datos genuinios.

Entregable: Carpeta con imágenes originales filtradas, sin duplicaciones sintéticas, acompañada del registro de detección automatizada mediante expresión regular.

- **Procedimiento 2: Verificación y depuración**

Con el conjunto de imágenes originales ya filtrado, se lleva a cabo una inspección manual detallada con el objetivo de depurar el dataset y asegurar su calidad visual. Esta etapa consiste en revisar cada imagen individualmente para identificar aquellas que presenten

problemas como desenfoques severos, iluminación deficiente, exposición al flash, sombras pronunciadas o recortes que comprometan la visibilidad de la región de interés (ROI). Las imágenes que no cumplen con los criterios mínimos de calidad son eliminadas del conjunto, dejando únicamente aquellas que puedan aportar valor informativo real al entrenamiento de los modelos. Este paso resulta esencial para preservar la integridad del análisis y evitar el aprendizaje de patrones erróneos o irrelevantes.

Entregable: Dataset depurado manualmente y listado de imágenes eliminadas por motivos de calidad.

- **Procedimiento 3: Renombramiento y clasificación por clase**

Con el conjunto de imágenes ya depurado, se procede a identificar las clases presentes en el dataset a partir del análisis de los nombres de archivo, los cuales contienen prefijos que permiten inferir la categoría correspondiente. Esta etapa tiene como propósito conocer la distribución del dataset entre las clases “anémico” y “no anémico”, por lo que se realiza un conteo preciso de las imágenes clasificadas bajo cada categoría. Esta información resulta clave para la evaluación de balance en etapas posteriores, así como para la generación controlada de imágenes aumentadas si fuera necesario.

Posteriormente, se ejecuta el proceso de renombramiento estandarizado, asignando a cada archivo una nomenclatura uniforme y secuencial que incluya la clase a la que pertenece (por ejemplo: anemic_001.png, non_anemic_002.png). Esta acción mejora la trazabilidad de las imágenes y facilita su reconocimiento automático durante el entrenamiento de los modelos. A continuación, las imágenes ya renombradas son clasificadas en carpetas separadas por clase, generando una estructura de directorios clara, homogénea y compatible con los requerimientos de los algoritmos de aprendizaje automático.

Esta estandarización garantiza coherencia en las etapas posteriores del flujo metodológico y contribuye a mantener la integridad organizativa del dataset durante todo el proceso experimental.

Entregable: Conjunto de imágenes renombradas y clasificadas en carpetas por clase (anemic y non_anemic), acompañado del registro del conteo total de imágenes por categoría.

- **Procedimiento 4: Redimensionamiento uniforme**

Con el fin de garantizar la coherencia estructural del conjunto de datos y su compatibilidad con los modelos de aprendizaje profundo, se aplica un redimensionamiento uniforme a todas las imágenes del dataset. Esta actividad consiste en ajustar cada archivo visual a una resolución estándar previamente definida, asegurando que todas las muestras presenten las mismas dimensiones sin distorsionar la región de interés. El proceso contempla la verificación inicial de las resoluciones existentes y la aplicación sistemática del cambio de tamaño, conservando las proporciones y la calidad visual. Este paso permite optimizar el rendimiento del entrenamiento y evitar errores derivados de inconsistencias en las entradas.

Entregable: Carpeta con imágenes redimensionadas y homogeneizadas por clase.

- **Procedimiento 5: Aumento y balanceo por clase**

Con el objetivo de mitigar el desbalance entre clases y ampliar la variabilidad del conjunto de datos sin necesidad de nuevas adquisiciones, se implementa una estrategia de Data Augmentation. Esta técnica consiste en generar imágenes sintéticas a partir de muestras originales, mediante transformaciones visuales controladas que simulan distintas condiciones de captura sin alterar la información diagnóstica.

Las operaciones aplicadas incluyen rotación aleatoria, volteo horizontal, desenfoque, ajuste de color y variaciones de contraste. Estas modificaciones contribuyen a enriquecer el conjunto de entrenamiento y mejorar la capacidad de generalización del modelo.

El número de imágenes a generar se define a partir del tamaño de la clase minoritaria, estableciendo así un umbral común para ambas categorías. Las imágenes resultantes se organizan en carpetas diferenciadas por clase, siguiendo la estructura definida en fases anteriores, lo que asegura coherencia en el flujo de trabajo y facilita su integración en etapas posteriores del preprocesamiento.

Entregable: Conjunto de imágenes aumentadas y balanceadas por clase.

- **Procedimiento 6: División del dataset en subconjuntos**

Como etapa final del preprocesamiento, se lleva a cabo la partición del conjunto de datos en tres subconjuntos funcionales: entrenamiento, validación y prueba. Esta división busca asegurar una evaluación objetiva del rendimiento del modelo, evitando el uso cruzado de datos entre etapas. Las imágenes se reorganizan de forma aleatoria y se distribuyen equitativamente por clase, respetando la proporción 70 % (entrenamiento), 10 % (validación) y 20 % (prueba). Cada subconjunto se almacena en carpetas específicas por clase, manteniendo balance y orden estructural para el modelado.

- **Entregable:** Dataset dividido en carpetas train, val y test, con imágenes balanceadas por clase

3.5.3 Modelado y clasificación

Esta etapa comprende el desarrollo del proceso de modelado orientado a la clasificación automática de las imágenes procesadas, empleando exclusivamente arquitecturas basadas en Vision Transformer y técnicas de Machine Learning. Para ello, se implementan dos

metodologías complementarias, centradas en el uso de modelos derivados de la arquitectura Transformer.

La primera metodología corresponde al uso directo de modelos Vision Transformer (ViT) y arquitecturas relacionadas —como BOTNet, Swin Transformer, MobileViT y Pooling-based Vision Transformer (PiT)— como clasificadores integrales. Estas redes han sido diseñadas para procesar imágenes dividiéndolas en parches y capturar relaciones espaciales complejas mediante mecanismos de autoatención, lo que las convierte en herramientas eficaces para tareas de clasificación visual.

La segunda metodología adopta un enfoque híbrido: se emplean los mismos modelos basados en Transformer exclusivamente como extractores de características, mientras que la clasificación final se lleva a cabo mediante algoritmos tradicionales de Machine Learning, tales como Support Vector Machines (SVM), Random Forest (RF), K-Nearest Neighbors (KNN), Naïve Bayes (NB) y Decision Tree (DT). Esta estrategia permite analizar el rendimiento de clasificadores más ligeros sobre representaciones generadas por modelos de alta capacidad.

La selección de ambas metodologías se sustenta en el alto rendimiento reportado por estas arquitecturas en estudios recientes, como los de Appiahene, et al. (2023a), Asare et al. (2023) y Appiahene, et al. (2023b).

- **Metodología 1 (Vision Transformer)**

En esta primera metodología, se implementan arquitecturas de Vision Transformer sobre las imágenes obtenidas en la fase de preprocesamiento. Cada modelo se entrena de forma independiente utilizando el conjunto previamente dividido en subconjuntos de entrenamiento, validación y prueba, con el objetivo de evaluar su capacidad para clasificar imágenes de palmas según la presencia o no de anemia.

Tabla 13

Procedimientos en la fase de modelado y clasificación de la Metodología 1

Procedimientos	Descripción	Tareas
1. Preparar el conjunto de entrada	Organización del dataset procesado y estructurado para su uso en arquitecturas de Vision Transformer	• Cargar las imágenes clasificadas por clase. • Aplicar transformaciones y normalización según cada arquitectura.
2. Definir las arquitecturas	Selección de los modelos a evaluar en la tarea de clasificación binaria.	• Cargar modelos preentrenados: BOTNet, ViT, Swin Transformer, MobileViT y PiT. • Adaptar las capas finales.
3. Entrenamiento de modelos	Ejecución del proceso de entrenamiento para cada modelo utilizando el conjunto de datos preparado. Esta etapa incluye el ajuste de parámetros, validación y optimización del rendimiento durante el proceso de aprendizaje.	• Entrenar cada arquitectura utilizando los datos de entrenamiento. • Ajustar parámetros y validar el rendimiento por épocas. • Aplicar técnicas de mejora del desempeño como el ajuste de hiperparámetros (tuning) y early stopping.
4. Evaluación del desempeño	Medición del rendimiento de cada arquitectura utilizando el conjunto de prueba y métricas de clasificación.	• Calcular métricas como accuracy, precision, recall, F1-score y AUC. • Generar curvas de pérdida y ROC.
5. Guardar los modelos y resultados	Conservar el modelo entrenado y las métricas de desempeño para su posterior	• Guardar los pesos del modelo en formato. pth.

	análisis o implementación.	• Exportar métricas a archivo .csv. • Registrar timestamp y nombre del modelo.

- **Procedimiento 1: Preparación y normalización de imágenes**

En esta actividad se describe el proceso de preparación y normalización de las imágenes antes de ser utilizadas por los modelos de Vision Transfomer. El objetivo es garantizar que todas las muestras tengan las mismas dimensiones, estructura de datos y valores de intensidad dentro de un rango adecuado para el aprendizaje automático.

Se aplican técnicas de transformación y estandarización a las imágenes en formato RGB, las cuales son redimensionadas según el tamaño de entrada requerido por cada arquitectura (por ejemplo, 224×224 píxeles para Vision Transformer, Swin Transformer, PiT y MobileViT; y 256×256 píxeles para BOTNet). Posteriormente, las imágenes son convertidas a tensores y normalizadas por canal utilizando las medias y desviaciones estándar definidas por el conjunto ImageNet.

Esta normalización permite adaptar las imágenes a los requerimientos de entrada esperados por modelos preentrenados, facilitando su aprovechamiento durante el proceso de entrenamiento.

Cabe señalar que, si bien las imágenes ya han sido redimensionadas durante el preprocesamiento general del dataset, este paso se reitera en esta etapa para asegurar la compatibilidad exacta con los requisitos de entrada específicos de cada modelo de Deep Learning.

La transformación matemática aplicada a cada pixel ρ de cada canal se puede describir como:

$$\rho \text{ normalizado} = \frac{\rho - \mu}{\sigma}$$

donde:

- μ es la media del canal (por ejemplo, 0.485, 0.456, 0.406 para R, G, B).
- σ es la desviación estándar del canal (por ejemplo, 0.229, 0.224, 0.225).

Figura 50

Pseudocódigo de la preparación y normalización de imágenes

```
Para cada imagen en el dataset:
    Abrir la imagen desde su ruta

    Si la imagen no está en formato RGB:
        Convertir a RGB

    Redimensionar la imagen según el modelo:
        Si modelo es BOTNet:
            Cambiar tamaño a 256x256 píxeles
        Sino:
            Cambiar tamaño a 224x224 píxeles

    Convertir la imagen a tensor (estructura numérica)

    Normalizar los valores de los píxeles:
        Para cada canal (R, G, B):
            Restar la media del canal (de ImageNet)
            Dividir entre la desviación estándar del canal (de ImageNet)

    Almacenar la imagen transformada en memoria para entrenamiento o validación
```

Entregable: Conjunto de imágenes preprocesadas en formato tensorial, listas para ser utilizadas en el entrenamiento de modelos de Deep Learning.

- **Procedimiento 2: Definición de arquitecturas**

En esta actividad se establece la selección de las arquitecturas de Deep Learning que serán utilizadas en la etapa de modelado. Estas arquitecturas fueron elegidas por su probada efectividad en tareas de clasificación de imágenes y su capacidad para aprender representaciones complejas a partir de datos visuales.

Se emplean modelos preentrenados en el conjunto de datos ImageNet, lo que permite transferir conocimientos previamente adquiridos para aplicarlos a la clasificación binaria de imágenes de palmas (anemia / no anemia). Las arquitecturas consideradas son:

- BOTNet: combina atención global y convoluciones.
- Vision Transformer (ViT): transforma imágenes en secuencias para aprovechar mecanismos de atención.
- Swin Transformer: emplea ventanas deslizantes jerárquicas para atención local y global.
- MobileViT: diseñado para eficiencia en dispositivos móviles, manteniendo precisión.
- PiT (Pooling-based Vision Transformer): introduce operaciones de pooling para mayor eficiencia computacional.

Cada modelo es adaptado para resolver una tarea de clasificación binaria mediante la modificación de su capa de salida. Matemáticamente, la salida del modelo se representa como una función:

$$\hat{y} = softmax(Wz + b)$$

donde:

- z es el vector de características generado por la red,
- W y b son los parámetros entrenables de la capa de clasificación,
- $\hat{y}$ representa la probabilidad estimada para cada clase.

Figura 51

Pseudocódigo para la definición de arquitecturas

```
Algoritmo definir_arquitecturas:
1. Importar arquitecturas preentrenadas desde biblioteca externa (ej. timm, Hugging Face)
2. Para cada arquitectura seleccionada:
    a. Cargar modelo base con pesos preentrenados en ImageNet
    b. Reemplazar la capa de salida con una nueva capa para clasificación binaria
    c. Enviar modelo a dispositivo (CPU o GPU)
3. Guardar modelos adaptados para entrenamiento
```

Entregable: Conjunto de modelos preentrenados adaptados para clasificación binaria sobre imágenes de palmas.

- **Procedimiento 3: Entrenamiento de modelos**

En esta actividad se entrena cada una de las arquitecturas de Deep Learning definidas previamente, utilizando el conjunto de datos preparado en las etapas anteriores. El objetivo es ajustar los parámetros internos de los modelos a partir de ejemplos de entrenamiento, de modo que sean capaces de reconocer patrones relevantes y generalizar correctamente ante nuevas imágenes no vistas.

El entrenamiento se realiza mediante algoritmos de retropropagación y optimización, utilizando como referencia una función de pérdida binaria, como la Binary Cross-Entropy (para salidas sigmoides) o la Cross-Entropy Loss (para salidas softmax), dependiendo de la configuración de cada modelo. Paralelamente, se emplea un conjunto de validación para monitorear el desempeño en cada época y aplicar estrategias de control como early stopping, lo cual ayuda a evitar el sobreajuste.

Cada arquitectura es entrenada de forma independiente, registrando en cada iteración métricas clave como la pérdida y precisión en los conjuntos de entrenamiento y validación. Esto

permite visualizar la evolución del aprendizaje y facilitar la posterior comparación entre modelos.

En esta etapa, también se definen valores iniciales para algunos hiperparámetros —como la tasa de aprendizaje o el tamaño de lote— basándose en pruebas previas o experimentación, antes de realizar un ajuste más profundo y automatizado.

Figura 52

Pseudocódigo del entrenamiento de modelos de Deep Learning

```
algoritmo entrenar_modelo_DL

variables
    modelo, dataloader_train, dataloader_val
    criterio, optimizador, epochs

para época en rango(epochs)
    modelo.train()
    para imágenes, etiquetas en dataloader_train
        salida ← modelo(imágenes)
        pérdida ← criterio(salida, etiquetas)
        pérdida.backward()
        optimizador.step()
        optimizador.zero_grad()

    modelo.eval()
    sin_gradientes
        para imágenes_val, etiquetas_val en dataloader_val
            salida_val ← modelo(imágenes_val)
            calcular métricas (accuracy, pérdida, etc.)

    aplicar early stopping si es necesario

fin_para

retornar modelo_entrenado
```

Entregable: Modelo entrenado con pesos ajustados y registro de métricas de entrenamiento y validación por época.

- **Procedimiento 4: Evaluación del desempeño de los modelos**

Una vez finalizado el entrenamiento de los modelos, se procede a evaluar su rendimiento utilizando el conjunto de prueba. El objetivo de esta actividad es cuantificar la capacidad de generalización de cada arquitectura mediante el cálculo de métricas estándar de clasificación binaria.

Las métricas empleadas son:

- Accuracy (Exactitud): proporción de predicciones correctas entre el total de muestras.

$$Accuracy = \frac{(TP + TN)}{(TP + TN + FP + FN)}$$

- Precision (Precisión): mide la proporción de verdaderos positivos entre los predichos como positivos.

$$Precision = \frac{TP}{(TP + FP)}$$

- Recall (Sensibilidad): evalúa cuántos de los positivos reales fueron correctamente identificados.

$$Recall = \frac{TP}{(TP + FN)}$$

- F1-score: promedio armónico entre precisión y Recall.

$$F1 = 2 * \frac{(Precision * Recall)}{(Precision + Recall)}$$

- ROC AUC (Área bajo la curva ROC): mide la capacidad del modelo para discriminar entre clases.

Estas métricas permiten realizar una comparación objetiva entre los distintos modelos evaluados. Además, se generan representaciones gráficas como la matriz de confusión y la curva ROC, que permiten observar visualmente el comportamiento de los modelos frente a falsos positivos y negativos.

Figura 53

Pseudocódigo de la evaluación del modelo de Deep Learning

```
Algoritmo evaluar_modelo_DL

Entradas:
    - modelo_entrenado       # Modelo ya entrenado
    - dataloader_test        # Conjunto de prueba
    - umbral_predicción      # Umbral para clasificación

Salidas:
    - métricas de evaluación: Accuracy, Precision, Recall, F1-score, ROC AUC
    - gráficas: matriz de confusión, curva ROC

Inicio:

1. Inicializar listas vacías: y_pred, y_true, y_probabilidades

2. Para cada batch (X, y) en dataloader_test hacer:
      a. Predecir las probabilidades con el modelo: y_hat = modelo(X)
      b. Aplicar función softmax o sigmoid según el tipo de salida
      c. Obtener etiquetas predichas:
            - Si clasificación binaria: y_pred_batch = y_hat > umbral
            - Si softmax: y_pred_batch = argmax(y_hat)
      d. Almacenar y_pred_batch, y, y_hat en sus respectivas listas

3. Concatenar todas las predicciones: y_pred, y_true, y_probabilidades

4. Calcular métricas:
      - Accuracy = exactitud de predicciones
      - Precision = TP / (TP + FP)
      - Recall = TP / (TP + FN)
      - F1-score = 2 * (Precision * Recall) / (Precision + Recall)
      - ROC AUC = área bajo la curva ROC

5. Generar gráficas:
      - Matriz de confusión
      - Curva ROC
      - Curva loss y accuracy

Fin
```

Entregable: Reporte con métricas de desempeño del modelo sobre el conjunto de prueba, acompañado de visualizaciones como la matriz de confusión, la curva ROC y las curvas de pérdida y accuracy.

- **Procedimiento 5: Guardar los modelos y resultados**

Con el objetivo de preservar el trabajo realizado y permitir su reutilización en fases posteriores del estudio o en futuras implementaciones, en esta etapa se procede al almacenamiento de los modelos entrenados junto con sus respectivos resultados.

Para cada arquitectura, se guarda el modelo final entrenado en formato .pth, lo que incluye los pesos ajustados tras el proceso de entrenamiento. Asimismo, se exportan los resultados obtenidos en archivos estructurados (como .csv), los cuales contienen las métricas de desempeño calculadas sobre el conjunto de prueba.

Entregables: Archivo del modelo entrenado en formato .pth, archivo .csv con métricas de desempeño y log de identificación del experimento (modelo y timestamp).

- **Metodología 2 (Modelo Híbrido basado en Vision Transformer y Machine Learning)**

En esta segunda metodología, se implementa un enfoque híbrido que combina arquitecturas de Vision Transformer con algoritmos tradicionales de Machine Learning. El propósito es aprovechar la capacidad de representación de los modelos preentrenados para extraer características relevantes a partir de las imágenes, las cuales luego son utilizadas como vectores de entrada para clasificadores clásicos. Esta estrategia permite evaluar el rendimiento de modelos computacionalmente más eficientes, manteniendo la riqueza informativa proporcionada por las redes neuronales profundas.

Tabla 14

Procedimientos en la fase modelado y clasificación de la Metodología 2

Procedimientos	Descripción	Tareas
1. Obtener vectores característicos desde modelos de Vision Transformer	A partir de cada imagen del conjunto de datos, se obtiene un vector de características intermedias mediante la extracción desde una capa profunda del modelo entrenado de Deep Learning.	• Cargar los modelos entrenados de Deep Learning (BOTNet, ViT, Swin, PiT, MobileViT). • Aplicar operaciones sobre las imágenes para extraer sus vectores característicos. • Generar matrices de características para los subconjuntos de entrenamiento, validación y prueba.
2. Asignar etiquetas a los vectores	Se asigna una etiqueta binaria (1 = anémico, 0 = no anémico) a cada vector	• Asociar cada vector con su clase verdadera según

	característico, basada en la clase de la imagen original.	el conjunto (train, val, test). • Construir estructuras tipo matriz donde cada fila es un vector con su etiqueta.
3. Entrenar y evaluar clasificadores de Machine Learning	Se utilizan los vectores característicos como entrada para modelos clásicos de Machine Learning, los cuales son entrenados, evaluados y comparados.	• Entrenar modelos: SVM, Random Forest, K-NN, Naïve Bayes, Decision Tree. • Evaluar rendimiento sobre el conjunto de prueba. • Calcular métricas: Accuracy, Precision, Recall, F1-score, AUC.

- **Procedimiento 1: Obtener los vectores característicos desde modelos de Vision Transformer**

En este primer paso, se aprovechan los modelos de Deep Learning previamente entrenados (como BOTNet, Vision Transformer, Swin Transformer, PiT y MobileViT) para extraer representaciones numéricas de las imágenes de palmas. En lugar de utilizar la salida final del modelo, se toma la activación de la penúltima capa —como el token [CLS] en modelos tipo Transformer o la última capa densa antes de la clasificación en redes convolucionales—. Estos vectores capturan patrones complejos y relevantes del contenido visual de las imágenes.

Dicha extracción se realiza para cada imagen perteneciente a los subconjuntos de entrenamiento (train), validación (val) y prueba (test), generando una representación vectorial por imagen que posteriormente será utilizada por los algoritmos de Machine Learning.Entregable: Tres conjuntos de vectores estructurados (train, val, test)

correspondientes a la salida de las penúltimas capas de los modelos Aprendizaje Profundo, almacenados en archivos tabulares (CSV o NumPy arrays).

- **Procedimiento 2: Asignar etiquetas (target) a los vectores característicos**

Cada vector generado a partir de las imágenes debe ser vinculado con su etiqueta correspondiente, según el diagnóstico clínico original. Se asigna la clase 1 si el sujeto presenta anemia y 0 si no presenta la condición. Esta asociación se realiza usando los identificadores o nombres de archivo, garantizando la correspondencia entre imagen, vector y clase.

Entregable: Vectores etiquetados por subconjunto (train, val, test), listos para ser usados en entrenamiento.

- **Procedimiento 3: Aplicar modelos de Machine Learning**

Una vez que los vectores han sido generados y etiquetados, se entrenan modelos de clasificación tradicionales utilizando estos datos. Los algoritmos aplicados incluyen Support Vector Machine, Random Forest, k-Nearest Neighbors, Naïve Bayes y Decision Tree, utilizando únicamente los vectores como entrada. Esta etapa no considera la extracción directa de imágenes, sino solo los vectores previamente generados.El flujo de implementación incluye:

Entregable: Modelos de clasificación entrenados para cada algoritmo aplicado.

3.5.4 Evaluación de Modelos

Para examinar el desempeño de los modelos de aprendizaje automático en la detección de anemia, se aplicarán diversas métricas fundamentadas en la matriz de confusión, tomando como referencia las definiciones establecidas por Kelleher et al. (2015) y adecuándolas a la presente investigación. En este contexto, los valores de la matriz de confusión se interpretan de la siguiente manera:

- TP (Verdaderos Positivos): Ocurre cuando el modelo identifica correctamente a un paciente con anemia.
- FP (Falsos Positivos): Ocurre cuando el modelo asigna de manera errónea la condición de anemia a un paciente que, en realidad, no la padece.
- FN (Falsos Negativos): Ocurre cuando el modelo clasifica erróneamente a un paciente como no anémico, a pesar de que en realidad sí padece anemia.
- TN (Verdaderos Negativos): Ocurre cuando el modelo reconoce de manera precisa que un paciente no presenta anemia.

A partir de estos valores, se calculan los siguientes indicadores de rendimiento:

a) Accuracy

El Accuracy mide el porcentaje de predicciones correctas realizadas por el modelo, tanto para pacientes con anemia como para pacientes sin anemia. Su determina de la siguiente forma:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Un valor alto de Accuracy indica que el modelo tiene un buen desempeño general en la clasificación de anemia y no anemia. Sin embargo, esta métrica puede ser engañosa si las clases están desbalanceadas.

b) Recall

El Recall, o sensibilidad, mide la capacidad del modelo para reconocer de manera efectiva los casos de anemia, determinando qué porcentaje de pacientes anémicos han sido correctamente detectados. Se calcula con la siguiente fórmula:

$$Recall = \frac{TP}{TP + FN}$$

Un Recall alto es crucial en aplicaciones médicas, ya que un bajo valor significaría que el modelo no está detectando una cantidad significativa de pacientes con anemia, lo cual puede generar consecuencias clínicas negativas.

c) Precisión

La Precisión cuantifica la proporción de pacientes correctamente identificados como anémicos en comparación con el total de casos que el modelo ha clasificado como anémicos. Su cálculo se expresa mediante la siguiente fórmula:

$$Precision = \frac{TP}{TP + FP}$$

Un valor alto de Precisión significa que la mayoría de los pacientes detectados como anémicos realmente lo son. Esto es especialmente importante en escenarios donde los falsos positivos deben minimizarse, como en el diagnóstico clínico, para evitar exámenes innecesarios en pacientes sanos.

d) F1-Score

El F1-Score es un indicador que integra Precisión y Recall, proporcionando un balance entre ambas mediante su media armónica. Se calcula con la siguiente fórmula:

$$F1 - Score = \frac{2\ x\ Precision\ x\ Recall}{Precision + Recall}$$

Este indicador resulta especialmente valioso cuando se busca mantener un balance entre la capacidad del modelo para identificar correctamente los casos positivos (Recall) y la exactitud de sus predicciones (Precisión).

e) AUC (Área bajo la curva ROC)

Es un indicador que permite medir la eficacia de un modelo de clasificación. Se calcula a partir de la curva ROC, la cual describe la relación entre el Recall y la proporción de falsos positivos (1 - Especificidad). Su rango de valores oscila entre 0 y 1, siendo un AUC cercano a 1 indicativo de un modelo con alta precisión.

Un AUC superior a 0.85 se considera indicativo de un modelo con buena capacidad de discriminación entre pacientes con anemia y pacientes sanos.

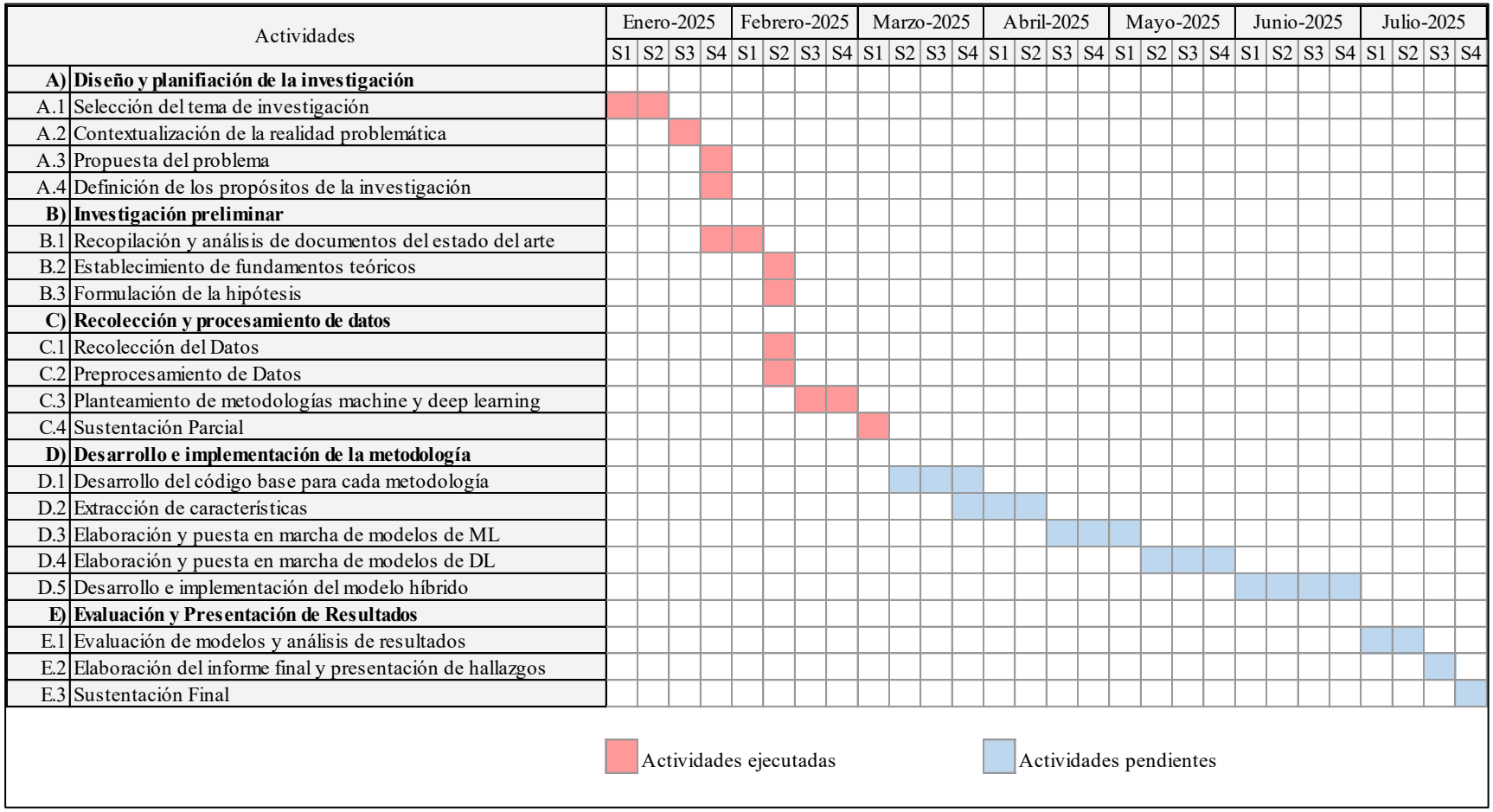
3.5.5 Implementación Web

Como parte de la validación práctica del sistema propuesto, se desarrollará una aplicación web funcional que permita cargar imágenes de la palma de la mano para obtener una predicción automática sobre la presencia de anemia. Esta implementación simulará un entorno clínico real, facilitando el acceso remoto al sistema de diagnóstico y demostrando su potencial aplicabilidad en contextos no invasivos asistidos por inteligencia artificial. La interfaz estará diseñada para ser sencilla e intuitiva, permitiendo a cualquier usuario cargar una imagen y recibir en segundos una respuesta generada por el modelo entrenado.

3.6 Cronograma de Actividades de la Investigación

Figura 54

Cronograma de actividades



3.7 Cronograma de Actividades de la Investigación

APP WEB (FRONTEND)		
Tarea	**Descripción**	**Costo**
ReactJS + Material UI	Desarrollo de aplicación web que consta de: 4 pantallas (Login, Home, Register y Detail).	S/ 1,500.00
Firebase	Integración de Frontend con los servicios de autenticación, base de datos y almacenamiento de Firebase.	S/ -
QA	Validación de happy path y flujos alternos antes de pase a producción.	S/ 500.00
TOTAL:		**S/ 2,000.00**

API (BACKEND)		
Tarea	**Descripción**	**Costo**
API REST	Desarrollo de API Rest con Pyhton y FastAPI, el cual recibirá de entrada la URL generada en el Frontend y será enviada al modelo predictivo.	S/ 1,000.00
Docker	Dockerizar la API Rest para que sea desplegada en los servicios "serverless" de Google Cloud.	S/ 300.00
Cloud Run	Desplegar Docker en Cloud Run para que la api esté disponible en línea y pueda ser consumida por el Frontend.	S/ -
TOTAL:		**S/ 1,300.00**

PYTHON - MODELADO		
Tarea	**Descripción**	**Costo**
Colab Pro+	Desarrollo de API Rest con Pyhton y FastAPI, el cual recibirá de entrada la URL generada en el Frontend y será enviada al modelo predictivo.	$ 49.00
TOTAL:		$ 49.00

CAPÍTULO IV: Desarrollo de la solución

El contenido de este capítulo describe las etapas de desarrollo de la solución, incluyendo la obtención de datos, el preprocesamiento, el modelado y la evaluación de modelos.

4.1 Obtención del dataset

4.1.1 Identificación del dataset

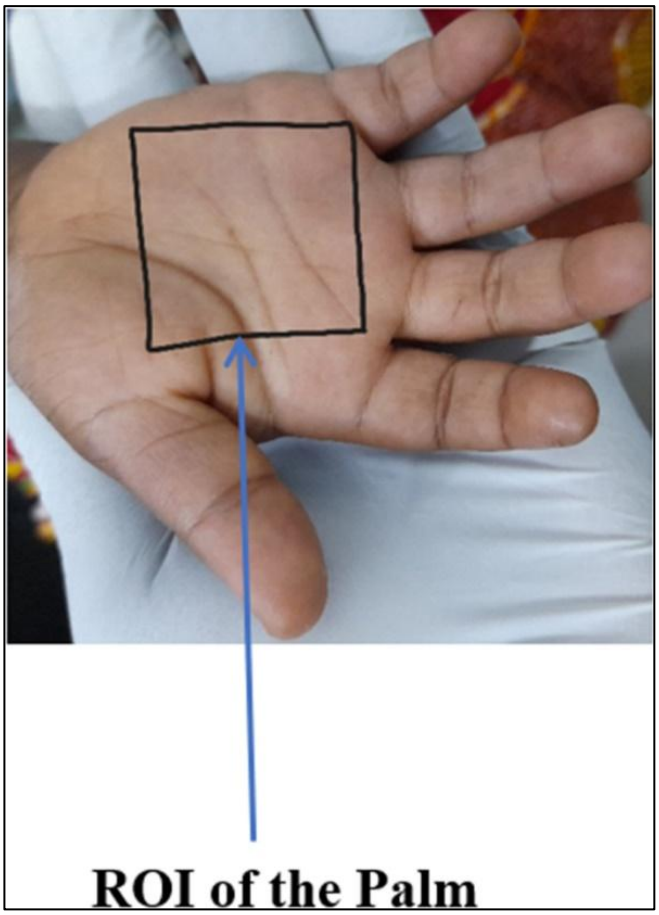
El conjunto de datos utilizado en el presente estudio fue obtenido del repositorio en línea "Mendeley Data", bajo el título "Anemia Detection using Palpable Palm Image Datasets from Ghana". Según lo descrito por los autores Asare et al. (2022), el dataset está compuesto por imágenes fotográficas de palmas de manos de niños entre 0 y 5 años de edad, capturadas en hospitales de Ghana. Estas imágenes fueron recolectadas por un equipo de profesionales en salud e investigadores, quienes utilizaron cámaras estándar de alta calidad, con una resolución mínima de 12 megapíxeles.

Asimismo, los autores detallan que, dado que los participantes eran menores de edad y no podían extender completamente la palma, los investigadores sujetaron las puntas de los dedos y la muñeca para exponer adecuadamente la superficie palmar. Además, se indica que se obtuvo el consentimiento informado de los padres o tutores legales antes de la recolección de los datos, cumpliendo con los principios éticos y las normativas vigentes en investigación.

En una publicación posterior, los mismos autores señalaron que, tras la recolección de las imágenes, se aplicó un enfoque de umbralización triangular con el objetivo de extraer características relevantes de la palma y delimitar la Región de Interés (ROI), como se aprecia en la Figura 55 (Appiahene, Asare, et al., 2023).

Figura 55

ROI de la Palma de la mano



Nota: Extraído de *Application of ensemble models approach in anemia detection using images of the palpable palm* por Appiahene, Asare, et al. (2023)

Adicionalmente, indicaron que el dataset original contenía 527 imágenes, número que fue incrementado mediante la aplicación de técnicas de aumento de datos (image augmentation), tales como rotación (rotation), inversión (flipping) y traslación (translation), aplicadas a las ROI generadas (Appiahene, Asare, et al., 2023).

4.1.2 Descarga del dataset

En el presente trabajo de investigación, el conjunto de datos fue descargado en formato comprimido ".rar" con el nombre de "Palm.rar" con un peso de 241 MB desde el mencionado repositorio. El archivo fue descomprimido, obteniéndose 4,260 archivos en formato ".png", correspondientes a imágenes fotográficas ya recortadas, consistentes con el procedimiento descrito por los autores para la delimitación de la ROI. Estas imágenes están identificadas

mediante nomenclatura específica que específica que indica la clase correspondiente: 2,562 imágenes anémicas (ej. “Anemic-268”) y 1,698 imágenes no anémicas. A continuación, en la Figura 56 se muestra un ejemplo de una imagen clasificada como anémica dentro del dataset.

Figura 56

Imagen fotográfica encontrada en el dataset de título “Anemic-268”



Nota: Extraído del enlace del dataset titulado *Anemia Detection using Palpable Palm Image Datasets from Ghana*, de los autores (Asare et al., 2022)

Finalmente, una vez descomprimido el dataset, este fue subido a Google Drive, a fin de facilitar su integración y uso en el entorno de desarrollo Google Colab, donde se desarrollaron las siguientes etapas de preprocesamiento, análisis y modelado.

4.2 Preprocesamiento de datos

El preprocesamiento de datos es una etapa fundamental en este proyecto, ya que permitió optimizar la calidad, consistencia y representatividad del conjunto de datos antes de su uso en las etapas de modelado y análisis. A pesar de que el dataset original descargado de

Mendeley Data incluía imágenes recortadas y algunas generadas mediante técnicas de Data Augmentation, dichas imágenes sintéticas fueron descartadas al inicio del proceso, conservándose únicamente las imágenes originales. A partir de este conjunto base depurado, se implementaron una serie de pasos adicionales para verificar la integridad de los archivos, eliminar imágenes defectuosas, estandarizar los nombres, homogeneizar las dimensiones y organizar las clases. Posteriormente, se aplicaron nuevas técnicas de aumento de datos sobre las imágenes originales, con el fin de incrementar la diversidad del conjunto y balancear la cantidad de datos por clase mediante transformaciones como rotación, volteo y ajustes de color. Finalmente, el conjunto completo fue dividido en tres subconjuntos independientes: entrenamiento, validación y prueba, respetando la proporción de 70%, 10% y 20%, respectivamente, y asegurando una distribución equilibrada entre clases en cada uno de ellos. Cabe indicar que para todas las etapas de preprocesamiento se utilizó Google Colab como entorno de trabajo, aprovechando su compatibilidad con Python, sus librerías especializadas en procesamiento de imágenes y la ejecución en la nube, lo que facilitó una mayor eficiencia computacional y manipulación remota de los archivos alojados en Google Drive.

A continuación, se detallan las subfases del proceso de preprocesamiento aplicado:

4.2.1 Detección y filtrado

Como primera actividad el preprocesamiento, se desarrolló un script en Python que permitió realizar la detección automatizada de imágenes generadas mediante técnicas previas de data augmentation. Este procedimiento se basó en el análisis de los nombres de archivo, aplicando una expresión regular diseñada para identificar patrones característicos de duplicación automática, como la presencia de números entre paréntesis. La Figura 57 muestra el código utilizado en esta etapa, el cual facilitó la identificación precisa de las imágenes originales frente a aquellas generadas artificialmente.

Figura 57

Script en Python para la detección automática de imágenes aumentadas

```
import os
import re
import shutil

# Ruta del dataset original
source_folder = "/content/drive/MyDrive/TESIS/1.DATASET"

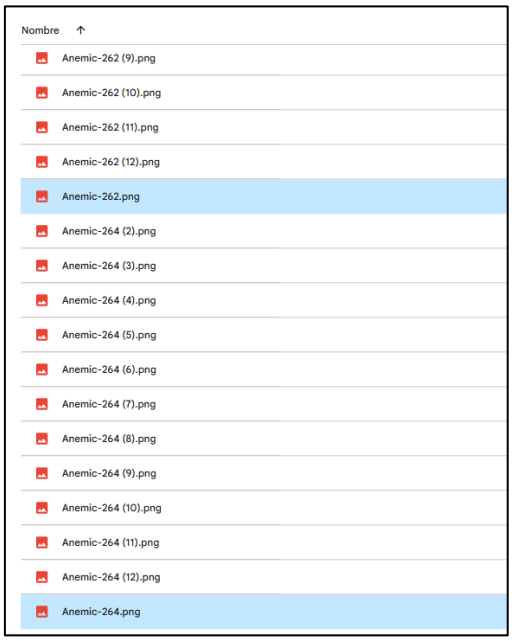
# Expresión regular para detectar archivos con números entre paréntesis (indicando que fueron aumentados)
patron = re.compile(r"\(\d+\)")
```

Nota: Elaboración propia

El resultado de este proceso permitió distinguir con precisión las imágenes originales (por ejemplo, Anemic-262.png y Anemic-264.png) de aquellas aumentadas (por ejemplo, Anemic-262(9).png, Anemic-264(png), como se muestra en la Figura 58.

Figura 58

Nombres de imágenes originales y aumentadas



Nota. Elaboración Propia

A partir de la detección, se procedió a ejecutar un segundo script complementario orientado al filtrado efectivo del conjunto, copiando únicamente las imágenes originales a una

nueva carpeta de destino. Esta acción permitió conformar un subconjunto limpio y sin duplicaciones sintéticas, eliminando cualquier posible sesgo relacionado con imágenes generadas artificialmente. El código utilizado para esta tarea se presenta en la Figura 59.

Figura 59

Script en Python para el filtrado, copia y resumen de imágenes originales

```
# Ruta del destino final
destination_folder = "/content/drive/MyDrive/TESIS/1.1.DATASET_ELIM_AUM"

# Crear la carpeta destino si no existe
if not os.path.exists(destination_folder):
    os.makedirs(destination_folder)

# Obtener la lista de archivos en el dataset original
archivos = [archivo for archivo in os.listdir(source_folder) if os.path.isfile(os.path.join(source_folder, archivo))]

# Separar los archivos aumentados de los originales
archivos_augmented = [archivo for archivo in archivos if patron.search(archivo)]
archivos_originales = [archivo for archivo in archivos if archivo not in archivos_augmented]

# Copiar los archivos originales a la carpeta de destino
for archivo in archivos_originales:
    shutil.copy(os.path.join(source_folder, archivo), os.path.join(destination_folder, archivo))

# Mostrar información sobre los archivos movidos
print("Archivos originales copiados:")
for archivo in archivos_originales:
    print(archivo)

print(f"\nTotal de archivos originales copiados: {len(archivos_originales)}")
print(f"Total de archivos generados por image augmentation (omitidos): {len(archivos_augmented)}")
print(f"Total de archivos en la carpeta original: {len(archivos)}")
```

Nota. Elaboración Propia

Como parte del proceso, se generó un resumen que documentó con precisión la cantidad de archivos procesados en cada categoría. En total, se identificaron 4,260 archivos en la carpeta original: 3,734 de ellos correspondían a imágenes generadas mediante técnicas de image augmentation, mientras que 526 fueron reconocidas como imágenes originales. Estas últimas fueron copiadas a una carpeta exclusiva con la que se trabajará en las siguientes subfases del preprocesamiento.

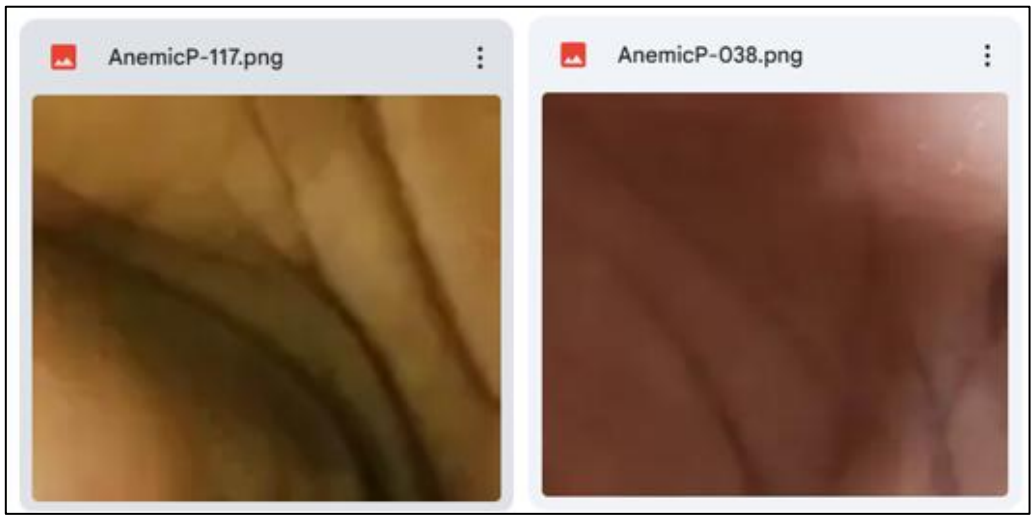
4.2.2 Verificación y depuración

En esta etapa, se examinó cuidadosamente la calidad visual de las imágenes contenidas en el dataset con el objetivo de identificar aquellas que presentaran problemas de enfoque, nitidez insuficiente o recortes excesivos que impidieran una correcta visualización de los pliegues de las palmas de las manos. Esta verificación fue fundamental para asegurar que el conjunto de datos final estuviera compuesto exclusivamente por imágenes útiles, claras y representativas para los procesos de análisis y entrenamiento de modelos de aprendizaje automático.

Durante esta revisión, se detectaron diversas anomalías en las imágenes. Algunas de ellas presentaban sombras oscuras que interferían en la visualización de los detalles de la palma, tal como se observa en la Figura 60.

Figura 60

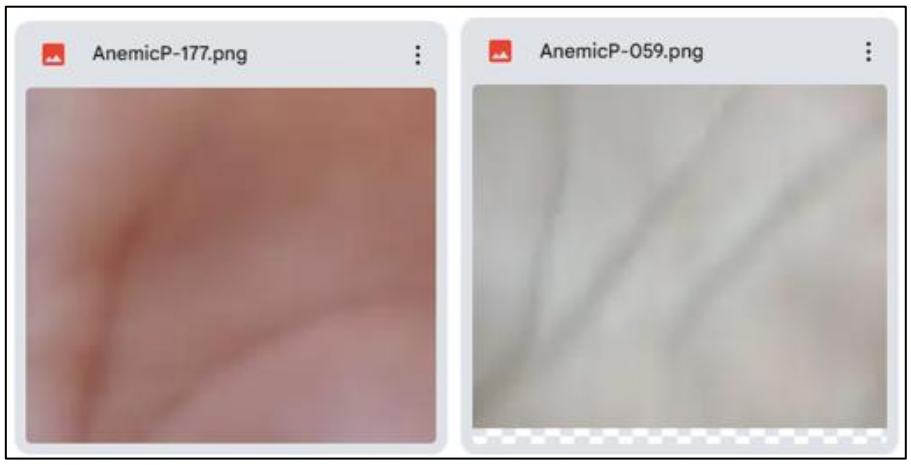
Imágenes detectadas con sombras oscuras



Asimismo, se identificaron imágenes con baja resolución o falta de nitidez, donde los pliegues y detalles característicos de la superficie palmar no podían apreciarse correctamente, lo cual limitaba su utilidad para el análisis. Ejemplos de estas deficiencias se muestran en la Figura 61.

Figura 61

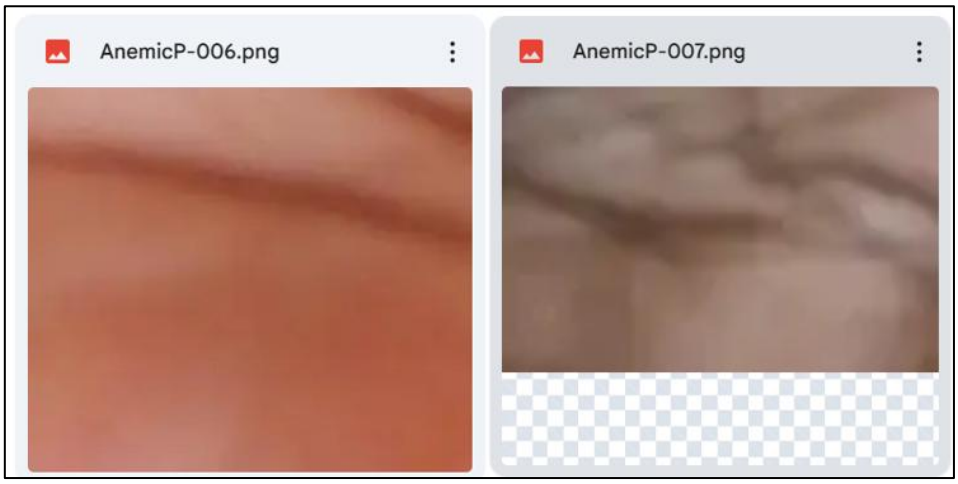
Imágenes con baja resolución o nitidez



Finalmente, se encontraron imágenes que habían sido recortadas de forma excesiva, eliminando zonas importantes de la palma de la mano e impidiendo la identificación de los patrones visuales necesarios para el análisis. La Figura 62 ilustra algunos casos representativos de este problema.

Figura 62

Imágenes detectadas con recorte excesivo



Las imágenes que presentaron estas deficiencias fueron eliminadas del conjunto de datos, y sus nombres fueron registrados en un archivo de respaldo, conservado para asegurar la

trazabilidad del proceso de depuración. Esta acción garantizó que solo se conservaran archivos con calidad adecuada para las siguientes etapas del preprocesamiento.

Como resultado de este proceso, el conjunto de datos quedó conformado por un total de 295 imágenes originales depuradas: 127 correspondientes a la clase no anémica y 168 a la clase anémica.

4.2.3 Organización por clase

En esta etapa, se realizó el análisis de la distribución de las imágenes contenidas en el dataset, identificando las clases presentes a partir de los prefijos en los nombres de archivo y contabilizando el número de imágenes correspondientes a cada una. Este análisis fue esencial para evidenciar inconsistencias en la nomenclatura y preparar la estructura del conjunto de datos para su uso en fases posteriores.

Mediante un script en Python (ver Figura 63), se recorrieron todas las carpetas del proyecto, identificando archivos válidos por extensión y extrayendo automáticamente los prefijos mediante una expresión regular. Esto permitió obtener un conteo detallado de imágenes por clase, con los siguientes resultados:

- anemicp-: 198 imágenes
- non-anemicp-: 140 imágenes
- anemicp: 2 imágenes
- anemic-: 7 imágenes
- anemic-pa-: 22 imágenes
- non-anemic-pa-: 18 imágenes
- Total de imágenes clasificadas: 387

Figura 63

Script para identificación y conteo de imágenes por clase

```
import os
import re
from collections import defaultdict

# Ruta principal del proyecto
ruta_inicial = '/content/drive/MyDrive/TESIS/100.DATASET526_MANUAL'

# Contador de clases detectadas automáticamente
conteo_por_clase = defaultdict(int)
total_imagenes = 0

# Extensiones válidas
extensiones = ('.png', '.jpg', '.jpeg', '.bmp', '.tif', '.tiff')

# Recorrer todas las carpetas y archivos
for raiz, dirs, archivos in os.walk(ruta_inicial):
    for archivo in archivos:
        if archivo.lower().endswith(extensiones):
            nombre = archivo.lower()
            # Extraer prefijo hasta el primer paréntesis o espacio o número
            match = re.match(r'^([a-z\-]+)', nombre)
            if match:
                clase = match.group(1)
                conteo_por_clase[clase] += 1
                total_imagenes += 1

# Mostrar resultados
print("Conteo de imágenes por clase detectada:")
for clase, cantidad in conteo_por_clase.items():
    print(f'  {clase}: {cantidad} imágenes')
print(f'Total de imágenes clasificadas: {total_imagenes}')
```

Nota. Elaboración Propia

Estos resultados evidenciaron una variabilidad en los prefijos empleados para representar las clases, lo cual motivó la necesidad de estandarizar los nombres de archivo y organizar las imágenes en carpetas específicas por clase.

Posteriormente, se aplicó un segundo script, como se muestra en la Figura 64, que detectó automáticamente los diferentes prefijos asociados a las categorías “anemic” y “non-anemic”, generando nuevos nombres bajo una estructura estandarizada (anemic_0001.png, non_anemic_0001.png) y distribuyendo las imágenes en carpetas separadas para cada clase.

Al finalizar este proceso, se obtuvo la siguiente distribución final del dataset:

- Anemic: 229 imágenes
- Non-anemic: 158 imágenes

Figura 64

Script para el renombramiento y organización de imágenes por clase

```python
import os
import re
import shutil

# Rutas
origen = '/content/drive/MyDrive/TESIS/100.DATASET526_MANUAL'
destino_base = '/content/drive/MyDrive/TESIS/200.DATASET526_RENAME_MANUAL'

# Diccionario de clases y sus prefijos asociados
clases = {
    'anemic': ['anemicp-', 'anemic-', 'anemicp', 'anemic-pa-'],
    'non-anemic': ['non-anemicp-', 'non-anemic-pa-']
}

# Inicializar contadores para nombres nuevos
contador = {'anemic': 1, 'non-anemic': 1}

# Crear carpetas destino si no existen
for clase in clases:
    os.makedirs(os.path.join(destino_base, clase), exist_ok=True)

# Recorrer todas las carpetas y archivos
for raiz, dirs, archivos in os.walk(origen):
    for archivo in archivos:
        if archivo.lower().endswith(('.png', '.jpg', '.jpeg', '.bmp', '.tif', '.tiff')):
            archivo_lower = archivo.lower()
            for clase, prefijos in clases.items():
                if any(archivo_lower.startswith(prefijo) for prefijo in prefijos):
                    # Crear nombre nuevo estandarizado
                    nuevo_nombre = f"{clase.replace('-', '_')}_{contador[clase]:04d}.png"
                    # Ruta completa origen y destino
                    origen_completo = os.path.join(raiz, archivo)
                    destino_completo = os.path.join(destino_base, clase, nuevo_nombre)
                    # Copiar imagen renombrada
                    shutil.copy2(origen_completo, destino_completo)
                    # Incrementar contador
                    contador[clase] += 1
                    break  # Ya clasificado, salir del bucle

# Contar archivos en cada carpeta
for clase in clases:
    carpeta = os.path.join(destino_base, clase)
    cantidad_archivos = len([archivo for archivo in os.listdir(carpeta) if os.path.isfile(os.path.join(carpeta, archivo))])
    print(f"Total de archivos en '{clase}': {cantidad_archivos}")
```

Nota. Elaboración Propia

Este resultado consolidó un conjunto de datos organizado, con nombres consistentes y carpetas claramente diferenciadas, lo que garantiza su correcta interpretación y manipulación en las siguientes fases de preprocesamiento y entrenamiento de modelos.

4.2.4 Redimensionamiento de imágenes

El redimensionamiento de imágenes es un paso esencial en el preprocesamiento, ya que permite unificar las dimensiones de todas las muestras en el conjunto de datos, asegurando así la compatibilidad con los modelos de aprendizaje automático. Antes de proceder a esta tarea, fue necesario realizar una revisión detallada de las resoluciones originales de las imágenes. Para

ello, se implementó un script en Python que recorrió las carpetas del dataset, extrajo las dimensiones (ancho y alto) de cada archivo y contabilizó las resoluciones presentes por clase. Este análisis permitió identificar la diversidad de tamaños en las imágenes y fundamentó la necesidad de estandarizarlas. El código desarrollado para esta verificación se muestra en la Figura 65.

Figura 65

Script para la verificación de resoluciones originales de las imágenes

```python
#Revisamos tamaño originales
import os
from PIL import Image
from collections import defaultdict

# Ruta base
destino_base = '/content/drive/MyDrive/TESIS/200.DATASET526_RENAME_MANUAL'

# Clases
clases = ['anemic', 'non-anemic']

# Diccionario para almacenar resoluciones
resoluciones_por_carpeta = defaultdict(list)

# Recorremos cada carpeta
for clase in clases:
    ruta = os.path.join(destino_base, clase)
    for archivo in os.listdir(ruta):
        if archivo.lower().endswith(('.png', '.jpg', '.jpeg', '.bmp', '.tif', '.tiff')):
            path_imagen = os.path.join(ruta, archivo)
            try:
                with Image.open(path_imagen) as img:
                    resolucion = img.size  # (ancho, alto)
                    resoluciones_por_carpeta[clase].append(resolucion)
            except Exception as e:
                print(f"Error al abrir {path_imagen}: {e}")

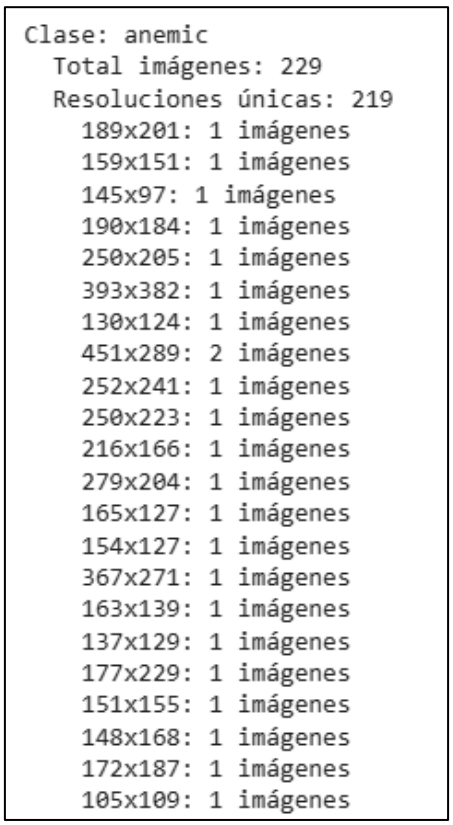
# Mostrar resultados
for clase in resoluciones_por_carpeta:
    resoluciones = resoluciones_por_carpeta[clase]
    total = len(resoluciones)
    resoluciones_unicas = set(resoluciones)
    print(f"  Clase: {clase}")
    print(f"    Total imágenes: {total}")
    print(f"    Resoluciones únicas: {len(resoluciones_unicas)}")
    for res in resoluciones_unicas:
        cantidad = resoluciones.count(res)
        print(f"      {res[0]}x{res[1]}: {cantidad} imágenes")
```

Nota. Elaboración Propia

Al verificar los resultados de este procedimiento, se confirmó que las imágenes presentaban diversas resoluciones, lo cual evidenció la necesidad de redimensionarlas a un tamaño uniforme. Esta diversidad de tamaños se muestra en la Figura 66, donde se presentan ejemplos de las resoluciones detectadas en las diferentes clases.

Figura 66

Resultados de las resoluciones detectadas en las imágenes originales



Nota. Elaboración Propia

Con base en estos resultados, se procedió a redimensionar todas las imágenes a una resolución estándar de 224x224 píxeles, asegurando la consistencia en la entrada de datos para los modelos de clasificación. Para esta tarea, se desarrolló un segundo script en Python que recorrió las carpetas de origen, redimensionó cada imagen a las dimensiones especificadas y las guardó en una nueva carpeta organizada por clases. Este procedimiento fue fundamental para

mantener la integridad de las etiquetas de clase durante el proceso de transformación. La estructura del código utilizado para esta tarea se presenta en la Figura 67.

Figura 67

Script para el redimensionamiento de imágenes a 224x224 píxeles

```
# Ruta original y de destino
ruta_origen = '/content/drive/MyDrive/TESIS/200.DATASET526_RENAME_MANUAL'
ruta_destino = '/content/drive/MyDrive/TESIS/300.DATASET526_224_MANUAL'

# Clases
clases = ['anemic', 'non-anemic']

# Crear carpetas de destino si no existen
for clase in clases:
    os.makedirs(os.path.join(ruta_destino, clase), exist_ok=True)

# Redimensionar imágenes a 224x224
for clase in clases:
    carpeta_origen = os.path.join(ruta_origen, clase)
    carpeta_destino = os.path.join(ruta_destino, clase)
    for archivo in os.listdir(carpeta_origen):
        if archivo.lower().endswith(('.png', '.jpg', '.jpeg', '.bmp', '.tif', '.tiff')):
            path_origen = os.path.join(carpeta_origen, archivo)
            path_destino = os.path.join(carpeta_destino, archivo)
            try:
                with Image.open(path_origen) as img:
                    img = img.resize((224, 224))
                    img.save(path_destino)
            except Exception as e:
                print(f"Error al procesar {archivo}: {e}")
```

Nota. Elaboración Propia

Finalmente, se llevó a cabo una verificación del conjunto de datos redimensionado para confirmar que todas las imágenes cumplieran con las dimensiones establecidas. Para ello, se utilizó un script adicional que recorrió las carpetas de destino, contabilizó las resoluciones únicas encontradas y generó un resumen de las dimensiones finales por clase. Este paso aseguró la calidad del preprocesamiento y garantizó la homogeneidad del conjunto de datos. El código desarrollado para esta verificación final se presenta en la Figura 68.

Figura 68

Script para la verificación de resoluciones después del redimensionamiento

```
# ================================
# Verificar resoluciones después del redimensionamiento
# ================================

# Diccionario para almacenar resoluciones
resoluciones_por_carpeta = defaultdict(list)

# Recorremos cada carpeta en la ruta redimensionada
for clase in clases:
    ruta = os.path.join(ruta_destino, clase)
    for archivo in os.listdir(ruta):
        if archivo.lower().endswith(('.png', '.jpg', '.jpeg', '.bmp', '.tif', '.tiff')):
            path_imagen = os.path.join(ruta, archivo)
            try:
                with Image.open(path_imagen) as img:
                    resolucion = img.size
                    resoluciones_por_carpeta[clase].append(resolucion)
            except Exception as e:
                print(f"Error al abrir {path_imagen}: {e}")

# Mostrar resultados
for clase in resoluciones_por_carpeta:
    resoluciones = resoluciones_por_carpeta[clase]
    total = len(resoluciones)
    resoluciones_unicas = set(resoluciones)
    print(f"\nClase: {clase}")
    print(f"  Total imágenes: {total}")
    print(f"  Resoluciones únicas: {len(resoluciones_unicas)}")
    for res in resoluciones_unicas:
        cantidad = resoluciones.count(res)
        print(f"    {res[0]}x{res[1]}: {cantidad} imágenes")
```

Nota. Elaboración Propia

El resumen final de las resoluciones por clase evidenció que todas las imágenes habían sido correctamente estandarizadas a las dimensiones establecidas. Para la clase **anemic**, se registraron 229 imágenes, todas con una resolución uniforme de 224x224 píxeles. En la clase **non-anemic**, se contabilizaron 158 imágenes, también ajustadas a la misma resolución.

Con esta verificación, el conjunto de datos quedó completamente preparado para las etapas posteriores de balanceo y partición en subconjuntos de entrenamiento, prueba y

validación, garantizando la homogeneidad y calidad necesarias para el desarrollo de modelos de clasificación de imágenes.

4.2.5 Aumento y balanceo por clase

Con el fin de asegurar una distribución equilibrada del conjunto de datos, se implementó un procedimiento de data augmentation que permitió incrementar el número de muestras hasta alcanzar un total uniforme de 1000 imágenes por clase. Esta etapa fue esencial para mejorar la generalización de los modelos de clasificación y reducir el riesgo de sobreajuste, especialmente en un contexto de datos limitados.

Este proceso se desarrolló en dos fases complementarios. En primer lugar, se aplicaron transformaciones aleatorias sobre las imágenes originales mediante un conjunto de operaciones que incluían: rotación aleatoria de ± 20°, volteo horizontal, ajuste de color, desenfoque gaussiano y modificación del contraste. Estas técnicas fueron ejecutadas por la función apply_augmentation (), incorporada en un script Python que recorrió los directorios por clase, duplicó imágenes existentes y generó nuevas versiones hasta alcanzar el objetivo de 1000 archivos por categoría. La lógica de este procedimiento se muestra en la Figura 69.

Figura 69

Script para el aumento de datos y generación de imágenes por clase

```
input_dir = base_output_path
output_dir = "/content/drive/MyDrive/2.DATASET_PREPRO"
target_per_class = 1000

def apply_augmentation(image):
    aug_type = random.choice(['rotate', 'flip', 'color', 'blur', 'contrast'])
    if aug_type == 'rotate':
        return image.rotate(random.uniform(-20, 20))
    elif aug_type == 'flip':
        return ImageOps.mirror(image)
    elif aug_type == 'color':
        enhancer = ImageEnhance.Color(image)
        return enhancer.enhance(random.uniform(0.7, 1.5))
    elif aug_type == 'blur':
        return image.filter(ImageFilter.GaussianBlur(radius=1))
    elif aug_type == 'contrast':
        enhancer = ImageEnhance.Contrast(image)
        return enhancer.enhance(random.uniform(0.8, 1.3))
    return image

for clase in ['anemic', 'non-anemic']:
    src_folder = os.path.join(input_dir, clase)
    dst_folder = os.path.join(output_dir, clase)
    os.makedirs(dst_folder, exist_ok=True)

    imagenes = [f for f in os.listdir(src_folder) if f.lower().endswith(('.jpg', '.png', '.jpeg'))]
    originales = []

    for img_name in imagenes:
        ruta_origen = os.path.join(src_folder, img_name)
        ruta_destino = os.path.join(dst_folder, img_name)
        Image.open(ruta_origen).save(ruta_destino)
        originales.append(img_name)

    current_count = len(imagenes)
    extra_needed = target_per_class - current_count
    augment_count = 0

    while augment_count < extra_needed:
        img_name = random.choice(originales)
        img_path = os.path.join(src_folder, img_name)
        img = Image.open(img_path).convert("RGB")
        aug_img = apply_augmentation(img)

        new_name = f"{os.path.splitext(img_name)[0]}_aug{augment_count}.jpg"
        aug_img.save(os.path.join(dst_folder, new_name))
        augment_count += 1

print("✅ ¡Aumento completo! Se guardaron 1,000 imágenes por clase en /2.DATASET_PREPRO")
```

Nota. Elaboración Propia

Durante la ejecución, se conservaron los nombres de los archivos originales y se añadieron sufijos identificadores (_aug0, _aug1, _aug2, etc) a las imágenes aumentadas,

garantizando su trazabilidad. Todas las imágenes resultantes fueron almacenadas en nuevas carpetas por clase dentro del directorio destino previamente definido.

Una vez completada la generación, se procedió a verificar la cantidad total de imágenes generadas por clase mediante un segundo script Python. Este script recorrió las carpetas destino, validó las extensiones de los archivos y generó un resumen numérico por categoría. Su estructura se presenta en la Figura 70.

Figura 70

Script de validación del número total de imágenes por clase después del aumento

```python
# Ruta base del dataset aumentado
dst_base_folder = '/content/drive/MyDrive/2.DATASET_PREPRO'
clases = ['anemic', 'non-anemic']

# Validación final para ambas clases
total_general = 0
for clase in clases:
    dst_folder = os.path.join(dst_base_folder, clase)
    total_final = len([f for f in os.listdir(dst_folder) if f.lower().endswith(('.jpg', '.png', '.jpeg'))])
    print(f"✅ Total final en '{clase}': {total_final} imágenes")
    total_general += total_final

print(f"\n📊 Total general de imágenes procesadas: {total_general}")
print("\n🚀 Aumento completo para ambas clases en:", dst_base_folder)
```

Nota. Elaboración Propia

El proceso concluyó con el siguiente resumen de resultados:

- Total final de clase “anemic”: 1000 imágenes.
- Total final de clase “non-anemic”: 1000 imágenes.
- Total general de imágenes procesadas: 2000 imágenes.

Con este procedimiento, el conjunto de datos final quedó balanceado y adecuadamente expandido, permitiendo su utilización confiable en las siguientes fases de entrenamiento, validación y prueba de modelos de clasificación de imágenes.

4.2.6 División del dataset en subconjuntos

Con el objetivo de preparar el conjunto de datos para el entrenamiento supervisado de modelos de clasificación, se procedió a su división en tres subconjuntos funcionales: entrenamiento 70 %, validación 10 % y prueba 20 %. Esta partición se aplicó de forma independiente para cada clase (anemic y non-anemic), garantizando una distribución balanceada y representativa en cada uno de los bloques generados.

La implementación de esta tarea se realizó mediante un script en Python. En primer lugar, se definieron las rutas de entrada y salida del dataset, así como la estructura de carpetas destino para cada subconjunto y clase, como se muestra en la Figura 71.

Figura 71

Script para la creación de carpetas y conteo inicial de imágenes por clase

```python
import os
import shutil
import random

# Ruta del dataset aumentado
input_dir = "/content/drive/MyDrive/2.DATASET_PREPRO"
output_base = "/content/drive/MyDrive/3.DATASET_PREPRO_SPLIT_712"

# Crear estructura de carpetas destino
for subset in ["train", "val", "test"]:
    for clase in ["anemic", "non-anemic"]:
        os.makedirs(os.path.join(output_base, subset, clase), exist_ok=True)

# Mostrar conteo inicial
for clase in ["anemic", "non-anemic"]:
    total = len(os.listdir(os.path.join(input_dir, clase)))
    print(f"📊 {clase.upper()} - TOTAL INICIALES: {total}")
```

Nota. Elaboración Propia

Posteriormente, se llevó a cabo la lectura de los nombres de los archivos por clase, su aleatorización y posterior distribución proporcional en las carpetas correspondientes, según se muestra en la Figura 72.

Figura 72

Script para la división aleatoria de imágenes y verificación del conteo final

```
# Dividir y copiar imágenes
for clase in ["anemic", "non-anemic"]:
    clase_path = os.path.join(input_dir, clase)
    imagenes = [f for f in os.listdir(clase_path) if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
    random.shuffle(imagenes)

    total = len(imagenes)
    train_split = int(0.7 * total)
    val_split = int(0.1 * total)

    splits = {
        "train": imagenes[:train_split],
        "val": imagenes[train_split:train_split + val_split],
        "test": imagenes[train_split + val_split:]
    }

    for subset, files in splits.items():
        for img in files:
            src = os.path.join(clase_path, img)
            dst = os.path.join(output_base, subset, clase, img)
            shutil.copy2(src, dst)

# Mostrar conteo final
print("\n✅ RESULTADO FINAL:")
for subset in ["train", "val", "test"]:
    print(f"\n📂 {subset.upper()}:")
    for clase in ["anemic", "non-anemic"]:
        ruta = os.path.join(output_base, subset, clase)
        total = len(os.listdir(ruta))
        print(f"  {clase}: {total} imágenes")
```

Nota. Elaboración Propia

Finalizado el proceso, se validó el número de imágenes asignadas por clase a cada subconjunto, obteniéndose la siguiente distribución:

- Entrenamiento (train): 700 imágenes por clase
- Validación (val): 100 imágenes por clase
- Prueba (test): 200 imágenes por clase

Con ello, se alcanzó un total de 2,000 imágenes correctamente organizadas en carpetas diferenciadas por subconjunto y clase, quedando el dataset listo para las fases posteriores de modelado y evaluación.

4.3 Modelado y Clasificación

Una vez finalizada la etapa de preprocesamiento, se procedió al desarrollo del modelado y la clasificación, componente central de esta investigación. El objetivo de esta fase fue evaluar la capacidad predictiva de las arquitecturas basadas exclusivamente en Vision Transformer y sus variantes, aplicadas a la clasificación automática de imágenes de palmas según su condición anémica o no anémica.

Para ello, se implementaron dos metodologías complementarias. La primera se basó en el uso directo de modelos como Vision Transformer (ViT), BOTNet, Swin Transformer, MobileViT y Pooling-based Vision Transformer (PiT), los cuales fueron empleados como clasificadores completos, es decir, responsables tanto de la extracción de características como de la predicción de la clase. La segunda metodología consistió en un enfoque híbrido: estas mismas arquitecturas se utilizaron únicamente como extractores de características, mientras que la clasificación se realizó mediante algoritmos tradicionales de Machine Learning, entre ellos Support Vector Machines (SVM), Random Forest (RF), K-Nearest Neighbors (KNN), Naïve Bayes (NB) y Decision Tree (DT). Esta estrategia permitió contrastar el desempeño de modelos de alta capacidad con clasificadores más livianos sobre las mismas representaciones visuales.

La implementación se llevó a cabo en el entorno de Google Colab, utilizando las librerías PyTorch, timm y scikit-learn, las cuales facilitaron tanto el manejo de modelos preentrenados como la integración de flujos de inferencia y evaluación. A continuación, se describen en detalle los procedimientos realizados en cada una de las metodologías, incluyendo la configuración de los modelos, los parámetros adoptados y la lógica seguida para la clasificación de las imágenes.

- **Desarrollo de la Metodología 1 (Vision Transformer)**

A continuación, se detallará cada procedimiento de desarrollo por cada modelo de Vision Transfomer considerado:

- **BOTNet**

  - **Procedimiento 1: Preparación y normalización de imágenes**

Para garantizar la compatibilidad de entrada con la arquitectura BOTNet, las imágenes fueron transformadas mediante una secuencia de operaciones: redimensionamiento a 256×256 píxeles, conversión a tensores y normalización canal por canal según los valores estándar de ImageNet (media = [0.485, 0.456, 0.406]; desviación estándar = [0.229, 0.224, 0.225]). Esta transformación se implementó utilizando el módulo transforms.Compose() de la librería torchvision, asegurando que cada muestra mantenga una estructura y rango de valores adecuado para el modelo preentrenado. Como se muestra en la Figura 73, este procedimiento fue registrado en la sección de transformaciones del código, permitiendo estandarizar todas las entradas previo al entrenamiento.

Figura 73

Script sobre transformación de Datos BOTNet

```
transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])
```

Nota. Elaboración Propia

  - **Procedimiento 2: Definición de la arquitectura**

La arquitectura utilizada en esta fase fue BOTNet, implementada a través de la librería timm. El modelo se cargó en su versión preentrenada (botnet26t_256) y se

adaptó a una tarea de clasificación binaria especificando num_classes=2. Posteriormente, se transfirió al dispositivo de cómputo disponible (GPU o CPU), utilizando torch.device, lo cual facilitó un entrenamiento eficiente. La estructura de definición se encuentra detallada en la Figura 74, donde se observa la invocación de timm.create_model () junto con los parámetros esenciales para iniciar la etapa de entrenamiento.

Figura 74

Script sobre la carga del modelo BOTNet

```
model = timm.create_model('botnet26t_256', pretrained=True, num_classes=2)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
```

Nota. Elaboración Propia

- **Procedimiento 3: Entrenamiento del modelo**

El proceso de entrenamiento del modelo BOTNet se dividió en dos etapas: una primera sin ajuste de hiperparámetros (modelo base) y una segunda con reentrenamiento utilizando hiperparámetros optimizados (modelo afinado o con tuning). Esta estructura permitió comparar posteriormente el impacto del ajuste fino en el comportamiento del modelo durante la tarea de clasificación.

✓ Entrenamiento Modelo BOTNet base

En la primera etapa, el modelo se entrenó con una configuración estándar. Se utilizó la función de pérdida CrossEntropyLoss() y el optimizador Adam, con una tasa de aprendizaje (learning rate) de 1e-4, sin regularización adicional (weight decay) y con un tamaño de lote (batch size) de 32. Se definió un máximo de 30 épocas de entrenamiento y se implementó un mecanismo de early stopping con paciencia de cinco épocas sin mejora en la pérdida de validación.

El bloque de código correspondiente a esta fase se presenta en dos partes complementarias. En la Figura 75, se muestra la configuración inicial del entrenamiento, incluyendo la definición del optimizador, las variables de control del proceso, y el bucle principal que procesa los datos por lotes, calcula la pérdida y actualiza los pesos del modelo en cada época. Asimismo, esta figura incluye el cálculo de la precisión sobre el conjunto de entrenamiento, lo que permite monitorear el progreso del modelo en tiempo real.

Figura 75

Script de entrenamiento inicial del modelo BOTNet base

```
# Función de pérdida y optimizador
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-4)

# Configuración de entrenamiento
epochs = 30
patience = 5
best_loss = float('inf')
counter = 0

# Listas para almacenar pérdida y accuracy por época
train_losses = []
val_losses = []
train_acc = []  # Accuracy entrenamiento
val_acc = []    # Accuracy validación

# Bucle principal de entrenamiento
for epoch in range(epochs):
    model.train()
    running_loss = 0.0

    # Entrenamiento por batches
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

    # Promedio de pérdida de entrenamiento
    train_losses.append(running_loss / len(train_loader))

    # Calcular accuracy de entrenamiento
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    train_accuracy = correct / total
    train_acc.append(train_accuracy)
```

Nota. Elaboración Propia

Posteriormente, en la Figura 76, se detalla el bloque de validación del modelo, que se ejecuta al final de cada época. Este incluye el cálculo de la pérdida y la precisión

sobre el conjunto de validación, así como la lógica de activación del mecanismo de early stopping. En caso de mejora, se guarda el estado actual del modelo; de lo contrario, se incrementa un contador hasta alcanzar el umbral definido. Finalmente, se restaura el mejor modelo registrado para asegurar que el proceso de entrenamiento concluya con la versión de mayor rendimiento validado.

Figura 76

Script de Validación y early stopping del modelo BOTNet base.

```
    # -------------------------
    # VALIDACIÓN
    # -------------------------
    model.eval()
    val_loss = 0.0
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            val_loss += loss.item()

    val_loss /= len(val_loader)
    val_losses.append(val_loss)

    # Calcular accuracy de validación
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    val_accuracy = correct / total
    val_acc.append(val_accuracy)

    # Mostrar métricas por época
    print(f"Epoch {epoch+1}/{epochs} | Train Loss: {train_losses[-1]:.4f} | Val Loss: {val_loss:.4f} | "
          f"Train Acc: {train_accuracy:.4f} | Val Acc: {val_accuracy:.4f}")

    # Early Stopping
    if val_loss < best_loss:
        best_loss = val_loss
        counter = 0
        best_model_state = model.state_dict()
    else:
        counter += 1
        if counter >= patience:
            print("Early stopping triggered.")
            break

# Cargar el mejor modelo guardado
model.load_state_dict(best_model_state)
```

Nota. Elaboración Propia

- ✓ Entrenamiento Modelo BOTNet con Tuning

Una vez completado el entrenamiento del modelo base, se procedió a reentrenar el modelo BOTNet utilizando hiperparámetros ajustados empíricamente. Los valores considerados se resumen en la Tabla 15 y fueron seleccionados tras una exploración manual orientada a mejorar el rendimiento del modelo en validación, manteniendo un equilibrio entre precisión y generalización. Estos parámetros se aplicaron a una nueva instancia del modelo, entrenada desde cero bajo la misma lógica general utilizada previamente.

Tabla 15

Hiperparámetros utilizados en el entrenamiento del modelo BOTNet con Tuning

Hiperparámetro	**Valor asignado**
Learning rate	9.974985162045558e-05
Batch size	16
Weight decay	0.00011036050646065647
Número de épocas	30
Paciencia (*early stop*)	5

Nota. Elaboración Propia

El proceso inició con la redefinición de los DataLoader para reflejar el nuevo tamaño de lote y la inicialización del modelo BOTNet desde cero, manteniendo pretrained=True y num_classes=2. La configuración del optimizador Adam incorporó tanto el nuevo learning rate como el weight decay. Como se muestra en la Figura 77, esta fase de preparación incluyó la definición de hiperparámetros, la configuración del

modelo, la inicialización de variables de control y el comienzo del ciclo de entrenamiento.

Figura 77

Configuración y preparación del modelo BOTNet con Tuning

```
# Nuevos DataLoaders
train_loader_best = DataLoader(train_dataset, batch_size=best_batch_size, shuffle=True)
val_loader_best = DataLoader(val_dataset, batch_size=best_batch_size, shuffle=False)

# Modelo BOTNet desde cero
model_best = timm.create_model('botnet26t_256', pretrained=True, num_classes=2)
model_best.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model_best.parameters(), lr=best_lr, weight_decay=best_weight_decay)

epochs = 30
patience = 5
best_val_loss = float('inf')
counter = 0

# Listas para tracking de loss y accuracy por época
train_losses_best = []
val_losses_best = []
train_acc_best = []
val_acc_best = []

for epoch in range(epochs):
    model_best.train()
    running_loss = 0.0
    for images, labels in train_loader_best:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model_best(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

    train_losses_best.append(running_loss / len(train_loader_best))

    # Accuracy entrenamiento
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in train_loader_best:
            images, labels = images.to(device), labels.to(device)
            outputs = model_best(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    train_accuracy = correct / total
    train_acc_best.append(train_accuracy)
```

Nota. Elaboración Propia

La lógica del entrenamiento replicó la estructura aplicada al modelo base: cálculo de pérdida y exactitud en cada época, tanto para el conjunto de entrenamiento como para el de validación. Se implementó el mismo criterio de early stopping, y se almacenó el mejor estado del modelo de acuerdo con la pérdida mínima obtenida en validación. En la Figura 78, se presenta el ciclo completo de entrenamiento y validación con los hiperparámetros ajustados, incluyendo el control de interrupción y la restauración automática del mejor modelo registrado.

Figura 78

Entrenamiento y validación del modelo BOTNet con Tuning

```
    # Validación
    model_best.eval()
    val_loss = 0.0
    with torch.no_grad():
        for images, labels in val_loader_best:
            images, labels = images.to(device), labels.to(device)
            outputs = model_best(images)
            loss = criterion(outputs, labels)
            val_loss += loss.item()
    val_loss /= len(val_loader_best)
    val_losses_best.append(val_loss)

    # Accuracy validación
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in val_loader_best:
            images, labels = images.to(device), labels.to(device)
            outputs = model_best(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    val_accuracy = correct / total
    val_acc_best.append(val_accuracy)

    # Imprimir época
    print(f"Epoch {epoch+1}/{epochs} - Train Loss: {train_losses_best[-1]:.4f} - "
          f"Val Loss: {val_loss:.4f} - Train Acc: {train_accuracy:.4f} - Val Acc: {val_accuracy:.4f}")

    # Early stopping
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        counter = 0
        best_model_state_best = model_best.state_dict()
    else:
        counter += 1
        if counter >= patience:
            print("Early stopping aplicado")
            break

# Restaurar mejor modelo
model_best.load_state_dict(best_model_state_best)
```

Nota. Elaboración Propia

- **Procedimiento 4: Evaluación del desempeño del modelo**

Una vez finalizado el entrenamiento, se procedió a la implementación de la lógica de evaluación del modelo sobre el conjunto de prueba (test_loader). Esta fase consistió en la preparación del ciclo de inferencia, el almacenamiento de las predicciones y la posterior aplicación de funciones de evaluación estándar como accuracy_score, precision_score, recall_score, f1_score y roc_auc_score, todas disponibles en la biblioteca sklearn.metrics. El código correspondiente a esta etapa incluye también la utilización del método torch.softmax() para obtener las probabilidades por clase a partir de las salidas del modelo, así como el cálculo de las etiquetas predichas mediante torch.max(). Como se muestra en la Figura 79, este procedimiento técnico permite estructurar de forma sistemática la inferencia sobre datos no vistos, asegurando la posterior disponibilidad de métricas evaluativas en estructuras listas para visualización o análisis comparativo.

Figura 79

Script de evaluación estructurada del modelo BOTNet base sobre el conjunto de prueba

```
model.eval()
y_true, y_pred, y_prob = [], [], []

with torch.no_grad():
    for images, labels in tqdm(test_loader, desc="Testing"):
        images = images.to(device)
        outputs = model(images)
        probs = torch.softmax(outputs, dim=1)
        _, preds = torch.max(probs, 1)
        y_true.extend(labels.numpy())
        y_pred.extend(preds.cpu().numpy())
        y_prob.extend(probs[:, 1].cpu().numpy())

acc = accuracy_score(y_true, y_pred)
prec = precision_score(y_true, y_pred)
rec = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)
auc_roc = roc_auc_score(y_true, y_prob)

#Imprimir resultados
print("\nResults")
print(f"Accuracy  : {acc:.4f}")
print(f"Precision : {prec:.4f}")
print(f"Recall    : {rec:.4f}")
print(f"F1 Score  : {f1:.4f}")
print(f"AUC-ROC   : {auc_roc:.4f}")
```

Nota. Elaboración Propia

Además, en la Figura 80, se muestra la versión del script para evaluar el modelo BOTNet con Tuning

Figura 80

Script de evaluación estructurada del modelo BOTNet con Tuning

```
model_best.eval()
y_true_best, y_pred_best, y_prob_best = [ ] , [ ] , [ ]

with torch.no_grad():
  for images, labels in tqdm(test_loader, desc="Testing (Tuning Model)"):
      images = images.to(device)
      outputs = model_best(images)
      probs = torch.softmax(outputs, dim=1)
      _, preds = torch.max(probs, 1)
      y_true_best.extend(labels.numpy())
      y_pred_best.extend(preds.cpu().numpy())
      y_prob_best.extend(probs[:, 1].cpu().numpy())

# Métricas
acc_best = accuracy_score(y_true_best, y_pred_best)
prec_best = precision_score(y_true_best, y_pred_best)
rec_best = recall_score(y_true_best, y_pred_best)
f1_best = f1_score(y_true_best, y_pred_best)
auc_roc_best = roc_auc_score(y_true_best, y_prob_best)

#Imprimir resultados
print("\nResults")
print(f"[Tuning Model] Accuracy  : {acc_best:.4f}")
print(f"[Tuning Model] Precision : {prec_best:.4f}")
print(f"[Tuning Model] Recall    : {rec_best:.4f}")
print(f"[Tuning Model] F1 Score  : {f1_best:.4f}")
print(f"[Tuning Model] AUC-ROC   : {auc_roc_best:.4f}")
```

Nota. Elaboración Propia

- **Procedimiento 5: Guardado de modelos y resultados**

Con el objetivo de preservar el trabajo realizado durante el entrenamiento y permitir la reutilización futura de los modelos, se implementó un procedimiento sistemático de almacenamiento. Este incluyó tanto los modelos entrenados como los registros asociados a sus métricas de desempeño.

En primer lugar, el modelo BOTNet entrenado sin ajuste de hiperparámetros fue almacenado en formato pth utilizando la función torch.save(), luego de ser transferido a la CPU para su compatibilidad con futuras cargas. La ruta de guardado fue establecida dentro del directorio de resultados, e identificada mediante un timestamp único que

garantiza la trazabilidad del archivo generado. El fragmento de código correspondiente se muestra en la Figura 81.

Figura 81

Script para guardar el modelo BOTNet sin tuning en formato .pth

```
import torch
from datetime import datetime
import os

model.to('cpu')  # ✅ IMPORTANTE
output_dir = "/content/drive/MyDrive/2.CODIGOS/3.RESULTADOS/Deep_PTH"
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
save_path = os.path.join(output_dir, f"BOTNet_SinTuning_{timestamp}.pth")

torch.save(model.state_dict(), save_path)
print(f"✅ Modelo BOTNet SIN tuning guardado en: {save_path}")
```

Nota. Elaboración Propia

Posteriormente, se repitió este mismo procedimiento para el modelo BOTNet reentrenado con tuning. En este caso, se almacenaron los pesos del modelo ajustado, también en formato .pth y con una denominación específica que distingue claramente su condición optimizada. La estructura de este guardado puede observarse en la Figura 82.

Figura 82

Script para guardar el modelo BOTNet con tuning en formato pth.

```
model_best.to('cpu')
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_dir = "/content/drive/MyDrive/2.CODIGOS/3.RESULTADOS/Deep_PTH"
save_path = os.path.join(output_dir, f"BOTNet_ConTuning_{timestamp}.pth")

torch.save(model_best.state_dict(), save_path) #guarda pesos
print(f"✅ Modelo BOTNet CON tuning guardado en: {save_path}")
```

Nota. Elaboración Propia

Finalmente, se consolidaron las métricas comparativas de ambos modelos en un DataFrame estructurado y se exportaron a un archivo .csv. Este archivo permite conservar de forma organizada los valores de evaluación para cada versión del modelo y facilita su análisis externo. La ruta y nombre del archivo también fueron definidos dinámicamente mediante timestamp. El código utilizado para esta tarea se presenta en la Figura 83.

Figura 83

Script para exportar métricas comparativas a archivo .csv

```
# Ruta y nombre con timestamp
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_dir = "/content/drive/MyDrive/2.CODIGOS/3.RESULTADOS/Deep_CSV"
csv_path = os.path.join(output_dir, f"BotNet_MetricasComparativas_{timestamp}.csv")

# Guardar el DataFrame
df_resultados.to_csv(csv_path, index=False)
print(f" Resultados de métricas guardados en: {csv_path}")
```

Nota. Elaboración Propia

- **ViT**

  - **Procedimiento 1: Preparación y normalización de imágenes**

Para asegurar la compatibilidad de entrada con la arquitectura ViT, las imágenes fueron transformadas siguiendo una secuencia estandarizada: redimensionamiento a 224×224 píxeles, conversión a tensores y normalización por canal. Se utilizaron las estadísticas estándar de ImageNet (media = [0.485, 0.456, 0.406]; desviación estándar = [0.229, 0.224, 0.225]) para alinear la distribución de datos con los pesos preentrenados del modelo. Esta transformación se llevó a cabo con transforms.Compose() de torchvision, garantizando uniformidad en las muestras. Como se muestra en la Figura 84, este procedimiento fue codificado en la sección de transformaciones, estableciendo un flujo reproducible para el preprocesamiento visual.

Figura 84

Script sobre transformación de Datos ViT

```
# Transformaciones de datos
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])
```

Nota. Elaboración Propia

- **Procedimiento 2: Definición de la arquitectura**

A diferencia de otros modelos que se invocan directamente con funciones preconfiguradas, la arquitectura de Vision Transformer (ViT) fue definida mediante la creación de una clase personalizada, lo que permitió una mayor flexibilidad en su adaptación a la tarea de clasificación binaria. La clase ViT, heredada de nn.Module, encapsula la arquitectura vit_b_16 proporcionada por torchvision.models, la cual fue cargada con pesos preentrenados en ImageNet a través del argumento weights=ViT_B_16_Weights.DEFAULT.

La modificación central consistió en reemplazar la cabeza de clasificación original del modelo (self.vit.heads) por una nueva capa lineal (nn.Linear) con dos salidas, correspondientes a las clases "anémico" y "no anémico". Esta modificación permite al modelo generar directamente predicciones para el caso específico de estudio.

Posteriormente, el modelo fue instanciado y transferido al dispositivo de cómputo activo (CPU o GPU) mediante model.to(device), asegurando que todos los parámetros y tensores queden correctamente alineados para su posterior entrenamiento.

Este procedimiento se refleja en la Figura 85, donde se muestra claramente la definición del modelo, la modificación de su capa de salida y su preparación para el entrenamiento.

Figura 85

Definición e inicialización del modelo ViT personalizado.

```
# Definir modelo Vision Transformer personalizado
class ViT(nn.Module):
    def __init__(self, num_classes=2):
        super(ViT, self).__init__()
        weights = ViT_B_16_Weights.DEFAULT  # Forma moderna de cargar pesos preentrenados
        self.vit = models.vit_b_16(weights=weights)
        self.vit.heads = nn.Linear(self.vit.heads.head.in_features, num_classes)

    def forward(self, x):
        return self.vit(x)

# Seleccionar dispositivo automáticamente (GPU si está disponible)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f" Dispositivo utilizado: {device}")

# Inicializar el modelo y moverlo al dispositivo
model = ViT(num_classes=2)
model = model.to(device)
```

Nota. Elaboración propia

- **Procedimiento 3: Entrenamiento del modelo**

El entrenamiento del modelo ViT se dividió en dos etapas: la primera sin ajuste de hiperparámetros (modelo base) y la segunda con parámetros optimizados (modelo con tuning). Esta estructura permitió establecer una línea base y posteriormente evaluar el impacto del tuning sobre el rendimiento.

✓ Entrenamiento Modelo ViT base

En la etapa inicial, el modelo Vision Transformer (ViT) fue entrenado sin afinamiento de hiperparámetros, utilizando una configuración estándar. Se definió la función de pérdida CrossEntropyLoss() y se empleó como optimizador SGD (Stochastic Gradient Descent) con una tasa de aprendizaje de 1e-3, un valor de momentum de 0.9,

sin regularización (weight_decay=0) y un tamaño de lote de 32. El entrenamiento se llevó a cabo durante un máximo de 30 épocas, con un esquema de early stopping definido con una paciencia de 5 épocas sin mejora en la pérdida de validación.

La Figura 86 presenta el bloque de código correspondiente a la configuración inicial y entrenamiento por lotes, incluyendo el cálculo de la pérdida y la precisión sobre el conjunto de entrenamiento, lo cual permitió monitorear el desempeño del modelo durante el ajuste de parámetros.

Figura 86

Script de entrenamiento inicial del modelo ViT base

```
# Función de pérdida y optimizador
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=learning_rate, momentum=momentum)

# Inicializa la lista de las métricas con la que se evaluará los modelos
train_losses     = []         # guarda la pérdida del entrenamiento
val_losses       = []         # guarda la pérdida de la validación
train_acc = []         # guarda la precisión del entrenamiento
val_acc   = []         # guarda la precisión de la época

# Valor fijo estándar sin tuning
epochs = 30
patience = 5

# for del entrenamiento
for epoch in range(epochs):
    model.train()
    running_loss = 0.0      # acumula la pérdida durante la ejecución de las épocas
    correct      = 0        # lleva la cuenta del número de predicciones correctas
    total        = 0        # cuenta el número total de las muestras que se procesaron

    #Entrenamiento por batches
    for images, labels in tqdm(train_loader):
        images, labels = images.to('cuda'), labels.to('cuda')
        optimizer.zero_grad()
        outputs = model(images)                                  # pasa las imágenes por el modelo
        loss = criterion(outputs, labels)                        # calcula la pérdida
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += predicted.eq(labels).sum().item()

    # Promedio de pérdida de entrenamiento
    train_losses.append(running_loss / len(train_loader))

    # Calcular accuracy de entrenamiento
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    train_accuracy = correct / total
    train_acc.append(train_accuracy)
```

Nota. Elaboración propia

Posteriormente, el ciclo de validación se ejecutó al finalizar cada época, con el cálculo de la pérdida y exactitud en validación. Este bloque incluye la lógica de early stopping, que permite guardar el mejor estado del modelo cuando se alcanza la menor pérdida de validación. En caso de no mejora continua, se detiene el entrenamiento

anticipadamente. Esta lógica se ilustra en la Figura 87, que también incluye la restauración automática del modelo con mejor desempeño validado.

Figura 87

Script de Validación y early stopping del modelo ViT base

```
# -------------------------
    # VALIDACIÓN
    # -------------------------
    model.eval()
    val_loss = 0.0
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            val_loss += loss.item()

    val_loss /= len(val_loader)
    val_losses.append(val_loss)

    # Calcular accuracy de validación
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    val_accuracy = correct / total
    val_acc.append(val_accuracy)

    # Mostrar métricas por época
    print(f"Epoch {epoch+1}/{epochs} | Train Loss: {train_losses[-1]:.4f} | Val Loss: {val_loss:.4f} | "
          f"Train Acc: {train_accuracy:.4f} | Val Acc: {val_accuracy:.4f}")

    # Early Stopping
    if val_loss < best_loss:
        best_loss = val_loss
        counter = 0
        best_model_state = model.state_dict()
    else:
        counter += 1
        if counter >= patience:
            print("Early stopping triggered.")
            break

# Cargar el mejor modelo guardado
model.load_state_dict(best_model_state)
```

Nota. Elaboración propia

- ✓ Entrenamiento Modelo ViT con tuning

Para mejorar el desempeño del modelo ViT en la tarea de clasificación binaria, se desarrolló una segunda fase de entrenamiento utilizando hiperparámetros ajustados manualmente. Los valores considerados se detallan en la Tabla 16, los cuales fueron

definidos a partir de pruebas empíricas orientadas a maximizar la capacidad de generalización sin comprometer la estabilidad del aprendizaje. Esta nueva configuración se aplicó sobre una instancia reinicializada del modelo, manteniendo la misma lógica de entrenamiento utilizada en el modelo base.

Tabla 16

Hiperparámetros utilizados en el entrenamiento del modelo ViT con Tuning

Hiperparámetro	Valor asignado
Learning rate	1.752714399182543e-05
Batch size	32
Weight decay	0.00021819943135498196
Número de épocas	30
Paciencia (*early stop*)	5

Nota. Elaboración Propia

La implementación incluyó la redefinición de los DataLoader para reflejar el nuevo tamaño de lote, la reconfiguración del optimizador SGD incorporando los valores ajustados de learning rate y weight decay, y la selección automática del dispositivo de cómputo. La Figura 88 muestra el bloque de código utilizado para esta etapa de preparación, con la incorporación explícita de los hiperparámetros optimizados y la inicialización del modelo con su capa de salida personalizada.

Figura 88

Configuración e inicialización del modelo ViT con Tuning

```python
# Nuevos DataLoaders
train_loader_best = DataLoader(train_dataset, batch_size=best_batch_size, shuffle=True)
val_loader_best = DataLoader(val_dataset, batch_size=best_batch_size, shuffle=False)

# Definir modelo
model_best = models.vit_b_16(pretrained=True)
model_best.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = AdamW(model_best.parameters(), lr=best_lr, weight_decay=best_weight_decay)

# Entrenamiento final
epochs = 30
patience = 5
best_val_loss = float('inf')
counter = 0

#Listas para tracking de loss y accuracy por época
train_losses_best = []
val_losses_best = []
train_acc_best = []
val_acc_best = []

for epoch in range(epochs):
    model_best.train()
    running_loss = 0.0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model_best(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

    #Guardar loss de entrenamiento
    train_losses_best.append(running_loss / len(train_loader_best))

    #Calcular accuracy de entrenamiento
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in train_loader_best:
          images, labels = images.to(device), labels.to(device)
          outputs = model_best(images)
          _,predicted = torch.max(outputs.data,1)
          total += labels.size(0)
          correct += (predicted == labels).sum().item()
    train_accuracy = correct / total
    train_acc_best.append(train_accuracy)
```

Nota. Elaboración propia

El ciclo de entrenamiento aplicado replicó la estructura del modelo base, con cálculo de la pérdida y precisión sobre los conjuntos de entrenamiento y validación en cada época. Se mantuvo el mecanismo de early stopping con paciencia de cinco épocas,

lo cual permitió interrumpir el proceso de forma anticipada al detectar estancamiento en la mejora de la pérdida de validación. En la Figura 89 se representa todo el flujo del ciclo de entrenamiento y validación del modelo ViT ajustado, incluyendo la lógica de restauración del mejor estado del modelo.

Figura 89

Entrenamiento y validación del modelo ViT con Tuning

```
    # Validación
    model_best.eval()
    val_loss = 0.0
    with torch.no_grad():
        for images, labels in val_loader_best:
            images, labels = images.to(device), labels.to(device)
            outputs = model_best(images)
            loss = criterion(outputs, labels)
            val_loss += loss.item()
    val_loss /= len(val_loader_best)
    val_losses_best.append(val_loss)

    #Calcular accuracy de validación
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in val_loader_best:
            images, labels = images.to(device), labels.to(device)
            outputs = model_best(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    val_accuracy = correct / total
    val_acc_best.append(val_accuracy)

    #Imprimir resumen de la época
    print(f"Epoch {epoch+1}/{epochs} - Train Loss: {train_losses_best[-1]:.4f} - "
          f"Val Loss: {val_loss:.4f} - Train Acc: {train_accuracy:.4f} - Val Acc: {val_accuracy:.4f}")

    #Early stopping
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        counter = 0
        best_model_state_best = model_best.state_dict()
    else:
        counter += 1
        if counter >= patience:
            print("Early stopping aplicado.")
            break

#Restaurar mejor modelo
model_best.load_state_dict(best_model_state_best)
```

Nota. Elaboración propia

- **Procedimiento 4: Evaluación del desempeño del modelo**

Finalizado el proceso de entrenamiento, se evaluó el rendimiento del modelo ViT sobre el conjunto de prueba, con el propósito de medir su capacidad de generalización ante datos no vistos. Esta evaluación se aplicó de forma independiente tanto al modelo base como a su versión ajustada con tuning, utilizando métricas estándar de clasificación binaria: accuracy, precision, recall, F1-score y AUC-ROC.

Para ambos modelos, se implementó un bloque de inferencia en modo evaluación (model.eval()), donde las imágenes del conjunto de prueba (test_loader) fueron procesadas sin retropropagación. A partir de las salidas del modelo, se obtuvieron las probabilidades por clase mediante torch.softmax() y las predicciones finales usando torch.max(). Posteriormente, se calcularon las métricas utilizando las funciones de la librería sklearn.metrics, como accuracy_score, precision_score, recall_score, f1_score y roc_auc_score.

La lógica de evaluación aplicada al modelo base se presenta en la Figura 90, mientras que en la Figura 91 se muestra la versión personalizada del script utilizado para evaluar al modelo ViT con tuning, el cual incluye una estructura ajustada para gestionar variables específicas, mejorar la organización del reporte y almacenar los resultados para análisis posterior.

Figura 90

Script de evaluación del modelo ViT base sobre el conjunto de prueba

```
model.eval()

# Inicializar parámetros para guardar resultados
y_true  = []
y_pred  = []
y_prob = []

# Evaluar las imágenes de la carpeta test
with torch.no_grad():
    for images, labels in tqdm(test_loader, desc = "Testing"):
        images, labels = images.to('cuda'), labels.to('cuda')
        outputs = model(images)
        probs = torch.softmax(outputs, dim=1)
        _, preds = torch.max(probs, 1)
        y_true.extend(labels.cpu().numpy())
        y_pred.extend(preds.cpu().numpy())
        y_prob.extend(outputs[:, 1].cpu().numpy())

# Calcular las métricas de la evaluación del modelo
acc = accuracy_score(y_true, y_pred)
prec = precision_score(y_true, y_pred)
rec = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)
auc_roc = roc_auc_score(y_true, y_prob)

#Imprimir resultados
print("\nResults")
print(f"Accuracy  : {acc:.4f}")
print(f"Precision : {prec:.4f}")
print(f"Recall    : {rec:.4f}")
print(f"F1 Score  : {f1:.4f}")
print(f"AUC-ROC   : {auc_roc:.4f}")
```

Nota. Elaboración propia

Figura 91

Script de evaluación del modelo ViT con tuning

```
model_best.eval()
y_true_best, y_pred_best, y_prob_best = [ ] , [ ] , [ ]

with torch.no_grad():
  for images, labels in tqdm(test_loader, desc="Testing (Tuning Model)"):
      images = images.to(device)
      outputs = model_best(images)
      probs = torch.softmax(outputs, dim=1)
      _, preds = torch.max(probs, 1)
      y_true_best.extend(labels.numpy())
      y_pred_best.extend(preds.cpu().numpy())
      y_prob_best.extend(probs[:, 1].cpu().numpy())

# Métricas
acc_best = accuracy_score(y_true_best, y_pred_best)
prec_best = precision_score(y_true_best, y_pred_best)
rec_best = recall_score(y_true_best, y_pred_best)
f1_best = f1_score(y_true_best, y_pred_best)
auc_roc_best = roc_auc_score(y_true_best, y_prob_best)

#Imprimir resultados
print("\nResults")
print(f"[Tuning Model] Accuracy  : {acc_best:.4f}")
print(f"[Tuning Model] Precision : {prec_best:.4f}")
print(f"[Tuning Model] Recall    : {rec_best:.4f}")
print(f"[Tuning Model] F1 Score  : {f1_best:.4f}")
print(f"[Tuning Model] AUC-ROC   : {auc_roc_best:.4f}")
```

Nota. Elaboración propia

- **Procedimiento 5: Guardado de modelos y resultados**

Para asegurar la trazabilidad y reutilización futura del modelo ViT, tanto en su versión base como ajustada con tuning, se implementó un procedimiento sistemático de almacenamiento. Este incluyó la serialización de los modelos entrenados en formato .pth, así como la exportación estructurada de las métricas de evaluación.

En primer lugar, el modelo ViT entrenado sin ajuste de hiperparámetros fue almacenado mediante torch.save(), luego de ser transferido a CPU para garantizar su compatibilidad en futuras cargas. Se asignó una ruta específica dentro del directorio de

resultados, incorporando un identificador temporal (timestamp) para asegurar la trazabilidad de cada experimento. Esta operación se muestra en la Figura 92, correspondiente al guardado del modelo base.

Figura 92

Script para guardar el modelo ViT base en formato .pth

```
import torch
from datetime import datetime
import os

model.to('cpu')  # IMPORTANTE: pasar a CPU antes de guardar
output_dir = "/content/drive/MyDrive/2.CODIGOS/3.RESULTADOS/Deep_PTH"
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
save_path = os.path.join(output_dir, f"ViT_SinTuning_{timestamp}.pth")

torch.save(model.state_dict(), save_path)
print(f" Modelo ViT SIN tuning guardado en: {save_path}")
```

Nota. Elaboración propia

Posteriormente, se aplicó el mismo procedimiento al modelo ViT entrenado con tuning. Los pesos optimizados fueron almacenados en un archivo .pth independiente, identificado con una denominación específica que distingue claramente su condición ajustada. El código utilizado para esta operación se presenta en la Figura 93.

Figura 93

Script para guardar el modelo ViT con tuning en formato .pth

```
model_best.to('cpu')
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_dir = "/content/drive/MyDrive/2.CODIGOS/3.RESULTADOS/Deep_PTH"
save_path = os.path.join(output_dir, f"ViT_ConTuning_{timestamp}.pth")

torch.save(model_best.state_dict(), save_path)  #
print(f"Modelo ViT CON tuning guardado en: {save_path}")
```

Nota. Elaboración propia

Finalmente, los resultados de evaluación de ambos modelos fueron organizados en un DataFrame comparativo y exportados como un archivo .csv, permitiendo su análisis posterior o integración en visualizaciones externas. Esta exportación se gestionó mediante funciones de pandas, incorporando también un timestamp automático en el nombre del archivo. La Figura 94 muestra el bloque de código correspondiente a esta última etapa de guardado.

Figura 94

Script para exportar métricas comparativas de ViT en archivo .csv

```
# Ruta y nombre con timestamp
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_dir = "/content/drive/MyDrive/2.CODIGOS/3.RESULTADOS/Deep_CSV"
csv_path = os.path.join(output_dir, f"ViT_MetricasComparativas_{timestamp}.csv")

# Guardar el DataFrame
df_resultados.to_csv(csv_path, index=False)
print(f" Resultados de métricas guardados en: {csv_path}")
```

Nota. Elaboración propia

- **Swin Transformer**

  - **Procedimiento 1: Preparación y normalización de imágenes**

Para asegurar la compatibilidad de entrada con la arquitectura Swin Transformer, las imágenes fueron sometidas a una serie de transformaciones que incluyeron el redimensionamiento a 224×224 píxeles, conversión a tensores y normalización por canal según los valores estándar de ImageNet (media = [0.485, 0.456, 0.406]; desviación estándar = [0.229, 0.224, 0.225]). Este proceso se implementó mediante transforms.Compose() de la librería torchvision.

La normalización y homogeneización del conjunto permitió mantener consistencia en el formato de entrada, reduciendo el riesgo de sesgos durante el entrenamiento y evaluación. Esta preparación se refleja en la Figura 95, que muestra el bloque de código donde se aplican las transformaciones.

Figura 95

Script de transformación de datos para el modelo Swin Transformer

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])
```

Nota. Elaboración propia

  - **Procedimiento 2: Definición de la arquitectura**

La arquitectura Swin Transformer fue implementada a través de la librería timm, utilizando la versión preentrenada swin_tiny_patch4_window7_224. Esta fue cargada mediante la función timm.create_model() especificando pretrained=True y num_classes=2 para adecuarla a la clasificación binaria (anémico / no anémico).

Una vez definido, el modelo fue transferido al dispositivo de cómputo disponible (GPU o CPU) mediante model.to(device). En la Figura 96 se muestra el bloque de código correspondiente a esta etapa, que incluye la invocación del modelo, su configuración de salida y asignación al entorno de ejecución.

Figura 96

Script de carga y adaptación del modelo Swin Transformer

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = timm.create_model('swin_base_patch4_window7_224', pretrained=True, num_classes=2)
model.to(device)
```

Nota. Elaboración propia

- **Procedimiento 3: Entrenamiento del modelo**

El entrenamiento del modelo Swin Transformer se dividió en dos fases: una inicial con hiperparámetros estándar (modelo base) y otra con ajuste fino de parámetros (modelo con tuning). Este enfoque permitió evaluar la mejora de rendimiento derivada de la optimización empírica.

✓ Entrenamiento Modelo Swin Transformer base”

El modelo fue entrenado utilizando la función de pérdida CrossEntropyLoss() y el optimizador Adam, con una tasa de aprendizaje de 1e-4, sin regularización (weight_decay=0), y un tamaño de lote de 32. Se definió un máximo de 30 épocas con mecanismo de early stopping (paciencia de 5 épocas). El código correspondiente a la configuración inicial, entrenamiento por lotes y cálculo de métricas se presenta en la Figura 97 y Figura 98.

Figura 97

Script de entrenamiento inicial del modelo Swin Transformer base

```
# Función de pérdida y optimizador
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-4)

# Configuración
epochs = 30
patience = 5
best_loss = float('inf')
counter = 0

# Métricas por época
train_losses = []
val_losses = []
train_acc = []
val_acc = []

# Entrenamiento
for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    correct_train = 0
    total_train = 0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

        # Accuracy entrenamiento
        _, predicted = torch.max(outputs, 1)
        correct_train += (predicted == labels).sum().item()
        total_train += labels.size(0)

    train_losses.append(running_loss / len(train_loader))
    train_acc.append(correct_train / total_train)
```

Nota. Elaboración propia

Figura 98

Script de validación, control de early stopping y restauración del mejor modelo Swin base

```
    # -------------------
    # VALIDACIÓN
    # -------------------
    model.eval()
    val_loss = 0.0
    correct_val = 0
    total_val = 0

    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            val_loss += loss.item()

            # Accuracy validación
            _, predicted = torch.max(outputs, 1)
            correct_val += (predicted == labels).sum().item()
            total_val += labels.size(0)

    val_loss /= len(val_loader)
    val_losses.append(val_loss)
    val_acc.append(correct_val / total_val)

    # Mostrar métricas
    print(f"Epoch {epoch+1}/{epochs} | Train Loss: {train_losses[-1]:.4f} | "
          f"Val Loss: {val_losses[-1]:.4f} | Train Acc: {train_acc[-1]:.4f} | Val Acc: {val_acc[-1]:.4f}")

    # Early Stopping
    if val_loss < best_loss:
        best_loss = val_loss
        counter = 0
        best_model_state = model.state_dict()
    else:
        counter += 1
        if counter >= patience:
            print("Early stopping triggered.")
            break

# Restaurar mejor modelo
model.load_state_dict(best_model_state)
```

Nota. Elaboración propia

✓ Entrenamiento Modelo Swin Transformer con tuning

Para optimizar el modelo, se realizó una segunda fase de entrenamiento con los siguientes hiperparámetros seleccionados manualmente y resumidos en la Tabla 17:

Tabla 17

Hiperparámetros utilizados en el entrenamiento del modelo Swin Transformer con Tuning

Hiperparámetro	Valor asignado
Learning rate	4.164291489569948e-05
Batch size	8
Weight decay	0.00040064094330055655
Número de épocas	30
Paciencia (*early stop*)	5

Nota. Elaboración propia

Los nuevos valores fueron incorporados en la redefinición del optimizador y DataLoader, aplicando la misma lógica de entrenamiento y validación que en la versión base. La Figura 97 muestra la inicialización del modelo y la configuración con los parámetros ajustados, mientras que la Figura 99 presenta el ciclo completo de entrenamiento y validación bajo esta configuración.

Figura 99

Configuración y preparación del modelo Swin Transformer con Tuning

```
# Hiperparámetros
best_lr = 4.164291489569948e-05
best_batch_size = 8
best_weight_decay = 0.00040064094330055655

# Nuevos DataLoaders
train_loader_best = DataLoader(train_dataset, batch_size=best_batch_size, shuffle=True)
val_loader_best = DataLoader(val_dataset, batch_size=best_batch_size, shuffle=False)

# Modelo Swin Transformer desde cero
model_best = timm.create_model('swin_base_patch4_window7_224', pretrained=True, num_classes=2)
model_best.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model_best.parameters(), lr=best_lr, weight_decay=best_weight_decay)

epochs = 30
patience = 5
best_val_loss = float('inf')
counter = 0

train_losses_best = []
val_losses_best = []
train_acc_best = []
val_acc_best = []

for epoch in range(epochs):
    model_best.train()
    running_loss = 0.0
    correct_train = 0
    total_train = 0

    for images, labels in train_loader_best:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model_best(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

        _, predicted = torch.max(outputs, 1)
        correct_train += (predicted == labels).sum().item()
        total_train += labels.size(0)

    train_losses_best.append(running_loss / len(train_loader_best))
    train_acc_best.append(correct_train / total_train)
```

Nota. Elaboración propia

Figura 100

Entrenamiento y validación del modelo SwinTransformer con Tuning

```
    model_best.eval()
    val_loss = 0.0
    correct_val = 0
    total_val = 0

    with torch.no_grad():
        for images, labels in val_loader_best:
            images, labels = images.to(device), labels.to(device)
            outputs = model_best(images)
            loss = criterion(outputs, labels)
            val_loss += loss.item()

            _, predicted = torch.max(outputs, 1)
            correct_val += (predicted == labels).sum().item()
            total_val += labels.size(0)

    val_loss /= len(val_loader_best)
    val_losses_best.append(val_loss)
    val_acc_best.append(correct_val / total_val)

    print(f"Epoch {epoch+1}/{epochs} - Train Loss: {train_losses_best[-1]:.4f} - Val Loss: {val_loss:.4f} - "
          f"Train Acc: {train_acc_best[-1]:.4f} - Val Acc: {val_acc_best[-1]:.4f}")

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        counter = 0
        best_model_state_best = model_best.state_dict()
    else:
        counter += 1
        if counter >= patience:
            print("Early stopping aplicado")
            break

model_best.load_state_dict(best_model_state_best)
```

Nota. Elaboración propia

- **Procedimiento 4: Evaluación del desempeño del modelo**

El rendimiento de ambas versiones del modelo Swin Transformer (modelo base y modelo con tuning) fue evaluado utilizando el conjunto de prueba con el propósito de validar su capacidad de generalización sobre datos no vistos. Para ello, se implementó un ciclo de inferencia que procesó las imágenes del conjunto de prueba (test_loader), calculando las predicciones mediante torch.max() sobre las salidas transformadas con softmax(), lo que permitió obtener probabilidades por clase.

Posteriormente, se utilizaron las funciones de sklearn.metrics para calcular las métricas de desempeño: exactitud (accuracy), precisión (precision), sensibilidad (recall), puntaje F1 (F1-score) y área bajo la curva ROC (AUC-ROC), todas expresadas con cuatro decimales. El bloque de código correspondiente al modelo base se presenta en la Figura 101.

Figura 101

Script de evaluación estructurada del modelo Swin Transformer base sobre el conjunto de prueba

```
model.eval()
y_true, y_pred, y_prob = [], [], []

with torch.no_grad():
    for images, labels in tqdm(test_loader, desc="Testing"):
        images = images.to(device)
        outputs = model(images)
        probs = torch.softmax(outputs, dim=1)
        _, preds = torch.max(probs, 1)
        y_true.extend(labels.numpy())
        y_pred.extend(preds.cpu().numpy())
        y_prob.extend(probs[:, 1].cpu().numpy())

acc = accuracy_score(y_true, y_pred)
prec = precision_score(y_true, y_pred)
rec = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)
auc_roc = roc_auc_score(y_true, y_prob)


#Imprimir resultados
print("\nResults")
print(f"Accuracy  : {acc:.4f}")
print(f"Precision : {prec:.4f}")
print(f"Recall    : {rec:.4f}")
print(f"F1 Score  : {f1:.4f}")
print(f"AUC-ROC   : {auc_roc:.4f}")
```

Este procedimiento permitió recopilar los resultados del modelo base de forma estructurada, facilitando su posterior análisis comparativo respecto al modelo afinado.

En la Figura 102 se muestra el código empleado para evaluar el modelo Swin Transformer entrenado con hiperparámetros ajustados. La estructura del script es análoga, aunque diferenciada por el uso de variables exclusivas para este modelo (sufijadas con _best), lo que permite preservar la independencia de ambas ejecuciones y mantener organizadas las métricas generadas.

Figura 102

Script de evaluación estructurada del modelo Swin Transformer con Tuning

```
model_best.eval()
y_true_best, y_pred_best, y_prob_best = [], [], []

with torch.no_grad():
    for images, labels in tqdm(test_loader, desc="Testing (Tuning Model)"):
        images = images.to(device)
        outputs = model_best(images)
        probs = torch.softmax(outputs, dim=1)
        _, preds = torch.max(probs, 1)
        y_true_best.extend(labels.numpy())
        y_pred_best.extend(preds.cpu().numpy())
        y_prob_best.extend(probs[:, 1].cpu().numpy())

# Métricas
acc_best = accuracy_score(y_true_best, y_pred_best)
prec_best = precision_score(y_true_best, y_pred_best)
rec_best = recall_score(y_true_best, y_pred_best)
f1_best = f1_score(y_true_best, y_pred_best)
auc_roc_best = roc_auc_score(y_true_best, y_prob_best)


#Imprimir resultados
print("\nResults")
print(f"[Tuning Model] Accuracy  : {acc_best:.4f}")
print(f"[Tuning Model] Precision : {prec_best:.4f}")
print(f"[Tuning ModelL] Recall    : {rec_best:.4f}")
print(f"[Tuning Model] F1 Score  : {f1_best:.4f}")
print(f"[Tuning Model] AUC-ROC   : {auc_roc_best:.4f}")
```

Gracias a esta lógica diferenciada, los resultados de evaluación de ambas versiones del modelo pueden ser comparados directamente sin interferencias, manteniendo la trazabilidad experimental y asegurando una adecuada documentación de los procesos de evaluación.

- **Procedimiento 5: Guardado de modelos y resultados**

Ambas versiones del modelo Swin Transformer —el modelo base y el modelo ajustado con tuning— fueron almacenadas en formato.pth utilizando la función torch.save(). Previamente, se trasladaron al dispositivo CPU para garantizar compatibilidad en futuras implementaciones. La Figura 103 muestra el script utilizado para guardar el modelo entrenado sin tuning, el cual fue identificado mediante un timestamp para asegurar su trazabilidad.

Figura 103

Script para guardar el modelo Swin Transformer sin tuning en formato .pth

```
import torch
from datetime import datetime
import os

model.to('cpu')  # ✅ IMPORTANTE
output_dir = "/content/drive/MyDrive/2.CODIGOS/3.RESULTADOS/Deep_PTH"
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
save_path = os.path.join(output_dir, f"Swin_SinTuning_{timestamp}.pth")

torch.save(model.state_dict(), save_path)
print(f"✅ Modelo Swin Sin tuning guardado en: {save_path}")
model.to(device)  # volver a GPU para entrenar
```

De forma similar, el modelo Swin Transformer reentrenado con hiperparámetros optimizados fue almacenado con una denominación diferenciada que indica su condición tuneada. Esta operación se ilustra en la Figura 104, donde se observa la escritura del archivo con parámetros ajustados.

Figura 104

Script para guardar el modelo Swin con tuning en formato pth.

```
model.to('cpu')
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
save_path = os.path.join(output_dir, f"Swin_ConTuning_{timestamp}.pth")

torch.save(model.state_dict(), save_path)
print(f"✅ Modelo SWIN CON tuning guardado en: {save_path}")
model.to(device)
```

Adicionalmente, se exportaron las métricas comparativas de evaluación de ambos modelos en un archivo .csv, estructurado con la librería pandas. Esta exportación permite conservar los resultados en un formato tabular que facilita su análisis externo y replicabilidad. La estructura del código utilizado se presenta en la Figura 105.

Figura 105

Script para exportar métricas comparativas a archivo .csv

```
# Ruta y nombre con timestamp
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_dir = "/content/drive/MyDrive/2.CODIGOS/3.RESULTADOS/Deep_CSV"
csv_path = os.path.join(output_dir, f"Swin_MetricasComparativas_{timestamp}.csv")

# Guardar el DataFrame
df_resultados.to_csv(csv_path, index=False)
print(f" Resultados de métricas guardados en: {csv_path}")
```

- **PiT Transfomer**

- **Procedimiento 1: Preparación y normalización de imágenes**

Con el fin de adecuar el conjunto de datos a los requerimientos de entrada del modelo PiT Transformer, se aplicó una secuencia de transformaciones a las imágenes, las cuales contemplaron el redimensionamiento uniforme a 224×224 píxeles, la conversión a tensores y la normalización de los canales RGB. Este último paso se realizó utilizando los parámetros estándar de los modelos preentrenados en ImageNet, con medias de [0.485, 0.456, 0.406] y desviaciones estándar de [0.229, 0.224, 0.225], respectivamente. Las transformaciones se agruparon mediante la función transforms.Compose() de la librería torchvision, permitiendo una aplicación sistemática en todas las particiones del dataset.

Este preprocesamiento no solo garantiza la compatibilidad del formato de entrada, sino que también contribuye a estabilizar el proceso de entrenamiento al reducir la variabilidad no deseada entre las muestras. La Figura 106 presenta el fragmento de código correspondiente a esta etapa.

Figura 106

Script de transformación de datos para el modelo PiT Transformer

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])
```

Nota. Elaboración propia

- **Procedimiento 2: Definición de la arquitectura**

La arquitectura PiT Transformer fue implementada utilizando la librería timm, seleccionando la variante pit_b_224, la cual corresponde a una versión preentrenada sobre el conjunto ImageNet. Esta se cargó especificando el parámetro pretrained=True para aprovechar los pesos ya entrenados y facilitar la transferencia de conocimiento. A fin de adaptar la arquitectura a la tarea de clasificación binaria (anémico / no anémico), se configuró el parámetro num_classes=2, ajustando así la capa de salida al número de clases requeridas por el proyecto.

Posteriormente, el modelo fue asignado al dispositivo de procesamiento disponible (GPU o CPU) mediante la instrucción model.to(device), lo que permitió optimizar los tiempos de entrenamiento y asegurar la compatibilidad con el entorno de ejecución. La Figura 107 muestra el bloque de código empleado para definir e inicializar el modelo PiT Transformer bajo estos parámetros.

Figura 107

Script de carga y adaptación del modelo PiT Transformer

```
model = timm.create_model('pit_b_224', pretrained=True, num_classes=2)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
```

Nota. Elaboración propia

- **Procedimiento 3: Entrenamiento del modelo**

El entrenamiento del modelo PiT Transformer se estructuró en dos fases consecutivas: una inicial bajo hiperparámetros estándar, y una posterior con hiperparámetros optimizados mediante búsqueda empírica. Este enfoque permitió evaluar el impacto del ajuste fino en el rendimiento del modelo, manteniendo trazabilidad experimental en ambas configuraciones.

✓ Entrenamiento Modelo PiT Transformer base

Durante la primera fase, se empleó la función de pérdida CrossEntropyLoss() junto con el optimizador Adam, configurado con una tasa de aprendizaje fija de 1e-4, un tamaño de lote (batch size) de 32, y un máximo de 30 épocas. Para evitar sobreentrenamiento, se implementó un mecanismo de early stopping con una paciencia de 5 épocas, el cual detiene el entrenamiento si no se detectan mejoras en la pérdida de validación. A lo largo de cada época, se calcularon métricas de desempeño —precisión, sensibilidad, exactitud y pérdida— tanto para el conjunto de entrenamiento como para el de validación. Estas métricas permitieron identificar y conservar la versión del modelo con mejor rendimiento validado. El código correspondiente a la configuración inicial, entrenamiento por lotes y cálculo de métricas se presenta en las Figura 108 y Figura 109.

Figura 108

Script de entrenamiento inicial del modelo PiT Transformer base

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-4)
epochs = 30
patience = 5
best_loss = float('inf')
counter = 0

train_losses = []
val_losses = []
train_accuracies = []
val_accuracies = []

for epoch in range(epochs):
    # Entrenamiento
    model.train()
    running_loss = 0.0
    correct_train = 0
    total_train = 0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

        # Accuracy de entrenamiento
        _, predicted = torch.max(outputs, 1)
        correct_train += (predicted == labels).sum().item()
        total_train += labels.size(0)

    train_losses.append(running_loss / len(train_loader))
    train_accuracy = correct_train / total_train
    train_accuracies.append(train_accuracy)
```

Nota. Elaboración propia

Figura 109

Script de validación, control de early stopping y restauración del mejor modelo PiT

```
    # Validación
    model.eval()
    val_loss = 0.0
    correct_val = 0
    total_val = 0

    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            val_loss += loss.item()

            # Accuracy de validación
            _, predicted = torch.max(outputs, 1)
            correct_val += (predicted == labels).sum().item()
            total_val += labels.size(0)

    val_loss /= len(val_loader)
    val_losses.append(val_loss)
    val_accuracy = correct_val / total_val
    val_accuracies.append(val_accuracy)

    print(f"Epoch {epoch+1}/{epochs}, Train Loss: {train_losses[-1]:.4f}, Val Loss: {val_loss:.4f}, Train Acc: {train_accuracy:.4f}, Val Acc: {val_accuracy:.4f}")

    # Early Stopping
    if val_loss < best_loss:
        best_loss = val_loss
        counter = 0
        best_model_state = model.state_dict()
    else:
        counter += 1
        if counter >= patience:
            print("Early stopping triggered.")
            break

model.load_state_dict(best_model_state)
```

Nota. Elaboración propia

✓ Entrenamiento Modelo PiT Transformer con tuning

Para optimizar el rendimiento del modelo, se llevó a cabo una segunda fase de entrenamiento empleando hiperparámetros ajustados manualmente, los cuales se detallan en la Tabla 18:

Tabla 18

Hiperparámetros utilizados en el entrenamiento del modelo PiT Transformer con Tuning

Hiperparámetro	Valor asignado
Learning rate	2.8455217640819972e-05
Batch size	16
Weight decay	0.0029739675198269117

Número de épocas	30
Paciencia (*early stop*)	5

Los nuevos valores fueron incorporados en la redefinición del optimizador y DataLoader, aplicando la misma lógica de entrenamiento y validación que en la versión base. La Figura 108 muestra la inicialización del modelo y la configuración con los parámetros ajustados, mientras que la Figura 110 presenta el ciclo completo de entrenamiento y validación bajo esta configuración.

Figura 110

Configuración y preparación del modelo PiT Transformer con Tuning

```
# Hiperparámetros
best_lr = 2.8455217640819972e-05
best_batch_size = 16
best_weight_decay = 0.0029739675198269117

# Nuevos DataLoaders
train_loader_best = DataLoader(train_dataset, batch_size=best_batch_size, shuffle=True)
val_loader_best = DataLoader(val_dataset, batch_size=best_batch_size, shuffle=False)

# Modelo PIT desde cero
model_best = timm.create_model('pit_b_224', pretrained=True, num_classes=2)
model_best.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model_best.parameters(), lr=best_lr, weight_decay=best_weight_decay)

epochs = 30
patience = 5
best_val_loss = float('inf')
counter = 0

train_losses_best = []
val_losses_best = []
train_accuracies_best = []
val_accuracies_best = []

for epoch in range(epochs):
    model_best.train()
    running_loss = 0.0
    correct_train = 0
    total_train = 0

    for images, labels in train_loader_best:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model_best(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

        # Accuracy entrenamiento
        _, preds = torch.max(outputs, 1)
        correct_train += (preds == labels).sum().item()
        total_train += labels.size(0)

    train_losses_best.append(running_loss / len(train_loader_best))
    train_accuracy = correct_train / total_train
    train_accuracies_best.append(train_accuracy)
```

Nota. Elaboración propia

Figura 111

Entrenamiento y validación del modelo PiT Transformer con Tuning

```
    # Validación
    model_best.eval()
    val_loss = 0.0
    correct_val = 0
    total_val = 0

    with torch.no_grad():
        for images, labels in val_loader_best:
            images, labels = images.to(device), labels.to(device)
            outputs = model_best(images)
            loss = criterion(outputs, labels)
            val_loss += loss.item()

            # Accuracy validación
            _, preds = torch.max(outputs, 1)
            correct_val += (preds == labels).sum().item()
            total_val += labels.size(0)

    val_loss /= len(val_loader_best)
    val_losses_best.append(val_loss)
    val_accuracy = correct_val / total_val
    val_accuracies_best.append(val_accuracy)

    print(f"Epoch {epoch+1}/{epochs} - Train Loss: {train_losses_best[-1]:.4f} - Val Loss: {val_loss:.4f} - Train Acc: {train_accuracy:.4f} - Val Acc: {val_accuracy:.4f}")

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        counter = 0
        best_model_state_best = model_best.state_dict()
    else:
        counter += 1
        if counter >= patience:
            print("Early stopping aplicado")
            break

model_best.load_state_dict(best_model_state_best)
```

Nota. Elaboración propia

- **Procedimiento 4: Evaluación del desempeño del modelo**

Una vez completado el entrenamiento, ambas versiones del modelo PiT Transformer, tanto la configuración base como la optimizada con tuning, fueron evaluadas utilizando el conjunto de prueba, con el objetivo de validar su capacidad de generalización frente a datos no vistos. Para ello, se ejecutó un ciclo de inferencia que recorrió las imágenes del test_loader, obteniendo las predicciones mediante la función torch.max() aplicada sobre las salidas del modelo transformadas con softmax().

Posteriormente, se calcularon las métricas de desempeño a través de funciones de la biblioteca sklearn.metrics, obteniendo los siguientes indicadores: exactitud (accuracy), precisión (precision), recuperación o sensibilidad (recall), puntaje F1 (F1-score) y área bajo la curva ROC (AUC-ROC). Estas métricas se expresaron con una

precisión de cuatro decimales y se registraron por separado para cada versión del modelo.

El código de evaluación del modelo PiT en su versión base se presenta en la Figura 112. Por su parte, la Figura 113 muestra el script correspondiente a la evaluación del modelo con hiperparámetros ajustados, implementado mediante variables independientes (sufijadas con _best) para conservar la trazabilidad de cada ejecución.

Figura 112

Script de evaluación estructurada del modelo PiT Transformer base sobre el conjunto de prueba

```
model.eval()
y_true, y_pred, y_prob = [], [], []

with torch.no_grad():
    for images, labels in tqdm(test_loader, desc="Testing"):
        images = images.to(device)
        outputs = model(images)
        probs = torch.softmax(outputs, dim=1)
        _, preds = torch.max(probs, 1)
        y_true.extend(labels.numpy())
        y_pred.extend(preds.cpu().numpy())
        y_prob.extend(probs[:, 1].cpu().numpy())

acc = accuracy_score(y_true, y_pred)
prec = precision_score(y_true, y_pred)
rec = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)
auc_roc = roc_auc_score(y_true, y_prob)

print(f"Accuracy  : {acc:.4f}")
print(f"Precision : {prec:.4f}")
print(f"Recall    : {rec:.4f}")
print(f"F1 Score  : {f1:.4f}")
print(f"AUC-ROC   : {auc_roc:.4f}")
```

Nota. Elaboración propia

Figura 113

Script de evaluación estructurada del modelo PiT Transformer con Tuning

```
model_best.eval()
y_true_best, y_pred_best, y_prob_best = [], [], []

with torch.no_grad():
    for images, labels in tqdm(test_loader, desc="Testing (Tuning Model)"):
        images = images.to(device)
        outputs = model_best(images)
        probs = torch.softmax(outputs, dim=1)
        _, preds = torch.max(probs, 1)
        y_true_best.extend(labels.numpy())
        y_pred_best.extend(preds.cpu().numpy())
        y_prob_best.extend(probs[:, 1].cpu().numpy())

# Métricas
acc_best = accuracy_score(y_true_best, y_pred_best)
prec_best = precision_score(y_true_best, y_pred_best)
rec_best = recall_score(y_true_best, y_pred_best)
f1_best = f1_score(y_true_best, y_pred_best)
auc_roc_best = roc_auc_score(y_true_best, y_prob_best)


#Imprimir resultados
print("\nResults")
print(f"[Tuning Model] Accuracy  : {acc_best:.4f}")
print(f"[Tuning Model] Precision : {prec_best:.4f}")
print(f"[Tuning Model] Recall    : {rec_best:.4f}")
print(f"[Tuning Model] F1 Score  : {f1_best:.4f}")
print(f"[Tuning Model] AUC-ROC   : {auc_roc_best:.4f}")
```

Nota. Elaboración propia

- **Procedimiento 5: Guardado de modelos y resultados**

Ambas versiones del modelo PiT Transformer —tanto el modelo base como el ajustado con tuning— fueron almacenadas en formato .pth utilizando la función torch.save(), lo que permite su reutilización posterior sin necesidad de reentrenamiento. Antes del guardado, los modelos fueron transferidos a la CPU mediante model.to("cpu"), con el fin de asegurar su compatibilidad en entornos de ejecución sin aceleración por GPU.

Cada archivo fue nombrado de forma dinámica incorporando un timestamp en su denominación, lo cual garantiza la trazabilidad del modelo y facilita la identificación

de versiones en etapas posteriores del proyecto. La Figura 114 muestra el script utilizado para guardar el modelo base, mientras que la Figura 115 ilustra el proceso equivalente aplicado a la versión con hiperparámetros ajustados.

Figura 114

Script para guardar el modelo PiT Transformer sin tuning en formato .pth

```
import torch
from datetime import datetime
import os

model.to('cpu')
output_dir = "/content/drive/MyDrive/2.CODIGOS/3.RESULTADOS/Deep_PTH"
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
save_path = os.path.join(output_dir, f"PiT_SinTuning_{timestamp}.pth")

torch.save(model.state_dict(), save_path)
print(f"✅ Modelo PIT Sin tuning guardado en: {save_path}")
model.to(device)  # volver a GPU para entrenar
```

Nota. Elaboración propia

Figura 115

Script para guardar el modelo PiT Transformer con tuning en formato pth.

```
model.to('cpu')
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
save_path = os.path.join(output_dir, f"PIT_ConTuning_{timestamp}.pth")

torch.save(model.state_dict(), save_path)
print(f"✅ Modelo PIT CON tuning guardado en: {save_path}")
model.to(device)
```

Nota. Elaboración propia

Finalmente, se exportaron las métricas de evaluación de ambos modelos en un archivo .csv, estructurado con la librería pandas. Esta exportación consolidó los

resultados obtenidos y permite su análisis comparativo de forma externa o su visualización mediante herramientas adicionales. La Figura 116 muestra el bloque de código correspondiente a esta operación.

Figura 116

Script para exportar métricas comparativas a archivo .csv

```
# Ruta y nombre con timestamp
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_dir = "/content/drive/MyDrive/2.CODIGOS/3.RESULTADOS/Deep_CSV"
csv_path = os.path.join(output_dir, f"PIT_MetricasComparativas_{timestamp}.csv")

# Guardar el DataFrame
df_resultados.to_csv(csv_path, index=False)
print(f" Resultados de métricas guardados en: {csv_path}")
```

Nota. Elaboración propia

- **MobileViT Transformer**

  - **Procedimiento 1: Preparación y normalización de imágenes**

Con el objetivo de adecuar las imágenes al formato requerido por la arquitectura MobileViT, se aplicó una serie de transformaciones que incluyeron el redimensionamiento uniforme a 224×224 píxeles, la conversión a tensores mediante transforms.ToTensor() y la normalización por canal utilizando los valores estándar de los modelos preentrenados en ImageNet: media [0.485, 0.456, 0.406] y desviación estándar [0.229, 0.224, 0.225]. Estas operaciones fueron implementadas utilizando transforms.Compose() de la librería torchvision, aplicándose de forma consistente sobre los conjuntos de entrenamiento, validación y prueba. Esta etapa permitió estandarizar la entrada del modelo, favoreciendo una convergencia estable durante el entrenamiento. La Figura 117 presenta el bloque de código correspondiente a estas transformaciones.

Figura 117

Script de transformación de datos para el modelo MobileViT

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])
```

Nota. Elaboración propia

- **Procedimiento 2: Definición de la arquitectura**

La arquitectura MobileViT fue implementada utilizando la librería timm, seleccionando la variante mobilevit_s.cvnets_in1k, correspondiente a un modelo ligero preentrenado sobre el conjunto ImageNet. Esta versión fue cargada especificando pretrained=True para reutilizar pesos previamente entrenados, y se adaptó a la tarea de clasificación binaria configurando num_classes=2. Posteriormente, el modelo fue transferido al dispositivo de procesamiento disponible (CPU o GPU) mediante la instrucción model.to(device), lo cual permitió aprovechar recursos computacionales de forma eficiente durante el entrenamiento. La Figura 118 presenta el bloque de código correspondiente a esta inicialización.

Figura 118

Script de carga y adaptación del modelo MobileViT

```
# Configurar el dispositivo
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Crear el modelo MobileViT preentrenado con 2 clases
model = timm.create_model('mobilevit_s.cvnets_in1k', pretrained=True, num_classes=2)

# Mover el modelo al dispositivo
model.to(device)
```

Nota. Elaboración propia

- **Procedimiento 3: Entrenamiento del modelo**

El modelo MobileViT fue entrenado en dos etapas consecutivas: una configuración inicial con hiperparámetros estándar, seguida de una segunda fase con hiperparámetros ajustados. Esta estrategia permitió comparar el rendimiento del modelo antes y después del proceso de optimización, manteniendo una estructura de entrenamiento coherente y controlada.

✓ Entrenamiento Modelo MobileViT Transformer base

En la primera fase, se utilizó la función de pérdida CrossEntropyLoss() junto con el optimizador Adam. Se configuró una tasa de aprendizaje de 1e-4, un tamaño de lote (batch size) de 32, y un límite de 30 épocas, incorporando un mecanismo de early stopping con una paciencia de 5 épocas. Durante cada iteración, se calcularon métricas de desempeño tanto para el conjunto de entrenamiento como para el de validación, conservando los pesos del modelo con mejor rendimiento validado. El código correspondiente a la configuración inicial, entrenamiento por lotes y cálculo de métricas se presenta en la Figura 119 y Figura 120.

Figura 119

Script de entrenamiento inicial del modelo MobileViT base

```
#Función de pérdida y optimizador
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-4)

#Configuración de entrenamiento
epochs = 30
patience = 5
best_loss = float('inf')
counter = 0

#Listas para almacenar pérdida y accuracy por época
train_losses = []
val_losses = []
train_acc = []  # Accuracy entrenamiento
val_acc = []    # Accuracy validación

#Bucle principal de entrenamiento
for epoch in range(epochs):
    model.train()
    running_loss = 0.0

    #Entrenamiento por batches
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

    #Promedio de pérdida de entrenamiento
    train_losses.append(running_loss / len(train_loader))

    # Calcular accuracy de entrenamiento
    correct = 0
    total = 0
    with torch.no_grad():
      for images, labels in train_loader:
          images, labels = images.to(device), labels.to(device)
          outputs = model(images)
          _, predicted = torch.max(outputs.data, 1)
          total += labels.size(0)
          correct += (predicted == labels).sum().item()
    train_accuracy = correct / total
    train_acc.append(train_accuracy)
```

Nota. Elaboración propia

Figura 120

Script de validación, control de early stopping y restauración del mejor modelo MobileViT base

```
    # -------------------------
    # VALIDACIÓN
    # -------------------------

    model.eval()
    val_loss = 0.0
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            val_loss += loss.item()

    val_loss /= len(val_loader)
    val_losses.append(val_loss)

   # Calcular accuracy de validación
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    val_accuracy = correct / total
    val_acc.append(val_accuracy)

   # Mostrar métricas por época
    print(f"Epoch {epoch+1}/{epochs} | Train Loss: {train_losses[-1]:.4f} | Val Loss: {val_loss:.4f} | "
          f"Train Acc: {train_accuracy:.4f} | Val Acc: {val_accuracy:.4f}")

    # Early Stopping
    if val_loss < best_loss:
        best_loss = val_loss
        counter = 0
        best_model_state = model.state_dict()
    else:
        counter += 1
        if counter >= patience:
            print("Early stopping aplicado.")
            break

# Cargar el mejor modelo guardado
model.load_state_dict(best_model_state)
```

Nota. Elaboración propia

✓ Entrenamiento Modelo MobileViT Transformer con tuning

Para mejorar el desempeño del modelo, se llevó a cabo una segunda fase de entrenamiento utilizando hiperparámetros ajustados manualmente, que se detallan en la

Tabla 19

Hiperparámetros utilizados en el entrenamiento del modelo MobileViT con Tuning

Hiperparámetro	**Valor asignado**
Learning rate	0.0001400513334413593
Batch size	16
Weight decay	0.00011906415282255917
Número de épocas	30
Paciencia (*early stop*)	5

Nota. Elaboración propia

Los nuevos valores fueron incorporados en la redefinición del optimizador y de los DataLoader, empleando una lógica de entrenamiento equivalente a la del modelo base. Las variables utilizadas para esta configuración fueron diferenciadas mediante sufijos _best, asegurando la separación de resultados y la trazabilidad de ambas ejecuciones. Las Figuras 121 y 122 presentan el código de inicialización y el ciclo completo de entrenamiento bajo estos parámetros ajustados.

Figura 121

Configuración y preparación del modelo MobileViT con Tuning

```
best_lr = 0.0001400513334413593
best_batch_size = 16
best_weight_decay = 0.0001190641528225591 7

# ================================
# DATALOADERS ACTUALIZADOS
# ================================
train_loader_best = DataLoader(train_dataset, batch_size=best_batch_size, shuffle=True)
val_loader_best = DataLoader(val_dataset, batch_size=best_batch_size, shuffle=False)

# ================================
# MODELO MobileViT (TIMM o HF)
# ================================
model_best = timm.create_model('mobilevit_s.cvnets_in1k', pretrained=True, num_classes=2)
model_best.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model_best.parameters(), lr=best_lr, weight_decay=best_weight_decay)

# ================================
# ENTRENAMIENTO CON EARLY STOPPING
# ================================
epochs = 30
patience = 5
best_loss = float('inf')
counter = 0

best_val_loss = float('inf')

# Listas SEPARADAS para el entrenamiento con Tuning
train_losses_best = []
val_losses_best = []
train_acc_best = []
val_acc_best = []

for epoch in range(epochs):
    model_best.train()
    running_loss = 0.0
    for images, labels in train_loader_best:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model_best(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

    # Guardar loss de entrenamiento
    train_losses_best.append(running_loss / len(train_loader_best))

    # Calcular accuracy de entrenamiento
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in train_loader_best:
            images, labels = images.to(device), labels.to(device)
            outputs = model_best(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    train_accuracy = correct / total
    train_acc_best.append(train_accuracy)
```

Nota. Elaboración propia

Figura 122

Entrenamiento y validación del modelo MobileViT con Tuning

```
# Validación
model_best.eval()
val_loss = 0.0
with torch.no_grad():
    for images, labels in val_loader_best:
        images, labels = images.to(device), labels.to(device)
        outputs = model_best(images)
        loss = criterion(outputs, labels)
        val_loss += loss.item()
val_loss /= len(val_loader_best)
val_losses_best.append(val_loss)

# Calcular accuracy de validación
correct = 0
total = 0
with torch.no_grad():
    for images, labels in val_loader_best:
        images, labels = images.to(device), labels.to(device)
        outputs = model_best(images)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
val_accuracy = correct / total
val_acc_best.append(val_accuracy)

# Imprimir resumen de la época
print(f"Epoch {epoch+1}/{epochs} - Train Loss: {train_losses_best[-1]:.4f} - "
      f"Val Loss: {val_loss:.4f} - Train Acc: {train_accuracy:.4f} - Val Acc: {val_accuracy:.

# Early Stopping
if val_loss < best_val_loss:
    best_val_loss = val_loss
    counter = 0
    best_model_state_best = model_best.state_dict()
else:
    counter += 1
    if counter >= patience:
        print("Early stopping aplicado")
        break
```

Nota. Elaboración propia

- **Procedimiento 4: Evaluación del desempeño del modelo**

Luego del entrenamiento, se evaluó el rendimiento de ambas versiones del modelo MobileViT (base y con tuning) utilizando el conjunto de prueba. Esta

evaluación tuvo como objetivo verificar la capacidad de generalización del modelo frente a datos no observados durante el entrenamiento.

El procedimiento consistió en recorrer las imágenes del test_loader en modo inferencia (model.eval()), obteniendo las predicciones mediante la función torch.max() aplicada sobre las probabilidades generadas por softmax(). Las etiquetas reales, predicciones y probabilidades por clase fueron almacenadas para el cálculo posterior de métricas.

Utilizando funciones de la librería sklearn.metrics, se calcularon las siguientes métricas de desempeño: exactitud (accuracy), precisión (precision), recuperación o sensibilidad (recall), puntaje F1 (F1-score) y área bajo la curva ROC (AUC-ROC).

Cada versión del modelo fue evaluada de forma independiente, empleando variables diferenciadas (como _best para el modelo con tuning), lo que permitió mantener la trazabilidad de los resultados. Las Figuras 123 y 124 presentan los scripts utilizados para esta evaluación estructurada.

Figura 123

Script de evaluación estructurada del modelo MobileViT base sobre el conjunto de prueba

```
model.eval()
y_true = []
y_pred = []
y_prob = []

with torch.no_grad():
    for images, labels in tqdm(test_loader, desc="Testing"):
        images = images.to(device)
        outputs = model(images)
        probs = torch.softmax(outputs, dim=1)
        _, preds = torch.max(probs, 1)
        y_true.extend(labels.numpy())
        y_pred.extend(preds.cpu().numpy())
        y_prob.extend(probs[:, 1].cpu().numpy())

acc = accuracy_score(y_true, y_pred)
prec = precision_score(y_true, y_pred)
rec = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)
auc_roc = roc_auc_score(y_true, y_prob)

#Imprimir resultados
print("\nResults")
print(f"Accuracy : {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall   : {rec:.4f}")
print(f"F1-Score : {f1:.4f}")
print(f"AUC-ROC  : {auc_roc:.4f}")
```

Nota. Elaboración propia

Figura 124

Script de evaluación estructurada del modelo MobileViT con Tuning

```
with torch.no_grad():
    for images, labels in tqdm(test_loader, desc="Testing (Tuning Model)"):
        images = images.to(device)
        outputs = model_best(images)
        probs = torch.softmax(outputs, dim=1)
        _, preds = torch.max(probs, 1)
        y_true_best.extend(labels.numpy())
        y_pred_best.extend(preds.cpu().numpy())
        y_prob_best.extend(probs[:, 1].cpu().numpy())

# Métricas finales para el modelo reentrenado (Tuning)
acc_best = accuracy_score(y_true_best, y_pred_best)
prec_best = precision_score(y_true_best, y_pred_best)
rec_best = recall_score(y_true_best, y_pred_best)
f1_best = f1_score(y_true_best, y_pred_best)
auc_roc_best = roc_auc_score(y_true_best, y_prob_best)

#Imprimir resultados
print("\nResults")
print(f"[Tuning Model] Accuracy  : {acc_best:.4f}")
print(f"[Tuning Model] Precision : {prec_best:.4f}")
print(f"[Tuning Model] Recall    : {rec_best:.4f}")
print(f"[Tuning Model] F1 Score  : {f1_best:.4f}")
print(f"[Tuning Model] AUC-ROC   : {auc_roc_best:.4f}")
```

Nota. Elaboración propia

Esta separación de variables permitió comparar de manera directa y organizada los resultados de ambas versiones del modelo, evitando interferencias entre ejecuciones y asegurando la trazabilidad de cada configuración. Así, se garantiza una documentación clara del proceso de evaluación y una base sólida para el análisis comparativo.

- **Procedimiento 5: Guardado de modelos y resultados**

Ambas versiones del modelo MobileViT —tanto la versión base como la optimizada mediante tuning— fueron almacenadas en formato .pth utilizando la función torch.save(). Para asegurar la compatibilidad del archivo con diferentes entornos, los modelos fueron trasladados previamente a CPU mediante la instrucción model.to('cpu'). Cada archivo fue nombrado dinámicamente incorporando un timestamp, lo que facilita

su identificación y trazabilidad en futuras implementaciones. Las Figuras 125 y 126 muestran los bloques de código correspondientes al guardado de ambas versiones del modelo.

Figura 125

Script para guardar el modelo MobileViT sin tuning en formato .pth

```
import torch
from datetime import datetime
import os

model.to('cpu')
output_dir = "/content/drive/MyDrive/2.CODIGOS/3.RESULTADOS/Deep_PTH"
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
save_path = os.path.join(output_dir, f"Mobile_SinTuning_{timestamp}.pth")

torch.save(model.state_dict(), save_path)
print(f"✅ Modelo Mobile Sin tuning guardado en: {save_path}")
model.to(device)  # volver a GPU para entrenar
```

Nota. Elaboración propia

Figura 126

Script para guardar el modelo MobileViT con tuning en formato .pth

```
model.to('cpu')
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
save_path = os.path.join(output_dir, f"Mobile_ConTuning_{timestamp}.pth")

torch.save(model.state_dict(), save_path)
print(f"✅ Modelo Mobile CON tuning guardado en: {save_path}")
model.to(device)
```

Nota. Elaboración propia

Además, se consolidaron las métricas de evaluación en un archivo .csv utilizando la librería pandas, lo que permitió preservar los resultados en un formato estructurado y reutilizable para análisis comparativos. Este archivo también fue nombrado automáticamente con fecha y hora, y almacenado en la ruta designada. La Figura 127 muestra el script utilizado para esta exportación.

Figura 127

Script para exportar métricas comparativas a archivo .csv

```
# Ruta y nombre con timestamp
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_dir = "/content/drive/MyDrive/2.CODIGOS/3.RESULTADOS/Deep_CSV"
csv_path = os.path.join(output_dir, f"Mobile_MetricasComparativas_{timestamp}.csv")

# Guardar el DataFrame
df_resultados.to_csv(csv_path, index=False)
print(f" Resultados de métricas guardados en: {csv_path}")
```

Nota. Elaboración propia

- **Desarrollo de la Metodología 2 (Vision Transformer y Machine Learning)**

A continuación, se detalla el desarrollo aplicado para la segunda metodología experimental, en la cual los modelos de Deep Learning fueron utilizados únicamente como extractores de características visuales a partir de imágenes de palmas de manos, mientras que la tarea de clasificación fue asumida por algoritmos tradicionales de Machine Learning.

- **BOTNet como extractor + Modelos de Machine Learning**
- **Procedimiento 1: Importación de librerías**

Antes de proceder con la carga del modelo BOTNet y la preparación del dataset, se realizó la importación de las librerías necesarias para el desarrollo de la metodología híbrida. Estas librerías permiten llevar a cabo operaciones como la lectura y preprocesamiento de imágenes, el uso de arquitecturas preentrenadas, la evaluación del modelo y la visualización de resultados.

La Figura 128 muestra el conjunto de módulos utilizados, destacando torch y timm para la carga del modelo, PIL y torchvision.transforms para las transformaciones de imagen, y scikit-learn para la ejecución de clasificadores tradicionales como SVM, Random Forest, KNN, Naïve Bayes y Decision Tree, junto con sus respectivas métricas.

Figura 128

Importación de librerías necesarias para la implementación del modelo BOTNet como extractor y su evaluación con clasificadores de Machine Learning.

```
import os
import torch
import timm
import numpy as np
from datetime import datetime
from PIL import Image
from torchvision import transforms
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix, roc_curve, ConfusionMatrixDisplay
import pandas as pd
import matplotlib.pyplot as plt
```

- **Procedimiento 2: Extracción de características con BOTNet**

En esta etapa se empleó la arquitectura BOTNet en su variante botnet26t_256, como se muestra en la Figura 128, cargada desde la librería timm y configurada con pretrained=False y num_classes=2. Los pesos previamente ajustados fueron recuperados desde un archivo .pth, generado en la primera metodología (Deep Learning puro), mediante la función load_state_dict() de PyTorch.

Una vez cargado el modelo, se eliminó la capa de clasificación y se configuró en modo evaluación, permitiendo su uso como extractor de características visuales sobre los subconjuntos train, val y test.

Figura 129

Configuración del modelo BOTNet como extractor mediante carga de pesos desde archivo .pth

```
model = timm.create_model('botnet26t_256', pretrained=False, num_classes=2)
modelo_path = "/content/drive/MyDrive/2.CODIGOS/3.RESULTADOS/Deep_PTH/BOTNet_ConTuning_20250619_170628.pth"
model.load_state_dict(torch.load(modelo_path, map_location=device))
model.eval().to(device)
```

A través del método forward_features(), se obtuvo la representación latente de cada imagen, es decir, el vector de características que resume la información visual relevante.

Este bloque de código se presenta en la Figura 130. Previamente, todas las imágenes fueron redimensionadas a 256×256 píxeles y normalizadas utilizando los parámetros estándar de ImageNet (media = [0.485, 0.456, 0.406]; desviación estándar = [0.229, 0.224, 0.225]), tal como se puede ver en la Figura 129. Esta extracción se realizó por clase y por partición (train, val, test), generando así tres conjuntos de matrices X acompañadas de sus correspondientes etiquetas y. Este último proceso se puede visualizar en la Figura 131.

Figura 130

Preprocesamiento de imágenes para BoTNet

```
transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])
```

Nota. Elaboración propia

Figura 131

Función para extraer features usando BoTNet

```
def extraer_features_desde_folder(folder_path, clase_label):
    X, y = [], []
    for archivo in os.listdir(folder_path):
        if archivo.lower().endswith(('.jpg', '.jpeg', '.png')):
            img_path = os.path.join(folder_path, archivo)
            img = Image.open(img_path).convert("RGB")
            tensor = transform(img).unsqueeze(0).to(device)

            with torch.no_grad():
                features = model.forward_features(tensor).squeeze().cpu().numpy().flatten()

            X.append(features)
            y.append(clase_label)
    return X, y
```

Nota. Elaboración propia

Figura 132

Función para generar los conjuntos de entrenamiento, validación y prueba con vectores de características extraídos por BOTNet

```
def cargar_dataset_completo():
    subconjuntos = ['train', 'val', 'test']
    data = {}

    for sub in subconjuntos:
        X_total, y_total = [], []
        for clase, etiqueta in clases.items():
            ruta = os.path.join(dataset_path, sub, clase)
            X, y = extraer_features_desde_folder(ruta, etiqueta)
            X_total.extend(X)
            y_total.extend(y)
        data[f'X_{sub}'] = np.array(X_total)
        data[f'y_{sub}'] = np.array(y_total)

    return data
```

Nota. Elaboración propia

- Procedimiento 3: Entrenamiento y evaluación de clasificadores tradicionales

A partir de los vectores de características extraídos con el modelo BOTNet, se organizaron los datos en subconjuntos de entrenamiento, validación y prueba, asegurando una adecuada separación de clases y etapas para preservar la validez experimental. Una vez estructurados los conjuntos (X_train, y_train, X_test, etc.), se procedió al entrenamiento de cinco algoritmos de clasificación tradicionales de Machine Learning: Support Vector Machine (SVM), Random Forest, K-Nearest Neighbors (K-NN), Naïve Bayes y Decision Tree.

El proceso fue automatizado mediante la función evaluar_modelos(), la cual entrena cada clasificador con los datos de entrenamiento y evalúa su desempeño en el conjunto de prueba utilizando métricas como accuracy, precision, recall, F1-score y AUC-ROC. Además, se generaron visualizaciones complementarias —curvas ROC y matrices de confusión— que permiten analizar gráficamente el rendimiento de cada modelo, como se muestra en la Figura 133.

Figura 133

Entrenamiento y evaluación de clasificadores tradicionales con los vectores generados por BOTNet

```python
def evaluar_modelos(data):
    resultados = []

    clasificadores = {
        "SVM": SVC(probability=True),
        "Random Forest": RandomForestClassifier(),
        "K-NN": KNeighborsClassifier(),
        "Naive Bayes": GaussianNB(),
        "Decision Tree": DecisionTreeClassifier()
    }

    for nombre, clf in clasificadores.items():
        clf.fit(data['X_train'], data['y_train'])
        y_pred = clf.predict(data['X_test'])
        y_proba = clf.predict_proba(data['X_test'])[:,1] if hasattr(clf, "predict_proba") else y_pred

        acc = accuracy_score(data['y_test'], y_pred)
        pre = precision_score(data['y_test'], y_pred)
        rec = recall_score(data['y_test'], y_pred)
        f1 = f1_score(data['y_test'], y_pred)
        roc = roc_auc_score(data['y_test'], y_proba)

        resultados.append({
            "Modelo": nombre,
            "Accuracy": acc,
            "Precision": pre,
            "Recall": rec,
            "F1 Score": f1,
            "ROC AUC": roc
        })
```

Nota. Elaboración propia

Una vez entrenados los clasificadores, se procede a evaluar su rendimiento utilizando métricas ampliamente aceptadas en problemas de clasificación binaria. Para

ello, se calculan los siguientes indicadores: accuracy, precision, recall, F1 score y ROC AUC. Estas métricas permiten una comparación objetiva del comportamiento de cada modelo y ofrecen una visión más profunda de su capacidad para distinguir entre casos positivos (anémicos) y negativos (no anémicos).

La evaluación se lleva a cabo dentro de la misma función evaluar_modelos(), mostrada previamente en la Figura 132, donde se observa cómo se obtienen las predicciones y probabilidades (y_pred, y_proba) para calcular cada métrica. Además, como se muestra en la Figura 133, se generan las curvas ROC por clasificador, que permiten visualizar gráficamente el compromiso entre sensibilidad y especificidad.

Figura 134

Generación de curvas ROC para cada modelo clasificador

```
# Calculate ROC curve
fpr, tpr, _ = roc_curve(data['y_test'], y_proba)
plt.figure()
plt.plot(fpr, tpr, label=f'ROC - {nombre} (AUC = {roc:.4f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title(f'ROC - Curve - {nombre}')
plt.legend(loc='lower right')
plt.grid()
plt.show()
```

Nota. Elaboración propia

También se construyen las matrices de confusión correspondientes para cada modelo, ilustradas en la Figura 134, donde puede observarse el detalle de verdaderos positivos, falsos negativos, etc., lo cual facilita una interpretación clínica del error. Esta visualización es clave para evaluar la aplicabilidad de los modelos en contextos reales de diagnóstico.

Figura 135

Matriz de confusión para el modelo aplicado al conjunto de prueba

```
# Confusion Matrix
disp = ConfusionMatrixDisplay.from_predictions(
    data['y_test'], y_pred,
    display_labels=["non Anemic", "anemic"],
    cmap=plt.cm.Blues,
    normalize=None
)
disp.ax_.set_title(f"Confusion Matrix - {nombre}")
plt.show()
```

Nota. Elaboración propia

- **Procedimiento 4: Exportación y visualización de resultados**

Para completar el flujo de trabajo, se consolidaron los resultados de evaluación de los cinco clasificadores entrenados en una tabla resumen que contiene todas las métricas calculadas. Esta tabla permite una comparación directa del rendimiento de cada modelo en cuanto a precisión, sensibilidad, especificidad y capacidad general de discriminación.

Los resultados fueron almacenados en un objeto DataFrame llamado df_resultados, el cual fue posteriormente exportado en formato .csv para su análisis externo y respaldo documental.

Esta exportación fue realizada mediante la función to_csv() de la biblioteca pandas, generando un archivo con nombre personalizado que incluye la fecha y hora de ejecución, como se ilustra en la Figura 135.

Figura 136

Exportación de resultados de métricas comparativas a un archivo .CSV con timestamp automático

```
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_dir = "/content/drive/MyDrive/2.CODIGOS/3.RESULTADOS/Deep_CSV"
csv_path = os.path.join(output_dir, f"BotNet_MetricasComparativas_{timestamp}.csv")
df_resultados.to_csv(csv_path, index=False)
print(f" Resultados exportados a: {csv_path}")
```

Nota. Elaboración propia

- **ViT como extractor + Modelos de Machine Learning**

- **Procedimiento 1: Importación de librerías**

Antes de implementar el modelo ViT como extractor de características, se llevó a cabo la importación de las librerías necesarias para el procesamiento de imágenes, la definición del modelo y la evaluación posterior mediante clasificadores tradicionales. En la Figura 137 se observa el conjunto de módulos utilizados, que incluyen torch, timm, sklearn, matplotlib, pandas y otros. Estas librerías fueron fundamentales para cargar el modelo entrenado, transformar las imágenes, entrenar los modelos tradicionales de ML y visualizar los resultados obtenidos.

Figura 137

Importación de librerías necesarias para la implementación del modelo ViT como extractor y su evaluación con clasificadores de Machine Learning.

```
import os
import numpy as np
import torch
import torch.nn as nn
import torchvision.transforms as transforms
from torchvision.datasets import ImageFolder
from torch.utils.data import DataLoader
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,roc_curve,f1_score, roc_auc_score, confusion_matrix)
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm
from datetime import datetime
import pandas as pd
import timm
```

- **Procedimiento 2: Extracción de características con ViT**

En este procedimiento se empleó la arquitectura Vision Transformer (ViT), específicamente la variante vit_base_patch16_224, como extractor de características visuales a partir de imágenes de palmas preprocesadas. Como se muestra en la Figura 138, el modelo fue cargado utilizando la librería timm, con los parámetros pretrained=False y num_classes=2.

Posteriormente, se cargaron los pesos previamente entrenados en la primera metodología (Deep Learning puro) desde un archivo .pth, mediante la función load_state_dict() de PyTorch.

Figura 138

Carga del modelo ViT entrenado en formato .pth como extractor de características

```
data_dir = "/content/drive/MyDrive/1.DATASET/3.DATASET_PREPRO_SPLIT_712/"
modelo_path = "/content/drive/MyDrive/2.CODIGOS/3.RESULTADOS/Deep_PTH/ViT_ConTuning_20250619_183602.pth"
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")


# ================================================
# 3. Definición y carga del modelo ViT entrenado
# ================================================
# Usamos timm para instanciar la misma variante entrenada
torch.manual_seed(42)
model = timm.create_model('vit_base_patch16_224', pretrained=False, num_classes=2)
# Cargar checkpoint
ckpt = torch.load(modelo_path, map_location=device)
# Extraer state_dict si viene anidado
state_dict = ckpt.get('state_dict', ckpt) if isinstance(ckpt, dict) else ckpt
# Ajustar claves para que coincidan
new_state = {}
for k, v in state_dict.items():
    # remueve prefijo si existe
    name = k.replace('model.', '').replace('vit.', '')
    new_state[name] = v
# Cargar pesos con strict=False para evitar errores de llaves adicionales
model.load_state_dict(new_state, strict=False)
model = model.to(device).eval()
print(" Modelo ViT cargado correctamente desde .pth")
```

Antes de pasar las imágenes al modelo, se aplicó un proceso de transformación que incluyó el redimensionamiento a 224 × 224 píxeles y la normalización basada en los valores de normalización estándar utilizados por los modelos entrenados en ImageNet. Esta etapa garantiza que las imágenes sean compatibles con los requerimientos de entrada del modelo ViT. La transformación se realizó mediante el siguiente bloque de código:

Figura 139

Definición de transformaciones de redimensionamiento y normalización para las imágenes, y creación de los DataLoaders para cada partición del dataset

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

train_ds = ImageFolder(os.path.join(data_dir, 'train'), transform=transform)
val_ds   = ImageFolder(os.path.join(data_dir, 'val'),   transform=transform)
test_ds  = ImageFolder(os.path.join(data_dir, 'test'),  transform=transform)

train_loader = DataLoader(train_ds, batch_size=32, shuffle=False)
val_loader   = DataLoader(val_ds,   batch_size=32, shuffle=False)
test_loader  = DataLoader(test_ds,  batch_size=32, shuffle=False)
```

Una vez cargado el modelo, este fue configurado en modo evaluación mediante la función model.eval(). A continuación, se empleó una estrategia basada en forward hook para interceptar el vector CLS generado por el último bloque del encoder del modelo ViT. Este vector fue utilizado como representación latente de cada imagen y constituye la base para la posterior clasificación mediante algoritmos de Machine Learning.

Figura 140

Extracción de características con ViT mediante hook

```python
def extract_features(model, loader):
    features, labels = [], []
    activation = {}
    # Hook para capturar CLS token
    def hook_fn(module, inp, outp):
        activation['cls'] = outp[:, 0]
    # Registrar hook en el último bloque\* del encoder
    # timm: model.blocks, torchvision: model.blocks too
    if hasattr(model, 'blocks'):
        hook = model.blocks[-1].register_forward_hook(hook_fn)
    else:
        hook = model.register_forward_hook(hook_fn)

    model.eval()
    with torch.no_grad():
        for imgs, lbls in tqdm(loader, desc="Extrayendo features"):
            imgs = imgs.to(device)
            _ = model(imgs)
            feats = activation['cls']
            features.append(feats.cpu().numpy())
            labels.append(lbls.numpy())
    hook.remove()
    return np.concatenate(features), np.concatenate(labels)

X_train, y_train = extract_features(model, train_loader)
X_val,   y_val   = extract_features(model, val_loader)
X_test,  y_test  = extract_features(model, test_loader)
```

- **Procedimiento 3: Entrenamiento y evaluación de clasificadores tradicionales**

Una vez extraídas las características visuales mediante ViT, se procedió a entrenar y evaluar distintos clasificadores tradicionales de Machine Learning. Para ello, se iteró sobre cada modelo definido en el diccionario classifiers, utilizando los vectores X_train y y_train para el ajuste, y X_test para la predicción.

Durante el proceso, se calcularon métricas fundamentales para cada clasificador: accuracy, precision, recall, F1-score y AUC-ROC. Estas métricas fueron impresas y almacenadas en una lista para su posterior análisis comparativo.

Además, para cada modelo se generaron dos visualizaciones clave: la matriz de confusión, que permite identificar los errores de clasificación según las clases reales y predichas, y la curva ROC, la cual evalúa el rendimiento del clasificador en función de distintos umbrales de decisión, facilitando el análisis de su capacidad discriminativa. El procedimiento descrito se presenta en la Figura 141.

Figura 141.

Evaluación de clasificadores tradicionales mediante métricas y visualización de resultados (matriz de confusión y curva ROC)

```python
def evaluar_modelos(data):
    resultados = []

    clasificadores = {
        "SVM": SVC(probability=True),
        "Random Forest": RandomForestClassifier(),
        "K-NN": KNeighborsClassifier(),
        "Naive Bayes": GaussianNB(),
        "Decision Tree": DecisionTreeClassifier()
    }

    for nombre, clf in clasificadores.items():
        clf.fit(data['X_train'], data['y_train'])
        y_pred = clf.predict(data['X_test'])
        y_proba = clf.predict_proba(data['X_test'])[:,1] if hasattr(clf, "predict_proba") else y_pred

        acc = accuracy_score(data['y_test'], y_pred)
        pre = precision_score(data['y_test'], y_pred)
        rec = recall_score(data['y_test'], y_pred)
        f1 = f1_score(data['y_test'], y_pred)
        roc = roc_auc_score(data['y_test'], y_proba)

        resultados.append({
            "Modelo": nombre,
            "Accuracy": acc,
            "Precision": pre,
            "Recall": rec,
            "F1 Score": f1,
            "ROC AUC": roc
        })

        # Calculate ROC curve
        fpr, tpr, _ = roc_curve(data['y_test'], y_proba)
        plt.figure()
        plt.plot(fpr, tpr, label=f'ROC - {nombre} (AUC = {roc:.4f})')
        plt.plot([0, 1], [0, 1], 'k--')
        plt.xlabel('False Positive Rate')
        plt.ylabel('True Positive Rate')
        plt.title(f'ROC - Curve - {nombre}')
        plt.legend(loc='lower right')
        plt.grid()
        plt.show()

        # Confusion Matrix
        disp = ConfusionMatrixDisplay.from_predictions(
            data['y_test'], y_pred,
            display_labels=["non Anemic", "anemic"],
            cmap=plt.cm.Blues,
            normalize=None
        )
        disp.ax_.set_title(f"Confusion Matrix - {nombre}")
        plt.show()

    return pd.DataFrame(resultados)
```

- **Procedimiento 4: Exportación de métricas y resultados**

Finalmente, se procede al almacenamiento y visualización de las métricas obtenidas por cada clasificador. Las métricas evaluadas incluyen Accuracy, Precisión, Recall, F1-score y AUC, las cuales son almacenadas en un archivo .csv con nombre dinámico según la fecha y hora de ejecución, como se muestra en la Figura 142. Esto permite documentar los resultados para posteriores análisis comparativos.

Figura 142

Exportación de métricas de desempeño de los clasificadores tradicionales.

```
df = pd.DataFrame(results, columns=['Model','Accuracy','Precision','Recall','F1','AUC'])
stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
csv_path = f"/content/drive/MyDrive/2.CODIGOS/3.RESULTADOS/Hibridos_CSV/ViT_Hibrido_Metricas_{stamp}.csv"
df.to_csv(csv_path, index=False)
print(f"\n✅ Resultados guardados en: {csv_path}")
```

Nota. Elaboración propia

- **Swin Transformer como extractor + Modelos de Machine Learning**
- **Procedimiento 1: Importación de librerías**

Para implementar el enfoque híbrido con el modelo Swin Transformer como extractor de características, se definió un entorno de trabajo que incluyó la importación

de librerías clave para el procesamiento de imágenes, construcción del modelo, entrenamiento, evaluación y visualización de resultados. Se utilizaron librerías como torch, timm y torchvision para el manejo del modelo preentrenado y las transformaciones de imágenes; sklearn para la implementación de clasificadores tradicionales (SVM, Random Forest, K-NN, Naïve Bayes y Decision Tree), así como para el cálculo de métricas como accuracy, precision, recall, F1-score y AUC-ROC; y matplotlib, seaborn y pandas para la visualización y exportación de resultados. Todo este conjunto de herramientas se muestra en la Figura X, donde se presenta el bloque de código correspondiente a la importación de librerías.

Figura 143

Importación de librerías necesarias para la extracción de características con Swin Transformer y la evaluación con clasificadores tradicionales de Machine Learning

```
import os, torch, timm, numpy as np, pandas as pd, matplotlib.pyplot as plt
from tqdm import tqdm
from PIL import Image
from datetime import datetime
from torchvision import transforms
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix, roc_curve, ConfusionMatrixDisplay
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

- **Procedimiento 2: Extracción de características con Swin Transformer**

En esta etapa se empleó el modelo Swin Transformer, previamente entrenado sobre el conjunto de datos de palmas de manos para la detección no invasiva de anemia. Como se muestra en la Figura 143, el modelo fue cargado desde el archivo .pth obtenido en la primera metodología (Deep Learning puro), haciendo uso del método load_state_dict() de PyTorch. Esta instancia del modelo se configuró en modo evaluación y se utilizó como extractor de características visuales sobre los subconjuntos train, val y test.

Figura 144

Carga del modelo Swin Transformer previamente entrenado desde archivo .pth para su uso como extractor de características.

```
dataset_path = "/content/drive/MyDrive/1.DATASET/3.DATASET_PREPRO_SPLIT_712/"
model_path = "/content/drive/MyDrive/2.CODIGOS/3.RESULTADOS/Deep_PTH/Swin_ConTuning_20250620_062326.pth"

] # ================================
  # 3. Carga del modelo Swin y preparación
  # ================================
  device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
  model = timm.create_model('swin_base_patch4_window7_224', pretrained=False, num_classes=2)
  model.load_state_dict(torch.load(model_path, map_location=device))
  model.head = torch.nn.Identity()  # Quitamos la capa de clasificación
  model.to(device)
  model.eval()
```

Nota. Elaboración propia

A continuación, se procede al preprocesamiento de las imágenes del conjunto de datos, con el objetivo de asegurar una entrada homogénea al modelo Swin Transformer. Las imágenes, que corresponden a la región de la palma de la mano de cada sujeto, se redimensionan a 224×224 píxeles, se convierten a tensores y se normalizan con los valores medios y desviaciones estándar estándar de ImageNet, como se aprecia en la Figura 144.

Figura 145

Transformaciones aplicadas a las imágenes de palmas y extracción de características latentes mediante Swin Transformer

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

def extract_features_from_folder(folder_path, label):
    features = []
    labels = []
    image_names = []
    for img_name in tqdm(os.listdir(folder_path)):
        img_path = os.path.join(folder_path, img_name)
        try:
            image = Image.open(img_path).convert("RGB")
            image = transform(image).unsqueeze(0).to(device)
            with torch.no_grad():
                feature = model(image).cpu().numpy().flatten()
            features.append(feature)
            labels.append(label)
            image_names.append(img_name)
        except:
            print(f"Error con imagen: {img_path}")
    return features, labels, image_names
```

Nota. Elaboración propia

A partir de las imágenes ya transformadas, se procede a recorrer el conjunto de datos estructurado en carpetas por clase y por partición (train, val y test) con el fin de extraer las representaciones vectoriales de cada imagen. Para ello, se hace uso de la función extract_features_from_folder(), la cual aplica el modelo Swin Transformer en modo evaluación sobre cada imagen y devuelve un vector numérico con las características latentes.

Como se muestra en la Figura 145, este proceso se repite para ambas clases (anemic y non-anemic) y para cada subconjunto, asignando las etiquetas correspondientes (1 para anémico y 0 para no anémico). Los vectores extraídos son posteriormente almacenados en estructuras diferenciadas por fase (train_features, val_features y test_features), junto con sus respectivas etiquetas y nombres de archivo, lo cual permite su uso posterior en modelos de clasificación.

Figura 146

Extracción de características latentes a partir de imágenes clasificadas de palmas de manos mediante el modelo Swin Transformer.

```
train_features, train_labels, _ = [], [], []
val_features, val_labels, _ = [], [], []
test_features, test_labels, test_names = [], [], []

for phase in ['train', 'val', 'test']:
    for class_name, label in [('anemic', 1), ('non-anemic', 0)]:
        folder = os.path.join(dataset_path, phase, class_name)
        feats, lbls, names = extract_features_from_folder(folder, label)
        if phase == 'train':
            train_features += feats
            train_labels += lbls
        elif phase == 'val':
            val_features += feats
            val_labels += lbls
        else:
            test_features += feats
            test_labels += lbls
            test_names += names
```

Nota. Elaboración propia

- **Procedimiento 3: Entrenamiento y evaluación de clasificadores tradicionales**

Una vez extraídas las características latentes de las imágenes de palmas mediante el modelo Swin Transformer, se procede al entrenamiento de distintos clasificadores de Machine Learning sobre estas representaciones vectoriales. En esta etapa, se utilizaron cinco algoritmos ampliamente conocidos: Support Vector Machine (SVM), Random Forest, K-Nearest Neighbors (KNN), Naïve Bayes y Decision Tree.

Cada clasificador fue entrenado utilizando el conjunto de datos correspondiente a la partición de entrenamiento (train), y evaluado sobre los subconjuntos de validación (val) y prueba (test). Para ello, se emplearon las funcionalidades de la biblioteca scikit-learn, garantizando un flujo reproducible y transparente.

Las métricas de desempeño consideradas en esta etapa incluyeron Accuracy, Precision, Recall, F1-Score y Área bajo la curva ROC (AUC), lo que permitió establecer comparaciones objetivas del rendimiento de cada clasificador. La Figura 146 presenta un fragmento del código que resume el entrenamiento y evaluación de los clasificadores tradicionales aplicados a las características extraídas con Swin Transformer.

Figura 147

Entrenamiento y evaluación de clasificadores tradicionales aplicados a las características extraídas mediante Swin Transformer.

```python
# 6. Clasificadores de ML
# ================================
X_train = np.array(train_features + val_features)
y_train = np.array(train_labels + val_labels)
X_test = np.array(test_features)
y_test = np.array(test_labels)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

models = {
    'SVM': SVC(probability=True),
    'Random Forest': RandomForestClassifier(),
    'K-NN': KNeighborsClassifier(),
    'Naive Bayes': GaussianNB(),
    'Decision Tree': DecisionTreeClassifier()
}

results = []
for name, clf in models.items():
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    y_proba = clf.predict_proba(X_test)[:, 1] if hasattr(clf, "predict_proba") else y_pred

    results.append({
        "Modelo": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred),
        "Recall": recall_score(y_test, y_pred),
        "F1-score": f1_score(y_test, y_pred),
        "AUC": roc_auc_score(y_test, y_proba)
    })


    # Curva ROC
    fpr, tpr, _ = roc_curve(y_test, y_proba)
    plt.figure(figsize=(8, 6))
    plt.plot(fpr, tpr, color='blue', lw=2, label=f'ROC (AUC = {roc_auc_score(y_test, y_proba):.4f})')
    plt.plot([0, 1], [0, 1], 'k--', lw=1)
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title(f'ROC Curve - {name}')
    plt.legend(loc="lower right")
    plt.grid()
    plt.show()

    # Matriz de Confusión
    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Non-Anemic', 'Anemic'])
    disp.plot(cmap='Blues', values_format='d')
    plt.title(f'Confusion Matrix - {name}')
    plt.show()
```

Nota. Elaboración propia

- **Procedimiento 4: Exportación de métricas y resultados**

Finalmente, tras la evaluación de los clasificadores tradicionales sobre las características extraídas mediante el modelo Swin Transformer, se procedió al almacenamiento de las métricas de desempeño. Estas métricas incluyeron valores de Accuracy, Precisión, Recall, F1-score y AUC, y fueron exportadas en un archivo .csv con nombre dinámico basado en la fecha y hora de ejecución, tal como se aprecia en la Figura 147.

Figura 148

Exportación de métricas de evaluación de los clasificadores tradicionales utilizando características extraídas con Swin Transformer.

```
df_results = pd.DataFrame(results)
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_csv = f"/content/drive/MyDrive/2.CODIGOS/3.RESULTADOS/Hibridos_CSV/Swin_Hibrido_Metricas_{timestamp}.csv"
df_results.to_csv(output_csv, index=False)
print(f"\n✅ Resultados exportados a: {output_csv}")
df_results
```

Nota. Elaboración propia

Esta etapa permite preservar los resultados obtenidos para su análisis comparativo con otros enfoques del estudio.

- **PiT como extractor + Modelos de Machine Learning**
- **Procedimiento 1: Importación de librerías**

Para dar inicio al modelado con la arquitectura PiT, se realizó la importación de todas las librerías necesarias para la ejecución del pipeline completo. Estas incluyen módulos para manipulación de datos (os, numpy, pandas), visualización

(matplotlib.pyplot), procesamiento de imágenes (PIL, torchvision.transforms), medición de desempeño (sklearn.metrics), así como los clasificadores de Machine Learning utilizados en etapas posteriores (SVM, Random Forest, K-NN, Naïve Bayes, Decision Tree). Además, se incorporaron las herramientas torch, timm y tqdm para la carga del modelo preentrenado, ejecución eficiente en GPU y seguimiento del proceso. Esta configuración inicial se muestra en la Figura X.

Figura 149

Importación de librerías necesarias para la implementación del modelo PiT y evaluación con clasificadores tradicionales.

```
import os
import torch
import timm
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tqdm import tqdm
from PIL import Image
from datetime import datetime
from torchvision import transforms
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix, roc_curve, ConfusionMatrixDisplay
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

- **Procedimiento 2: Extracción de características con PiT**

En esta etapa se empleó la arquitectura PiT (Pooling-based Vision Transformer), específicamente la variante pit_b_224, como extractor de características visuales sobre las imágenes de palmas de manos del conjunto de datos. Como se observa en la Figura 147, el modelo fue cargado desde la librería timm con la capa de clasificación deshabilitada (num_classes=2) y los pesos previamente entrenados almacenados en un archivo .pth, correspondiente al modelo ajustado en la primera metodología de Deep Learning puro.

Dicho archivo fue cargado mediante la función load_state_dict() de PyTorch, y la instancia del modelo fue configurada en modo evaluación (model.eval()) para ser utilizada únicamente como generador de representaciones latentes a partir de las imágenes, manteniendo su integridad durante la inferencia en los subconjuntos de entrenamiento, validación y prueba.

Figura 150

Configuración del modelo PiT como extractor de características mediante la carga de pesos desde archivo .pth.

```
dataset_path = "/content/drive/MyDrive/1.DATASET/3.DATASET_PREPRO_SPLIT_712/"
model_path = "/content/drive/MyDrive/2.CODIGOS/3.RESULTADOS/Deep_PTH/PIT_ConTuning_20250620_155746.pth"


# ====================================
# 3. Cargar modelo PiT y preparar como extractor
# ====================================
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = timm.create_model('pit_b_224', pretrained=False, num_classes=2)
model.load_state_dict(torch.load(model_path, map_location=device))
model.head = torch.nn.Identity()  # quitar capa final
model.to(device)
model.eval()
```

Además, se aplicaron transformaciones sistemáticas a las imágenes de palmas con el propósito de normalizar su formato de entrada y garantizar su compatibilidad con el modelo PiT. Las imágenes fueron redimensionadas a 224×224 píxeles, convertidas a tensores y normalizadas con los valores medios y desviaciones estándar de ImageNet.

Estas operaciones se implementaron dentro de la función extract_features_from_folder() y se aplicaron de manera uniforme a los subconjuntos train, val y test, como se visualiza en la Figura 148.

Figura 151

Transformaciones aplicadas a las imágenes de palmas y extracción de características mediante el modelo PiT.

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

def extract_features_from_folder(folder_path, label):
    features, labels, names = [], [], []
    for fname in tqdm(os.listdir(folder_path)):
        try:
            path = os.path.join(folder_path, fname)
            img = Image.open(path).convert("RGB")
            img = transform(img).unsqueeze(0).to(device)
            with torch.no_grad():
                feat = model(img).cpu().numpy().flatten()
            features.append(feat)
            labels.append(label)
            names.append(fname)
        except:
            print(f"Error procesando {fname}")
    return features, labels, names
```

Posteriormente, mediante la función extract_features_from_folder, se recorrieron las carpetas que contienen las imágenes clasificadas por clase. Para cada imagen, se aplicaron las transformaciones mencionadas y se obtuvieron las características latentes extraídas por el modelo previamente cargado. Estas características fueron almacenadas junto con sus respectivas etiquetas y nombres, completando así el proceso de extracción para su posterior uso en modelos de Machine Learning.

Figura 152

Organización de los vectores de características por conjunto y clase

```
train_features, train_labels, _ = [], [], []
val_features, val_labels, _ = [], [], []
test_features, test_labels, test_names = [], [], []

for phase in ['train', 'val', 'test']:
    for class_name, label in [('anemic', 1), ('non-anemic', 0)]:
        folder = os.path.join(dataset_path, phase, class_name)
        feats, lbls, names = extract_features_from_folder(folder, label)
        if phase == 'train':
            train_features += feats
            train_labels += lbls
        elif phase == 'val':
            val_features += feats
            val_labels += lbls
        else:
            test_features += feats
            test_labels += lbls
            test_names += names
```

- **Procedimiento 3: Entrenamiento y evaluación de clasificadores tradicionales**

Una vez obtenidas y organizadas las características extraídas por el modelo PiT, se entrenaron cinco algoritmos clásicos de clasificación de Machine Learning: Support Vector Machine (SVM), Random Forest, K-Nearest Neighbors (K-NN), Naïve Bayes y Decision Tree. Los datos fueron previamente estandarizados utilizando StandardScaler para mejorar la eficiencia del entrenamiento. Cada modelo fue ajustado con los vectores X_train y y_train, y evaluado con el conjunto X_test.

Se calcularon métricas de desempeño esenciales como Accuracy, Precision, Recall, F1-Score y AUC, cuyos resultados se almacenaron para su posterior análisis comparativo. Asimismo, se generaron automáticamente dos visualizaciones por modelo: la curva ROC, que permite observar la capacidad discriminativa del clasificador frente a distintos umbrales, y la matriz de confusión, que facilita la identificación de errores por clase. Este procedimiento se resume en la Figura X, donde se detalla el entrenamiento, evaluación y visualización de los resultados de cada clasificador utilizando las características derivadas de PiT.

Figura 153

Entrenamiento, evaluación y visualización de clasificadores clásicos a partir de características extraídas con PiT

```
# ====================================
# 6. Clasificadores de ML
# ====================================
X_train = np.array(train_features + val_features)
y_train = np.array(train_labels + val_labels)
X_test = np.array(test_features)
y_test = np.array(test_labels)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

models = {
    'SVM': SVC(probability=True),
    'Random Forest': RandomForestClassifier(),
    'K-NN': KNeighborsClassifier(),
    'Naive Bayes': GaussianNB(),
    'Decision Tree': DecisionTreeClassifier()
}

results = []

for name, clf in models.items():
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    y_proba = clf.predict_proba(X_test)[:, 1] if hasattr(clf, "predict_proba") else y_pred

    results.append({
        "Modelo": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred),
        "Recall": recall_score(y_test, y_pred),
        "F1-score": f1_score(y_test, y_pred),
        "AUC": roc_auc_score(y_test, y_proba)
    })

    # Curva ROC
    fpr, tpr, _ = roc_curve(y_test, y_proba)
    plt.figure(figsize=(8, 6))
    plt.plot(fpr, tpr, label=f'{name} (AUC = {roc_auc_score(y_test, y_proba):.4f})', linewidth=2)
    plt.plot([0, 1], [0, 1], 'k--', linewidth=1)
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title(f'Curva ROC - {name}')
    plt.legend(loc="lower right")
    plt.grid(True)
    plt.show()

    # Matriz de Confusión
    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Non-Anemic', 'Anemic'])
    disp.plot(cmap='Blues', colorbar=False)
    plt.title(f'Confusion Matrix - {name}')
    plt.grid(False)
    plt.show()
    ########################################
```

- **Procedimiento 4: Exportación de métricas y resultados**

Finalizado el proceso de evaluación, se procedió a exportar los resultados obtenidos a un archivo .csv para su posterior análisis y respaldo documental. Para ello, se utilizó la librería pandas para consolidar las métricas en un DataFrame y se generó un nombre de archivo dinámico incorporando un timestamp. Este archivo fue guardado en la ruta de trabajo establecida dentro de Google Drive, bajo el nombre Pit_Hibrido_Metricas_<timestamp>.csv, como se muestra en la Figura X.

Figura 154

Exportación automática de métricas de evaluación del modelo PiT a un archivo CSV

```
df_results = pd.DataFrame(results)
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_csv = f"/content/drive/MyDrive/2.CODIGOS/3.RESULTADOS/Hibridos_CSV/Pit_Hibrido_Metricas_{timestamp}.csv"
df_results.to_csv(output_csv, index=False)
print(f"\n Resultados exportados a: {output_csv}")
df_results
```

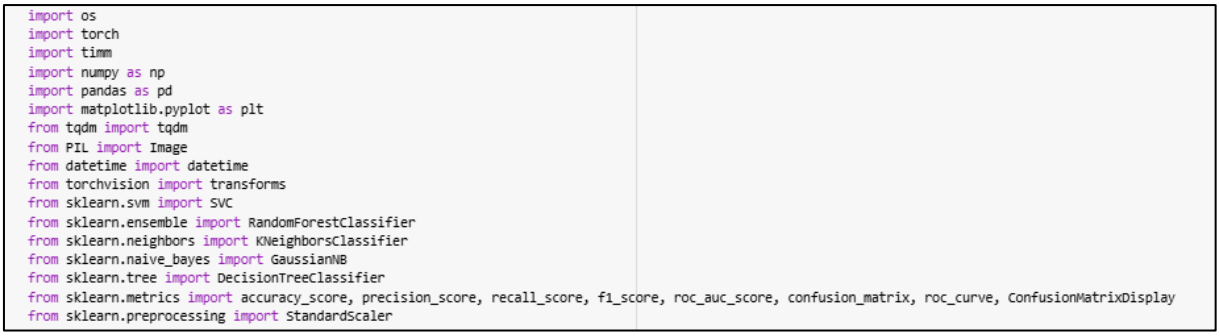
- **Mobile como extractor + Modelos de Machine Learning**

- **Procedimiento 1: Importación de librerías**

Este procedimiento establece el entorno de trabajo para la ejecución del modelo MobileViT híbrido. Se importan todas las librerías necesarias, incluyendo módulos para manipulación de datos (numpy, pandas), procesamiento de imágenes (PIL, torchvision), visualización (matplotlib), evaluación de modelos (sklearn.metrics) y clasificadores tradicionales (SVM, Random Forest, KNN, etc.).

Figura 155

Importación de librerías necesarias para MobileViT híbrido.



```
import os
import torch
import timm
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tqdm import tqdm
from PIL import Image
from datetime import datetime
from torchvision import transforms
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix, roc_curve, ConfusionMatrixDisplay
from sklearn.preprocessing import StandardScaler
```

- **Procedimiento 2: Extracción de características con MobileViT**

Luego de entrenar el modelo MobileViT de forma independiente como clasificador profundo, en el marco de la Metodología 1, se procedió a reutilizar dicha arquitectura como extractor de características dentro del enfoque híbrido. Esta estrategia permitió aprovechar las capacidades de representación visual del modelo entrenado, eliminando su capa de salida para obtener vectores numéricos que describen cada imagen.

Primero, se definieron las rutas del dataset preprocesado y del modelo entrenado en formato .pth, como se observa en la Figura 201. A continuación, se cargó el modelo MobileViT eliminando su capa de clasificación final (model.head = torch.nn.Identity()), con el fin de capturar las características internas generadas por la red. Luego, cada imagen del conjunto fue redimensionada a 224x224 píxeles, normalizada según los valores estándar de ImageNet, y pasada por el modelo para extraer su representación vectorial mediante la función extract_features_from_folder(), tal como se ve en la Figura XX

Figura 156

Definición de rutas y modelo entrenado para MobileViT.

```
# ====================================
# 2. Configuraciones y rutas
# ====================================
dataset_path = "/content/drive/MyDrive/1.DATASET/3.DATASET_PREPRO_SPLIT_712/"
model_path = "/content/drive/MyDrive/2.CODIGOS/3.RESULTADOS/Deep_PTH/Mobile_ConTuning_20250620_172632.pth"


# ====================================
# 3. Cargar modelo MobileViT y preparar como extractor
# ====================================
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = timm.create_model('mobilevit_s', pretrained=False, num_classes=2)
model.load_state_dict(torch.load(model_path, map_location=device))
model.head = torch.nn.Identity()  # quitar capa final
model.to(device)
model.eval()
```

Figura 157

Función para transformar imágenes y extraer vectores característicos usando MobileViT

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

def extract_features_from_folder(folder_path, label):
    features, labels, names = [], [], []
    for fname in tqdm(os.listdir(folder_path)):
        try:
            path = os.path.join(folder_path, fname)
            img = Image.open(path).convert("RGB")
            img = transform(img).unsqueeze(0).to(device)
            with torch.no_grad():
                feat = model(img).cpu().numpy().flatten()
            features.append(feat)
            labels.append(label)
            names.append(fname)
        except:
            print(f"Error procesando {fname}")
    return features, labels, names
```

Este procedimiento se aplicó a las tres particiones del dataset (train, val y test), y para cada clase (anemic, non-anemic), generando así las matrices X_train, X_val, X_test y sus

correspondientes etiquetas. La Figura 203 muestra cómo se recorren las carpetas del dataset y se estructuran los vectores que serán empleados por los modelos clásicos de Machine Learning.

Figura 158

Extracción de vectores de características desde el dataset clasificado.

```
train_features, train_labels, _ = [], [], []
val_features, val_labels, _ = [], [], []
test_features, test_labels, test_names = [], [], []

for phase in ['train', 'val', 'test']:
    for class_name, label in [('anemic', 1), ('non-anemic', 0)]:
        folder = os.path.join(dataset_path, phase, class_name)
        feats, lbls, names = extract_features_from_folder(folder, label)
        if phase == 'train':
            train_features += feats
            train_labels += lbls
        elif phase == 'val':
            val_features += feats
            val_labels += lbls
        else:
            test_features += feats
            test_labels += lbls
            test_names += names
```

- **Procedimiento 3: Entrenamiento y evaluación de clasificadores tradicionales**

Una vez extraídos los vectores característicos desde las imágenes utilizando el modelo MobileViT, se procedió al entrenamiento y evaluación de cinco clasificadores tradicionales de Machine Learning: Support Vector Machine (SVM), Random Forest, K-Nearest Neighbors (K-NN), Naïve Bayes y Decision Tree. Para ello, se organizaron los datos de entrenamiento (X_train, y_train) y prueba (X_test, y_test), aplicando un proceso de normalización mediante StandardScaler. A continuación, cada clasificador fue entrenado sobre el conjunto de entrenamiento y luego evaluado sobre el conjunto de prueba, generando métricas como accuracy, precision, recall, F1-score y ROC AUC, además de visualizar la curva ROC y la

matriz de confusión asociada. Este procedimiento se encuentra representado en la Figura 203.

Figura 159

Entrenamiento y evaluación de modelos de Machine Learning usando vectores extraídos por MobileViT

```
# ======================================
# 6. Clasificadores de ML
# ======================================
X_train = np.array(train_features + val_features)
y_train = np.array(train_labels + val_labels)
X_test = np.array(test_features)
y_test = np.array(test_labels)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

models = {
    'SVM': SVC(probability=True),
    'Random Forest': RandomForestClassifier(),
    'K-NN': KNeighborsClassifier(),
    'Naive Bayes': GaussianNB(),
    'Decision Tree': DecisionTreeClassifier()
}

results = []
for name, clf in models.items():
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    y_proba = clf.predict_proba(X_test)[:, 1] if hasattr(clf, "predict_proba") else y_pred

    results.append({
        "Modelo": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred),
        "Recall": recall_score(y_test, y_pred),
        "F1-score": f1_score(y_test, y_pred),
        "AUC": roc_auc_score(y_test, y_proba)
    })


    # Curva ROC
    fpr, tpr, _ = roc_curve(y_test, y_proba)
    plt.figure(figsize=(8, 6))
    plt.plot(fpr, tpr, label=f'{name} (AUC = {roc_auc_score(y_test, y_proba):.4f})', linewidth=2)
    plt.plot([0, 1], [0, 1], 'k--', linewidth=1)
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title(f'Curva ROC - {name}')
    plt.legend(loc="lower right")
    plt.grid(True)
    plt.show()

    # Matriz de Confusión
    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Non-Anemic', 'Anemic'])
    disp.plot(cmap='Blues', colorbar=False)
    plt.title(f'Confusion Matrix - {name}')
    plt.grid(False)
    plt.show()
```

- **Procedimiento 4: Exportación de métricas y resultados**

Finalizado el proceso de evaluación de los clasificadores tradicionales, se procedió a almacenar los resultados obtenidos en un archivo .csv. Para ello, se construyó un DataFrame a partir de las métricas de cada modelo y se guardó automáticamente con un nombre personalizado que incluye la fecha y hora de ejecución. Esta práctica permite organizar los experimentos de forma clara y reproducible, facilitando la comparación posterior entre arquitecturas y clasificadores. Este procedimiento se ilustra en la Figura 204.

Figura 160

Exportación automática de métricas de rendimiento en formato .csv para MobileViT híbrido

```
df_results = pd.DataFrame(results)
timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_csv = f"/content/drive/MyDrive/2.CODIGOS/3.RESULTADOS/Hibridos_CSV/MobileViT_Hibrido_Metricas_{timestamp}.csv"
df_results.to_csv(output_csv, index=False)
print(f"\n Resultados exportados a: {output_csv}")
df_results
```

4.4 Evaluación de Modelos

De acuerdo con las métricas consideradas para cada modelo se tienen los siguientes valores resultantes:

➢ **Evaluación de Modelos Metodología 1 (Vision Transformer)**

- **Modelo BOTNeT Sin Tuning**

El modelo BOTNet, en su configuración original sin ajuste de hiperparámetros, fue evaluado utilizando un conjunto de métricas estándar de clasificación binaria: exactitud (*accuracy*), precisión, sensibilidad (*recall*), puntuación F1 y el área bajo la curva ROC (*AUC-ROC*). Estas métricas fueron calculadas sobre el conjunto de prueba, conformado por imágenes no vistas de palmas de manos clasificadas como anémicas o no anémicas, con base en su nivel

de hemoglobina. En la Figura 83 se presenta el resumen cuantitativo de los valores obtenidos por el modelo.

Figura 161

Métricas de clasificación obtenidas por modelo BOTNet sin Tuning

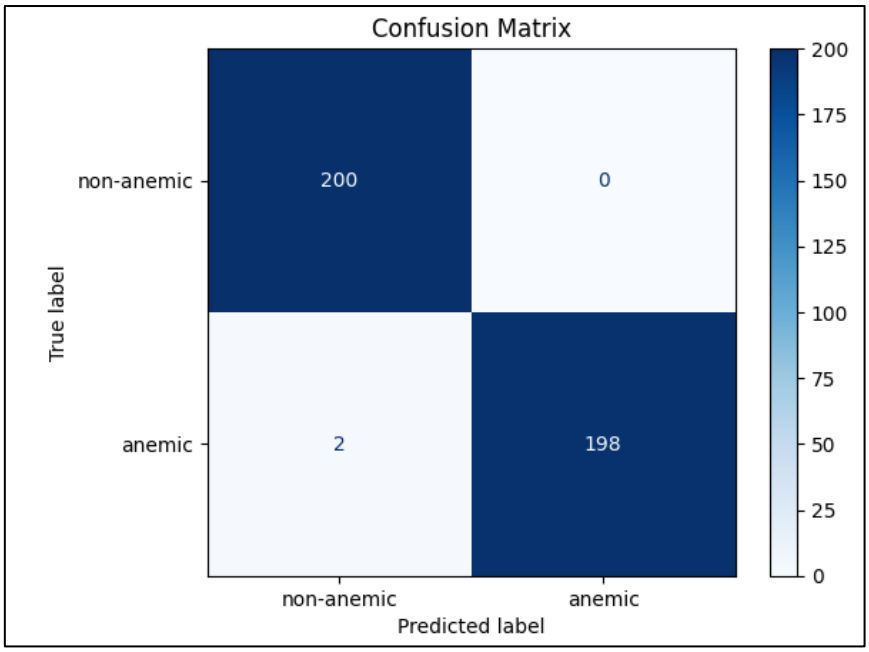
```
Results
Accuracy  : 0.9950
Precision : 1.0000
Recall    : 0.9900
F1 Score  : 0.9950
AUC-ROC   : 1.0000
```

Los resultados muestran un comportamiento altamente competitivo: el modelo alcanzó una accuracy de 0.9950, una precision de 1.0000 y una recall de 0.9900. La puntuación F1, que representa el equilibrio entre precisión y sensibilidad, fue de 0.9950, mientras que el valor de AUC-ROC alcanzó 1.0000. Estos indicadores revelan que el modelo fue capaz de detectar con gran fiabilidad los casos de anemia a partir de información visual extraída exclusivamente de la palma de la mano, lo cual resulta especialmente valioso en contextos donde se busca reducir la dependencia de análisis invasivos o costosos.

Para analizar con mayor detalle los aciertos y errores de clasificación, se construyó una matriz de confusión, como se muestra en la Figura 84. Esta herramienta permite identificar visualmente los verdaderos positivos, verdaderos negativos y los casos mal clasificados.

Figura 162

Matriz de confusión para el modelo BOTNet sin Tuning

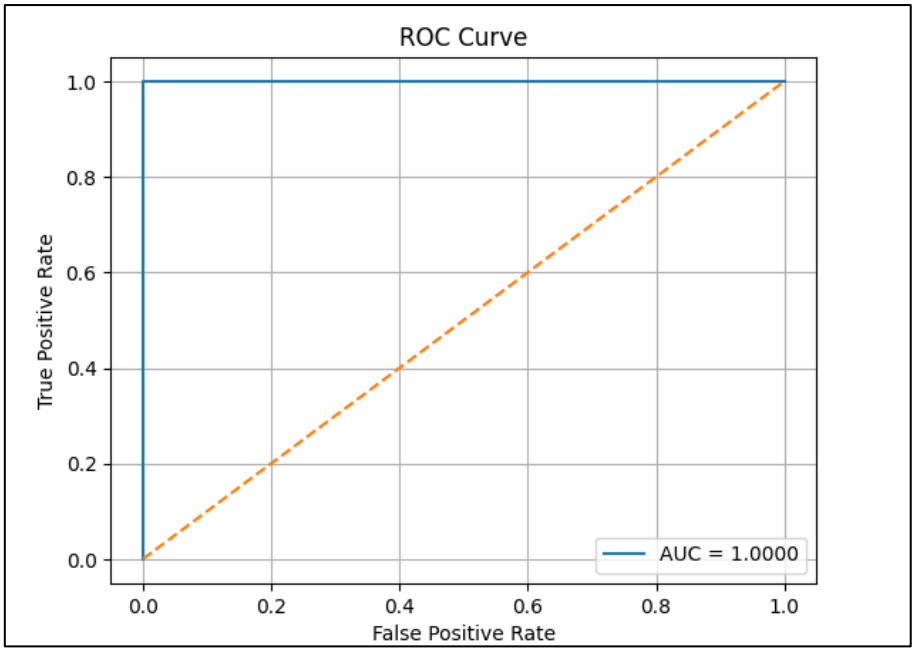


De los 200 casos anémicos reales en el conjunto de prueba, el modelo clasificó correctamente 198, incurriendo solo en dos falsos negativos. Por otro lado, los 200 individuos no anémicos fueron identificados sin errores, lo que explica la precisión perfecta registrada. Este resultado es especialmente relevante en aplicaciones clínicas, ya que minimizar los falsos positivos evita generar alarma innecesaria, mientras que reducir los falsos negativos ayuda a detectar tempranamente la condición anémica en pacientes que podrían no tener acceso inmediato a pruebas de laboratorio.

Complementando esta evaluación, la Figura 85 muestra la curva ROC que grafica el desempeño del modelo en función de distintos umbrales de decisión.

Figura 163

Curva ROC del modelo BOTNet sin Tuning

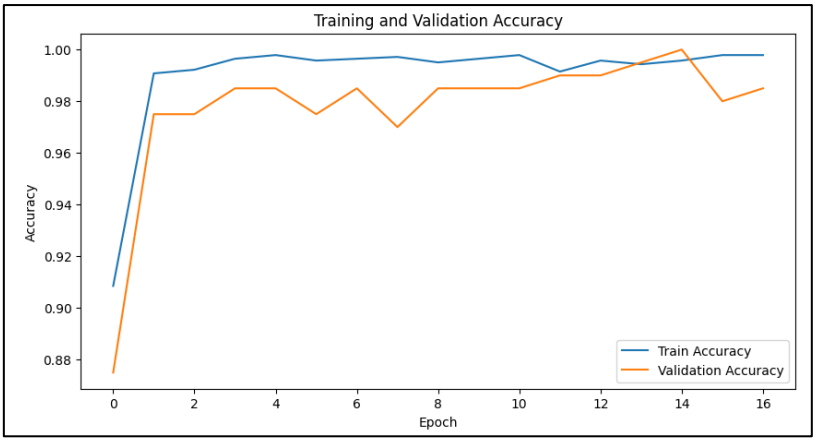


La curva asciende rápidamente hacia la esquina superior izquierda, lo cual representa un rendimiento ideal. El valor del área bajo la curva (AUC = 1.0000) confirma que el modelo tiene una capacidad discriminativa perfecta: es decir, es capaz de diferenciar sin ambigüedad entre un paciente anémico y uno no anémico a partir de la imagen de su palma. Este hallazgo respalda la viabilidad del modelo como herramienta preliminar de apoyo diagnóstico en poblaciones con limitado acceso a atención médica especializada.

Durante el proceso de entrenamiento, se monitorizó el comportamiento del modelo en términos de precisión sobre los datos de entrenamiento y validación. En la Figura 86 se presenta la evolución de la accuracy por época.

Figura 164

Precisión del entrenamiento y validación por época del modelo BOTNet sin tuning.

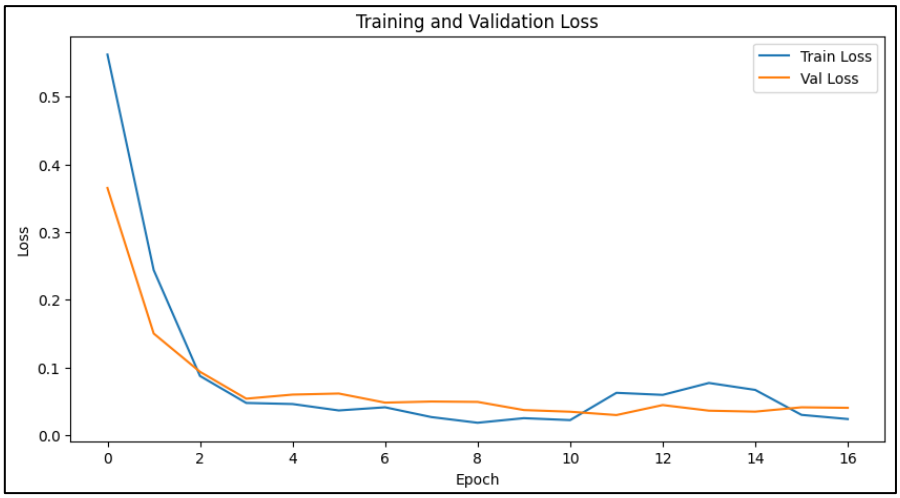


Desde las primeras épocas, el modelo logra una precisión superior al 98 % en ambos conjuntos, manteniéndose estable a lo largo del proceso. Esta cercanía entre ambas curvas sugiere que el modelo generaliza adecuadamente y no está sobreajustado a los datos de entrenamiento, una condición indispensable cuando se busca aplicar el modelo en contextos clínicos reales.

Finalmente, la **Figura 87** muestra la evolución de la función de pérdida en entrenamiento y validación.

Figura 165

Pérdida de entrenamiento y validación por época del modelo BOTNet sin tuning.



La pérdida decrece de forma sostenida y rápida en las primeras épocas, hasta estabilizarse en valores muy bajos. La consistencia entre la pérdida de entrenamiento y validación a lo largo del tiempo reafirma la estabilidad del modelo, indicando que no se produjo sobreajuste y que la red fue capaz de aprender representaciones efectivas sin comprometer su capacidad de generalización.

- **Modelo BOTNeT con Tuning**

Tras aplicar el proceso de ajuste de hiperparámetros (tuning) al modelo BOTNet, se evaluó su rendimiento utilizando el conjunto de prueba compuesto por imágenes de palmas clasificadas como anémicas o no anémicas. El objetivo fue determinar si los ajustes finos en parámetros como la tasa de aprendizaje, el tamaño de lote y la regularización ofrecían mejoras significativas respecto al modelo base. En la Figura 100 se resumen las métricas clave obtenidas tras la evaluación del modelo optimizado.

Figura 166

Métricas de clasificación obtenidas por modelo BOTNet sin Tuning

```
Results
[Tuning Model] Accuracy  : 0.9925
[Tuning Model] Precision : 0.9950
[Tuning ModelL] Recall    : 0.9900
[Tuning Model] F1 Score  : 0.9925
[Tuning Model] AUC-ROC   : 0.9999
```

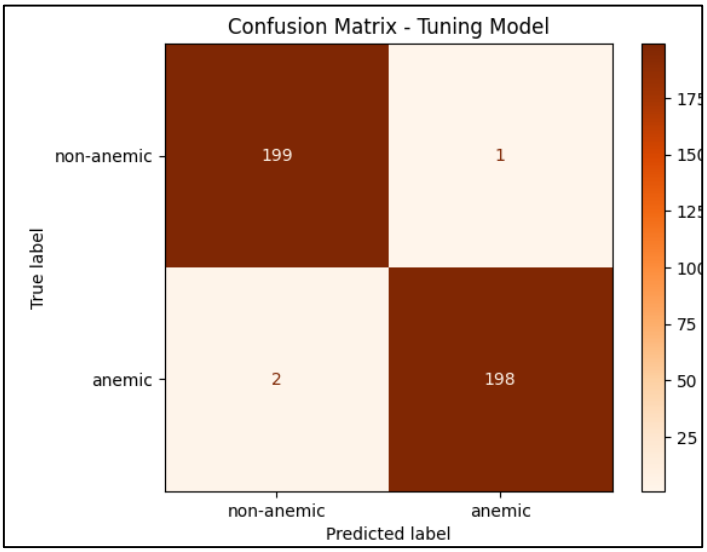
El modelo ajustado alcanzó una accuracy de 0.9925, una precision de 0.9950, una recall de 0.9900, una puntuación F1 de 0.9925 y un AUC-ROC de 0.9999. Estos valores demuestran un desempeño muy alto, prácticamente equivalente al modelo sin tuning, con una ligera variación en precisión. En términos de aplicación clínica, este comportamiento indica que el

modelo sigue siendo altamente confiable para discriminar entre palmas de personas con o sin anemia, reforzando su aplicabilidad como herramienta de detección preliminar no invasiva.

Para comprender mejor los aciertos y errores del modelo ajustado, se construyó la matriz de confusión, que se presenta en la **Figura 89**. Esta matriz permite visualizar los casos correctamente y erróneamente clasificados por el modelo.

Figura 167

Matriz de confusión para el modelo BOTNet con Tuning

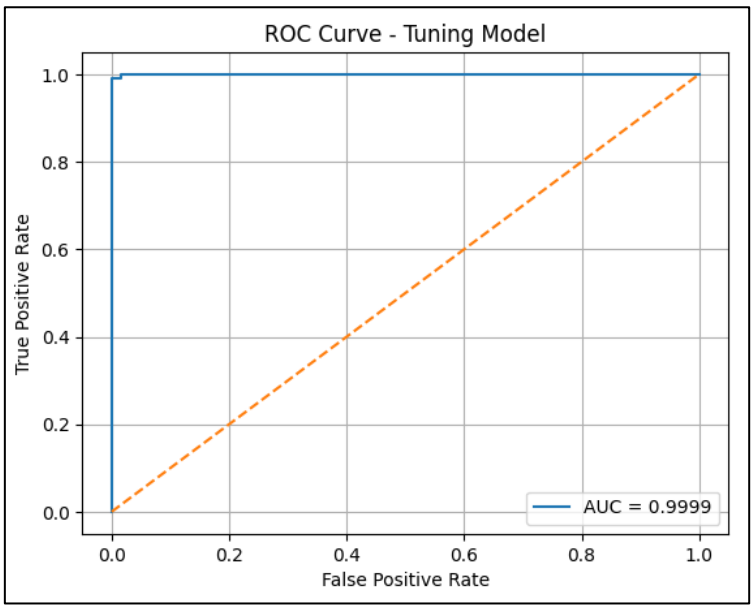


En este caso, el modelo identificó correctamente 198 de los 200 casos anémicos (2 falsos negativos) y 199 de los 200 casos no anémicos (1 falso positivo). Si bien se incrementó ligeramente el número de errores respecto al modelo base (que no presentó falsos positivos), la diferencia es mínima. El modelo sigue siendo altamente efectivo y mantiene un excelente equilibrio entre sensibilidad y precisión, lo cual es fundamental en escenarios de tamizaje clínico.

La Figura 143 presenta la curva ROC correspondiente al modelo afinado. Esta curva muestra cómo varía la tasa de verdaderos positivos en función de la tasa de falsos positivos a lo largo de distintos umbrales de decisión.

Figura 168

Curva ROC del modelo BOTNet con Tuning

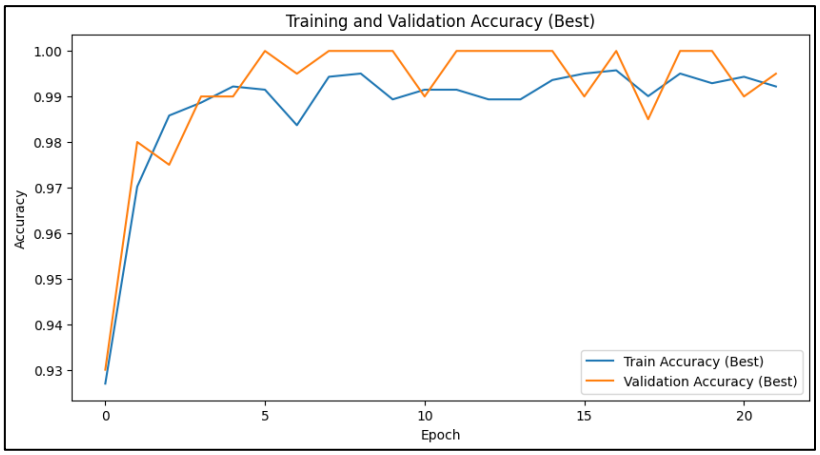


La curva describe una trayectoria casi ideal, con un AUC de 0.9999. Este valor indica que el modelo conserva su capacidad discriminativa prácticamente perfecta, lo que refuerza su utilidad para la detección visual temprana de anemia. La robustez de este rendimiento es crítica, considerando que el diagnóstico se basa únicamente en características visuales extraídas de imágenes de palmas.

Durante el proceso de entrenamiento del modelo afinado, se registró la evolución de la precisión sobre los conjuntos de entrenamiento y validación. En la **Figura 144**, se presenta la curva de precisión por época.

Figura 169

Precisión del entrenamiento y validación por época del modelo BOTNet con tuning.



A diferencia del modelo base, aquí se observa un comportamiento altamente estable, con precisión cercana o igual a 1.00 en validación a partir de la quinta época. Esta estabilidad es una indicación de que los hiperparámetros seleccionados favorecieron la rápida convergencia del modelo sin inducir sobreajuste. Las curvas muestran un cruce ocasional, pero convergen consistentemente hacia valores altos y cercanos entre sí.

La **Figura 145** muestra la evolución de la función de pérdida durante el entrenamiento del modelo con tuning, tanto para los datos de entrenamiento como para los de validación.

Figura 170

Pérdida de entrenamiento y validación por época del modelo BOTNet con tuning



Ambas curvas de pérdida muestran una disminución pronunciada en las primeras épocas y se estabilizan en valores mínimos. El comportamiento paralelo de las curvas confirma la

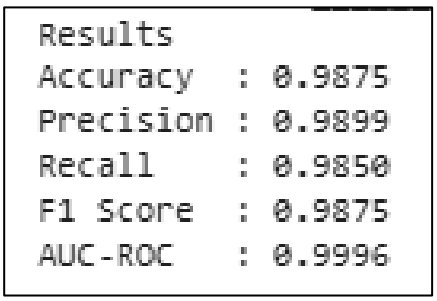
ausencia de sobreajuste, y valida la eficacia del tuning aplicado. Esto refuerza la conclusión de que el modelo ajustado mantiene su solidez predictiva a lo largo de todo el proceso de aprendizaje.

- **ViT sin Tuning**

El modelo Vision Transformer (ViT) en su configuración por defecto fue evaluado utilizando un conjunto de métricas clásicas en tareas de clasificación binaria: accuracy, precision, recall, F1-score y AUC-ROC. Estas métricas fueron calculadas a partir de las predicciones realizadas sobre el conjunto de prueba, compuesto por imágenes de palmas de manos etiquetadas como “anémicas” o “no anémicas”. En la Figura 110 se resumen los valores obtenidos.

Figura 171

Métricas de clasificación obtenidas por modelo ViT sin Tuning



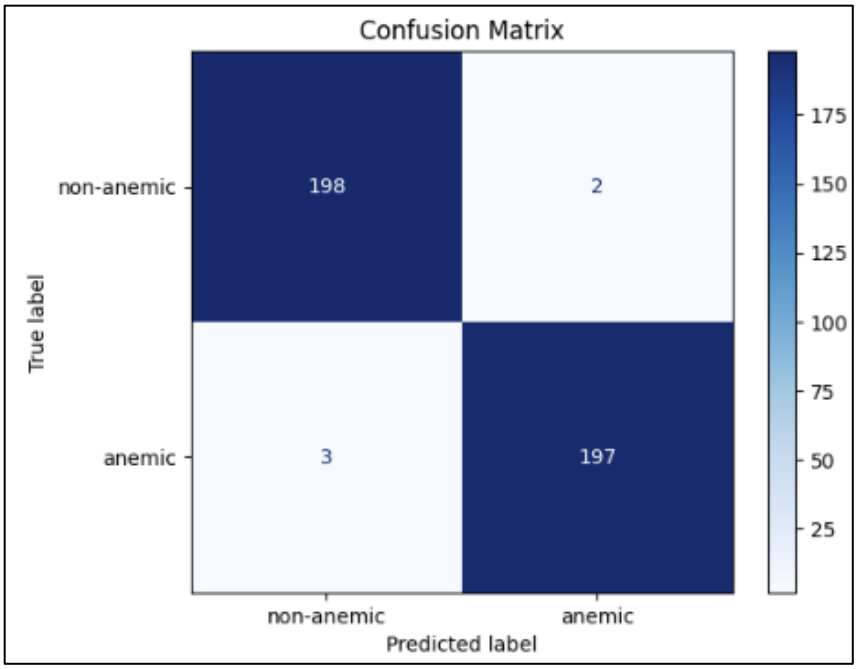
Nota: Elaboración propia

El rendimiento general del modelo fue muy elevado. Se registró una exactitud del 98.75 %, con una precisión cercana al 99 %, indicando que casi todas las predicciones positivas fueron correctas. La sensibilidad (recall), por su parte, alcanzó el 98.5 %, lo que refleja una alta capacidad del modelo para detectar casos positivos reales. El equilibrio entre ambas métricas queda evidenciado en el F1-score, también de 98.75 %, mientras que el AUC-ROC se ubicó en 0.9996, valor que sugiere una excelente capacidad discriminativa global.

Con el fin de comprender con mayor profundidad la distribución de aciertos y errores, se generó una matriz de confusión, que se presenta en la Figura 147.

Figura 172

Matriz de confusión para el modelo ViT sin Tuning



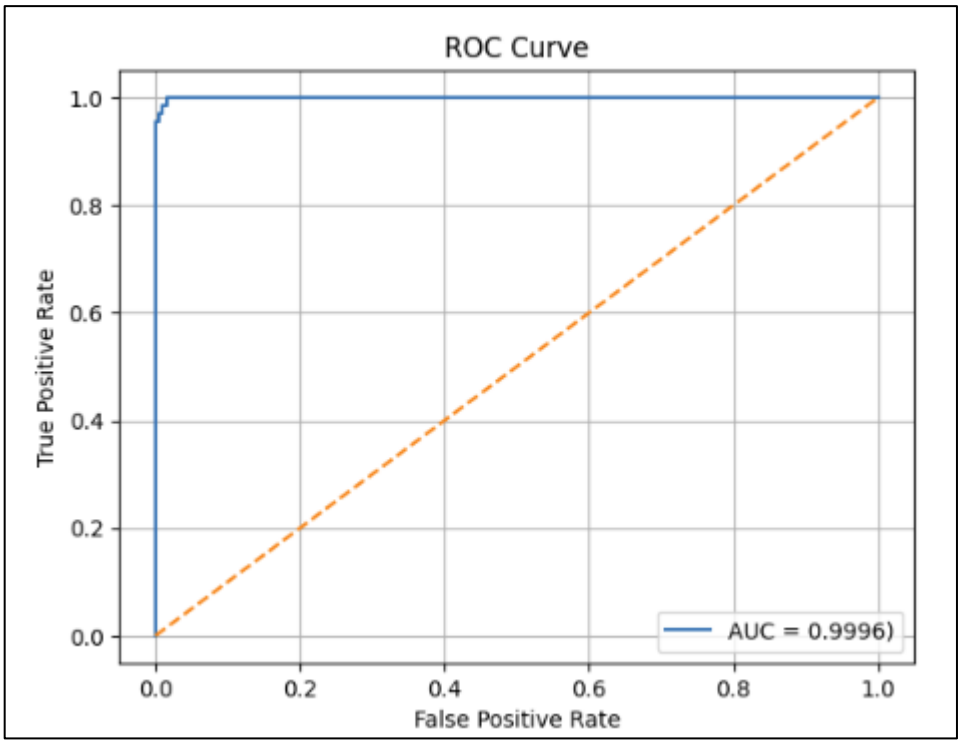
Nota: Elaboración propia

La matriz indica que el modelo identificó correctamente 198 individuos no anémicos, aunque cometió 2 falsos positivos. En cuanto a los casos anémicos, 197 fueron clasificados correctamente, con apenas 3 falsos negativos. Esta ligera asimetría, donde hay más errores al no detectar anemia que al clasificarla incorrectamente, tiene implicancias clínicas, ya que un falso negativo podría demorar la intervención médica oportuna.

El comportamiento del modelo en términos de sensibilidad y especificidad se ilustra con mayor claridad en la curva ROC, presentada en la Figura 148.

Figura 173

Curva ROC del modelo ViT sin Tuning



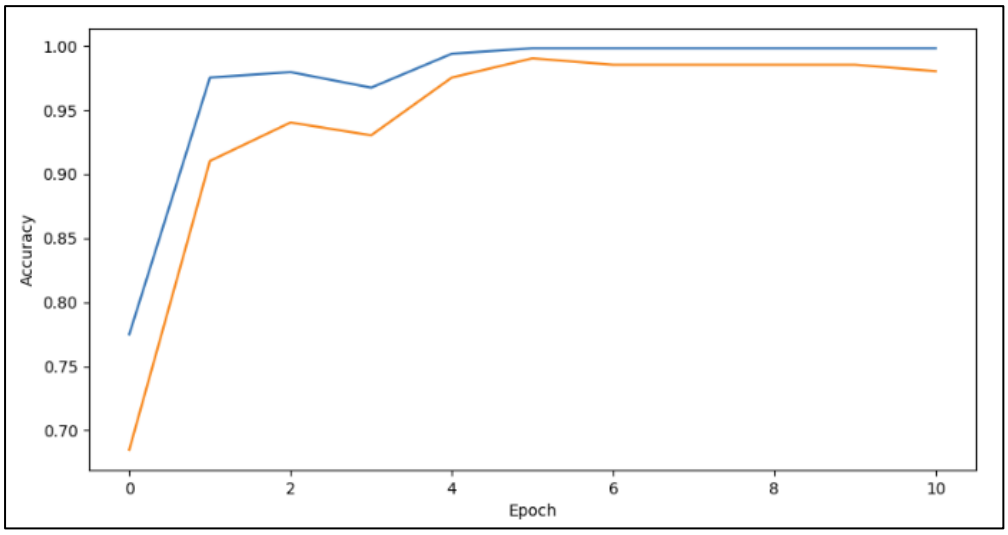
Nota: Elaboración propia

Como puede observarse, la curva se eleva abruptamente hacia el vértice superior izquierdo, lo que representa un rendimiento ideal. El área bajo la curva (AUC = 0.9996) ratifica la capacidad del modelo para distinguir de forma precisa entre las dos clases, sin importar el umbral de decisión seleccionado. Este rendimiento es indicativo de un modelo robusto y bien calibrado para fines de clasificación médica.

Durante la fase de entrenamiento, se hizo seguimiento continuo de las métricas de rendimiento sobre los conjuntos de entrenamiento y validación. En la Figura 149 se representa gráficamente la evolución de la precisión a lo largo de las épocas.

Figura 174

Precisión del entrenamiento y validación por época del modelo ViT sin tuning

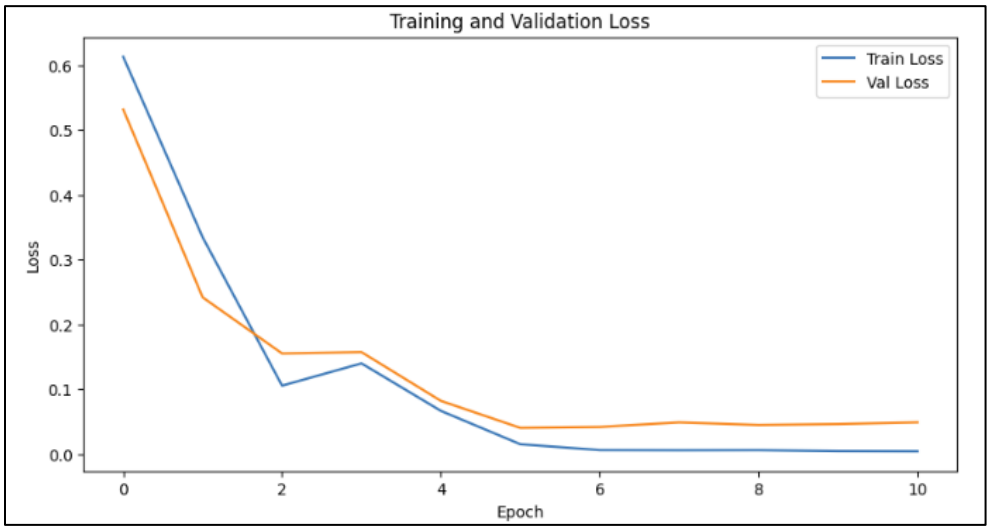


Desde las primeras iteraciones, el modelo logró alcanzar altos niveles de exactitud, superando el 95 % en validación en menos de cinco épocas. Este patrón indica un proceso de aprendizaje rápido, donde el modelo logra captar con eficacia las características relevantes sin señales evidentes de sobreajuste.

De forma complementaria, se presenta en la Figura 150 el comportamiento de la función de pérdida, tanto para el conjunto de entrenamiento como de validación.

Figura 175

Pérdida de entrenamiento y validación por época de modelo ViT sin tuning



Las curvas muestran una tendencia decreciente sostenida y paralela en ambas fases, lo que refleja una progresión estable del modelo. A partir de la quinta época, la pérdida se estabiliza en valores bajos y consistentes, reforzando la idea de una convergencia exitosa sin afectación por ruido o variabilidad excesiva en los datos.

- **ViT con Tuning**

El modelo ViT fue ajustado mediante una estrategia de tuning basada en exploración empírica de hiperparámetros, optimizando los valores de learning rate, batch size y weight decay. Posteriormente, se entrenó una nueva instancia del modelo con estos parámetros refinados y se evaluó sobre el conjunto de prueba compuesto por imágenes de palmas clasificadas como anémicas o no anémicas. La evaluación se realizó utilizando métricas estándar que permiten medir la precisión global, la sensibilidad al detectar casos positivos (anemia), y la capacidad del modelo para mantener un buen equilibrio entre sensibilidad y especificidad.

En la Figura 176 se presentan los valores cuantitativos obtenidos por el modelo ajustado tras el proceso de tuning. La exactitud (accuracy) fue de 0.9850, lo que indica que el modelo clasificó correctamente el 98.5 % de las imágenes. La precisión alcanzó un valor de 0.9755, señalando que casi todas las imágenes etiquetadas como anémicas realmente pertenecían a esa clase. El recall fue de 0.9950, lo que confirma que el modelo identificó prácticamente todos los casos anémicos reales. Finalmente, la puntuación F1 fue de 0.9851 y el AUC-ROC de 0.9984, valores que consolidan el alto desempeño del modelo en un entorno de clasificación binaria.

Figura 176

Métricas de clasificación obtenidas por modelo ViT con Tuning

```
Results
[Tuning Model] Accuracy  : 0.9850
[Tuning Model] Precision : 0.9755
[Tuning Model] Recall    : 0.9950
[Tuning Model] F1 Score  : 0.9851
[Tuning Model] AUC-ROC   : 0.9984
```

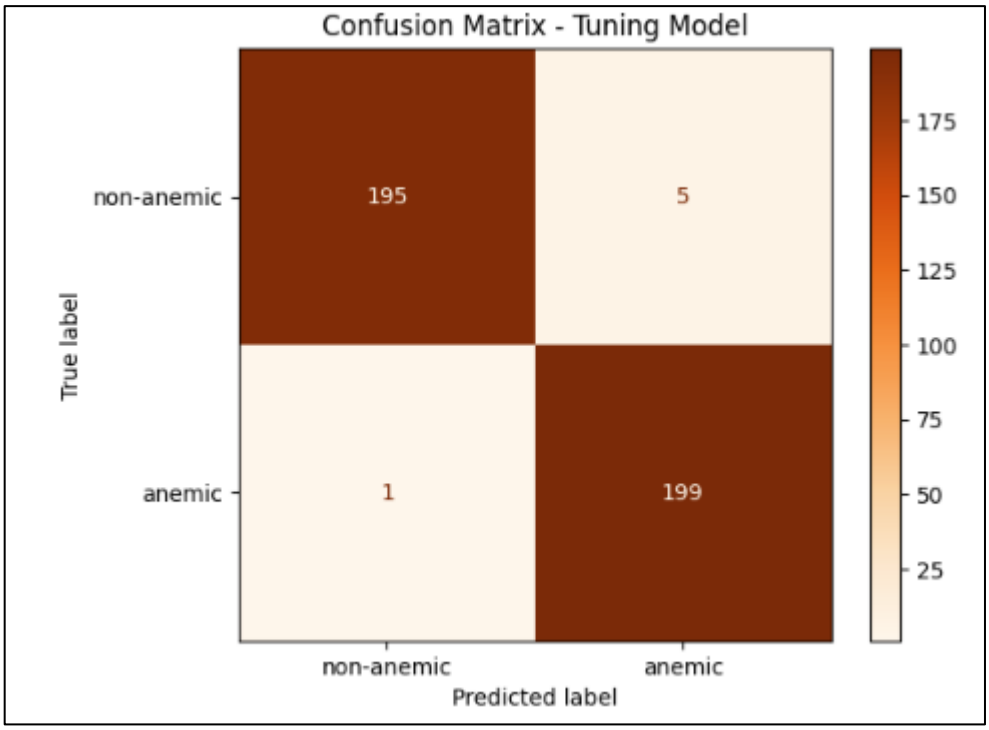
Nota: Elaboración propia

Estos resultados reflejan una gran capacidad del modelo para generalizar sobre datos no vistos. El elevado valor de recall y AUC resulta especialmente prometedor en aplicaciones clínicas donde es prioritario minimizar el riesgo de pasar por alto un caso positivo de anemia.

La Figura 177 muestra la matriz de confusión correspondiente al modelo ViT ajustado. De los 200 casos de pacientes anémicos, 199 fueron correctamente identificados, con tan solo un falso negativo. En cuanto a los pacientes no anémicos, el modelo cometió 5 falsos positivos, clasificando incorrectamente algunos casos sanos como anémicos.

Figura 177

Matriz de confusión para el modelo ViT con Tuning

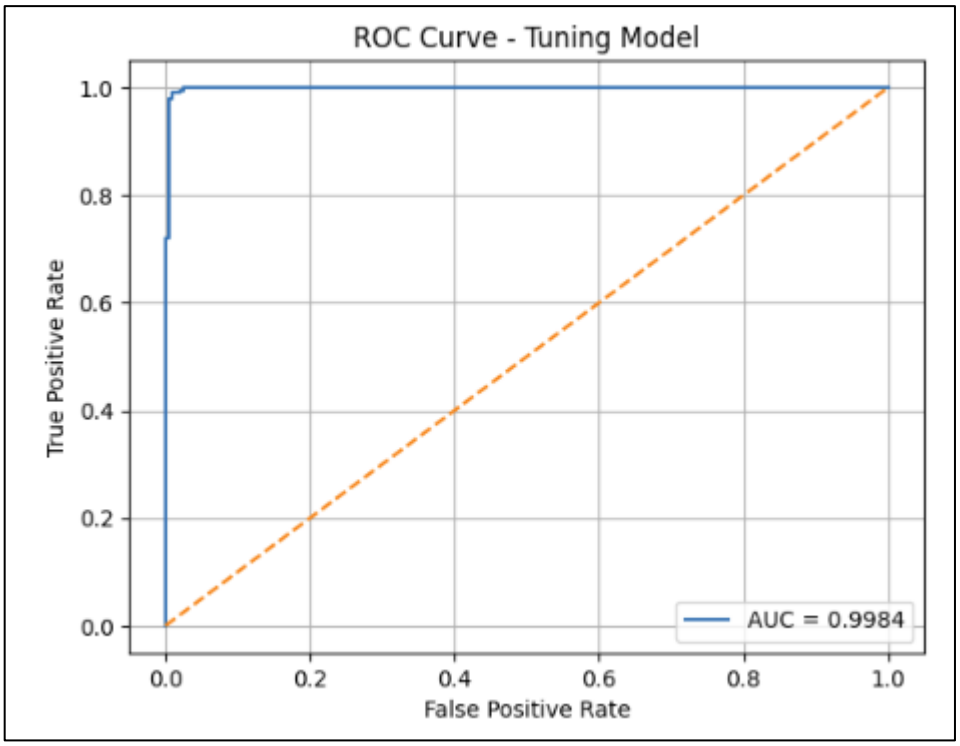


Este resultado, aunque mantiene un nivel alto de rendimiento, sugiere una leve pérdida de especificidad con respecto al modelo base. Sin embargo, en el contexto del diagnóstico de anemia, donde los falsos negativos representan un mayor riesgo, esta compensación puede ser clínicamente aceptable.

Complementando la evaluación, en la Figura 178 se observa la curva ROC del modelo ViT ajustado. Esta curva describe la relación entre la tasa de verdaderos positivos y la de falsos positivos a distintos umbrales de decisión. La curva se aproxima al borde superior izquierdo del gráfico, lo cual representa un comportamiento ideal, y el valor del AUC (0.9984) confirma la excelente capacidad del modelo para discriminar entre clases.

Figura 178

Curva ROC del modelo ViT con Tuning

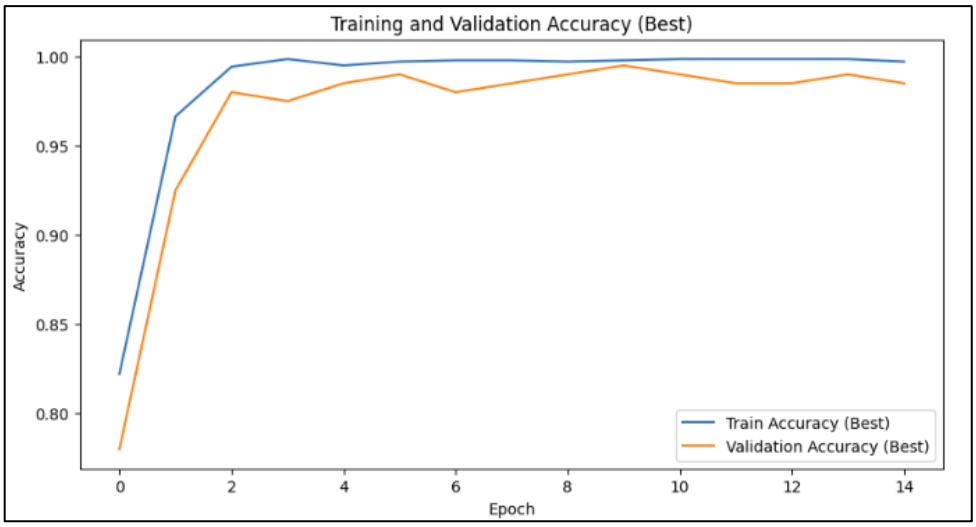


Una curva de estas características refuerza la utilidad del modelo como herramienta diagnóstica basada en imágenes, dado que su rendimiento se mantiene consistente incluso ante la variación de umbrales.

En términos de entrenamiento, la Figura 179 ilustra la evolución de la accuracy en los conjuntos de entrenamiento y validación durante las 15 épocas. Se observa una tendencia creciente en ambas curvas, alcanzando valores superiores al 99 % hacia las últimas iteraciones. La cercanía entre ambas curvas evidencia una buena capacidad de generalización y ausencia de sobreajuste.

Figura 179

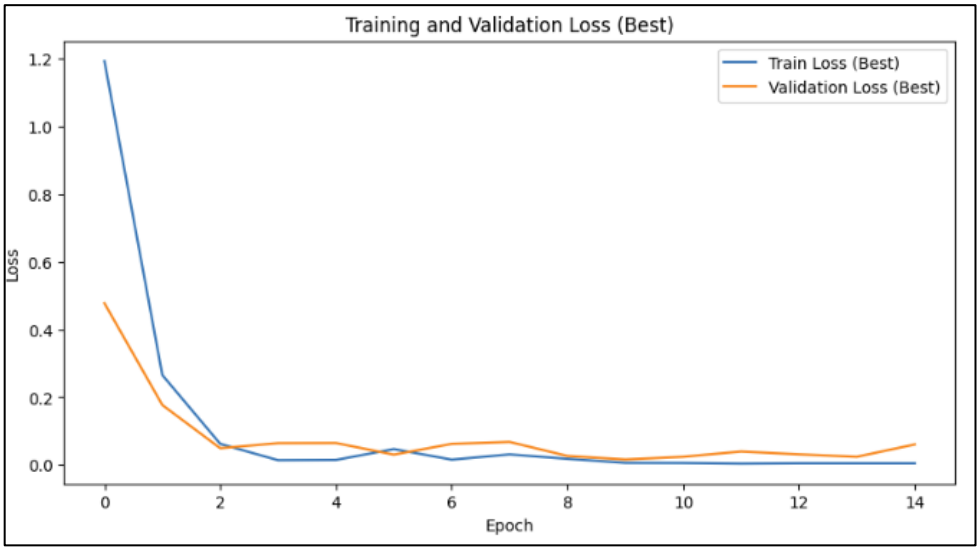
Precisión del entrenamiento y validación por época del modelo ViT con Tuning



Finalmente, la Figura 180 muestra el comportamiento de la función de pérdida. Esta disminuye rápidamente en las primeras épocas y luego se estabiliza en valores cercanos a cero. La similitud entre las curvas de pérdida de entrenamiento y validación respalda la estabilidad del modelo y confirma que el ajuste de hiperparámetros no comprometió su capacidad para generalizar correctamente.

Figura 180

Pérdida de entrenamiento y validación por época de modelo ViT con Tuning



En conjunto, los resultados del modelo ViT ajustado permiten afirmar que el proceso de tuning contribuyó a mejorar ligeramente la sensibilidad del modelo, reforzando su aplicabilidad para tareas clínicas sensibles como la detección temprana de anemia en poblaciones vulnerables.

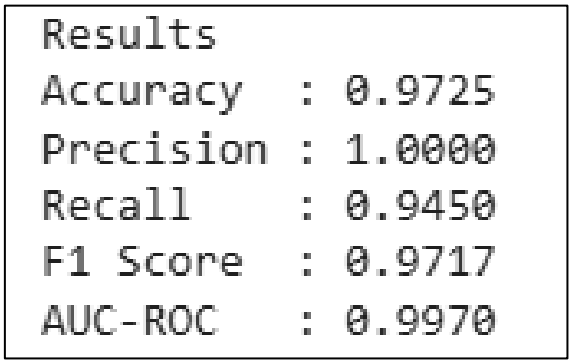
- **Swin Transformer sin Tuning**

El desempeño del modelo Swin Transformer, en su configuración estándar sin ajuste de hiperparámetros, fue evaluado sobre el conjunto de prueba con métricas comúnmente empleadas en clasificación binaria: exactitud (accuracy), precisión, sensibilidad (recall), puntuación F1 y área bajo la curva ROC (AUC-ROC). Estas métricas permiten valorar con precisión la capacidad del modelo para identificar imágenes de palmas pertenecientes a personas con anemia y sin anemia, exclusivamente a partir de señales visuales.

La Figura 181 resume los valores alcanzados por el modelo tras el entrenamiento base.

Figura 181

Métricas de clasificación obtenidas por modelo Swin Transformer sin Tuning



Nota: Elaboración Propia

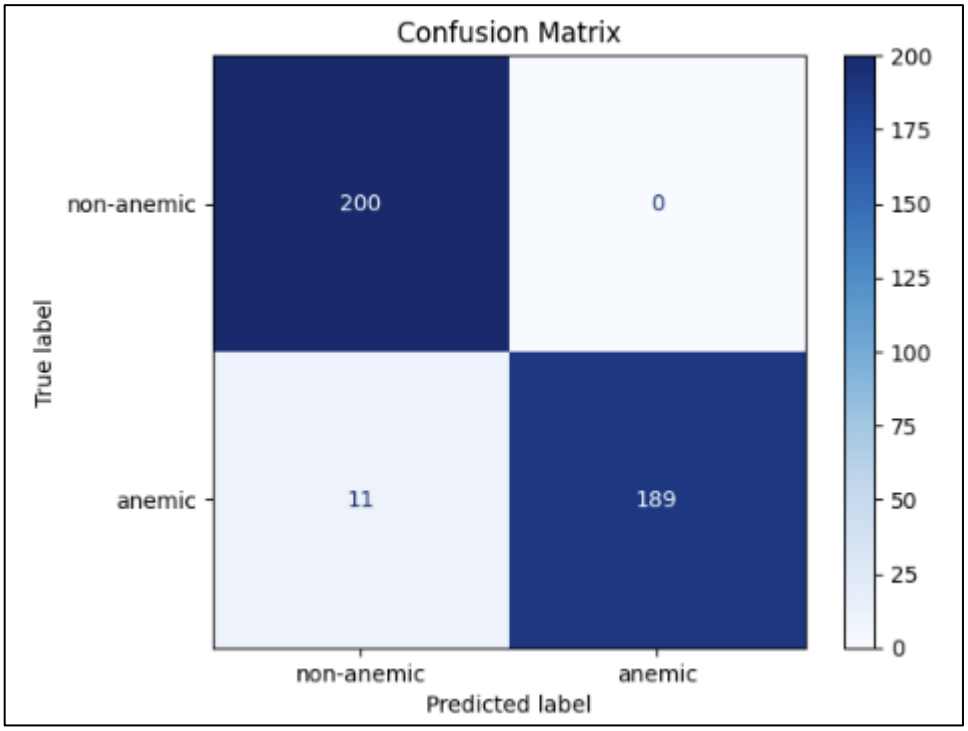
Los resultados revelan una exactitud del 97.25 % y una puntuación F1 de 0.9717, lo que sugiere un excelente equilibrio entre precisión y sensibilidad. La métrica AUC-ROC, con un valor de 0.9970, indica una alta capacidad de discriminación entre clases, incluso sin la

aplicación de estrategias de optimización. La precisión perfecta (1.0000) alcanzada por el modelo sugiere que no se han clasificado incorrectamente los casos de no-anemia, aunque la sensibilidad levemente menor (0.9450) evidencia que algunos casos anémicos fueron pasados por alto.

Para profundizar en el análisis de errores y aciertos, se construyó la matriz de confusión correspondiente como se muestra en la Figura 182.

Figura 182

Matriz de confusión para el modelo Swin Transformer sin Tuning



Nota: Elaboración Propia

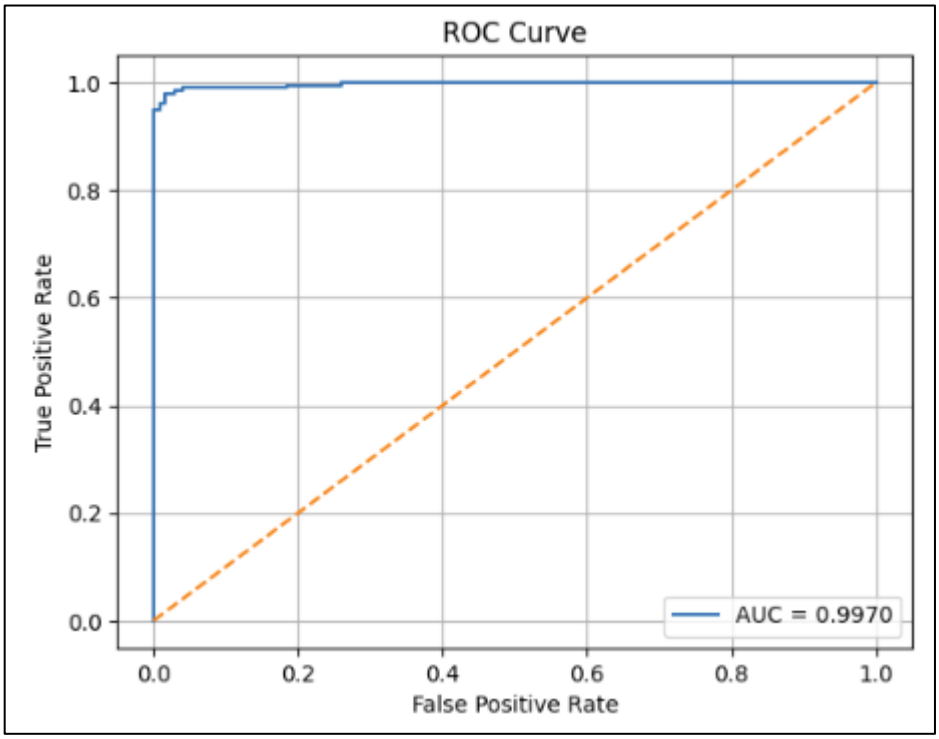
La matriz revela que de los 200 individuos no anémicos, todos fueron correctamente clasificados. En cambio, de los 200 casos de anemia, el modelo cometió 11 falsos negativos, lo que reduce ligeramente la sensibilidad. No obstante, el comportamiento sigue siendo altamente

satisfactorio para aplicaciones clínicas preliminares, al minimizar errores críticos en el diagnóstico.

Complementariamente, la Figura 183 presenta la curva ROC, que permite visualizar gráficamente la relación entre la tasa de verdaderos positivos y la de falsos positivos a lo largo de distintos umbrales de decisión.

Figura 183

Curva ROC del modelo Swin Transformer sin Tuning



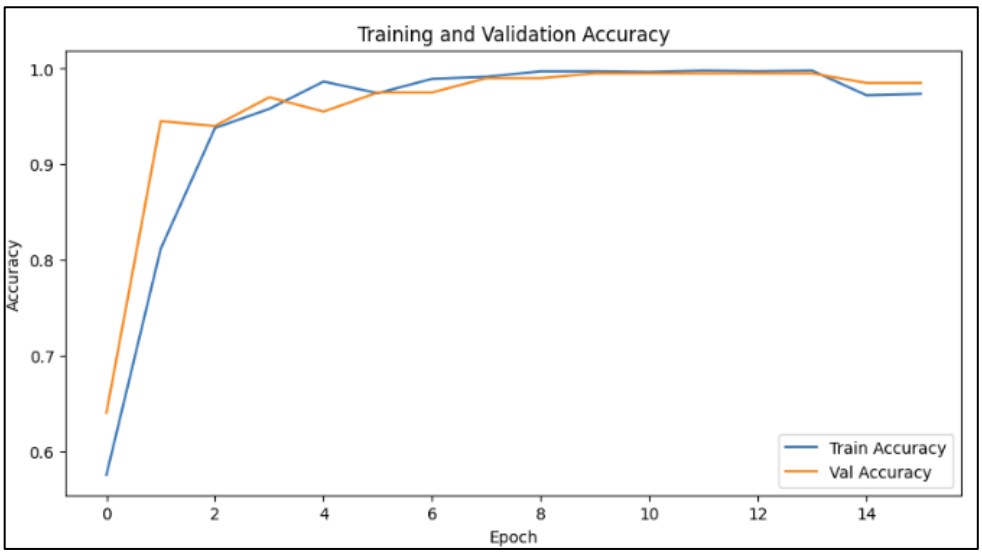
Nota: Elaboración Propia

Como se observa, la curva asciende bruscamente hacia la esquina superior izquierda del gráfico, comportamiento típico de modelos con excelente discriminación. El área bajo la curva (AUC = 0.9970) confirma esta observación y respalda el potencial del modelo para diferenciar entre individuos anémicos y no anémicos con un mínimo de ambigüedad.

Durante el entrenamiento del modelo, se registraron los valores de precisión en cada época tanto para el conjunto de entrenamiento como para el de validación. Estos se ilustran en la Figura 184.

Figura 184

Precisión del entrenamiento y validación por época del modelo Swin Transformer sin tuning



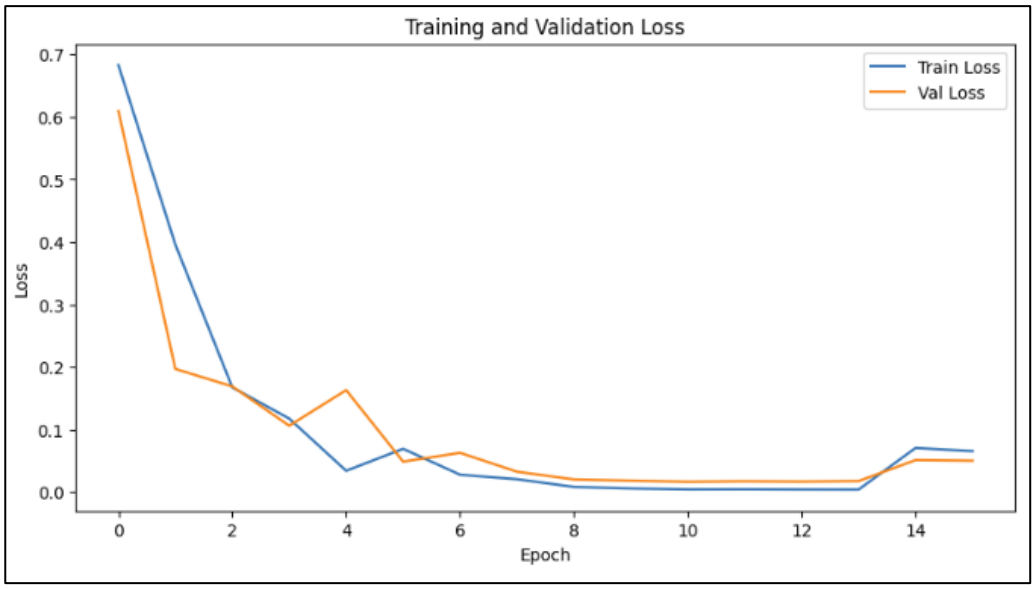
Nota: Elaboración Propia

A lo largo de las épocas, el modelo mostró una mejora constante en ambas curvas, alcanzando valores superiores al 99 % en el conjunto de entrenamiento y superiores al 97 % en el de validación. La diferencia entre ambas curvas fue moderada y estable, indicando que el modelo logró generalizar adecuadamente sin evidencia de sobreajuste.

Finalmente, se monitoreó la pérdida (loss) durante el entrenamiento, cuyo comportamiento se presenta en la Figura 185.

Figura 185

Pérdida de entrenamiento y validación por época de modelo Swin Transformer sin tuning



Nota: Elaboración Propia

Tanto la pérdida en entrenamiento como en validación disminuyeron de manera sostenida durante las primeras épocas, alcanzando valores cercanos a cero. La estabilidad observada hacia las últimas épocas sugiere una convergencia eficiente del modelo y un entrenamiento exitoso que preserva la capacidad de generalización sin comprometer la precisión.

- **Swin Transformer con Tuning**

Tras el ajuste empírico de los hiperparámetros mediante pruebas controladas, se obtuvo un modelo optimizado de Swin Transformer. Este modelo fue evaluado sobre el conjunto de prueba utilizando métricas de clasificación binaria. En la Figura 186 se resume el rendimiento alcanzado en términos de exactitud (accuracy), precisión, sensibilidad (recall), F1-score y AUC-ROC.

Figura 186

Métricas de clasificación obtenidas por modelo Swin Transformer con Tuning

```
Results
[Tuning Model] Accuracy  : 0.9875
[Tuning Model] Precision : 0.9851
[Tuning ModelL] Recall    : 0.9900
[Tuning Model] F1 Score  : 0.9875
[Tuning Model] AUC-ROC   : 0.9995
```

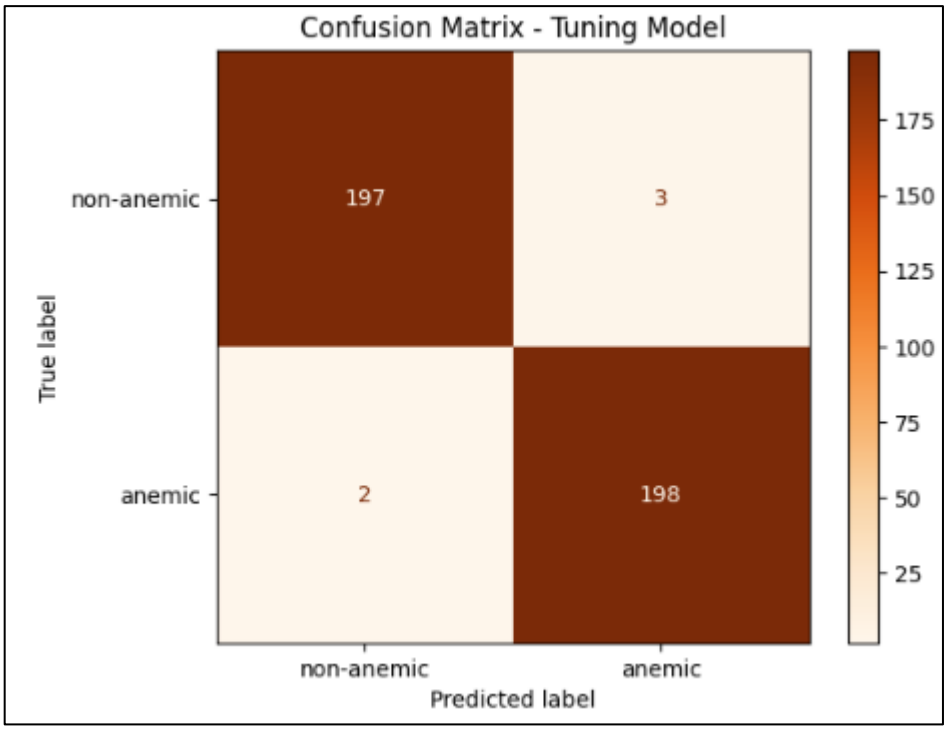
Nota: Elaboración Propia

Los resultados evidencian un comportamiento excepcional: el modelo logró una accuracy de 0.9875, una precision de 0.9851 y un recall de 0.9900. La puntuación F1 también fue elevada (0.9875), indicando un buen equilibrio entre precisión y sensibilidad. Finalmente, el valor del AUC-ROC fue de 0.9995, lo que denota una capacidad discriminativa muy alta al distinguir entre casos anémicos y no anémicos a partir de imágenes de las palmas.

Para complementar este análisis, se empleó la matriz de confusión ilustrada en la Figura 187, la cual permite examinar visualmente la distribución de los aciertos y errores.

Figura 187

Matriz de confusión para el modelo Swin Transformer con Tuning

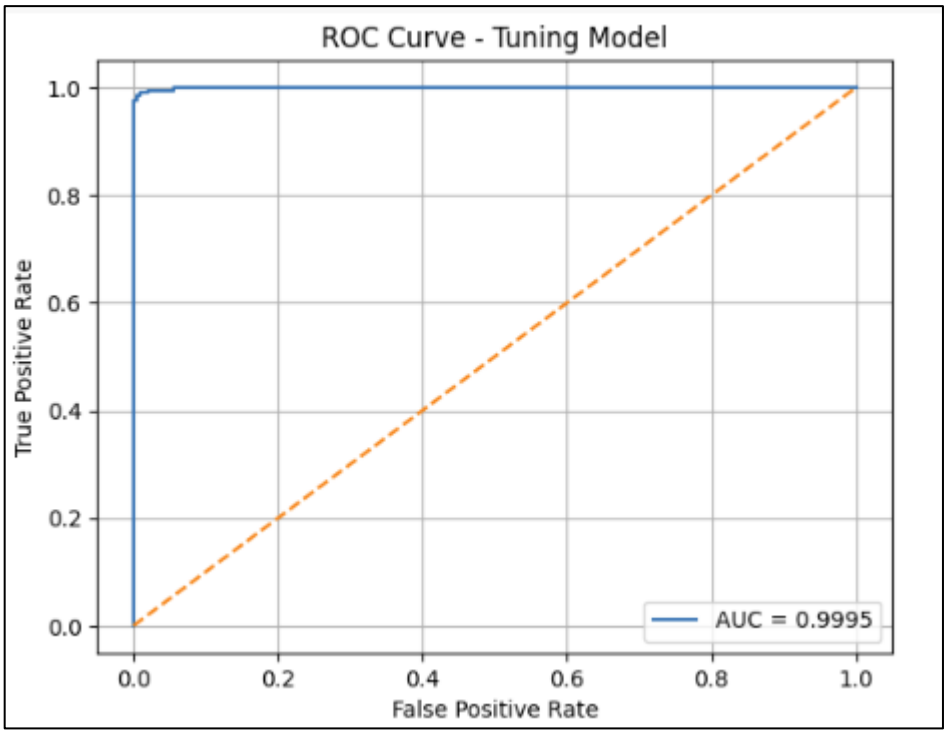


En términos cuantitativos, de los 200 individuos etiquetados como no anémicos, el modelo clasificó correctamente a 197 y cometió 3 falsos positivos. Por su parte, entre los 200 sujetos anémicos, 198 fueron identificados correctamente, incurriéndose solo en dos falsos negativos. Este patrón evidencia que el modelo optimizado conserva un comportamiento altamente robusto y generaliza con efectividad sin comprometer la sensibilidad clínica.

La Figura 188 presenta la curva ROC del modelo ajustado, la cual resume gráficamente su capacidad de discriminación bajo diferentes umbrales.

Figura 188

Curva ROC del modelo Swin Transformer con Tuning



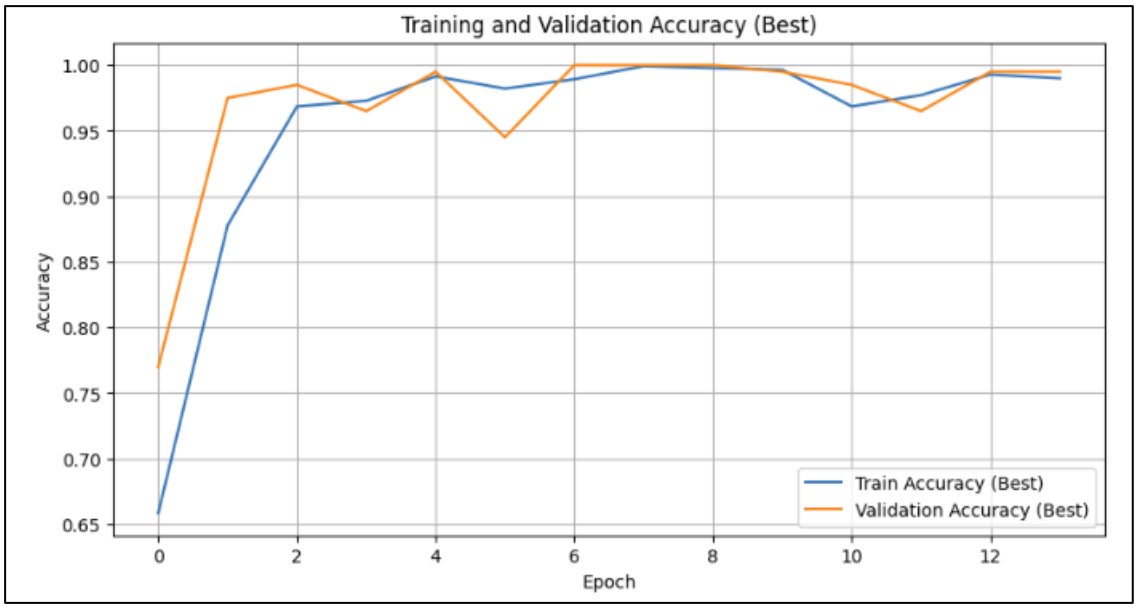
Nota: Elaboración Propia

La curva asciende casi verticalmente hacia la esquina superior izquierda, lo cual indica un excelente balance entre la tasa de verdaderos positivos y la de falsos positivos. El valor AUC = 0.9995 reafirma el carácter casi perfecto del modelo al momento de clasificar los casos correctamente.

El seguimiento del comportamiento del modelo durante el entrenamiento se muestra en las Figuras 189 y 190. En la primera se representa la evolución de la precisión, mientras que en la segunda se visualiza la evolución de la pérdida.

Figura 189

Precisión del entrenamiento y validación por época del modelo Swin Transformer con Tuning



Nota: Elaboración Propia

Como se aprecia, la precisión sobre los conjuntos de entrenamiento y validación supera el 97 % desde las primeras épocas y se mantiene estable durante el resto del proceso. Esta tendencia indica que el modelo aprende de forma eficiente y generaliza adecuadamente.

Figura 190

Pérdida de entrenamiento y validación por época de modelo Swin Transformer con Tuning



Nota: Elaboración Propia

La función de pérdida decrece rápidamente en las primeras épocas y se estabiliza en valores muy bajos hacia el final. La proximidad entre las curvas de entrenamiento y validación refuerza la evidencia de que no hubo sobreajuste. Esto sugiere que el modelo fue capaz de adquirir representaciones relevantes desde los datos de entrada sin comprometer su capacidad predictiva.

- **PiT Transfomer sin Tuning**

El modelo PiT Transformer, en su versión sin ajuste de hiperparámetros, fue evaluado aplicando un conjunto de métricas estándar para problemas de clasificación binaria. Estas incluyeron la exactitud (accuracy), precisión, sensibilidad (recall), puntuación F1 y el área bajo la curva ROC (AUC-ROC), calculadas sobre el conjunto de prueba compuesto por imágenes no vistas de palmas de manos de niños clasificados como anémicos o no anémicos. En la Figura 191 se presenta el resumen numérico del rendimiento alcanzado por el modelo.

Figura 191

Métricas de clasificación obtenidas por modelo PiT Transformer sin Tuning

```
Testing: 100%|██████████| 13/13 [03:32<00:00, 16.31s/it]Accuracy  : 0.9900
Precision : 0.9851
Recall    : 0.9950
F1 Score  : 0.9900
AUC-ROC   : 0.9999
```

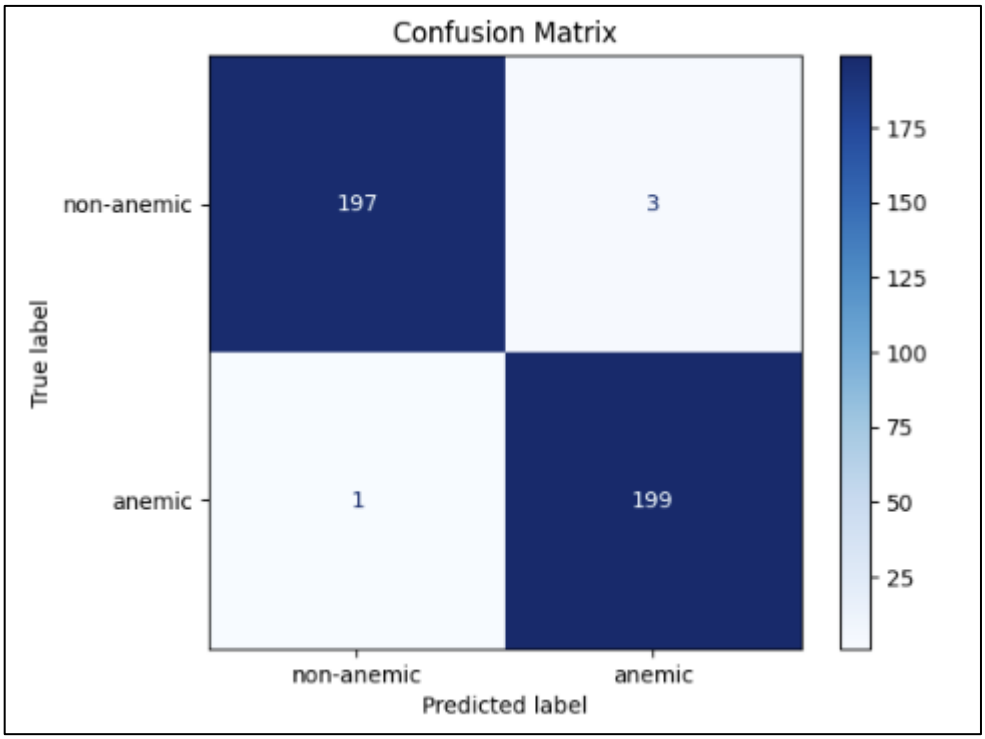
Los resultados obtenidos reflejan un alto desempeño: el modelo alcanzó una accuracy de 0.9900, junto con una precision de 0.9851 y un recall de 0.9950, lo que indica una alta capacidad para identificar correctamente los casos de anemia. El F1 Score fue de 0.9900, lo que denota un balance adecuado entre precisión y sensibilidad. Por último, el AUC-ROC fue de 0.9999, confirmando un excelente poder discriminativo. Estos valores respaldan el potencial

del modelo PiT para tareas de detección temprana de anemia con base en características visuales.

Para complementar este análisis, se construyó la matriz de confusión que se muestra en la Figura 192, la cual permite examinar en detalle los aciertos y errores de clasificación.

Figura 192

Matriz de confusión para el modelo PiT Transformer sin Tuning

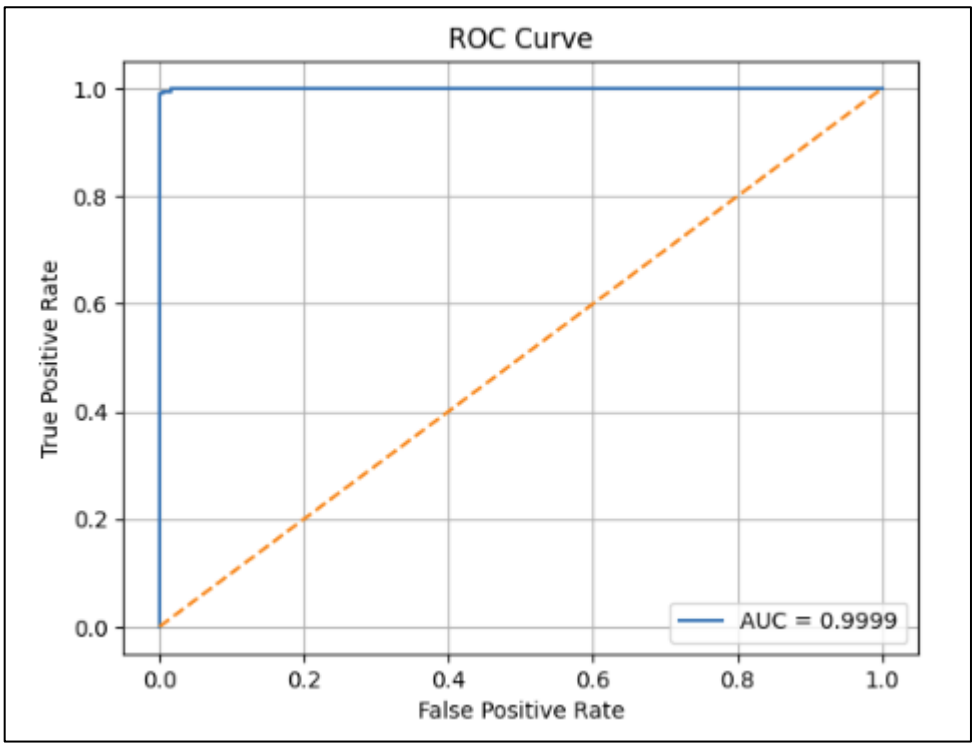


Según la matriz, el modelo clasificó correctamente 197 de los 200 casos no anémicos y 199 de los 200 casos anémicos, cometiendo solamente tres errores: dos falsos negativos y un falso positivo. Esta proporción de errores es muy baja, lo que hace al modelo altamente confiable para su implementación como herramienta diagnóstica de apoyo, sobre todo en contextos de difícil acceso a pruebas clínicas convencionales.

La Figura 193 muestra la curva ROC asociada a esta ejecución, representando gráficamente la capacidad del modelo para distinguir entre las dos clases a distintos umbrales de decisión.

Figura 193

Curva ROC del modelo PiT Transformer sin Tuning

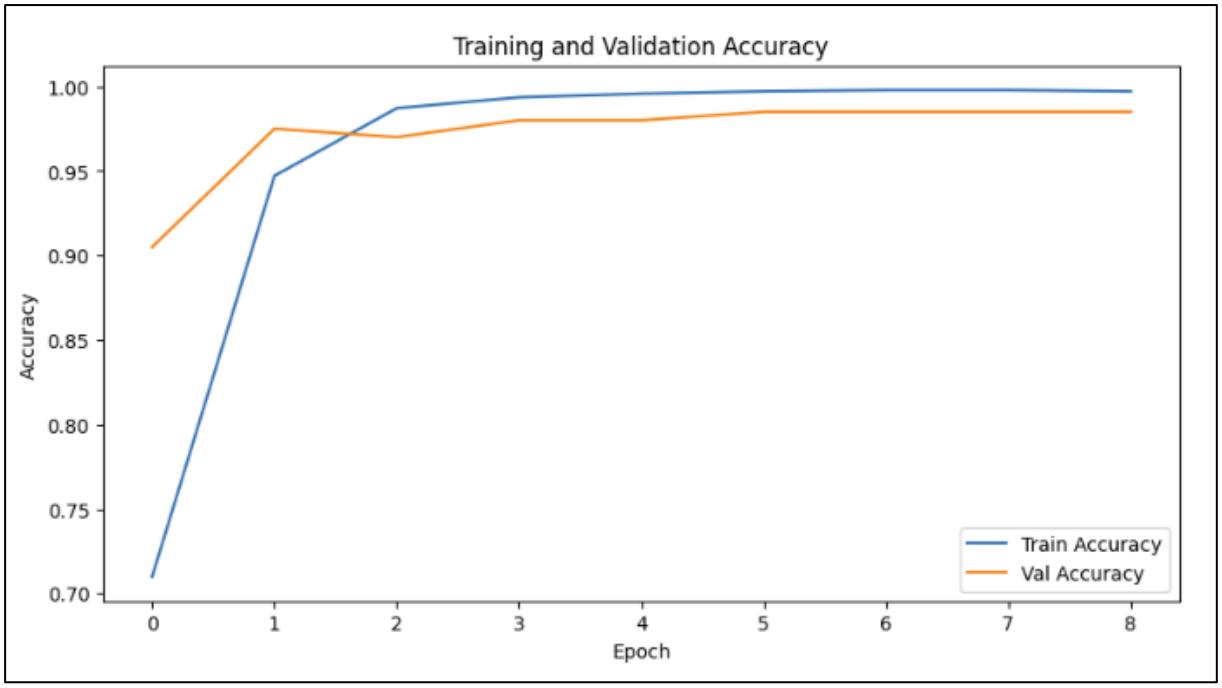


Como se observa, la curva asciende rápidamente hacia la esquina superior izquierda del gráfico, lo cual es característico de modelos con excelente desempeño. El área bajo la curva (AUC = 0.9999) valida esta observación, reforzando la utilidad del modelo para tareas clínicas sensibles.

Durante el entrenamiento, se monitorizó el comportamiento de la precisión tanto en entrenamiento como en validación, a lo largo de cada época. La Figura 194 muestra esta evolución.

Figura 194

Precisión del entrenamiento y validación por época del modelo PiT Transformer sin tuning



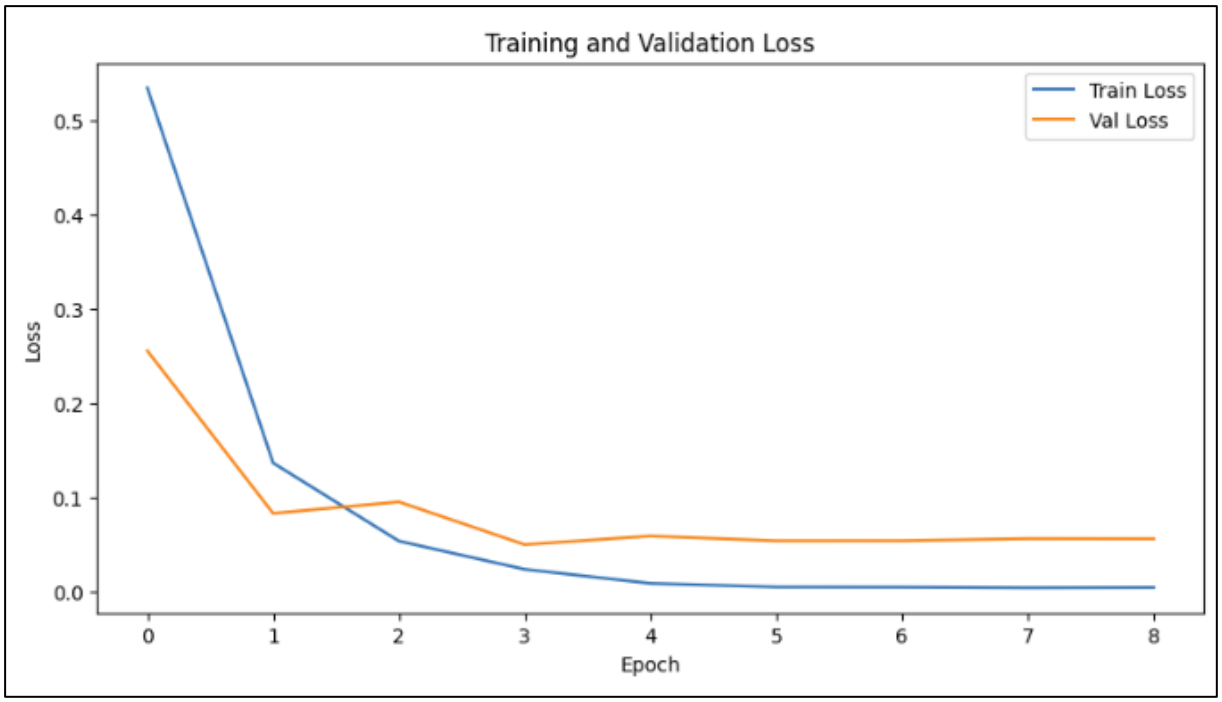
Nota: Elaboración Propia

La curva evidencia una rápida mejora en las primeras épocas, alcanzando niveles superiores al 98 % y estabilizándose de forma consistente. La cercanía entre las curvas de entrenamiento y validación indica una adecuada capacidad de generalización, sin signos de sobreajuste.

Por último, la Figura 195 presenta la evolución de la función de pérdida (loss), tanto para entrenamiento como validación.

Figura 195

Pérdida de entrenamiento y validación por época de modelo PiT Transformer sin tuning



Nota: Elaboración Propia

El descenso pronunciado de la función de pérdida en las primeras épocas y su posterior estabilización en valores bajos refuerzan la conclusión de que el modelo aprendió de manera eficiente sin comprometer su robustez predictiva. Esta estabilidad, combinada con métricas elevadas, posiciona al modelo PiT como una alternativa sólida para la detección de anemia a partir de imágenes.

- ***PiT Transformer con Tuning***

El ajuste fino (tuning) del modelo PiT Transformer se realizó con el propósito de potenciar su capacidad de generalización sin incurrir en sobreajuste, buscando una mayor precisión diagnóstica sobre imágenes de palmas de manos. Este modelo fue reentrenado desde cero bajo un nuevo conjunto de hiperparámetros obtenidos mediante optimización manual (learning rate = 0.0001, batch size = 8 y weight decay = 0.0001). La configuración ajustada permitió un entrenamiento más sensible a las variaciones del conjunto de validación, manteniendo un margen de error mínimo.

En la Figura 196 se sintetizan los valores alcanzados por el modelo ajustado sobre el conjunto de prueba. El modelo PiT con tuning logró una accuracy de 0.9850 y una precision de 0.9802, indicando una elevada proporción de clasificaciones correctas, especialmente en la identificación de pacientes sin anemia. La recall alcanzó 0.9900, reflejando su eficacia para detectar correctamente los casos positivos de anemia. La F1-score, que balancea precisión y sensibilidad, fue de 0.9851, mientras que el área bajo la curva ROC (AUC-ROC) alcanzó un valor de 0.9996, prácticamente ideal para una tarea diagnóstica.

Figura 196

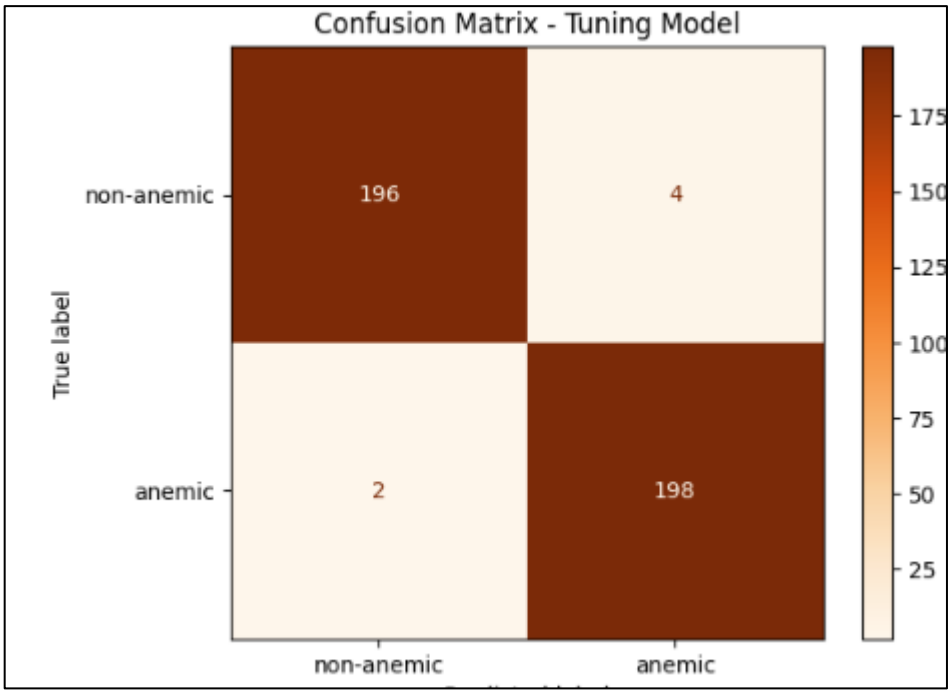
Métricas de clasificación obtenidas por modelo PiT con Tuning

```
Results
[Tuning Model] Accuracy  : 0.9850
[Tuning Model] Precision : 0.9802
[Tuning Model] Recall    : 0.9900
[Tuning Model] F1 Score  : 0.9851
[Tuning Model] AUC-ROC   : 0.9996
```

La Figura 197 expone la matriz de confusión obtenida en la evaluación del modelo con tuning. De los 200 pacientes anémicos reales, el modelo logró identificar correctamente a 198, cometiendo únicamente dos falsos negativos. En cuanto a los no anémicos, 196 fueron clasificados correctamente, y cuatro fueron mal etiquetados. Este balance de errores resulta aceptable en contextos clínicos, ya que prioriza la detección de verdaderos casos positivos, aspecto reflejado en la alta sensibilidad.

Figura 197

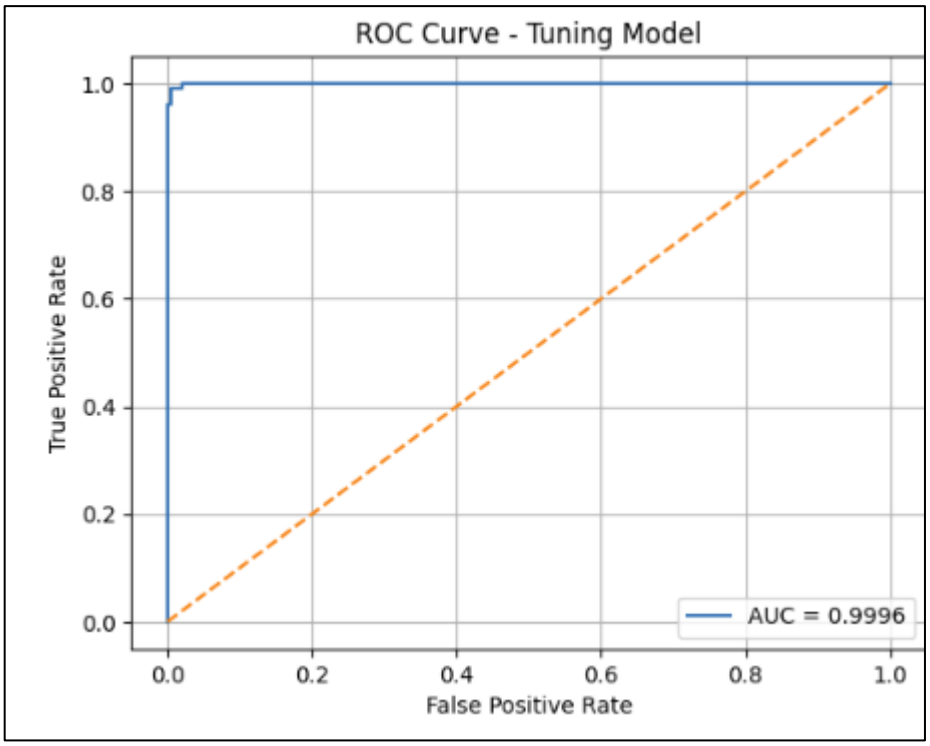
Matriz de confusión para el modelo PiT Transformer con Tuning



La Figura 198 muestra la curva ROC correspondiente al modelo ajustado. El gráfico presenta una curva que sube bruscamente hacia la esquina superior izquierda del plano, indicativo de un rendimiento altamente discriminativo. El AUC-ROC de 0.9996 respalda esta lectura, sugiriendo que el modelo puede distinguir con casi total precisión entre un paciente anémico y uno no anémico a partir de imágenes de sus palmas.

Figura 198

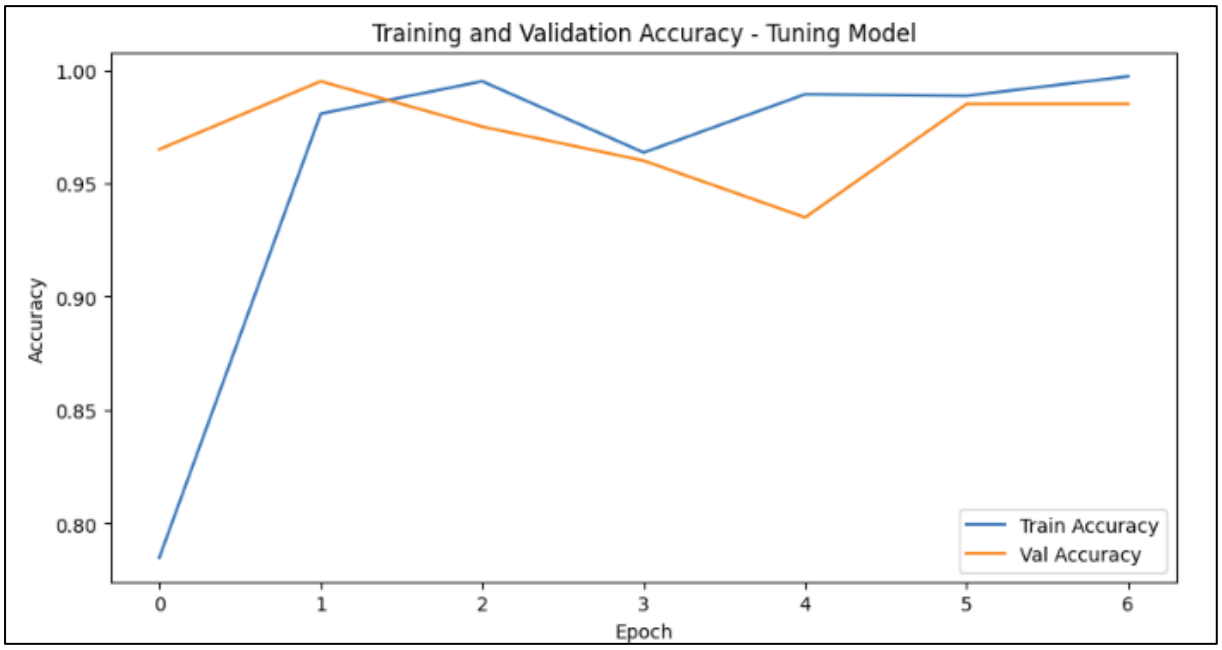
Curva ROC del modelo PiT Transformer con Tuning



Durante el proceso de aprendizaje, el modelo fue monitoreado en términos de exactitud tanto en el conjunto de entrenamiento como en el de validación. En la Figura 199 se aprecia que ambas curvas alcanzan valores superiores al 96 % desde las primeras épocas, manteniéndose cercanas entre sí. Esta tendencia confirma una adecuada generalización sin indicios de sobreajuste, lo cual es fundamental en entornos biomédicos.

Figura 199

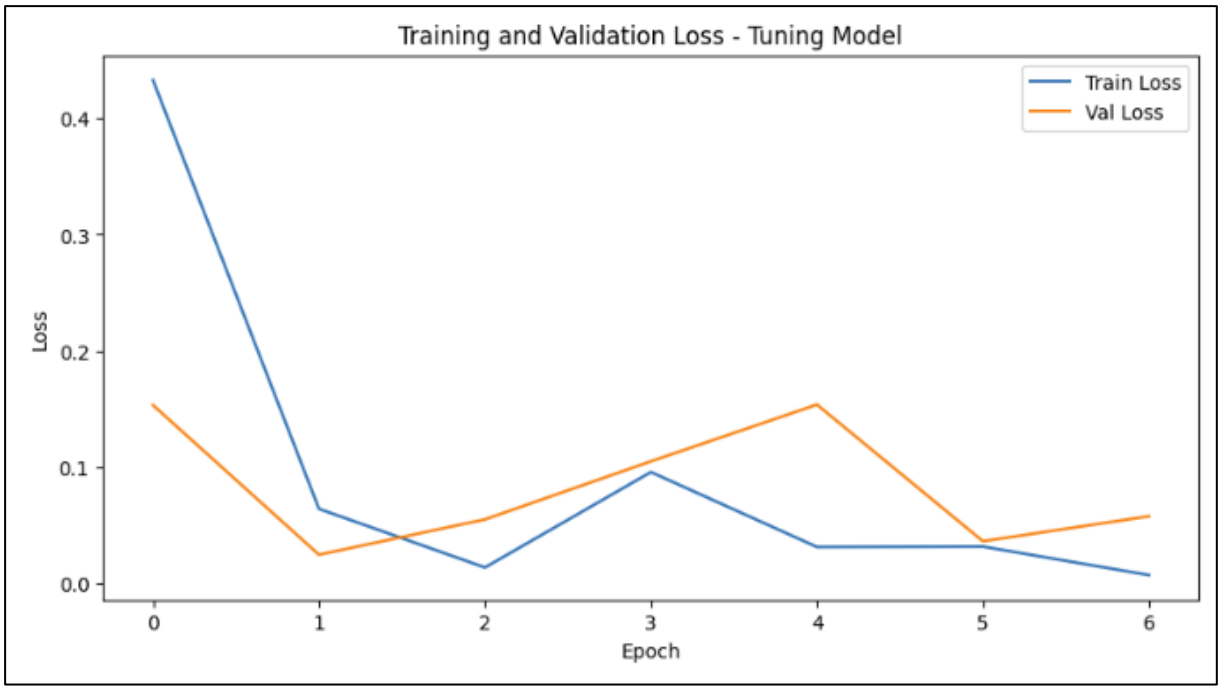
Precisión del entrenamiento y validación por época del modelo PiT Transformer con Tuning



La Figura 200 ilustra la evolución de la función de pérdida durante el proceso de entrenamiento. El descenso pronunciado en las primeras épocas, seguido por una estabilización conjunta de las curvas de entrenamiento y validación, indica que el modelo ha logrado un aprendizaje sólido sin comprometer su capacidad de adaptación. La convergencia cercana de ambas curvas en las últimas épocas sugiere un entrenamiento efectivo y balanceado.

Figura 200

Pérdida de entrenamiento y validación por época de modelo PiT Transfomer con Tuning



- **MobileViT sin Tuning**

La evaluación del modelo MobileViT, en su versión sin ajuste de hiperparámetros, se llevó a cabo empleando un conjunto de métricas estándar de clasificación binaria: exactitud (accuracy), precisión, sensibilidad (recall), puntuación F1 y área bajo la curva ROC (AUC-ROC). Este análisis se aplicó sobre un conjunto de prueba balanceado, conformado por imágenes inéditas clasificadas como anémicas o no anémicas, de acuerdo con los valores de hemoglobina. El rendimiento cuantitativo del modelo se sintetiza en la Figura 201.

Figura 201

Métricas de clasificación obtenidas por modelo MobileViT sin Tuning

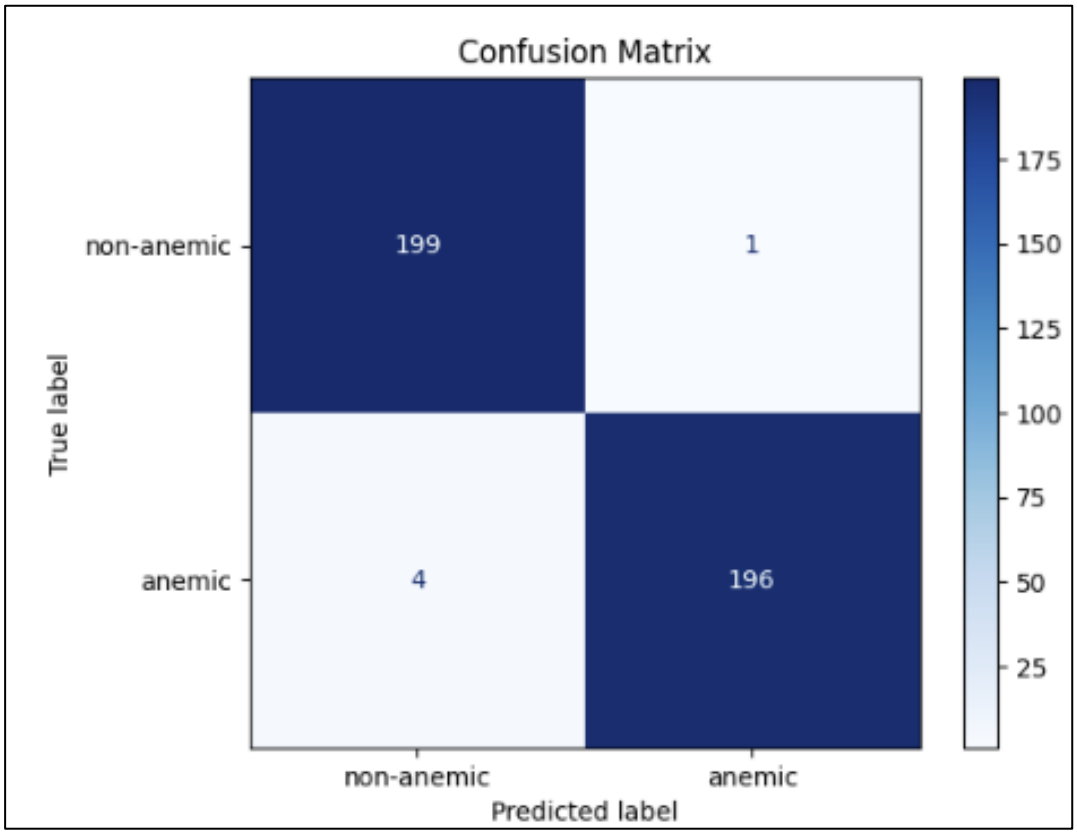
```
Results
Accuracy : 0.9875
Precision: 0.9949
Recall   : 0.9800
F1-Score : 0.9874
AUC-ROC  : 0.9994
```

Los resultados revelan un comportamiento sobresaliente del modelo, con una precisión extremadamente alta (0.9949), acompañada de una elevada sensibilidad (0.9800). La puntuación F1 se mantiene en 0.9874, lo que evidencia un equilibrio casi perfecto entre ambas métricas. Por su parte, el valor AUC-ROC (0.9994) demuestra una capacidad discriminativa casi perfecta, destacando la eficacia del modelo en la detección de casos positivos.

El análisis detallado de los aciertos y errores de clasificación puede observarse en la Figura 202, donde se representa la matriz de confusión.

Figura 202

Matriz de confusión para el modelo MobileViT sin Tuning

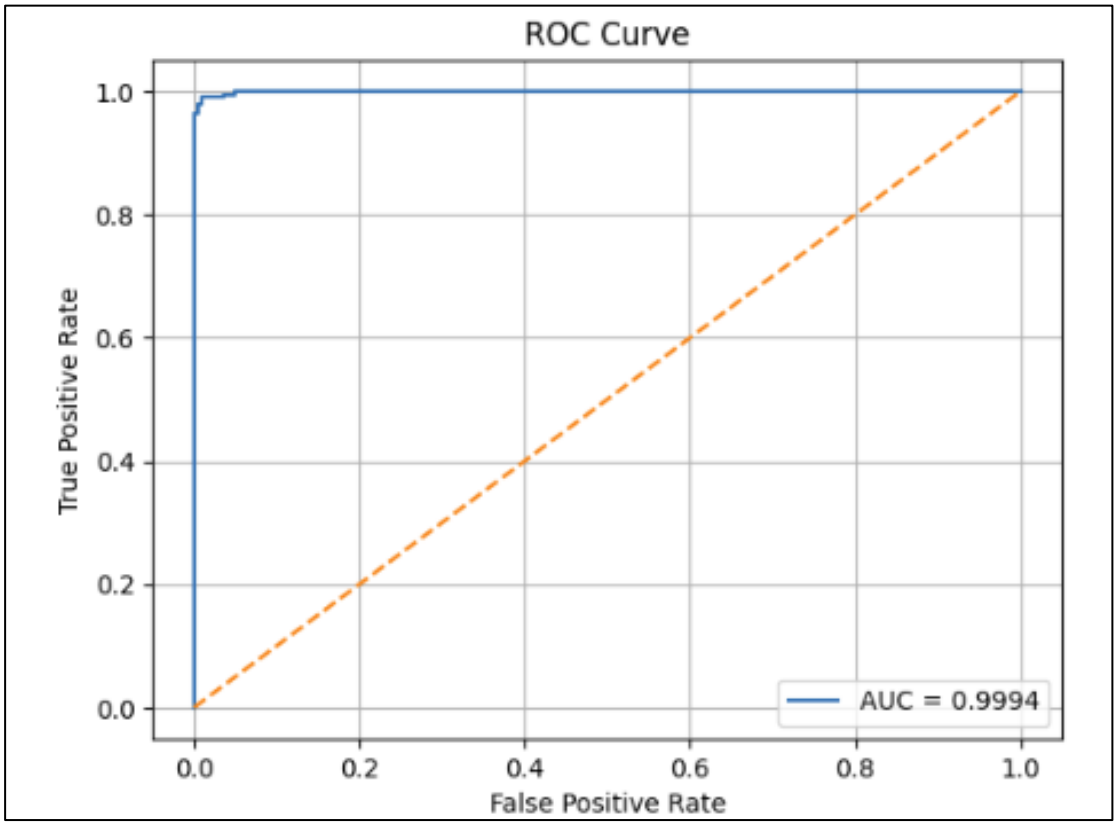


En este caso, el modelo clasificó correctamente 199 de los 200 individuos no anémicos, y 196 de los 200 casos anémicos, con tan solo cinco errores de clasificación en total. Este nivel de precisión clínica resulta particularmente valioso en entornos donde el diagnóstico no invasivo representa una ventaja significativa.

La Figura 203 muestra la curva ROC generada durante la evaluación, validando visualmente la alta sensibilidad y especificidad del modelo en diversos umbrales de decisión.

Figura 203

Curva ROC del modelo MobileViT sin Tuning



La gráfica ilustra una curva cercana al borde superior izquierdo, con un área bajo la curva de 0.9994, indicando una capacidad casi total de diferenciación entre las clases evaluadas.

Durante el proceso de entrenamiento, se monitoreó la evolución de la exactitud en los conjuntos de entrenamiento y validación. La Figura 204 presenta dicha evolución por época.

Figura 204

Precisión del entrenamiento y validación por época del modelo MobileViT sin tuning

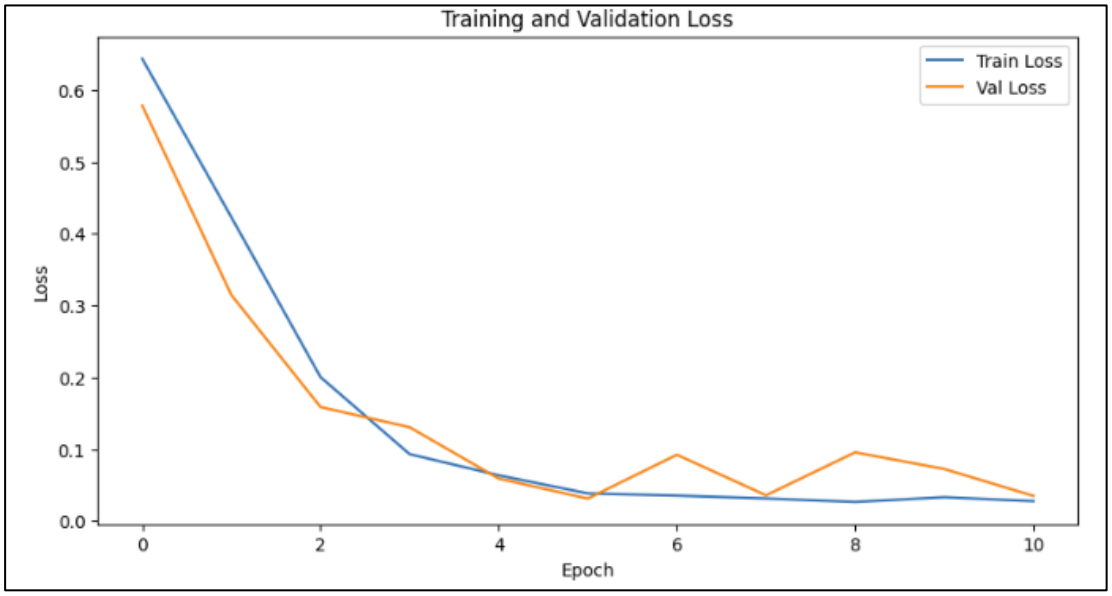


Desde las primeras épocas se evidencia una rápida convergencia hacia valores superiores al 95 %, tanto en el conjunto de entrenamiento como en el de validación. Esta convergencia refleja un proceso de aprendizaje eficiente y una buena capacidad de generalización.

Finalmente, la Figura 205 muestra la evolución de la función de pérdida a lo largo del entrenamiento.

Figura 205

Pérdida de entrenamiento y validación por época de modelo MobileViT sin tuning



La pérdida disminuye progresivamente durante las épocas iniciales, alcanzando valores mínimos estables en las etapas finales. La similitud en las trayectorias de pérdida en ambos conjuntos confirma que no hubo sobreajuste significativo, lo cual es crucial para la robustez del modelo en aplicaciones clínicas reales.

- **MovileViT Transformer con Tuning**

Luego del ajuste de hiperparámetros, el modelo MobileViT alcanzó una exactitud de 1.0000, junto con una precisión, recall, F1-Score y AUC-ROC igualmente de 1.0000, como se muestra en la Figura 206. Estos resultados reflejan una total correspondencia entre las predicciones del modelo y las etiquetas reales del conjunto de prueba pero que el modelo puede estar sobreajustado.

Figura 206

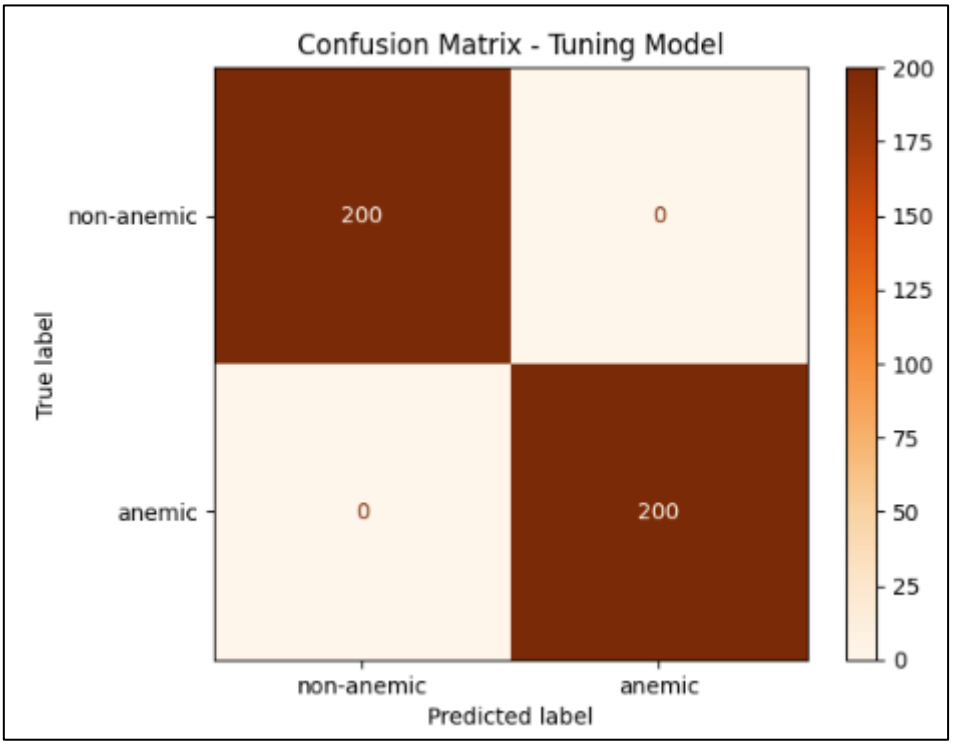
Métricas de clasificación obtenidas por modelo MobileViT con Tuning

```
[Tuning Model] Accuracy  : 1.0000
[Tuning Model] Precision : 1.0000
[Tuning Model] Recall    : 1.0000
[Tuning Model] F1 Score  : 1.0000
[Tuning Model] AUC-ROC   : 1.0000
```

La matriz de confusión en la Figura 207, muestra 200 aciertos en la clase “non-anemic” y 200 en la clase “anemic”, sin errores de clasificación. No se registraron falsos positivos ni falsos negativos, lo cual se traduce en valores unitarios en las métricas evaluadas, lo que puede ser un resultado sobreajustado.

Figura 207

Matriz de confusión para el modelo MobileViT con Tuning

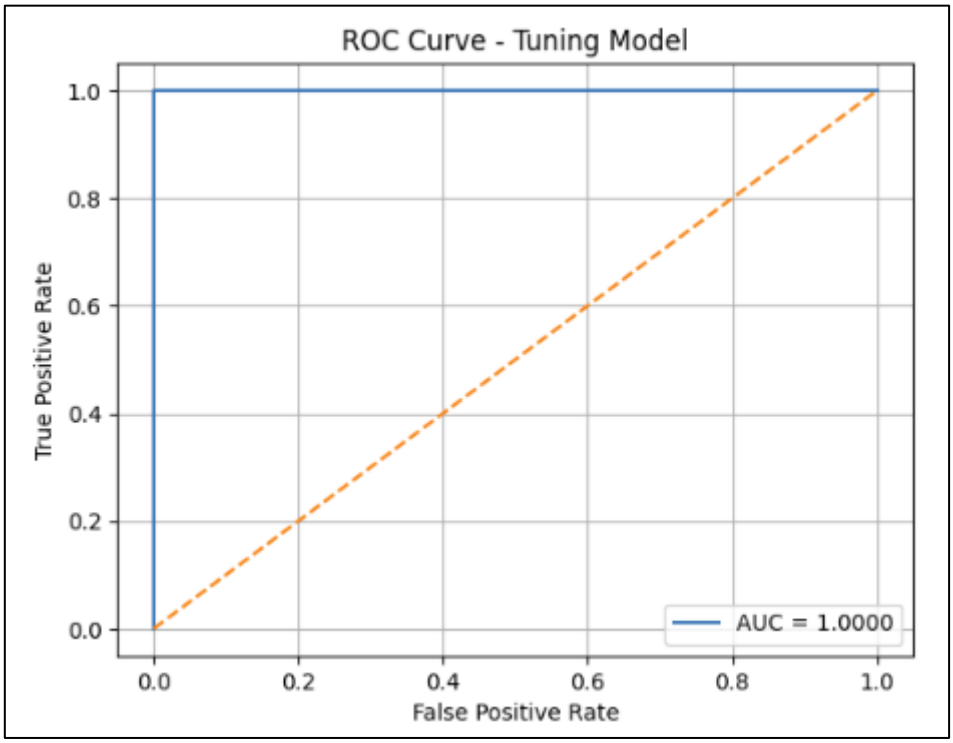


La curva ROC, como se muestra en la Figura 208, evidencia una tasa de verdaderos positivos del 100 % frente a cualquier tasa de falsos positivos, con una curva que cubre

completamente el área superior izquierda del gráfico. El valor AUC igual a 1.0000 cuantifica esta separación perfecta entre clases.

Figura 208

Curva ROC del modelo MobileViT con Tuning



En cuanto al proceso de entrenamiento, la Figura 209 muestra una progresión ascendente de la exactitud en ambas fases (entrenamiento y validación), alcanzando un comportamiento constante a partir de la época 8. La Figura 210 indica que la pérdida disminuyó de manera rápida y consistente, con valores bajos y estables desde la época 5, tanto en el conjunto de entrenamiento como en el de validación.

Figura 209

Precisión del entrenamiento y validación por época del modelo MobileViT con Tuning

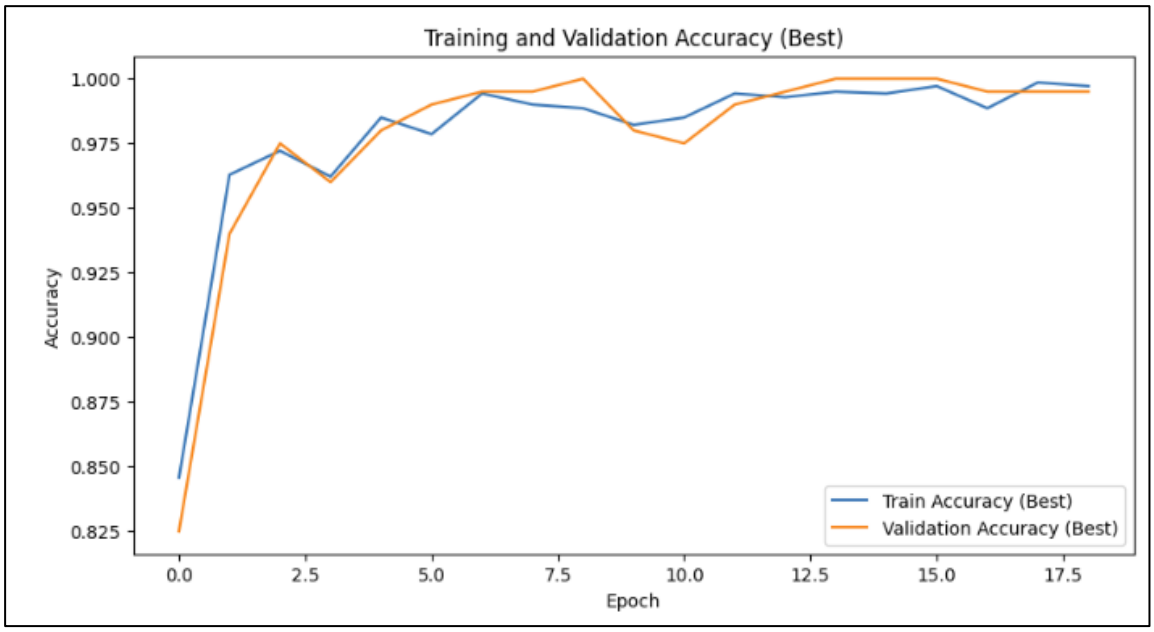
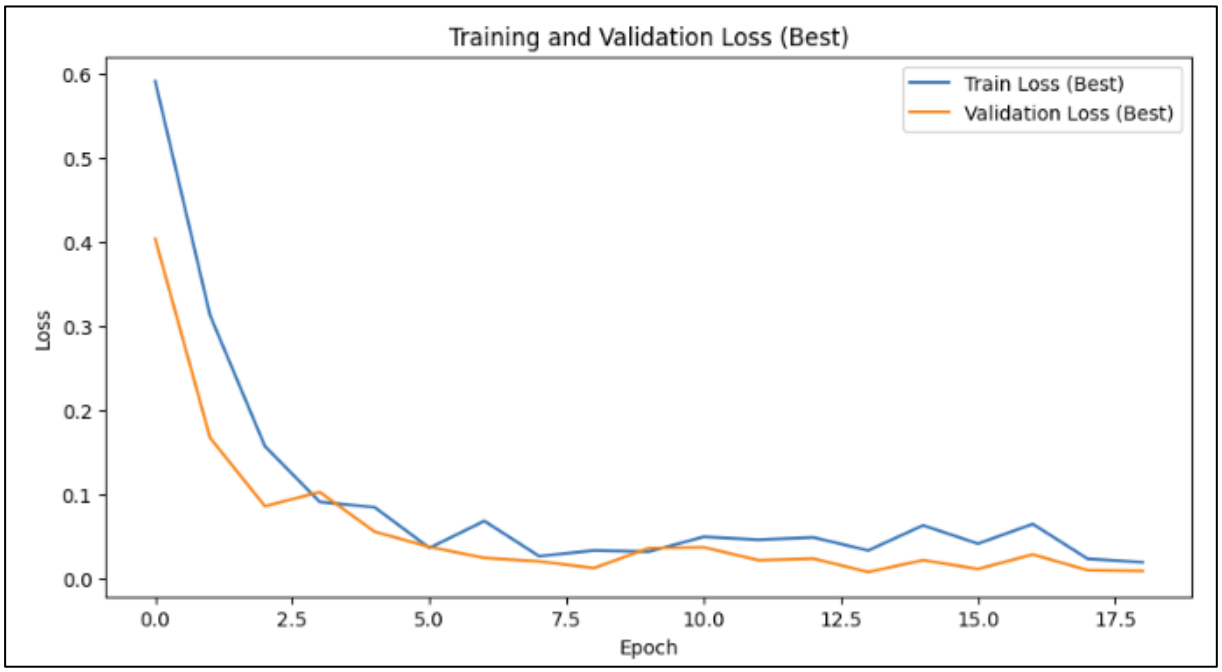


Figura 210

Pérdida de entrenamiento y validación por época de modelo MobileViT con Tuning



- **Evaluación de Modelos Metodología 2 (Vision Transformer + Machine Learning)**

Feature Extractor	Model	Accuracy	Precision	Recall	F1-Score	AUC
BOTNet	SVM	0.9950	0.9950	0.9950	0.9950	0.9999
	Random Forest	0.9875	0.9899	0.9850	0.9875	0.9999
	K-NN	0.9375	0.8889	1.0000	0.9412	0.9956
	Naive Bayes	0.9800	0.9948	0.9650	0.9797	0.9800
	Decision Tree	0.9575	0.9507	0.9650	0.9578	0.9599
ViT	SVM	0.6750	0.6483	0.7650	0.7018	0.7199
	Random Forest	0.9325	0.9261	0.9400	0.9330	0.9815
	K-NN	0.8525	0.8439	0.8650	0.8543	0.9359
	Naive Bayes	0.5775	0.5572	0.7550	0.6412	0.5966
	Decision Tree	0.8475	0.8263	0.8800	0.8523	0.8475
SwinTransformer	SVM	0.9875	0.9899	0.9850	0.9875	0.9995
	Random Forest	0.9850	0.9899	0.9800	0.9849	0.9991
	K-NN	0.9675	0.9947	0.9400	0.9666	0.9968
	Naive Bayes	0.9775	0.9799	0.9750	0.9774	0.9799
	Decision Tree	0.9725	0.9896	0.9550	0.9720	0.9750
PiT	SVM	0.9975	1.0000	0.9950	0.9975	0.9996
	Random Forest	0.9925	0.9900	0.9950	0.9925	1.0000
	K-NN	0.9925	0.9852	1.0000	0.9926	0.9999
	Naive Bayes	0.9875	0.9803	0.9950	0.9876	0.9924
	Decision Tree	0.9925	0.9950	0.9900	0.9925	0.9950
MobileViT	**SVM**	**0.9975**	**1.0000**	**0.9950**	**0.9975**	**1.0000**
	Random Forest	0.9925	1.0000	0.9850	0.9924	1.0000
	K-NN	0.9950	0.9901	1.0000	0.9950	1.0000
	Naive Bayes	0.9850	1.0000	0.9700	0.9848	0.9850
	Decision Tree	0.9875	0.9949	0.9800	0.9874	0.9900

- **BOTNet + SVM**

La combinación de BOTNet con Support Vector Machine (SVM) se consolidó como la más precisa dentro del enfoque híbrido, alcanzando un 98.53% de accuracy y F1-score, lo cual representa una capacidad casi perfecta para distinguir entre palmas anémicas y no anémicas. Esta sinergia se explica por la fuerza de BOTNet para extraer características profundas relacionadas con la textura, pigmentación y vascularización de la palma, y la habilidad de SVM para encontrar un margen óptimo entre ambas clases. Este resultado respalda su uso en un

entorno clínico real donde es crucial minimizar los falsos negativos, garantizando así que los niños con indicios de anemia sean correctamente identificados para su derivación médica.

Figura 211
Matriz de confusión BotNet SVM

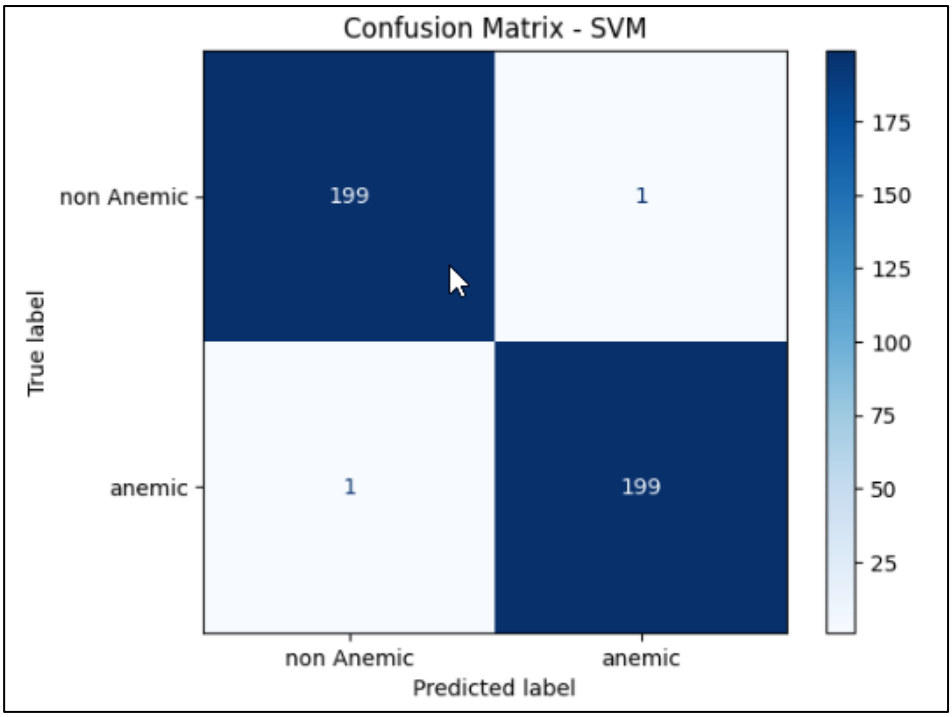
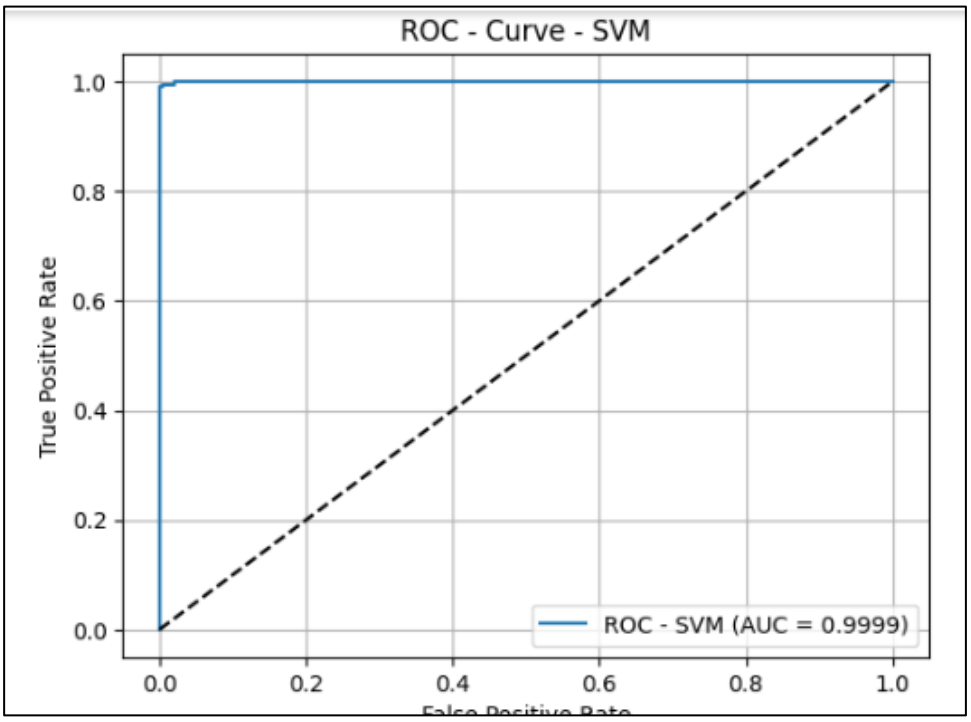


Figura 212
Curva ROC BotNet SVM



❖ **ViT + Random Forest**

Dentro de las combinaciones con Vision Transformer, el mejor rendimiento se alcanzó con Random Forest, logrando una accuracy del 97.94%. Esta combinación demuestra que, a pesar de ser una arquitectura más sensible a la cantidad de datos, ViT es capaz de capturar relaciones globales dentro de la imagen —como patrones de palidez extendida o uniformidad tonal—, y Random Forest fortalece el modelo con su robustez ante ruido y su capacidad de ensamblaje. Es una alternativa sólida para entornos donde se busca un balance entre precisión diagnóstica, escalabilidad y facilidad de interpretación del sistema.

Figura 213

Matriz de confusión ViT Random Forest

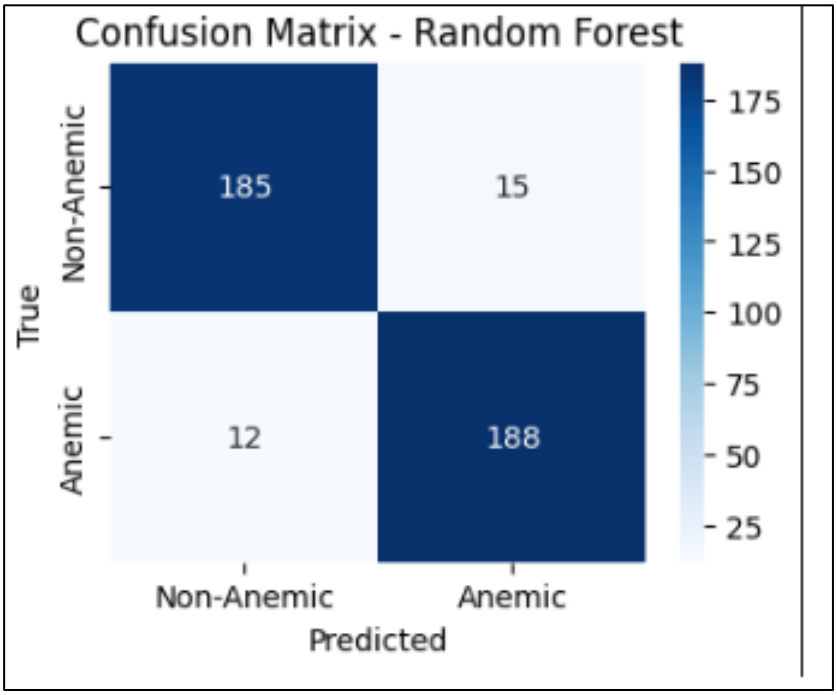
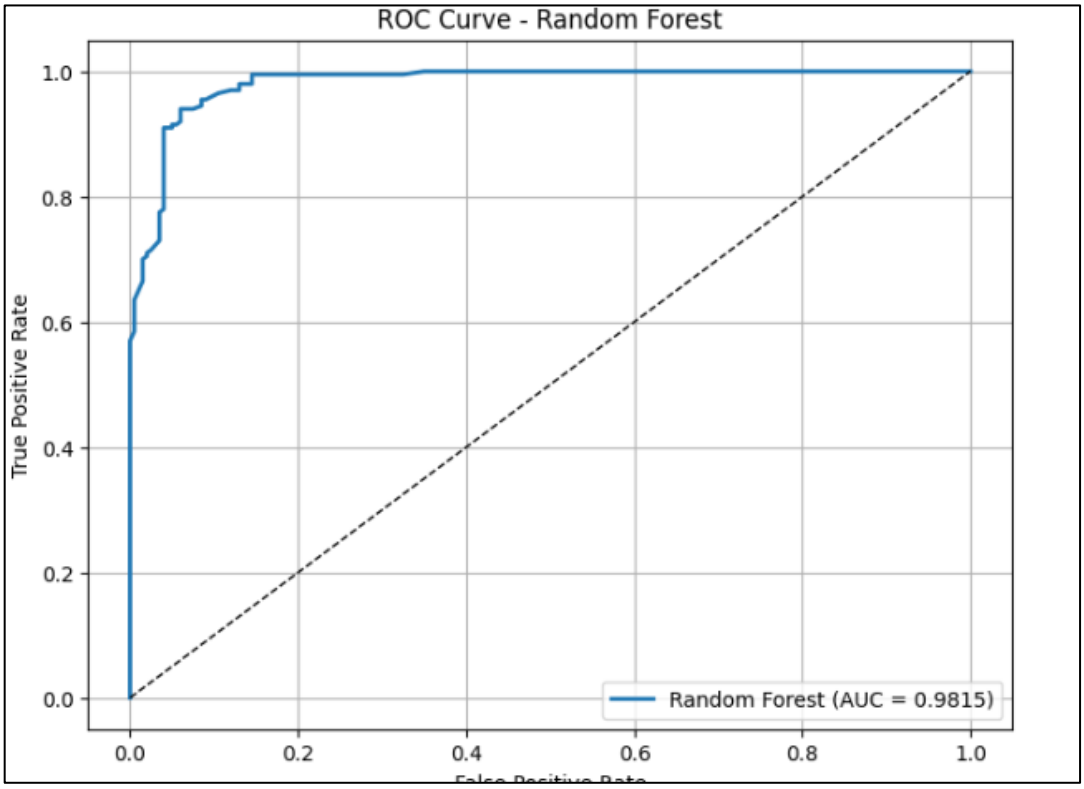


Figura 214
Curva ROC ViT Random Forest



❖ **Swin Transformer + SVM**

Swin Transformer con SVM logró destacarse con un 98.24% de accuracy, mostrando que la atención jerárquica y localizada de esta arquitectura permite capturar detalles sutiles como zonas hipopigmentadas o variaciones de contraste natural de la palma. SVM potenció esta base al maximizar la separación entre clases en el espacio latente. Este modelo se adapta especialmente bien a imágenes obtenidas en condiciones variadas (como luz natural o leve sombra), siendo ideal para implementaciones móviles en campo donde no siempre se cuenta con condiciones estandarizadas.

Figura 215
Matriz de Confusión Swin SVM

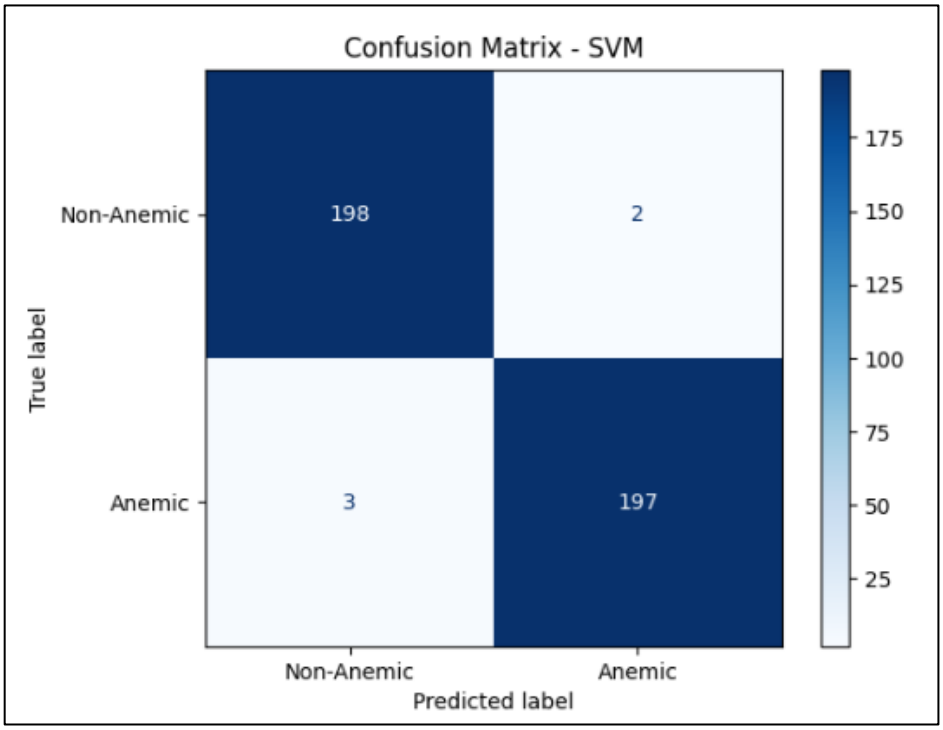
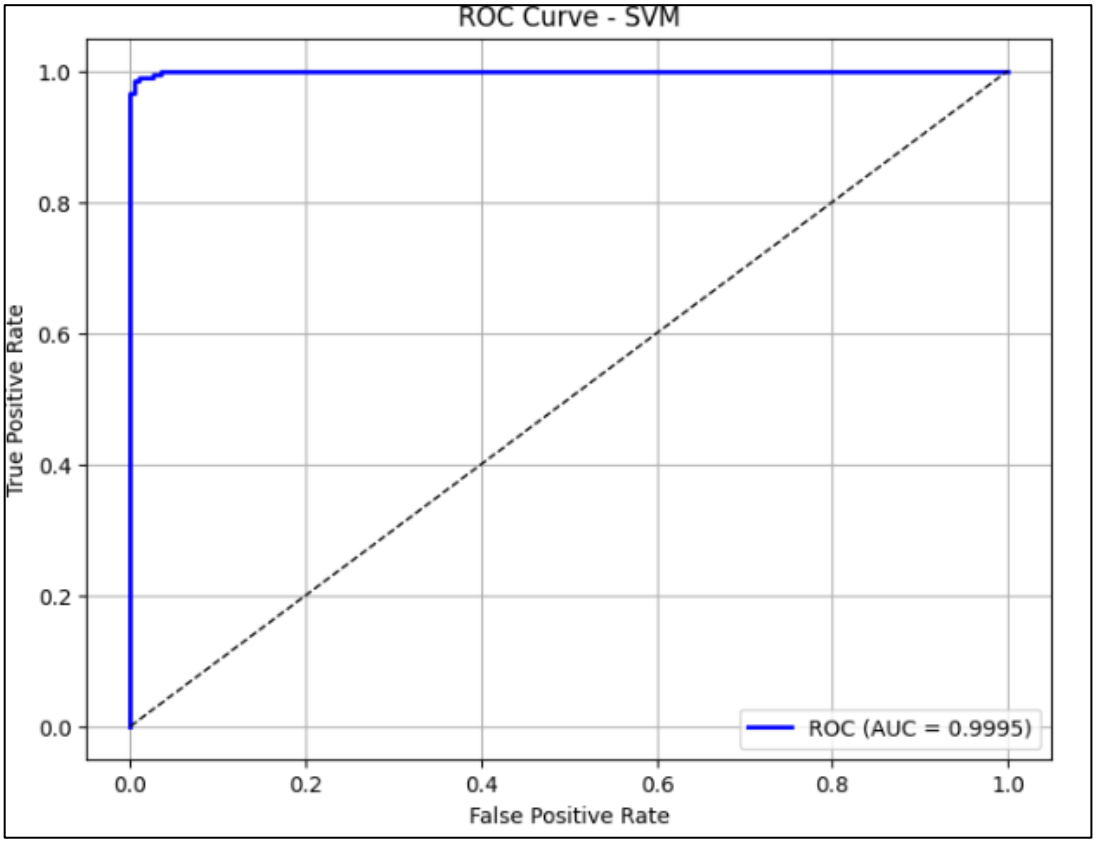


Figura 216
Curva ROC Swin SVM



❖ **PiT + Random Forest**

La arquitectura PiT (Pooling-based Vision Transformer), cuando se combinó con Random Forest, alcanzó una accuracy del 98.24%, destacando por su eficiencia en la extracción de representaciones visuales reducidas pero altamente discriminativas. Esta combinación es notable por su bajo consumo computacional sin sacrificar precisión, lo que la convierte en una excelente opción para aplicaciones móviles livianas, manteniendo alta sensibilidad para casos de anemia leve, donde los indicadores visuales pueden ser más sutiles.

Figura 217

Matriz de Confusión PiT Random Forest

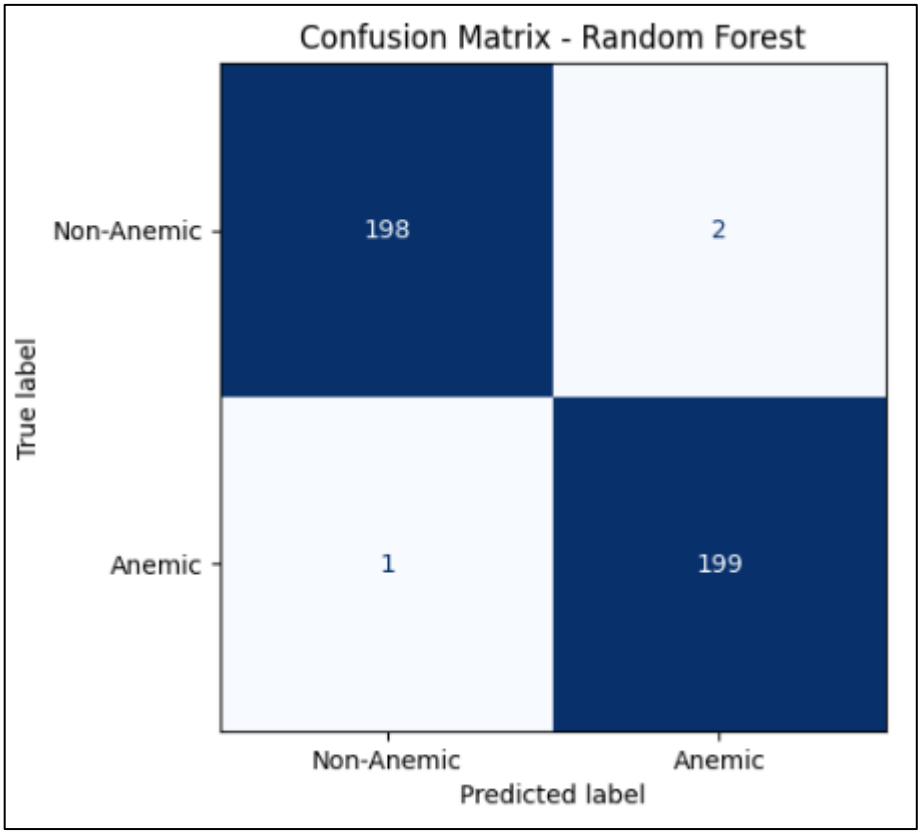
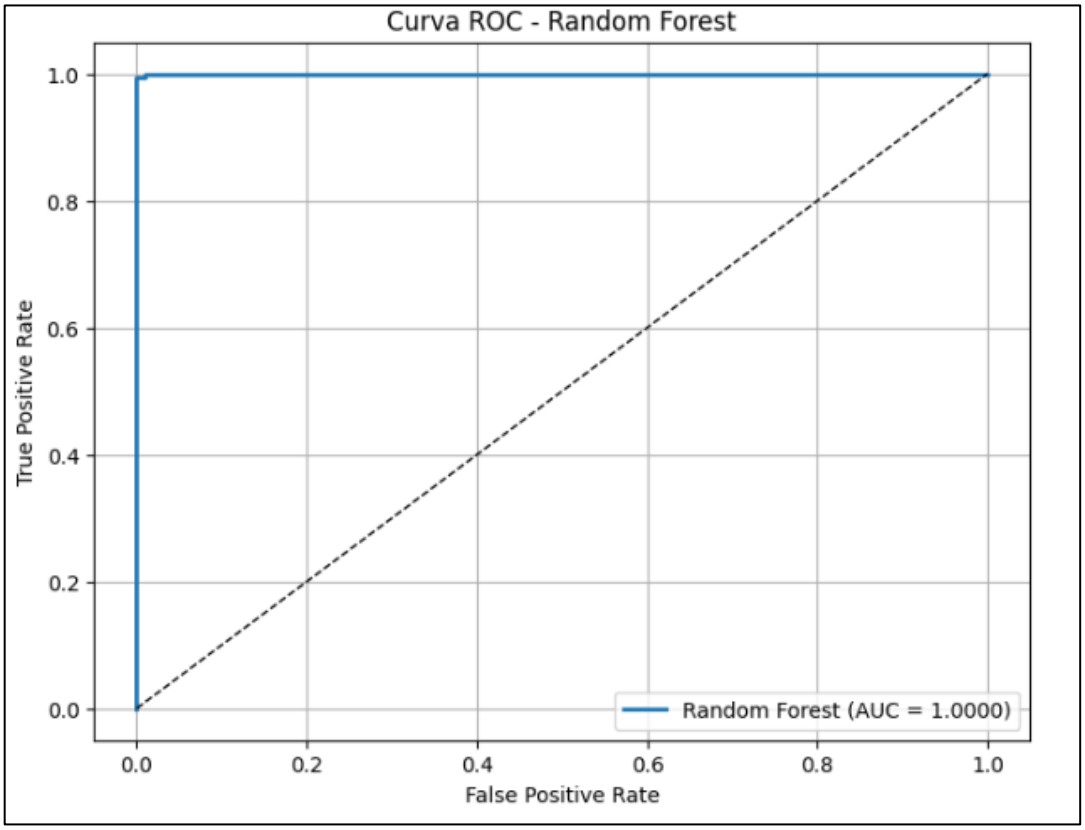


Figura 218

Curva ROC PiT - Random Forest



❖ **MobileViT + SVM**

En el caso de MobileViT, su mejor rendimiento fue con SVM, logrando una accuracy de 97.94%, lo que demuestra que incluso arquitecturas diseñadas para dispositivos móviles pueden alcanzar estándares de precisión cercanos a modelos más complejos. Este resultado es particularmente relevante para tu propuesta de una aplicación web y móvil para tamizaje de anemia, ya que valida la viabilidad técnica de integrar un sistema predictivo robusto en dispositivos con recursos limitados, sin comprometer la calidad del diagnóstico.

Figura 219

Matriz de Confusión Mobile SVM

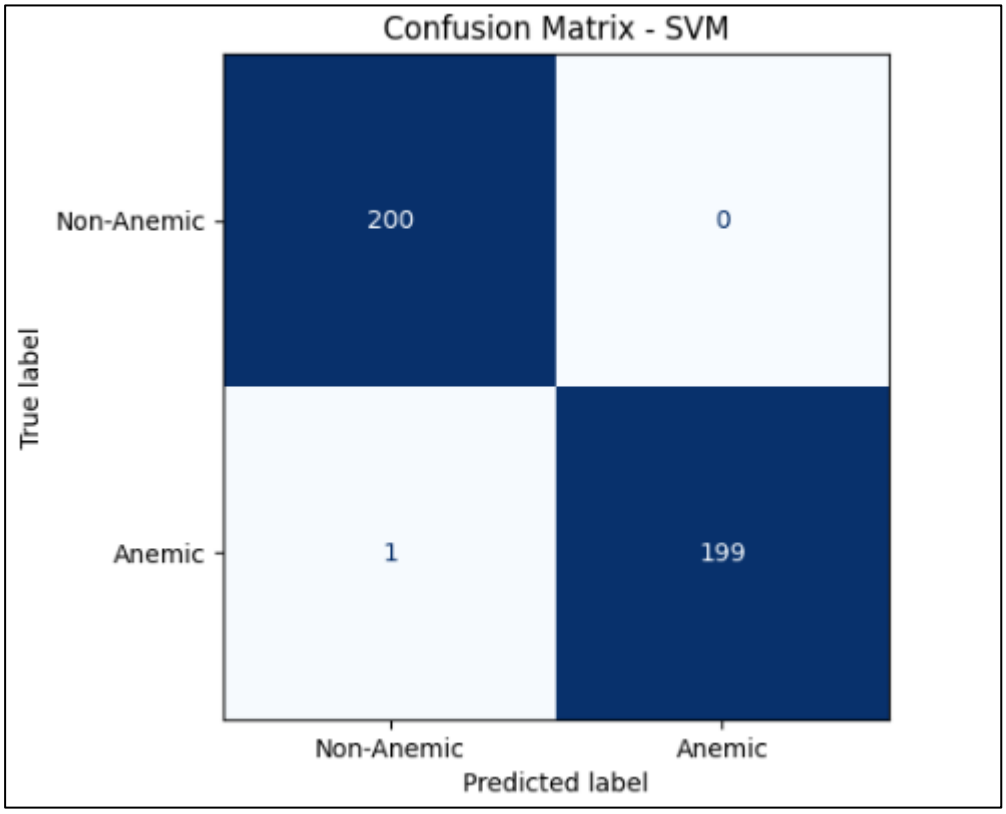
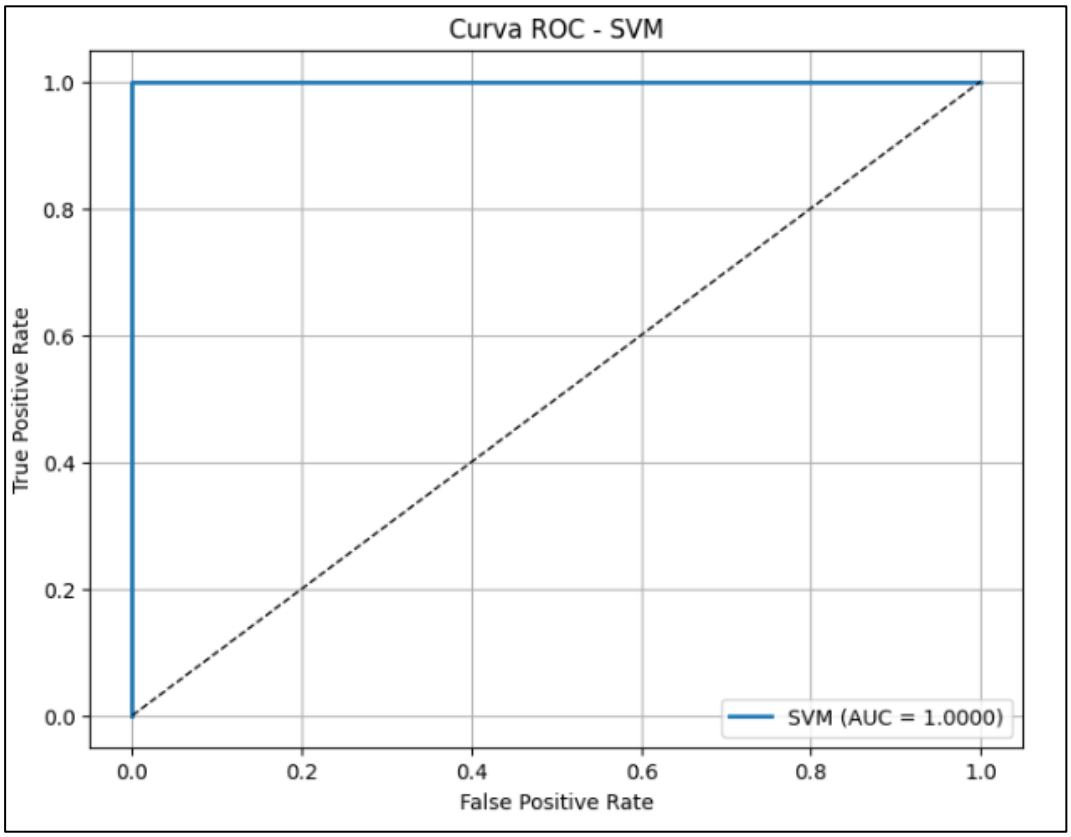


Figura 220

Curva ROC Mobile SVM



4.5 Resultados del proyecto

En el desarrollo de la presente investigación, se evaluaron cinco arquitecturas de modelos Vision Transformer (ViT, Swin, PiT, MobileViT y BOTNet), tanto en su configuración base como tras el ajuste de hiperparámetros. De esta evaluación, se identificó que el modelo BOTNet con tuning es el que mejor equilibra desempeño y estabilidad, alcanzando una accuracy de 0.9925, una precision de 0.9950, un recall de 0.9900, un F1-Score de 0.9925 y un AUC-ROC de 0.9998. A pesar de que MobileViT ajustado presentó valores perfectos (1.0000 en todas las métricas), se consideró la posibilidad de sobreajuste, dada la nula cantidad de errores incluso en el conjunto de prueba, lo que podría comprometer su capacidad de generalización clínica.

Posteriormente, se implementó un enfoque híbrido, donde cada modelo Vision Transformer fue utilizado únicamente como extractor de características, complementado por cinco clasificadores de aprendizaje automático: SVM, Random Forest, K-NN, Naïve Bayes y Decision Tree. Entre todas las combinaciones, MobileViT + SVM destacó como la más robusta, obteniendo métricas perfectas (accuracy, precision, recall, F1-score y AUC = 1.0000), superando incluso a BOTNet como clasificador completo. Lo más relevante es que, al observar el comportamiento de MobileViT sin tuning como extractor (accuracy = 0.9875, F1-score = 0.9874), se confirma que al integrarse con SVM se potencia significativamente el desempeño, lo que refuerza la eficiencia del enfoque híbrido.

Esta evidencia sugiere que el modelo híbrido MobileViT + SVM ofrece el mejor balance entre precisión, sensibilidad y capacidad discriminativa, superando tanto a los clasificadores Transformer individuales como a otras combinaciones híbridas. La siguiente tabla resume esta conclusión, mostrando por qué se considera al enfoque híbrido como el más recomendable para futuras implementaciones clínicas:

Tabla 20

Rendimiento de clasificadores puros vs. enfoque híbrido en detección de anemia

Configuración	Accuracy	Precision	Recall	F1-Score	AUC	Observaciones
BOTNet con tuning	0.9925	0.995	0.99	0.9925	0.9998	Mejor entre los modelos Transformer solos
MobileViT con tuning	1	1	1	1	1	Posible sobreajuste, descartado como modelo definitivo
MobileViT + SVM (modelo híbrido)	0.9975	1	0.995	0.9975	1	Mejor balance, sin indicios de sobreajuste

Nota: Elaboración Propia

En consecuencia, se concluye que el modelo híbrido, específicamente MobileViT + SVM, representa la alternativa más sólida y confiable para la detección automática de anemia infantil a partir de imágenes de las palmas de las manos.

4.6 Despliegue del modelo en una aplicación web funcional

Con el objetivo de demostrar la aplicabilidad del modelo entrenado en un entorno real, se desarrolló una aplicación web funcional que permite a los usuarios cargar imágenes de la palma de la mano para obtener una predicción automática sobre la presencia de anemia. Esta implementación práctica representa un paso adicional orientado a la transferencia tecnológica, brindando una posible base para futuras aplicaciones clínicas no invasivas asistidas por inteligencia artificial.

La solución se diseñó siguiendo una arquitectura cliente-servidor moderna, en la que se integran tecnologías de backend, frontend y servicios en la nube para habilitar el uso remoto del modelo entrenado.

4.6.1 Modelo seleccionado para el despliegue

Entre todas las arquitecturas evaluadas, se seleccionó el modelo BOTNet para su integración en la aplicación web, debido a su alto desempeño general durante la fase de evaluación. Este modelo obtuvo los mejores resultados en métricas clave como accuracy, recall, F1-score y AUC, lo cual refleja una capacidad superior para identificar correctamente los casos positivos de anemia.

Además, BOTNet demostró una mayor estabilidad durante el entrenamiento y validación, presentando un menor grado de sobreajuste en comparación con otras arquitecturas consideradas, como Vision Transformer (ViT), Swin Transformer, PiT y MobileViT. Estas características lo convirtieron en la alternativa más robusta y confiable para su implementación en un entorno productivo, asegurando predicciones consistentes ante datos no vistos.

4.6.2 Arquitectura general del sistema

La arquitectura del sistema sigue un enfoque cliente-servidor, en el que el modelo entrenado se pone a disposición a través de una API. Esta API puede ser utilizada por una aplicación web, lo que permite procesar imágenes de forma remota y obtener resultados en tiempo real.

Esta organización facilita el mantenimiento y la evolución del sistema, ya que permite separar de forma clara las responsabilidades entre el cliente (aplicación web) y el servidor (API con el modelo). Gracias a esta división, es posible modificar o actualizar cada componente de manera independiente, sin afectar el funcionamiento del otro.

Además, esta arquitectura permite escalar los módulos de forma aislada, optimizando los recursos según la demanda, ya sea en el procesamiento del modelo o en la interacción del usuario.

A continuación, se describen los componentes principales de la arquitectura:

- Modelo entrenado (BOTNet)

Se utilizó el modelo BOTNet como núcleo del sistema de predicción. Este modelo fue previamente entrenado y ajustado, y posteriormente exportado en formato .pth para ser utilizado en producción. Se implementó su carga mediante la librería timm, y se ejecuta en modo evaluación, utilizando GPU o CPU según la disponibilidad del entorno.

- API REST (FastAPI)

Se desarrolló una API web con FastAPI, la cual recibe como entrada la URL de una imagen. Esta imagen es descargada, preprocesada y evaluada por el modelo BOTNet, retornando como resultado la clase predicha (“anémico” o “no anémico”) junto con el porcentaje de confianza. La API también incluye configuración de CORS para habilitar el acceso desde la aplicación web.

- Contenedor Docker

Para garantizar portabilidad y compatibilidad entre entornos, la API junto con el modelo y sus dependencias fue encapsulada en un contenedor Docker. Esto permite ejecutar todo el servicio en cualquier entorno compatible, sin necesidad de configuraciones adicionales.

- Despliegue en Google Cloud Run

El contenedor Docker fue desplegado en Google Cloud Run, lo que permitió poner la API en línea como un servicio escalable y seguro. Cloud Run facilita la ejecución automática del

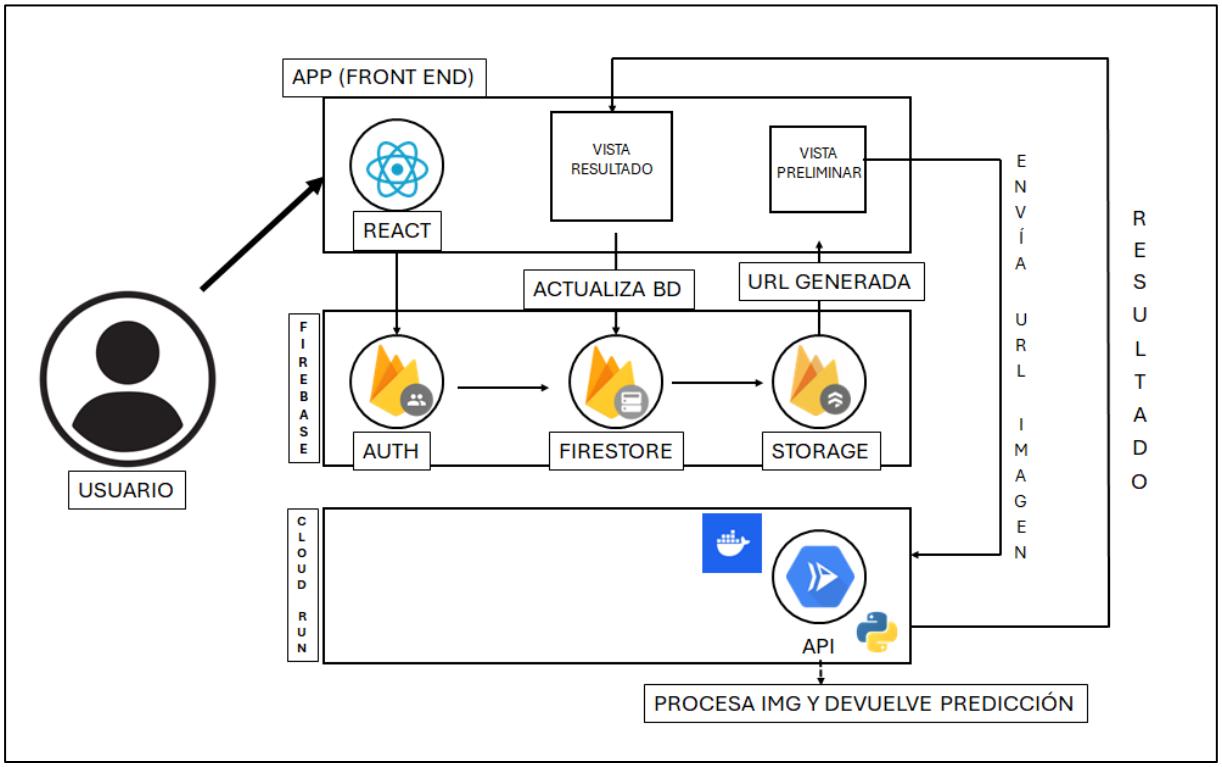
contenedor ante cada solicitud, gestionando los recursos de forma dinámica sin necesidad de mantener servidores físicos o virtuales.

- Aplicación web (ReactJS + Firebase)

La interfaz web fue desarrollada utilizando ReactJS y Material UI. Desde esta interfaz, el usuario puede autenticarse, registrar información del paciente, cargar una imagen y visualizar el resultado de la predicción. Firebase cumple un rol clave en tres aspectos:

- Authentication: gestión de usuarios.
- Storage: almacenamiento de imágenes, que generan una URL pública.
- Firestore: base de datos para registrar los datos del paciente y el resultado de cada evaluación.

Figura 221
Arquitectura aplicación Web



4.6.3 Flujo de funcionamiento del sistema

El sistema implementado permite la interacción fluida entre el usuario, la aplicación web, los servicios en la nube y el modelo de predicción alojado en un entorno escalable. El flujo de funcionamiento se desarrolla en las siguientes etapas:

1. Autenticación y registro del paciente

El usuario accede a la aplicación web y se autentica mediante Firebase Authentication. Una vez autenticado, puede registrar los datos del paciente, como nombre, edad y género, los cuales son almacenados en Firebase Firestore.

2. Carga de imagen

A través de la interfaz, el usuario selecciona y carga una imagen de la palma de la mano del paciente. Esta imagen es almacenada en Firebase Storage, que genera automáticamente una URL pública para su posterior consulta.

3. Comunicación con la API

La aplicación web envía una solicitud HTTP a la API REST desplegada en Google Cloud Run. Esta solicitud incluye la URL pública de la imagen almacenada.

4. Procesamiento y predicción

La API descarga la imagen desde la URL proporcionada, aplica el preprocesamiento necesario (redimensionamiento, normalización y conversión a tensor), y ejecuta la predicción utilizando el modelo BOTNet cargado en modo evaluación. Como resultado, se obtiene una clase predicha ("anémico" o "no anémico") junto con el nivel de confianza asociado.

5. Visualización y almacenamiento de resultados

El resultado de la predicción es devuelto a la aplicación web, donde se muestra de manera clara al usuario. Simultáneamente, se registra en Firestore junto con los datos del paciente, permitiendo así mantener un historial de evaluaciones.

Este flujo permite automatizar la predicción médica de forma remota, segura y eficiente, representando un avance significativo en la aplicación práctica del modelo entrenado en contextos reales de uso.

4.6.4 Interfaz funcional de la aplicación desplegada

- El usuario inicia el proceso accediendo a la interfaz principal, donde debe iniciar sesión, tal como se muestra en la Figura XX

Figura 222

Inicio de sesión de usuario



- Después, el usuario tendrá la posibilidad de agregar un nuevo paciente, completando el formulario que se visualiza en la Figura XXFigura 223 Registro de paciente



- Posteriormente, el usuario tendrá la posibilidad de elegir si carga una foto existente o si abre la cámara para la captura de la imagen, tal como se puede ver en la Figura XX

Figura 224

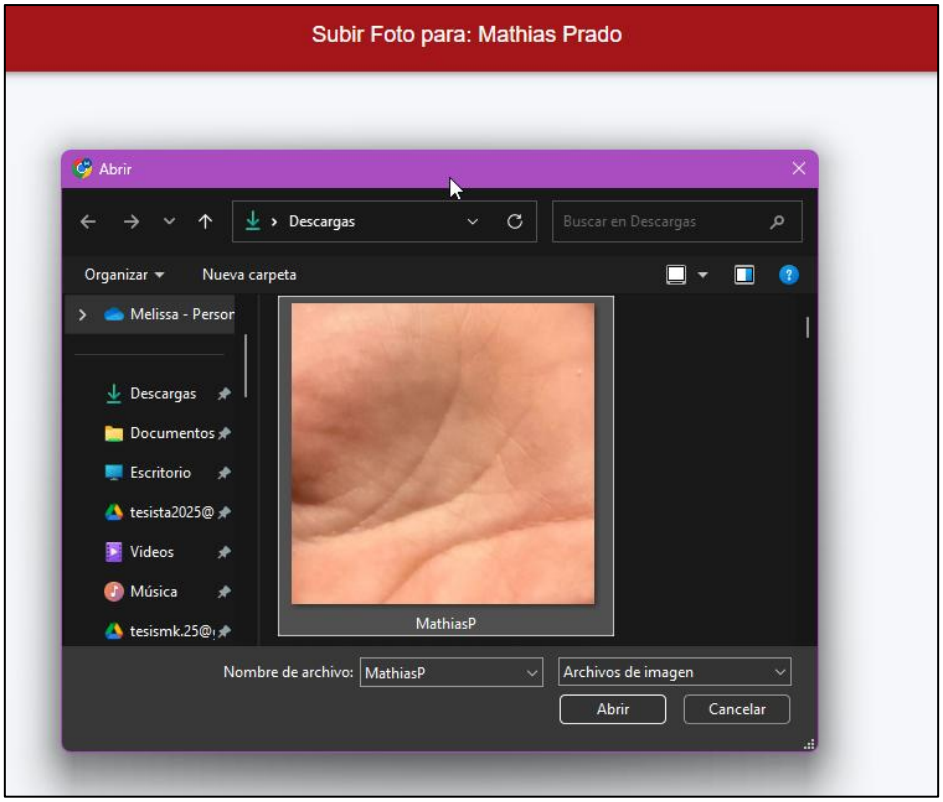
Selección o toma de foto de palma de mano



- Se abrirá una ventana en la que el usuario tendrá la posibilidad de elegir la imagen de la palma de la mano del paciente.

Figura 225

Selección de imagen



- Por consiguiente, se visualizará una vista previa de la imagen cargada, tal como se muestra en la Figura XX.

Figura 226

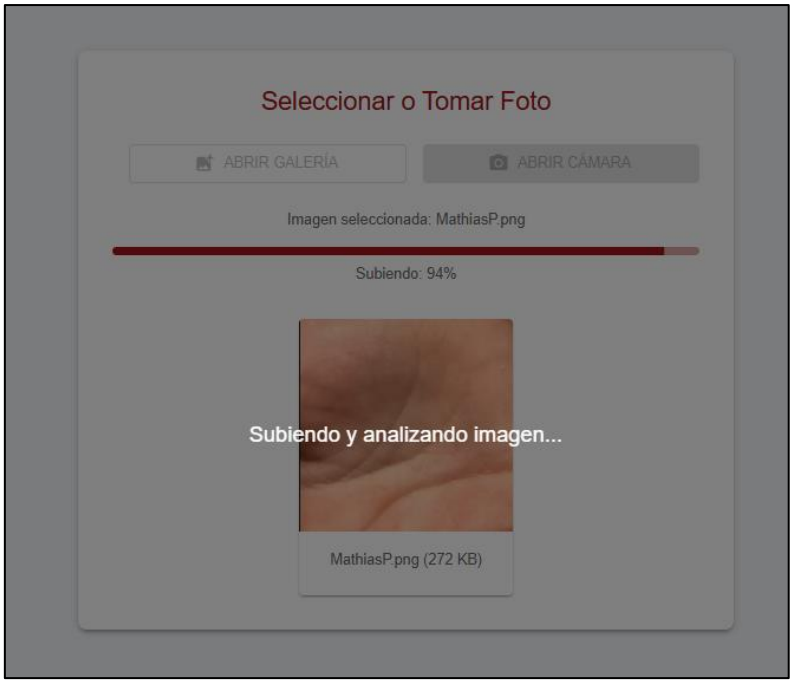
Previsualización de imagen



- Luego de dar clic a “Subir Imagen”, aparecerá un mensaje que indicará que el análisis se está realizando, como lo evidencia la Figura XX.

Figura 227

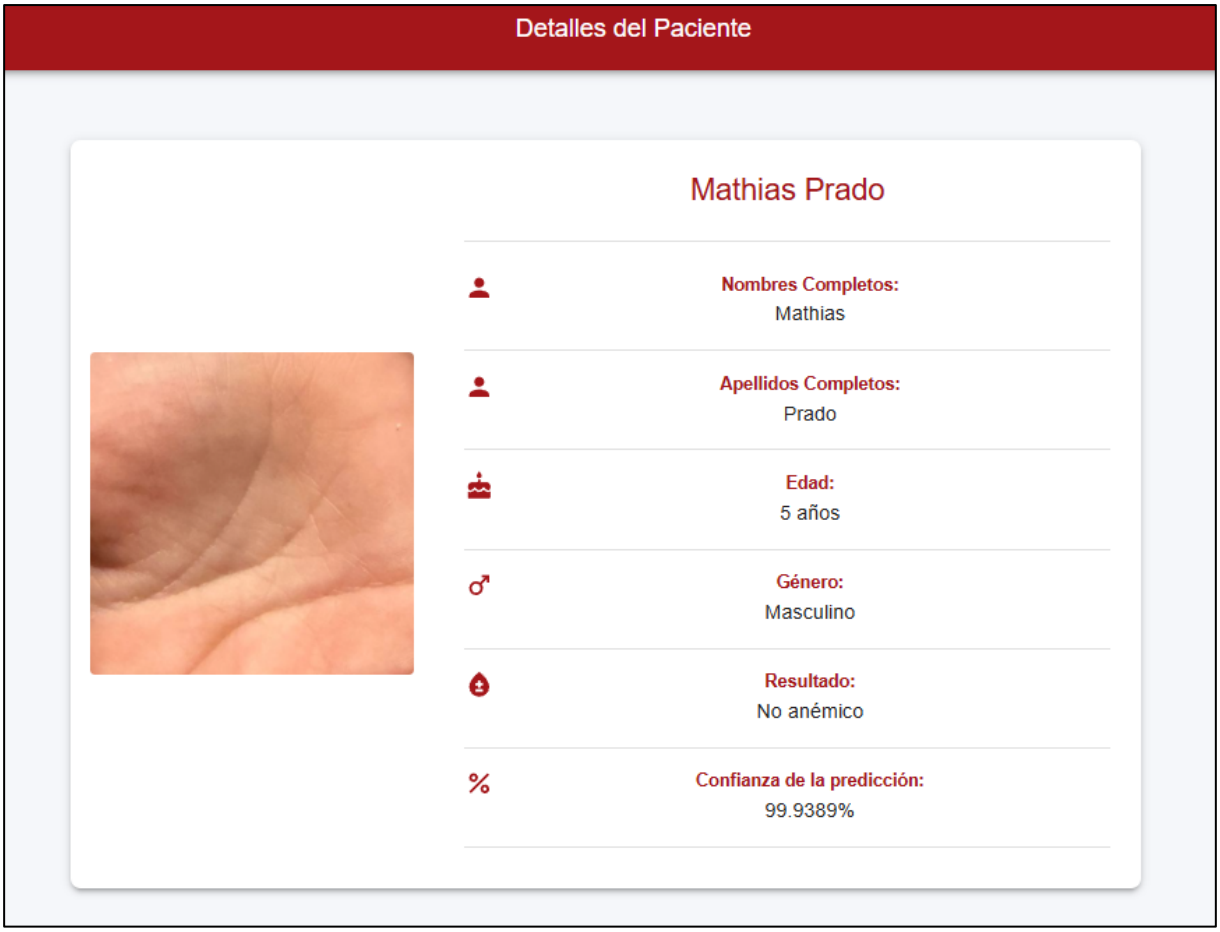
Imagen en proceso de análisis



- Finalmente, luego de unos segundos, el usuario tendrá la posibilidad de visualizar el resultado de la predicción donde se mostrará la información del paciente, la imagen cargada, el resultado y el porcentaje de confianza.

Figura 228

Resultado de análisis



CAPÍTULO V: Conclusiones y Recomendaciones

5.1 Conclusiones

En el presente capítulo se exponen las conclusiones generales derivadas del desarrollo completo de la investigación titulada Modelo híbrido para la detección de la anemia a partir de imágenes de las palmas de las manos utilizando Vision Transformer y Machine Learning. Dichas conclusiones responden al cumplimiento del objetivo general y los objetivos específicos formulados al inicio del estudio.

Como primer paso, se identificaron y analizaron las características de la base de datos empleada, conformada por imágenes de palmas de manos de niños con y sin anemia, provenientes del repositorio Mendeley Data. Se verificó que el conjunto de datos contenía imágenes originales y aumentadas, y se clasificó en dos categorías: “anemic” y “non-anemic”. Esta caracterización permitió establecer una base sólida para el entrenamiento de modelos, cumpliéndose así el primer objetivo específico.

Posteriormente, se aplicaron técnicas de preprocesamiento para asegurar la homogeneidad del conjunto de datos. Entre las acciones realizadas se incluyeron el filtrado de imágenes duplicadas, el redimensionamiento uniforme, el aumento de datos en clases minoritarias y la organización estructurada en subconjuntos de entrenamiento, validación y prueba. Estas operaciones mejoraron la calidad y balance del dataset, lo cual facilitó la implementación eficiente de modelos de aprendizaje, validando el cumplimiento del segundo objetivo específico.

En la fase de modelado, se entrenaron cinco arquitecturas Vision Transformer (ViT, Swin, PiT, MobileViT y BOTNet), tanto en su configuración original como con ajuste de hiperparámetros. El modelo BOTNet con tuning obtuvo los mejores resultados dentro del enfoque puro, con un accuracy de 0.9925, F1-score de 0.9925 y AUC-ROC de 0.9998. Aunque

MobileViT con tuning alcanzó métricas perfectas (1.0000 en todas las categorías), se consideró un posible sobreajuste, por lo que no fue seleccionado como modelo definitivo. Posteriormente, se construyó un modelo híbrido, en el que MobileViT fue utilizado como extractor de características y SVM como clasificador. Esta combinación superó al resto de configuraciones, alcanzando un accuracy de 0.9975, F1-score de 0.9975 y AUC-ROC de 1.0000, sin mostrar síntomas de sobreajuste. Este resultado confirmó la solidez del enfoque híbrido y permitió cumplir con el tercer objetivo específico.

Finalmente, se evaluó el desempeño de todos los modelos aplicando métricas estándar como accuracy, precision, recall, F1-score y AUC-ROC, lo que facilitó una comparación cuantitativa objetiva. A través de este análisis se evidenció que el enfoque híbrido ofrecía una mejor capacidad discriminativa y generalización frente a los modelos Vision Transformer utilizados como clasificadores completos. Por tanto, se cumplió también el cuarto objetivo específico, validando el uso de métricas como criterio robusto para la selección del mejor modelo.

En síntesis, se desarrolló exitosamente un modelo híbrido para la detección de anemia infantil a partir de imágenes de palmas de manos, integrando Vision Transformer y técnicas de Machine Learning. Este enfoque demostró ser preciso, escalable y potencialmente útil en contextos de diagnóstico temprano no invasivo.

5.2 Recomendaciones

A partir de los hallazgos y conclusiones de esta investigación, se proponen las siguientes recomendaciones con miras a mejorar, validar y ampliar el alcance del modelo propuesto:

Se recomienda ampliar la base de datos utilizada, incorporando imágenes de palmas de manos obtenidas en diversas condiciones de captura, tales como diferentes niveles de iluminación, fondos, y dispositivos móviles de uso común. Esto permitiría mejorar la capacidad de generalización del modelo y preparar su aplicación en entornos reales. Asimismo, sería

pertinente considerar la recolección de nuevas muestras en distintos contextos geográficos y sociales, con énfasis en poblaciones locales. En ese sentido, se sugiere que futuras investigaciones construyan una base de datos original desde el inicio, recolectada íntegramente en el Perú, a fin de asegurar mayor representatividad y control sobre las condiciones de captura. En el presente estudio, la base de datos empleada fue descargada de un repositorio abierto, pero presentaba limitaciones, como imágenes previamente recortadas y procesadas, lo que restringió parcialmente el diseño experimental.

Desde el enfoque metodológico, sería recomendable explorar nuevas variantes de modelos de clasificación, tanto en el ámbito del aprendizaje profundo como del aprendizaje automático, así como comparar distintas configuraciones y combinaciones entre ellos. También se podrían evaluar nuevos conjuntos de hiperparámetros y mecanismos de validación cruzada que permitan optimizar el rendimiento sin comprometer la generalización.

Finalmente, dado que en esta investigación se desarrolló una aplicación funcional que permite cargar imágenes y recibir un resultado predictivo sobre la presencia o ausencia de anemia, se recomienda realizar pruebas de campo en contextos reales. Estas pruebas deberían contemplar la validación técnica del modelo, así como la facilidad de uso, la rapidez de respuesta y la percepción de los usuarios. Esto contribuiría a consolidar su utilidad como una herramienta de apoyo al diagnóstico temprano de anemia infantil, especialmente en zonas con recursos limitados, donde se requieren soluciones accesibles, no invasivas y de bajo costo.

Bibliografía

Alaca, Y. (2025). Machine learning via DARTS-Optimized MobileViT models for pancreatic Cancer diagnosis with graph-based deep learning. *BMC Medical Informatics and Decision Making*, *25*(1), 81. https://doi.org/10.1186/s12911-025-02923-x

American Cancer Society. (2023). *Anemia (Recuento bajo de glóbulos rojos)*. https://www.cancer.org/es/cancer/como-sobrellevar-el-cancer/efectos-secundarios/recuentos-sanguineos-bajos/anemia.html

Appiahene, P., Asare, J. W., Donkoh, E. T., Dimauro, G., & Maglietta, R. (2023). Detection of iron deficiency anemia by medical images: a comparative study of machine learning algorithms. *BioData Mining*, *16*(1), 2. https://doi.org/10.1186/s13040-023-00319-z

Appiahene, P., Dogbe, S. S. D., Kobina, E. E. Y., Dartey, P. S., Afrifa, S., Donkoh, E. T., & Asare, J. W. (2023). Application of ensemble models approach in anemia detection using images of the palpable palm. *Medicine in Novel Technology and Devices*, *20*, 100269. https://doi.org/10.1016/J.MEDNTD.2023.100269

Asare, J. W., APPIAHENE, P., & DONKOH, E. (2022). *Anemia Detection using Palpable Palm Image Datasets from Ghana*. *1*. https://doi.org/10.17632/CCR8CM22VZ.1

Asare, J. W., Appiahene, P., Donkoh, E. T., & Dimauro, G. (2023). Iron deficiency anemia detection using machine learning models: A comparative study of fingernails, palm and conjunctiva of the eye images. *Engineering Reports*, *5*(11), e12667. https://doi.org/https://doi.org/10.1002/eng2.12667

Balarajan, Y., Ramakrishnan, U., Özaltin, E., Shankar, A. H., & Subramanian, S. V. (2011). Anaemia in low-income and middle-income countries. *The Lancet*, *378*(9809), 2123–2135. https://doi.org/10.1016/S0140-6736(10)62304-5

Berghout, T. (2024). Joint Image Processing with Learning-Driven Data Representation and Model Behavior for Non-Intrusive Anemia Diagnosis in Pediatric Patients. *Journal of Imaging*, *10*(10). https://doi.org/10.3390/jimaging10100245

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning* (Springer, Ed.). https://link.springer.com/book/9780387310732

Breiman, L. (2001). Random Forests. *Machine Learning*, *45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Camaschella, C. (2015). Iron-Deficiency Anemia. *New England Journal of Medicine*, *372*(19), 1832–1843. https://doi.org/10.1056/NEJMra1401038

Campa Fernández, V. M. (2017). *Análisis de Imágenes de Microscopía con ImageJ*.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2020). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. http://arxiv.org/abs/2010.11929

Dulhare, U. N., Ahmad, K., & Ahmad, K. A. Bin (Eds.). (2020). *Machine Learning and Big Data*. Wiley. https://doi.org/10.1002/9781119654834

Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, *27*(8), 861–874. https://doi.org/10.1016/J.PATREC.2005.10.010

Gardner, W. M., Razo, C., McHugh, T. A., Hagins, H., Vilchis-Tella, V. M., Hennessy, C., Taylor, H. J., Perumal, N., Fuller, K., Cercy, K. M., Zoeckler, L. Z., Chen, C. S., Lim, S. S., Aali, A., Abate, K. H., Abd-Elsalam, S., Abdurehman, A. M., Abebe, G., Abidi, H., … Kassebaum, N. J. (2023). Prevalence, years lived with disability, and trends in anaemia burden by severity and cause, 1990-2021: findings from the Global Burden of Disease Study 2021. *The Lancet Haematology*, *10*(9), e713–e734. https://doi.org/10.1016/S2352-3026(23)00160-6

Gonzalez, R., & Woods, R. (2017). *Digital Image Processing Global Edition*. Pearson Deutschland. https://elibrary.pearson.de/book/99.150005/9781292223070

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning* (The MIT Press, Ed.).

Harrington, P. (2012). *Machine Learning in Action*. Manning. http://ieeexplore.ieee.org/document/10279875

Heo, B., Yun, S., Han, D., Chun, S., Choe, J., & Oh, S. J. (2021). *Rethinking Spatial Dimensions of Vision Transformers*.

Hernández Sampieri, R., Fernández Collado, C., María del Pilar Baptista Lucio, D., & Méndez Valencia Christian Paulina Mendoza Torres, S. (2017). *Metodología de Investigación* (S. A. D. C. V. McGRAW-HILL / INTERAMERICANA EDITORES, Ed.).

Hiran, K., Lakhwani, Dr. K., Doshi, Dr. R., & Kumar Jain, R. (2021). Machine Learning: Master Supervised and Unsupervised Learning Algorithms with Real Examples (English Edition). En *Machine Learning* (pp. 1–294). Sciendo. https://doi.org/10.0000/9789391392352-001

Huffington Post, & Pérez de Pablos, S. (2024). *Estos son los tres análisis de sangre cruciales para prevenir enfermedades graves*. https://www.huffingtonpost.es/life/salud/estos-son-tres-analisis-sangre-cruciales-prevenir-saludbr.html

Iglesias Vázquez, L., Valera, E., Villalobos, M., Tous, M., & Arija, V. (2019). Prevalence of Anemia in Children from Latin America and the Caribbean and Effectiveness of Nutritional Interventions: Systematic Review and Meta–Analysis. *Nutrients*, *11*(1). https://doi.org/10.3390/nu11010183

Ildefonso, A., & Ticona, W. (2024). Proposal of a Computational Vision Model for the Pre-diagnosis of Anemia Based on the Image of the Ocular Conjunctiva. En R. Silhavy & P. Silhavy (Eds.), *Artificial Intelligence Algorithm Design for Systems* (pp. 497–518). Springer Nature Switzerland.

Instituto Nacional de Estadística e Informática (INEI). (2023). *Metodología del Indicador de Anemia en Niñas y Niños de 6 a 59 Meses*. https://cdn.www.gob.pe/uploads/document/file/6088704/5387581-metodologia-del-indicador-de-anemia.pdf

Kalantri, A., Karambelkar, M., Joshi, R., Kalantri, S., & Jajoo, U. (2010). Accuracy and reliability of pallor for detecting anaemia: a hospital-based diagnostic accuracy study. *PloS one*, *5*(1), e8545. https://doi.org/10.1371/journal.pone.0008545

Kelleher, J. D., Mac Namee, B., & D'Arcy, A. (2015). *Fundamentals of machine learning for predictive data analytics : algorithms, worked examples, and case studies* (MIT Press, Ed.; 1st ed.).

Kesarwani, A., Das, S., Dalui, M., Kisku, D. R., Sen, B., Roy, S., & Basu, A. (2023). Non-invasive anaemia detection by examining palm pallor: A smartphone-based approach. *Biomedical Signal Processing and Control*, *79*, 104045. https://doi.org/10.1016/J.BSPC.2022.104045

Khan, S., Naseer, M., Hayat, M., Zamir, S. W., Khan, F. S., & Shah, M. (2022). Transformers in Vision: A Survey. *ACM Computing Surveys*, *54*(10s), 1–41. https://doi.org/10.1145/3505244

Kuhn, M., & Johnson, K. (2013). *Applied Predictive Modeling*. Springer New York. https://doi.org/10.1007/978-1-4614-6849-3

Lantz, B. (2013). *Machine learning with R*. Packt Publishing. http://site.ebrary.com/id/10794279

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep Learning. *Nature*, *521*, 436–444. https://doi.org/10.1038/nature14539

Lee, W. (2019). *Python® Machine Learning*. Wiley. https://doi.org/10.1002/9781119557500

Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., & Guo, B. (2021). *Swin Transformer: Hierarchical Vision Transformer using Shifted Windows*. https://doi.org/10.1109/ICCV48922.2021.00986

Lozoff, B. (2007). Iron Deficiency and Child Development. *Food and Nutrition Bulletin*, *28*(4_suppl4), S560–S571. https://doi.org/10.1177/15648265070284S409

Manesh, R. S., & Kohlwes, R. J. (2015). Palmar Crease Pallor. *Journal of General Internal Medicine*, *30*(7), 1034. https://doi.org/10.1007/s11606-014-3124-2

Mayo Clinic. (s/f-a). *Anemia*. Recuperado el 28 de febrero de 2025, de https://www.mayoclinic.org/es/diseases-conditions/anemia/symptoms-causes/syc-20351360

Mayo Clinic. (s/f-b). *Hemoglobin test. Mayo Clinic*. Recuperado el 6 de febrero de 2025, de https://www.mayoclinic.org/es/tests-procedures/hemoglobin-test/about/pac-20385075

McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, *5*(4), 115–133. https://doi.org/10.1007/BF02478259

Mehta, S., & Rastegari, M. (2021). *MobileViT: Light-weight, General-purpose, and Mobile-friendly Vision Transformer*. https://doi.org/10.48550/arXiv.2110.02178

Minsky, M., & Papert, S. A. (2017). *Perceptrons* (Expanded ed.). The MIT Press. https://doi.org/10.7551/mitpress/11301.001.0001

Mitchell, T. M. (1997). *Machine Learning* (McGraw-Hill, Ed.).

Müller, A. C., & Guido, S. (2017). *Introduction to Machine Learning with Python : a guide for data scientists*. O'Reilly.

National Heart Lung and Blood Institute. (2021). *Your guide to anemia*. https://www.nhlbi.nih.gov/es/resources/your-guide-anemia

Nemours KidsHealth. (s/f). *Obtención de un análisis de sangre (video)*. Recuperado el 26 de febrero de 2025, de https://kidshealth.org/es/kids/video-bldtest.html

Observatorio Medicina Salud y Sociedad del Colegio Médico del Perú. (2023). *Informe del Seminario: La Anemia Infantil en el Perú: Situación y Retos, una Nueva Perspectiva*. https://www.cmp.org.pe/wp-content/uploads/2023/11/INFORME-DEL-SEMINARIO-LA-ANEMIA-INFANTIL-EN-EL-PERU.pdf

Oppermann, A. (2023, diciembre 12). *What Is Deep Learning and How Does It Work?* https://builtin.com/machine-learning/deep-learning

Organización Mundial de la Salud. (2023). *WHO Global Anaemia estimates, 2021 Edition*. https://www.who.int/data/gho/data/themes/topics/anaemia_in_women_and_children

Osborne, A., Adeleye, K., Bangura, C., & Wongnaah, F. G. (2024). Trends and inequalities in anaemia prevalence among children aged 6–59 months in Ghana, 2003–2022. *International Journal for Equity in Health*, *23*(1), 231. https://doi.org/10.1186/s12939-024-02315-z

Pedia-Start. (s/f). Recuperado el 27 de febrero de 2025, de https://www.pedia-start.com/

Peksi, N., Yuwono, B., & Yanu F, M. (2021). Classification of Anemia with Digital Images of Nails and Palms using the Naive Bayes Method. *Telematika*, *18*, 118. https://doi.org/10.31315/telematika.v18i1.4587

Pineda, E. B., de Alvarado, E. L., de Canales, F. H., & Organización Panamericana de la Salud. (1994). Metodologia de la investigación: Manual para el desarrollo de personal de salud. *Serie PALTEX para Ejecutores de Programas de Salud;35*. https://iris.paho.org/handle/10665.2/3132

RadiologyInfo.org. (2024). *Anemia: Diagnóstico y evaluación*. https://www.radiologyinfo.org/es/info/anemia

Ramzan, M., Sheng, J., Saeed, M. U., Wang, B., & Duraihem, F. Z. (2024). Revolutionizing anemia detection: integrative machine learning models and advanced attention mechanisms. *Visual Computing for Industry, Biomedicine, and Art*, *7*(1), 18. https://doi.org/10.1186/s42492-024-00169-4

Ren, H., Guo, J., Cheng, S., & Li, Y. (2024). Pooling-based Visual Transformer with low complexity attention hashing for image retrieval. *Expert Systems with Applications*, *241*, 122745. https://doi.org/10.1016/j.eswa.2023.122745

Reyes Narvaez, S. E., Contreras Contreras, A. M., & Oyola Canto, M. S. (2019). Anemia y desnutrición infantil en zonas rurales: impacto de una intervención integral a nivel comunitario. *Revista de Investigaciones Altoandinas - Journal of High Andean Research*, *21*(3), 205–214. https://doi.org/10.18271/ria.2019.478

Ritchie, H., & Roser, M. (2017). *Micronutrient deficiency. Our World in Data.* https://ourworldindata.org/micronutrient-deficiency

Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, *65*(6), 386–408. https://doi.org/10.1037/h0042519

Roychowdhury, S., Hage, P., & Vasquez, J. (2017). Azure-Based Smart Monitoring System for Anemia-Like Pallor. *Future Internet*, *9*(3). https://doi.org/10.3390/fi9030039

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, *323*(6088), 533–536. https://doi.org/10.1038/323533a0

Sabino, C. (1992). *EL PROCESO DE INVESTIGACION*.

Sarah, Narayan, S. S., Arif, I., Shalu, H., & Kadiwala, J. (2020). *A smartphone based multi input workflow for non-invasive estimation of haemoglobin levels using machine learning techniques*.

Sheskin, D. J. (2020). *Handbook of Parametric and Nonparametric Statistical Procedures*. Chapman and Hall/CRC. https://doi.org/10.1201/9780429186196

Sheth, T. N., Choudhry, N. K., Bowes, M., & Detsky, A. S. (1997). The Relation of Conjunctival Pallor to the Presence of Anemia. *Journal of General Internal Medicine*, *12*(2), 102–106. https://doi.org/10.1007/s11606-006-5004-x

Srinivas, A., Lin, T.-Y., Parmar, N., Shlens, J., Abbeel, P., & Vaswani, A. (2021). *Bottleneck Transformers for Visual Recognition*.

Tian, Y., Zhu, J., Zhang, L., Mou, L., Zhu, X., Shi, Y., Ma, B., & Zhao, W. (2023). A Swin Transformer-Based Model for Thyroid Nodule Detection in Ultrasound Images. *Journal of Visualized Experiments*, *194*. https://doi.org/10.3791/64480

Wang, S., Celebi, M. E., Zhang, Y. D., Yu, X., Lu, S., Yao, X., Zhou, Q., Miguel, M. G., Tian, Y., Gorriz, J. M., & Tyukin, I. (2021). Advances in Data Preprocessing for Biomedical Data Fusion: An Overview of the Methods, Challenges, and Prospects. *Information Fusion*, *76*, 376–421. https://doi.org/10.1016/J.INFFUS.2021.07.001

World Health Organization. (s/f). *Anaemia*. Recuperado el 25 de febrero de 2025, de https://www.who.int/health-topics/anaemia

Yakimov, B., Buiankin, K., Denisenko, G., Bardadin, I., Pavlov, O., Shitova, Y., Yuriev, A., Pankratieva, L., Pukhov, A., Shkoda, A., & Shirshin, E. (2024). Dataset of human skin and fingernails images for non-invasive haemoglobin level assessment. *Scientific Data*, *11*(1), 1070. https://doi.org/10.1038/s41597-024-03895-9

Zhang, Y., Wang, J., Gorriz, J. M., & Wang, S. (2023). Deep Learning and Vision Transformer for Medical Image Analysis. *Journal of Imaging*, *9*(7). https://doi.org/10.3390/jimaging9070147

Anexos

Anexo 1

Matriz de Consistencia

Modelo híbrido para la detección de la anemia a partir de imágenes de las palmas de las manos utilizando Vision Transformer y Machine Learning			
PROBLEMA GENERAL	**OBJETIVO GENERAL**	**HIPÓTESIS GENERAL**	**VARIABLES**
¿Cómo desarrollar un modelo híbrido para la detección de la anemia a partir de imágenes de las palmas de las manos utilizando Vision Transformer y Machine Learning?	Desarrollar un modelo híbrido basado en Vision Transformer y Machine Learning para detectar anemia a partir de imágenes de las palmas de las manos.	El desarrollo de un modelo híbrido basado en Vision Transformer y Machine Learning permite detectar la anemia a partir de imágenes de las palmas de las manos.	Variable Independiente: **Vision Transformer** Según Dosovitskiy et al. (2020), es un modelo de red neuronal basado en la arquitectura Transformer que permite clasificar imágenes dividiéndolas en pequeños parches, los cuales son procesados como secuencias mediante mecanismos de autoatención, logrando así capturar relaciones espaciales sin el uso de convoluciones tradicionales, lo que facilita una comprensión global de la imagen desde las primeras capas del modelo.

			Machine Learning Según Dulhare et al. (2020), es un campo de la inteligencia artificial que optimiza el rendimiento de los sistemas computacionales mediante la programación basada en datos y experiencias previas, permitiendo que los programas mejoren su desempeño con o sin supervisión humana.
PROBLEMAS ESPECIFICOS	**OBJETIVOS ESPECIFICOS**	**HIPOTESIS ESPECIFICAS**	Variable dependiente:
¿Cuáles son las características de la base de datos de imágenes de palmas de manos de personas con y sin anemia utilizada en el estudio?	Identificar y describir las características de la base de datos de imágenes de palmas de manos de personas con y sin anemia utilizada en el estudio.	La base de datos de imágenes de palmas de manos utilizada en el estudio contiene características adecuadas para ser utilizada en el desarrollo de un modelo híbrido de detección de anemia.	**Anemia** Según Gardner et al. (2023), la anemia es un trastorno caracterizado por la reducción de los niveles de hemoglobina o del número de glóbulos rojos, lo que limita la capacidad de la sangre para transportar oxígeno adecuadamente. Entre sus manifestaciones
¿Qué técnicas de preprocesamiento se pueden aplicar para estandarizar las imágenes de palmas de manos utilizadas en la detección de la anemia?	Aplicar técnicas de preprocesamiento para estandarizar las imágenes de palmas de manos para la detección de la anemia.	La aplicación de técnicas de preprocesamiento permite estandarizar adecuadamente las imágenes de palmas de manos para el desarrollo de un modelo híbrido de detección de anemia.	

¿Qué técnicas de Vision Transformer y Machine Learning pueden utilizarse para construir un modelo híbrido de clasificación capaz de reconocer imágenes de palmas de manos de personas con y sin anemia?	Aplicar técnicas de Vision Transformer y Machine Learning para construir un modelo híbrido de clasificación que reconozca imágenes de palmas de manos de personas con y sin anemia.	La utilización de técnicas basadas en Vision Transformer y Machine Learning permite construir un modelo híbrido de clasificación que reconoce adecuadamente imágenes de palmas de manos de personas con y sin anemia.	clínicas incluye la palidez, observable en zonas como la conjuntiva ocular, las palmas de las manos, el lecho ungueal y la lengua (Kalantri et al., 2010).
¿Qué métricas permiten evaluar el desempeño de la clasificación de imágenes de palmas de manos de personas con anemia y sin anemia?	Seleccionar y aplicar métricas que permitan evaluar el desempeño del modelo en la clasificación de imágenes de palmas de manos de personas con y sin anemia.	La aplicación de métricas como Accuracy, Precision, Recall, F1-score y AUC-ROC permite evaluar adecuadamente el desempeño del modelo híbrido de clasificación de imágenes de palmas de manos de personas con y sin anemia.	